

Fundamentos del análisis isogeométrico. Aplicación al problema de Poisson



Carlos Palomera Oliva
Trabajo de fin de grado de Matemáticas
Universidad de Zaragoza

Directores del trabajo: Álvaro Pé de la Riva y
Carmen Rodrigo Cardiel
Junio de 2026

Abstract

Isogeometric Analysis (IGA) was developed to close the gap between Computer-Aided Design (CAD) and Finite Element Analysis (FEA). In FEA, precise CAD geometries must be converted into approximate computational meshes, a process that introduces geometric errors and high computational costs [1]. By directly employing the same mathematical basis functions used in geometric modeling to approximate the solution of partial differential equations, IGA avoids these approximations and provides a truly unified framework for design and analysis [1, 2]. By employing the same mathematical basis functions used in geometric modeling to approximate the solution of partial differential equations, IGA allows an exact representation of the computational domain and provides a unified framework for design and analysis [1, 2].

This work explores the foundations of IGA, with particular emphasis on its application to the Poisson equation as a benchmark problem with the following formulations:

Strong Formulation Let $\Omega \subset \mathbb{R}^d$ be an open and bounded set (where $n \in \mathbb{N}$), with a sufficiently smooth boundary $\partial\Omega = \Gamma = \overline{\Gamma_D \cup \Gamma_R \cup \Gamma_N}$, such that $\Gamma_D \cap \Gamma_R = \Gamma_D \cap \Gamma_N = \Gamma_R \cap \Gamma_N = \emptyset$.

We seek a solution function $u : \Omega \rightarrow \mathbb{R}$, satisfying $u \in C^2(\Omega) \cap C^1(\overline{\Omega})$ such that:

$$\begin{cases} -\Delta u = f & \text{in } \Omega, \\ u = g & \text{on } \Gamma_D, \quad (\text{Dirichlet condition}), \\ \nabla u \cdot \mathbf{n} = h & \text{on } \Gamma_N, \quad (\text{Neumann condition}), \\ \alpha u + \beta \nabla u \cdot \mathbf{n} = r & \text{on } \Gamma_R, \quad (\text{Robin condition}). \end{cases}$$

Where the elements satisfy the following properties:

- $\mathcal{L}^{n-1}(\Gamma_D \cup \Gamma_R) > 0$.
- $f : \Omega \rightarrow \mathbb{R}, f \in L^2(\Omega)$.
- $g : \Gamma_D \rightarrow \mathbb{R}, g \in H^{1/2}(\Gamma_D)$.
- $h : \Gamma_N \rightarrow \mathbb{R}, h \in L^2(\Gamma_N)$.
- $r : \Gamma_R \rightarrow \mathbb{R}, r \in L^2(\Gamma_R)$.
- $\mathbf{n} : \Gamma \rightarrow \mathbb{R}^n$, outward unit normal vector to the boundary Γ .
- $\alpha, \beta \in \mathbb{R}$, such that $\frac{\alpha}{\beta} \geq 0$.

Weak Formulation We seek $u \in H^1(\Omega)$ with $u = g$ on Γ_D , such that for all test functions $v \in V_0(\Omega) = \{w \in H^1(\Omega) \mid w = 0 \text{ on } \Gamma_D\}$, the following holds:

$$a(u, v) = L(v), \tag{1}$$

where:

$$a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma.$$

$$\blacksquare L(v) = \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma.$$

The mathematical core of this work begins with a brief review of geometric modeling, progressing from Bézier curves to B-Splines and ultimately to Non-Uniform Rational B-Splines (NURBS). While Bézier curves provide a simple and intuitive framework, their lack of local control and the strong coupling between polynomial degree and the number of control points limit their applicability in complex geometries.

To overcome these issues, B-Splines are introduced [3]. Given a knot vector $U = \{u_0, u_1, \dots, u_s\}$, the corresponding basis functions $N_{i,p}(u)$ of degree $p \geq 1$ are defined recursively through the Cox-de Boor formula:

$$N_{i,p}(u) = \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u).$$

This construction provides local support and enables flexible refinement strategies.

Building on this framework, NURBS extend B-Splines by incorporating weights w_i , allowing for the exact representation of conic sections through a rational formulation [4]. The univariate NURBS basis functions are given by

$$\hat{R}_i^p(u) = \frac{N_{i,p}(u) w_i}{\sum_{j=0}^n N_{j,p}(u) w_j}.$$

For the construction of bidimensional domains, these basis functions are extended via a tensor-product approach. Given a control net $\mathbf{P}_{i,j}$ and a corresponding set of weights $w_{i,j}$, a NURBS surface is defined over the parametric domain $\hat{\Omega} = [u_0, u_s] \times [v_0, v_r]$ as

$$\mathbf{S}(u, v) = \sum_{i=0}^n \sum_{j=0}^m R_{i,j}^{p,q}(u, v) \mathbf{P}_{i,j} = \frac{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) M_{j,q}(v) w_{i,j} \mathbf{P}_{i,j}}{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) M_{j,q}(v) w_{i,j}}.$$

This tensor-product structure preserves the smoothness of the underlying representation while enabling the exact modeling of complex geometries. Moreover, refinement strategies such as h -refinement can be carried out through knot insertion, enriching the basis and increasing the number of degrees of freedom without modifying the original geometric parametrization.

The numerical analysis follows the isoparametric concept, where the solution space V_h is generated by the same NURBS bases used for the geometry. In this context, the open and bounded physical domain Ω is defined through the geometric mapping $\mathbf{G} : \hat{\Omega} \rightarrow \Omega$, ensuring a high global continuity of class C^{p-1} . The model problem addressed is the strong form of the Poisson equation, subject to mixed boundary conditions (Dirichlet on Γ_D , Neumann on Γ_N , and Robin on Γ_R):

$$-\Delta u = f \quad \text{in } \Omega.$$

To solve this problem numerically, its variational (or weak) formulation is derived. We seek a function $u \in V(\Omega)$ such that, $\forall v \in V_0(\Omega)$, the equation $a(u, v) = L(v)$ is satisfied, where the bilinear and linear forms are expanded as:

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} u v d\Gamma = \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma.$$

Next, the IGA framework is introduced within the isoparametric setting, where the same basis functions used to describe the geometry are also employed to approximate the solution. Following the Galerkin approach, the physical shape functions ϕ_j are obtained through the inverse geometric mapping, that is, $\phi_j = \hat{R}_j^p \circ \mathbf{G}^{-1}$. The discrete solution is then expressed as a linear combination of these functions,

$$u_h(\mathbf{x}) = \sum_{j=1}^{N_b} d_j \phi_j(\mathbf{x}),$$

where d_j denote the control variables and N_b is the total number of basis functions. Substituting this expression into the weak formulation leads to the linear system $\mathbf{Kd} = \mathbf{F}$.

All integrals in IGA are evaluated in the parametric domain $\hat{\Omega}$. For a two-dimensional element $\hat{\Omega}_e$, the entries of the local stiffness matrix involve the Jacobian matrix $\mathbf{J}$ associated with the geometric mapping:

$$\mathbf{K}_{i,j}^e = \int_{\hat{\Omega}_e} \left(\hat{\mathbf{v}} \hat{R}_j^{(p,q)} \right)^T (\mathbf{J}^{-1} \mathbf{J}^{-T}) \left(\hat{\mathbf{v}} \hat{R}_i^{(p,q)} \right) |\det \mathbf{J}| d\xi d\eta.$$

A key part of the analysis focuses on the properties of the resulting stiffness matrix $\mathbf{K}$, which is symmetric and sparse, with at most $(2p+1)^d$ non-zero entries per row. Its condition number scales as $\mathcal{O}(h^{-2})$, in line with classical theory. Moreover, error estimates guarantee optimal convergence rates of $\mathcal{O}(h^p)$ in the H^1 norm and $\mathcal{O}(h^{p+1})$ in the L^2 norm [5, 6].

Finally, the numerical implementation is described in detail. The code was developed in C++, making use of the Eigen library for sparse linear algebra, and includes an element-by-element assembly procedure. Numerical integration is carried out using Gaussian–Legendre quadrature with $p_i + 1$ points in each parametric direction, which ensures sufficient accuracy when evaluating both the rational basis functions and the non-constant Jacobian determinant [7].

Special care is required in the treatment of boundary conditions, since NURBS basis functions are generally non-interpolatory at interior control points. For open knot vectors, Dirichlet conditions are imposed strongly by modifying the corresponding rows of the stiffness matrix and adjusting the load vector accordingly, preserving symmetry. Neumann and Robin conditions, on the other hand, are incorporated naturally through boundary integrals along the parametric domain.

The resulting linear systems are solved using a sparse LU factorization in the one-dimensional case, while for two-dimensional problems a Preconditioned Conjugate Gradient (PCG) method with incomplete Cholesky preconditioning is employed to improve computational efficiency and reduce memory requirements.

Using this implementation, several numerical experiments were performed to assess the accuracy of the method. Rather than focusing on isolated error values, the analysis examines the asymptotic convergence behavior under h -refinement. In both one- and two-dimensional test cases, the results show optimal convergence rates of $\mathcal{O}(h^{p+1})$ in the L^2 norm and $\mathcal{O}(h^p)$ in the H^1 norm, in full agreement with theoretical predictions.

These results confirm the robustness of the approach and highlight the potential of IGA as an effective alternative to traditional FEA, particularly due to its higher geometric accuracy and global continuity, which are key features for modern integrated design and analysis frameworks.

Índice general

Abstract	III
1. Introducción	1
1.1. Contexto y relación con el método de elementos finitos	1
1.2. Evolución del análisis isogeométrico	2
1.3. Objetivos	3
1.4. El problema modelo: la ecuación de Poisson	3
1.4.1. Formulación fuerte (o clásica)	4
1.4.2. Formulación variacional (o débil)	4
1.5. Estructura del documento	4
2. Construcción de curvas y superficies mediante CAD	5
2.1. Curvas de Bézier	5
2.1.1. Polinomios de Bernstein	5
2.1.2. Definición y propiedades	6
2.2. B-Splines	7
2.2.1. Funciones base B-Spline	7
2.2.2. Curvas B-Spline	8
2.2.3. Superficies B-Splines	9
2.3. NURBS	9
2.3.1. Funciones base NURBS	9
2.3.2. Curvas NURBS	9
2.3.3. Superficies NURBS	10
2.4. Refinamiento de curvas y superficies	11
2.4.1. Inserción de nudos	11
2.4.2. Elevación del grado polinomial	12
3. Análisis isogeométrico	15
3.1. Formulación variacional	15
3.2. Formulación discreta de Galerkin	16
3.3. Propiedades del sistema	17
3.4. Caso 1D	17
3.4.1. Elementos de la matriz global y el vector de carga	17
3.4.2. Ensamblado de la matriz de rigidez y el vector de cargas	18
3.4.3. Imposición de condiciones de contorno.	18
3.5. Caso 2D	19
3.5.1. Elementos de la matriz global y el vector de carga	19
3.5.2. Ensamblado de la matriz de rigidez y el vector de cargas	19
3.5.3. Imposición de condiciones de contorno	20
3.6. Convergencia en L2 y H1 y tiempo de computación teórico	21
3.6.1. Convergencia	21

3.6.2. Coste computacional	21
4. Resultados.	23
4.1. Resultados en 1D	23
4.2. Resultados en 2D	25
4.2.1. Problema 1	25
4.2.2. Problema 2	26
5. Conclusiones	27
5.1. Trabajo futuro	27
Bibliografía	29
Anexos	33
A.1. Espacios funcionales empleados	33
A.2. Implementación computacional	33
A.2.1. Lenguaje y librerías utilizados	33
A.2.2. Métodos numéricos empleados	34
A.2.3. Estructura del programa	34
A.3. Método de los elementos finitos.	35
A.4. Demostraciones omitidas	39
A.4.1. Paso de Solución Fuerte a Solución Variacional	39
A.4.2. Existencia y unicidad de la solución variacional (lema de Lax-Milgram)	40
A.4.3. Equivalencia de la solución de la formulación variacional y fuerte	42
A.4.4. Equivalencia de normas	43
A.4.5. Continuidad de la forma lineal dual	44
A.4.6. Propiedades de Bézier	44
A.4.7. Propiedades de los B-Splines	46
A.4.8. Derivada de las funciones base NURBS	49
A.4.9. Derivada de la curva NURBS	49
A.4.10. Comportamiento direccional de las superficies NURBS	50
A.4.11. Igualdad de las nuevas funciones base bajo refinamiento	50
A.4.12. Equivalencia de curvas bajo refinamiento	51
A.4.13. Proposiciones elevación del grado polinomial	52
A.4.14. Propiedades del sistema	54
A.4.15. Convergencia en IGA	56
A.5. Base teórica de los algoritmos de condiciones de contorno	57
A.5.1. Tipo Dirichlet	58
A.5.2. Tipo Neumann	58
A.5.3. Tipo Robin o mixtas	58
A.6. Explicación y parámetros de las figuras	58
A.6.1. Figura 1.1	58
A.6.2. Figura 2.5	58
A.7. Código omitido	61
A.7.1. FEM	61
A.7.2. Figuras de Bézier	62
A.7.3. Código fundamental para el funcionamiento en C++	63
A.7.4. Curvas de NURBS	98
A.7.5. Superficies de NURBS	100
A.7.6. Implementaciones de IGA	101

Capítulo 1

Introducción

1.1. Contexto y relación con el método de elementos finitos

El análisis isogeométrico (IGA) es un método numérico cuya finalidad es la aproximación de problemas de ecuaciones en derivadas parciales (EDPs). Su característica principal se basa en la construcción del dominio geométrico a partir de funciones *spline* y el uso de estas mismas funciones base para la aproximación de la solución de las EDPs; para ello, partiendo de la formulación variacional se establece el correspondiente problema discreto sobre un espacio de aproximación generado por las propias funciones isogeométricas, donde la aproximación se construye mediante una combinación lineal de dichas funciones base [1, 2]. Propuesto originalmente por Hughes, Cottrell y Bazilevs en el año 2005 [1], este método nace con el objetivo fundamental de unificar los campos del diseño asistido por ordenador (CAD) y el método de elementos finitos (FEM).

La diferencia principal de IGA respecto al método de los elementos finitos (FEM) tradicional es la eliminación del error geométrico introducido por la discretización en mallas poligonales [1], ya que IGA permite trabajar directamente con la geometría exacta del dominio a lo largo de todo el proceso. Tradicionalmente, la base fundamental de este método han sido las *Non-Uniform Rational B-Splines* (NURBS) [4], aunque actualmente abarca una familia más amplia de funciones base [5].

Para evidenciar estas diferencias metodológicas, la Tabla 1.1 recoge una comparación técnica directa entre IGA y FEM tradicional. En ella se resumen los criterios clave que diferencian ambos métodos, tales como el tratamiento de la geometría, la naturaleza interpolatoria de las funciones base, el nivel de continuidad global y los tipos de refinamiento disponibles.

Tabla 1.1: Comparación técnica entre FEM tradicional e IGA.

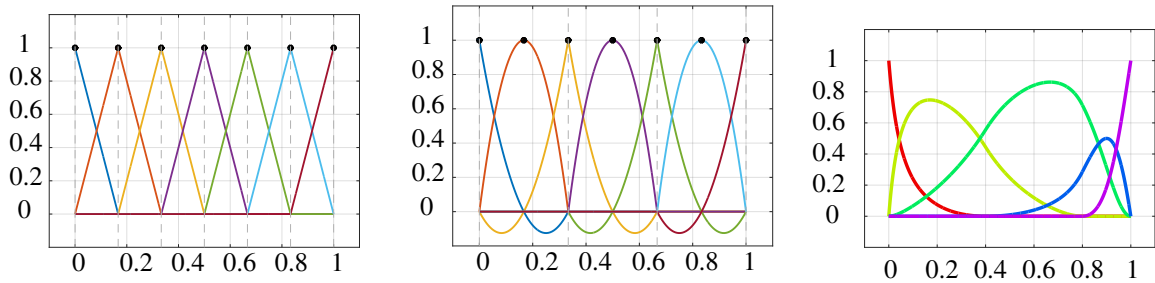
Criterio	FEM tradicional	Análisis isogeométrico (IGA)
Geometría	Aproximada. Malla poligonal que introduce error geométrico.	Exacta. Geometría idéntica a la del modelo empleado en CAD (NURBS, T-Splines, etc.).
Funciones base	Interpolatorias (nodos).	No interpolatorias (puntos de control).
Continuidad global	Baja ($\mathcal{C}^0$).	Alta (hasta $\mathcal{C}^{p-1}$, donde p es el grado polinomial empleado).
Refinamiento	h y p -refinamiento.	h , p y k -refinamiento.
Mapeo inverso	Directo. Inversión simple o analítica.	Complejo. Requiere evaluar el jacobiano de la transformación geométrica, lo cual aumenta el coste computacional.

Como complemento a la comparativa anterior, conviene analizar también el coste computacional asociado al ensamblaje de las matrices del sistema. Cuando en IGA se emplea una cuadratura estándar elemento a elemento (similar a la utilizada en FEM), dicho coste resulta elevado, ya que escala asintóticamente como $\mathcal{O}(p^{2d})$ por grado de libertad, donde p es el grado polinomial y d la dimensión espacial [7]. Este incremento se debe, en gran medida, al mayor soporte de las funciones base en comparación con las interpolatorias clásicas.

No obstante, esta dificultad puede mitigarse aprovechando la estructura de producto tensorial propia de los espacios isogeométricos. En particular, mediante técnicas como la factorización de sumas (*sum-factorization*), es posible reducir de forma significativa el coste de integración y ensamblaje hasta $\mathcal{O}(p^{d+1})$ [7]. De este modo, la eficiencia computacional de IGA resulta comparable (e incluso superior en regímenes de alto orden) a la de FEM tradicional.

Nota: Si se desea conocer más en detalle los distintos elementos de FEM para comparar más meticulosamente ambos métodos, refiérase al Anexo A.3.

Seguidamente se muestran las gráficas de distintas funciones base unidimensionales en la Figura 1.1, en la Figura 1.1a se muestran funciones base FEM para grado polinomial $p = 1$, y un número de elementos $h = 6$, en la Figura 1.1b se muestran funciones base FEM para grado polinomial $p = 2$ y $h = 3$ elementos y en la Figura 1.1c funciones base NURBS con grado polinomial $p = 2$ y $h = 3$ elementos.



(a) Funciones base FEM, $p = 1$. (b) Funciones base FEM, $p = 2$. (c) Funciones base NURBS, $p = 2$.

Figura 1.1: Comparativa de funciones base entre FEM y NURBS.

1.2. Evolución del análisis isogeométrico

IGA está íntimamente relacionado con el CAD, motivo por el cual resulta fundamental analizar la evolución histórica del diseño geométrico para comprender su formulación teórica. La formalización de las bases geométricas modernas comenzó a inicios de 1960, impulsada por las necesidades de representación y construcción de curvas, superficies, estructuras complejas... en los modelos empleados de las industrias de la aeronáutica y automoción. En este contexto destacan las contribuciones de Pierre Bézier y Paul de Casteljau, quienes sentaron las bases algorítmicas de curvas paramétricas. Posteriormente, en 1967, Ahlberg, Nilson y Walsh publicaron su obra *Theory of Splines and Their Applications* [8], pionera en formalizar la teoría de los splines. Durante las décadas de 1970 y 1980, estas herramientas matemáticas evolucionaron hasta el estándar de los *Non-Uniform Rational B-Splines* (NURBS) [4], consolidando las bases funcionales que, años más tarde, constituirían los fundamentos de IGA.

De forma paralela, el método de los elementos finitos surgió entre los años 50 y 60 como la principal herramienta para aproximar soluciones a problemas de ecuaciones en derivadas parciales en dominios complejos. Sin embargo, este método presenta la limitación inherente de trabajar sobre mallas poligonales discretas que aproximan la geometría del dominio, en lugar de utilizar la representación exacta que se estaba desarrollando simultáneamente en CAD. Esta separación generó una profunda brecha entre el análisis y el diseño, la cual perduró durante décadas, obligando a introducir un costoso proceso de conversión geométrica entre el diseño y el análisis computacional. Para resolver definitivamente este cuello de botella, Thomas J.R. Hughes, J. Austin Cottrell y Yuri Bazilevs introdujeron IGA en el año 2005 [1], unificando por primera vez el análisis y el diseño geométrico al emplear las propias bases paramétricas de CAD para aproximar la solución del problema físico.

Actualmente, el uso del análisis isogeométrico se ha consolidado en diversas industrias en las que la

exactitud geométrica desempeña un papel fundamental. En particular, en la industria automotriz, IGA se emplea en la simulación de colisiones y en el análisis de vibraciones estructurales [9, 10]. Asimismo, en los sectores aeroespacial y eólico permite modelar con gran precisión las interacciones fluido-estructura [11]. De forma similar, en biomedicina se utiliza para simular el flujo sanguíneo en modelos arteriales realistas [12], mientras que en ingeniería naval contribuye a optimizar la resistencia hidrodinámica en el diseño de cascos de buques [13]. Finalmente, en el ámbito del electromagnetismo, su precisión es crucial para predecir correctamente la dispersión de ondas de alta frecuencia [14].

En lo que respecta a su desarrollo teórico, la investigación actual en IGA abarca una amplia diversidad de áreas matemáticas fundamentales. Entre las líneas de estudio más destacadas se encuentran el análisis riguroso de la estabilidad y la verificación de las propiedades *inf-sup* para formulaciones mixtas [15], la deducción teórica de estimaciones de error para los distintos tipos de refinamiento [6] y el estudio exhaustivo de las propiedades espectrales de los operadores diferenciales discretizados [10]. Asimismo, se presta especial atención al estudio de las propiedades de los propios espacios de aproximación. En este contexto, han cobrado relevancia diversas técnicas orientadas a permitir un refinamiento local adaptado a los requerimientos físicos del problema. Entre ellas, destaca la introducción de «nodos en T» mediante T-Splines [5] y, de forma especialmente significativa, el uso de B-splines jerárquicos (HB-splines) y su variante truncada (THB-splines) [16, 17]. Estas últimas permiten superponer jerárquicamente dominios con distintos niveles de resolución, manteniendo al mismo tiempo la estructura tensorial.

Por otra parte, también se están desarrollando formulaciones de carácter espacio-temporal [18]. En lugar de recurrir a esquemas clásicos de discretización temporal, el tiempo se incorpora como una dimensión geométrica adicional dentro del propio dominio de análisis. Esta aproximación permite trabajar sobre un dominio unificado en el espacio-tiempo y aplicar sobre él las mismas bases isogeométricas, lo que puede traducirse en una mayor regularidad en la resolución de problemas dinámicos.

1.3. Objetivos

El objetivo principal del trabajo es desarrollar el marco teórico de IGA, así como su implementación para el problema de Poisson y la validación mediante resultados numéricos de resultados teóricos de convergencia; adicionalmente se tratarán algunas de las diferencias principales de IGA respecto a FEM, todo en el marco de problemas de contorno de ecuaciones en derivadas parciales.

Nos centraremos en el estudio de los casos 1D y 2D, pues la generalización a dimensiones superiores es análoga. El trabajo se estructura en las siguientes fases:

1. **Marco teórico:** Establecer las bases matemáticas de IGA, introduciendo la formulación variacional y el problema discreto empleando funciones base de tipo spline con la aproximación polinómica de FEM.
2. **Implementación computacional:** Desarrollo e implementación de código propio para la resolución de problemas de contorno de EDPs usando IGA.
3. **Validación y comparación:** Validar los resultados numéricos comparándolos con las soluciones analíticas de los problemas, así como comprobar que se cumpla el orden de convergencia teórico en las normas H^1 y L^2 .

1.4. El problema modelo: la ecuación de Poisson

Durante la realización del trabajo tomaremos el problema de Poisson como *benchmark* o problema modelo discretizado con IGA. Esta ecuación en derivadas parciales elíptica de segundo orden, que generaliza la ecuación de Laplace, se expresa matemáticamente como:

$$-\Delta u = f \quad \text{en } \Omega,$$

donde $\Omega \subset \mathbb{R}^d$ es un dominio abierto y acotado, Δ es el operador laplaciano, u es la función incógnita y f representa un término fuente conocido.

1.4.1. Formulación fuerte (o clásica)

La formulación fuerte de un problema de contorno implica que la ecuación diferencial debe cumplirse punto a punto en todo el dominio considerado, lo que conlleva imponer condiciones de regularidad exigentes sobre la función incógnita. En el caso de la ecuación de Poisson, la solución ha de ser suficientemente regular para que sus derivadas existan en el sentido clásico y el operador diferencial pueda evaluarse directamente; además, debe satisfacer de manera estricta las condiciones en la frontera.

Sea $\Omega \subset \mathbb{R}^d$ un conjunto abierto y acotado (donde $n \in \mathbb{N}$), con frontera $\partial\Omega = \Gamma = \overline{\Gamma_D \cup \Gamma_R \cup \Gamma_N}$, que sea suficientemente regular, tal que $\Gamma_D \cap \Gamma_R = \Gamma_D \cap \Gamma_N = \Gamma_R \cap \Gamma_N = \emptyset$.

Buscamos una función solución $u : \Omega \rightarrow \mathbb{R}$, que cumpla $u \in C^2(\Omega) \cap C^1(\overline{\Omega})$ y tal que:

$$\begin{cases} -\Delta u = f & \text{en } \Omega, \\ u = g & \text{en } \Gamma_D, \quad (\text{Condición de Dirichlet}), \\ \nabla u \cdot \mathbf{n} = h & \text{en } \Gamma_N, \quad (\text{Condición de Neumann}), \\ \alpha u + \beta \nabla u \cdot \mathbf{n} = r & \text{en } \Gamma_R, \quad (\text{Condición de Robin}). \end{cases}$$

Para garantizar que el problema esté bien planteado, se establecen las siguientes condiciones; exigimos que la unión de las fronteras de Dirichlet y Robin tenga medida no nula, $\mathcal{L}^{n-1}(\Gamma_D \cup \Gamma_R) > 0$, y denotamos por $\mathbf{n}$ al vector normal exterior unitario a la frontera Γ , a las funciones involucradas, asumimos que el término fuente $f : \Omega \rightarrow \mathbb{R}$ satisface $f \in L^2(\Omega)$, para las condiciones de contorno, requerimos que el dato de Dirichlet $g : \Gamma_D \rightarrow \mathbb{R}$ posea una regularidad $g \in H^{1/2}(\Gamma_D)$, los datos $h : \Gamma_N \rightarrow \mathbb{R}$ y $r : \Gamma_R \rightarrow \mathbb{R}$ deben pertenecer a $L^2(\Gamma_N)$ y $L^2(\Gamma_R)$, respectivamente y los parámetros reales $\alpha, \beta \in \mathbb{R}$ de la condición de Robin deben verificar $\frac{\alpha}{\beta} \geq 0$.

1.4.2. Formulación variacional (o débil)

La formulación débil puede entenderse como una reinterpretación del problema diferencial original en términos integrales, de modo que la ecuación ya no se exige punto a punto, sino en un sentido global sobre el dominio. Para obtenerla, se multiplica la ecuación en su forma fuerte por una función test y se integra en Ω , aplicando la primera identidad de Green. Como consecuencia, una de las derivadas se traslada a la función test¹, lo que permite rebajar las exigencias de regularidad sobre la incógnita, que pasa a considerarse en el espacio $H^1(\Omega)$. Además, en esta formulación las condiciones de Neumann y Robin aparecen de manera natural en los términos de frontera, mientras que la condición de Dirichlet se incorpora como una restricción esencial del espacio funcional.

Buscamos $u \in H^1(\Omega)$, con $u = g$ en Γ_D , tal que para toda función test $v \in V_0(\Omega) = \{w \in H^1(\Omega) \mid w = 0 \text{ en } \Gamma_D\}$, se cumple que:

$$a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} uv = \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v = L(v), \quad (1.1)$$

1.5. Estructura del documento

El resto del trabajo se organiza de la siguiente forma: En el capítulo 2 se introducen los fundamentos matemáticos del diseño CAD, desde las curvas de Bézier y los B-splines hasta los NURBS, describiendo tanto la construcción geométrica como las funciones base y los distintos refinamientos de curvas y superficies basados en inserción de nudos. En el capítulo 3 se desarrolla el marco teórico de IGA con el objetivo de discretizar y resolver el problema modelo, prestando especial atención a las propiedades del sistema obtenido y a las cotas teóricas de error en las normas L^2 y H^1 . En el capítulo 4 se presentan los resultados numéricos, que permiten contrastar los ordenes de convergencia esperados y analizar el coste computacional asociado al método.

¹ $\int_{\Omega} -\Delta u v dx = \int_{\Omega} \nabla u \cdot \nabla v dx - \int_{\partial\Omega} \frac{\partial u}{\partial \mathbf{n}} v ds$

Capítulo 2

Construcción de curvas y superficies mediante CAD

La integración entre el CAD y el análisis numérico requiere comprender cómo se construyen y parametrizan los dominios físicos. En la práctica, las curvas y superficies definidas mediante NURBS se han convertido en el estándar de la industria para la representación geométrica exacta. Dada la relevancia de las parametrizaciones de curvas y superficies mediante CAD en la representación geométrica, dedicamos este capítulo al estudio de las herramientas empleadas en el diseño siguiendo la evolución:

Bézier $\longrightarrow$ B-Splines $\longrightarrow$ NURBS.

2.1. Curvas de Bézier

En los años 60, impulsados por la industria de la automoción francesa y su necesidad de pasar los diseños en arcilla y papel a ordenador, de forma independiente, Pierre Bézier y Paul de Casteljaou formalizaron la construcción de curvas parametrizadas mediante el uso de polinomios de Bernstein como funciones base, combinándolos con una red de puntos de control [3]. La ventaja analítica de este método radica en el control geométrico que proporciona; el polinomio empleado no interpola simplemente los puntos de control, sino que, de manera intuitiva, el polígono formado por estos actúa como un esqueleto que moldea la forma. Esto nos garantiza propiedades como la invarianza bajo transformaciones afines.

A lo largo de esta sección se definirá la formulación analítica de las curvas de Bézier, se analizarán las propiedades de los polinomios de Bernstein y se tratará el algoritmo de De Casteljaou¹, el cual evita la inestabilidad numérica inherente a la evaluación de polinomios de grado elevado.

2.1.1. Polinomios de Bernstein

Definición. Los $p + 1$ polinomios de Bernstein de grado polinomial p se definen como:

$$B_{i,p}(u) = \binom{p}{i} u^i (1-u)^{p-i}, \quad i \in \{0, 1, \dots, p\}.$$

Proposición 2.1. Algunas de las propiedades fundamentales de los *polinomios de Bernstein* son:

- **No negatividad:** $B_{i,p}(u) \geq 0, \forall u \in [0, 1]$.
- **Partición de la unidad:** $\sum_{i=0}^p B_{i,p}(u) = 1, \quad \forall u \in [0, 1]$.
- **Interpolación de los extremos:** $B_{0,p}(0) = 1, \quad B_{p,p}(1) = 1$.
- **Soporte global:** $B_{i,p}(u) > 0, \quad \forall u \in (0, 1)$.

Demostración. La demostración se encuentra en el Anexo A.4.6. □

¹Implementación de este en el Anexo A.7.2.

2.1.2. Definición y propiedades

Definición. Las curvas de Bézier de grado p son curvas parametrizadas que emplean los polinomios de Bernstein como base para la parametrización en el espacio $\mathbb{R}^n$ con $n, p \in \mathbb{N}$, de la forma:

$$\mathbf{C}(u) = \sum_{i=0}^p B_{i,p}(u) \mathbf{P}_i, \quad u \in [0, 1],$$

donde: $\{\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_p\}^2 \subset \mathbb{R}^n$, es el conjunto formado por los **puntos de control**.

Para ilustrar las curvas de Bézier, en la Figura 2.1 se muestra una curva de Bézier cúbica empleando los puntos de control $\mathbf{P} = \{(0, 0, 0), (1, 5, 1), (2, 0, 2), (3, 4, 5, 4)\}$. En la Figura 2.1a se muestra la gráfica de la curva y en la Figura 2.1b se representan los correspondientes polinomios de Bernstein cúbicos.

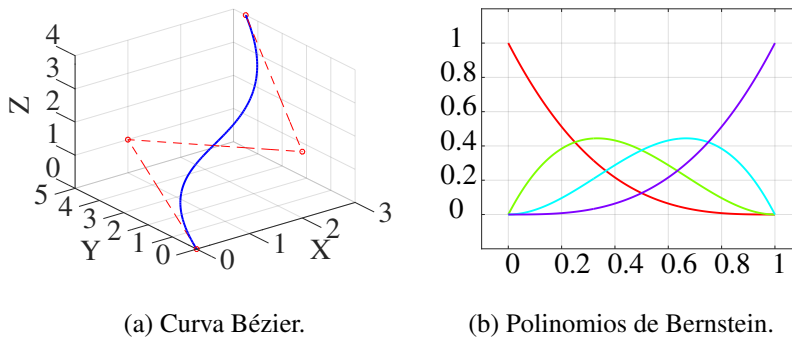


Figura 2.1: Curva cúbica de Bézier empleando los puntos de control $\mathbf{P} = \{(0, 0, 0), (1, 5, 1), (2, 0, 2), (3, 4, 5, 4)\}$ y sus correspondientes polinomios de Bernstein cúbicos.

Proposición 2.2. Sea $\mathbf{C}(u) = \sum_{i=0}^p B_{i,p}(u) \mathbf{P}_i$ una curva de Bézier de grado p , con $u \in [0, 1]$. Se satisfacen las siguientes propiedades:

1. **Interpolación en los extremos:** $\mathbf{C}(0) = \mathbf{P}_0$ y $\mathbf{C}(1) = \mathbf{P}_p$.
2. **Tangencia en los extremos:** $\mathbf{C}'(0) = p(\mathbf{P}_1 - \mathbf{P}_0)$, $\mathbf{C}'(1) = p(\mathbf{P}_p - \mathbf{P}_{p-1})$.
3. **Invarianza afín:** Sea $\Phi: \mathbb{R}^n \rightarrow \mathbb{R}^n$ una transformación afín ($\Phi(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$). Entonces:

$$\Phi \left(\sum_{i=0}^p B_{i,p}(u) \mathbf{P}_i \right) = \sum_{i=0}^p B_{i,p}(u) \Phi(\mathbf{P}_i).$$

Demostración. La demostración se encuentra en el Anexo A.4.6. □

Las principales limitaciones de las curvas de Bézier son la falta de control local sobre la curva (lo que hace que cualquier modificación en los puntos de control la afecte de manera global), y su incapacidad de representar cónicas de forma exacta. Para solucionar esto último se introduce la racionalización de las curvas de Bézier.

Proposición 2.3. Racionalización de las curvas de Bézier.

Se asigna un peso $w_i \in \mathbb{R}^+$ a cada punto de control $\mathbf{P}_i \in \mathbb{R}^d$, se considera el espacio extendido $\mathbb{R}^d \times \mathbb{R}$ mediante coordenadas homogéneas de la forma $(w_i \mathbf{P}_i, w_i)$. Posteriormente, se proyecta la curva resultante sobre el hiperplano $w = 1$, lo que analíticamente equivale a tomar las coordenadas referentes a $\mathbf{P}_i$ y dividir las entre la suma ponderada de los pesos activos:

$$\mathbf{C}^R(u) = \frac{\sum_{i=0}^p B_{i,p}(u) w_i \mathbf{P}_i}{\sum_{i=0}^p B_{i,p}(u) w_i}$$

²El conjunto de puntos de control puede contener repeticiones, es decir, **NO** satisfacen necesariamente $\mathbf{P}_i \neq \mathbf{P}_j$, si $i \neq j$.

2.2. B-Splines

Aunque la racionalización permitió representar con exactitud curvas cónicas, seguían existiendo las limitaciones relacionadas con el control global de la geometría y con la dependencia entre el grado del polinomio y el número de puntos de control. En respuesta a estas limitaciones, y motivada en gran medida por las necesidades de la industria aeroespacial y automotriz durante la década de 1970, la representación geométrica comenzó a transicionar hacia el uso de las curvas B-Splines [4]. Estas curvas se construyen a partir de funciones definidas a trozos mediante la fórmula de Cox-de Boor y permiten superar varias de las restricciones asociadas a los polinomios de Bernstein. En particular, introducen la propiedad de soporte local. Así, la modificación de un punto de control solo afecta a una parte local y limitada de la curva.

2.2.1. Funciones base B-Spline

La construcción de las curvas B-Spline se lleva a cabo de forma paramétrica sobre un dominio de referencia. La parametrización se establece fijando unos valores conocidos como nudos que dividen el dominio paramétrico. Dichos nudos se recogen en vectores de nudos.

Definición. Un **vector de nudos** es una sucesión no decreciente de números reales $U = \{u_0, u_1, \dots, u_m\}$. El número de nudos $(m + 1)$ está relacionado con p y n tal que cumple: $m = n + p + 1$.

Al intervalo entre dos nudos consecutivos se le denota como **knot span**.

Con el fin de realizar una interpolación de los extremos del dominio, capturando así la frontera física del problema y facilitando la imposición de condiciones de contorno, nos enfocaremos en los vectores de nudos **abiertos**, es decir, aquellos cuyos extremos se repiten $p + 1$ veces, $u_0 = u_1 = \dots = u_p$, $u_m = u_{m-1} = \dots = u_{m-p+1}$.

Definición. Sean $U = \{u_0, u_1, \dots, u_m\}$ un vector de nudos y $u \in [u_0, u_m]$. Las **funciones base B-Spline** $N_{i,p}(u)$ se definen recursivamente para grados polinomiales $p \geq 1$:

- Funciones constantes a trozos / Caso base ($p = 0$):

$$N_{i,0}(u) = \begin{cases} 1 & \text{si } u_i \leq u < u_{i+1}, \\ 0 & \text{en caso contrario.} \end{cases}$$

- Caso genérico, fórmula de **Cox-de Boor** ($p \geq 1$):

$$N_{i,p}(u) = \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u)^3.$$

Proposición 2.4. La derivada de las funciones base B-Splines pueden calcularse mediante la fórmula

$$N'_{i,p}(u) = \frac{p}{u_{i+p} - u_i} N_{i,p-1}(u) - \frac{p}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u).$$

Proposición 2.5. Algunas de las propiedades importantes de las funciones base B-Spline son:

1. **Soporte Local:** Cada función base $N_{i,p}(u)$ tiene soporte en un número limitado de intervalos de nudos, concretamente en $[u_i, u_{i+p+1})$. En consecuencia la curva adquiere la propiedad de **control local**: mover un punto de control $\mathbf{P}_i$ solo altera la forma de la curva en una región limitada.
2. **No Negatividad:** El valor de las funciones base B-Spline es siempre no negativo en todo el dominio paramétrico, es decir, $\forall u \in [u_0, u_m]$, $N_{i,p}(u) \geq 0$.

³Para el caso en el que los coeficientes toman valor $\frac{0}{0}$ se les asignará como valor 0.

3. **Número de Bases Activas:** Para cualquier intervalo de nudos $[u_i, u_{i+1})$, las únicas funciones base $N_{j,p}(u)$ cuyo soporte contiene dicho intervalo son aquellas con el índice $j \in \{i-p, i-p+1, \dots, i\}$. Por tanto el número de funciones base no nulas en el intervalo es $p+1$.

4. **Partición de la Unidad:** $\forall u \in [u_0, u_m]$, se cumple, $\sum_{i=0}^n N_{i,p}(u) = 1$.

Demostración. La demostración puede encontrarse en el Anexo A.4.7. □

2.2.2. Curvas B-Spline

Definición. Sean $U = \{u_0, u_1, \dots, u_m\}$ un vector de nudos, $u \in [u_0, u_m]$ y $\mathbf{P}_i \in \mathbb{R}^d$ la red de puntos de control, una curva B-Spline se define como

$$C(u) = \sum_{i=0}^n N_{i,p}(u) \cdot \mathbf{P}_i.$$

En la Figura 2.2a se muestra una curva B-spline cuadrática obtenida a partir del vector de nudos $U = \{0, 0, 0, 1/4, 1/4, 1/2, 1/2, 3/4, 3/4, 1, 1, 1\}$ y el conjunto de puntos de control $\mathbf{P} = \{(1, 0, 0), (1, 1, 0), (0, 1, 0), (-1, 1, 0), (-1, 0, 0), (-1, -1, 0), (0, -1, 0), (1, -1, 0), (1, 0, 0)\}$. Adicionalmente, en la Figura 2.2b se representan las funciones base B-spline con las que se construye dicha curva, mientras que en la Figura 2.2c se ilustran las derivadas analíticas de estas funciones (véase la composición global en la Figura 2.2).

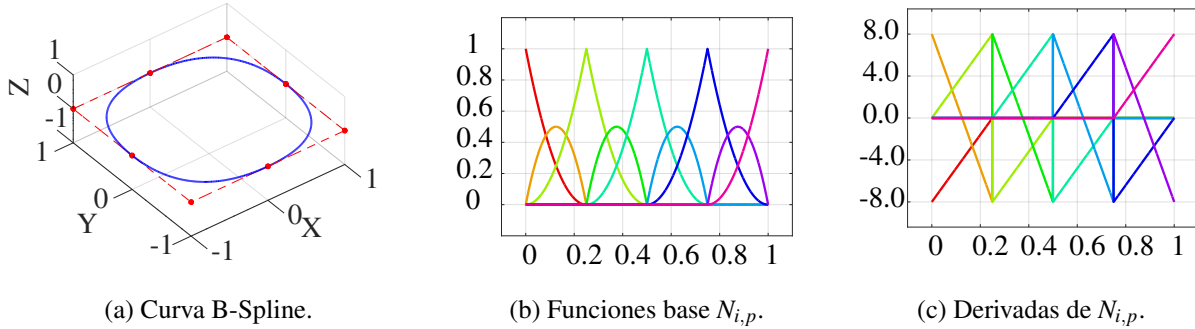


Figura 2.2: Curva B-spline cuadrática empleando el vector de nudos $U = \{0, 0, 0, 1/4, 1/4, 1/2, 1/2, 3/4, 3/4, 1, 1, 1\}$ y los puntos de control $\mathbf{P} = \{(1, 0, 0), (1, 1, 0), (0, 1, 0), (-1, 1, 0), (-1, 0, 0), (-1, -1, 0), (0, -1, 0), (1, -1, 0), (1, 0, 0)\}$, junto a sus funciones base y las derivadas de estas.

Proposición 2.6. Algunas de las propiedades importantes de las curvas B-Splines son:

1. **Invarianza bajo transformaciones afines:** La aplicación de una transformación afín al polígono de control resulta en la misma transformación aplicada directamente a la curva generada.
2. **Continuidad controlable:** La continuidad de la curva en cualquier unión (nudo) se controla localmente mediante la multiplicidad k^4 del nudo, sin depender de la manipulación compleja de los puntos de control. La continuidad es C^r , donde $r = p - k$.

Demostración. La demostración puede encontrarse en el Anexo A.4.7. □

⁴La multiplicidad k de un nudo u_i es el número de veces que el valor u_i aparece en el vector de nudos U , debe cumplirse que $k \leq p$ en los nudos interiores.

2.2.3. Superficies B-Splines

Para la generación de superficies, el enfoque más extendido es la implementación del **producto tensorial**, el cual constituye el estándar en los sistemas CAD. Para llevar a cabo esta extensión bidimensional, es necesario redefinir y generalizar los elementos unidimensionales que ya habíamos establecido para las curvas.

- **Grados polinomiales:** Grado p para la dirección paramétrica u , y grado q para la dirección v .
- **Vectores de nudos:** $U = \{u_0, \dots, u_{n+p+1}\}$ para la dirección u y $V = \{v_0, \dots, v_{m+q+1}\}$ para la dirección v .
- **Red de control:** matriz formada por $(n+1) \times (m+1)$ puntos de control $\mathbf{P}_{i,j}$.
- **Funciones base:** $\mathbf{B}_{i,j}(u, v) = N_{i,p}(u) \cdot M_{j,q}(v)$, donde $N_{i,p}(u)$ son las funciones base B-Spline en la dirección de u y $M_{j,q}(v)$ son las funciones base en la dirección de v .

Mediante este producto definimos la superficie generada por **B-Splines** usando la siguiente fórmula:

$$\mathbf{S}(u, v) = \sum_{i=0}^n \sum_{j=0}^m \mathbf{B}_{i,j}(u, v) \mathbf{P}_{i,j} = \sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) M_{j,q}(v) \mathbf{P}_{i,j},$$

donde $u \in [u_0, u_{n+p+1}]$ y $v \in [v_0, v_{m+q+1}]$.

2.3. NURBS

La transición de B-splines a NURBS no fue instantánea; se tuvo que esperar hasta 1979, cuando Boeing sufría las consecuencias de no poder representar cónicas exactas mediante B-splines, teniendo así su software segmentado, usando un programa basado en B-Splines para el fuselaje y otro basado en las cónicas de Rho para las partes cónicas, siendo incapaces de conectarlos sin perder precisión [19, 20], en este caos, Boeing inició el proyecto TIGER [19], reescribiendo todo su sistema CAD interno basándose en la tesis de Versprille [21] y propuso en 1981 a la IGES que las NURBS fueran el estándar universal, lo que llevó a la industria a transicionar de manera natural entre 1983 y 1990.

2.3.1. Funciones base NURBS

Definición. Dados los pesos $\{w_i\}$ con $w_i > 0$ para $i = 0, 1, \dots, n$, definimosas **funciones base NURBS** se definen como:

$$R_i^p(u) = \frac{N_{i,p}(u)w_i}{\sum_{j=0}^n N_{j,p}(u)w_j}, \quad i \in \{0, 1, \dots, n\}. \quad (2.1)$$

Proposición 2.7. La derivada de las funciones base NURBS viene dada por

$$(R_i^p)'(u) = \frac{w_i N_{i,p}'(u) - R_i^p(u) \sum_{j=0}^n N_{j,p}'(u)w_j}{\sum_{j=0}^n N_{j,p}(u)w_j}, \quad i \in \{0, 1, \dots, n\}.$$

Demostración. Demostración en el Anexo A.4.8. □

2.3.2. Curvas NURBS

Las curvas NURBS se definen de manera análoga a las curvas B-Spline, a partir de las funciones base NURBS.

Definición. Sean $U = \{u_0, u_1, \dots, u_m\}$ un vector de nudos, $u \in [u_0, u_m]$, $\{w_i\}$ los pesos y $\mathbf{P}_i \in \mathbb{R}^d$ la red de puntos de control, una curva NURBS se define como

$$\mathbf{C}(u) = \sum_{i=0}^n R_i^p(u) \cdot \mathbf{P}_i = \frac{\sum_{i=0}^n N_{i,p}(u)w_i \mathbf{P}_i}{\sum_{j=0}^n N_{j,p}(u)w_j}. \quad (2.2)$$

Proposición 2.8. La derivada de la curva NURBS viene dada por

$$\mathbf{C}'(u) = \frac{\sum_{i=0}^n N'_{i,p}(u) w_i (\mathbf{P}_i - \mathbf{C}(u))}{\sum_{i=0}^n N_{i,p}(u) w_i}.$$

Demostración. La demostración puede encontrarse en el Anexo A.4.9. □

Proposición 2.9. Imponiendo la no negatividad de los pesos w_i , las curvas NURBS también mantienen las propiedades de las curvas B-spline dadas en la proposición 2.6.

A continuación, para ilustrar la construcción de una curva NURBS, en la Figura 2.3a se representa gráficamente la circunferencia de radio unidad centrada en el origen, parametrizada como una curva NURBS cuadrática. Para esta construcción se han empleado el vector de nudos y los puntos de control de la Figura 2.2a, introduciendo en este caso los pesos $\mathbf{w} = \{1, \sqrt{2}/2, 1, \sqrt{2}/2, 1, \sqrt{2}/2, 1, \sqrt{2}/2, 1\}$. Por su parte, en la Figura 2.3b se muestran las funciones base racionales correspondientes, mientras que en la Figura 2.3c se ilustran sus respectivas derivadas analíticas (véase la composición global en la Figura 2.3).

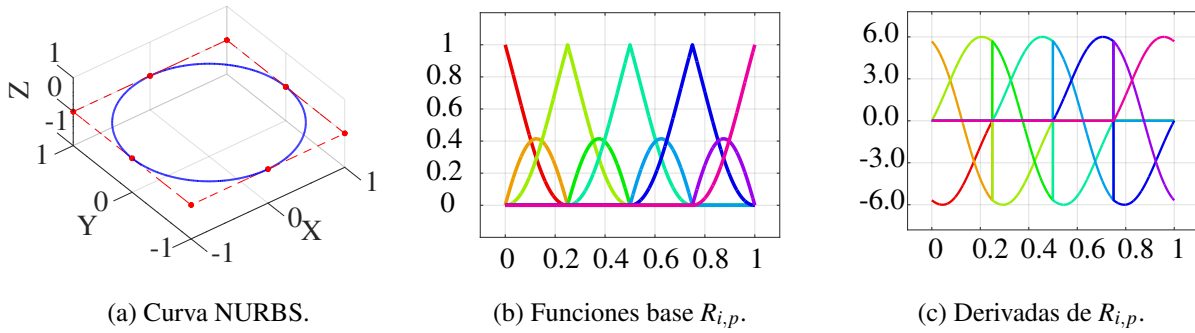


Figura 2.3: Curva NURBS cuadrática para representar la circunferencia unidad centrada en el origen con sus funciones base y las derivadas de estas.

2.3.3. Superficies NURBS

Partiendo de los elementos definidos en la subsección 2.2.3, definimos las funciones base de las superficies NURBS.

Definición. Las funciones base NURBS para superficies se definen mediante

$$R_{i,j}^{p,q}(u,v) = \frac{N_{i,p}(u) M_{j,q}(v) w_{i,j}}{\sum_{k=0}^n \sum_{l=0}^m N_{k,p}(u) M_{l,q}(v) w_{k,l}}, \quad i \in \{0, 1, \dots, n\}, \quad j \in \{0, 1, \dots, m\} \quad (2.3)$$

Utilizando estas podemos ahora definir las superficies NURBS.

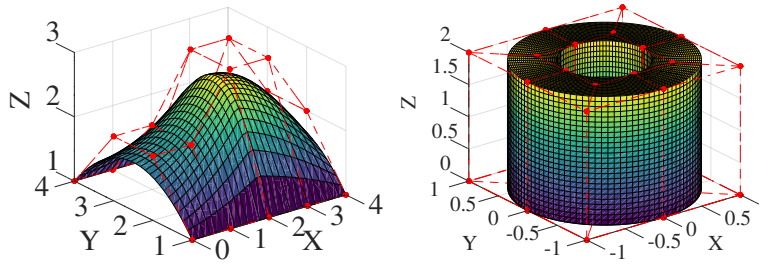
Definición. Sean $U = \{u_0, u_1, \dots, u_{n+p+1}\}$, $V = \{v_0, v_1, \dots, v_{m+q+1}\}$ vectores de nudos, $u \in [u_0, u_{n+p+1}]$, $v \in [v_0, v_{m+q+1}]$, w matriz de pesos y $\mathbf{P}_{i,j}$ una red de puntos de control, las superficies NURBS se definen como

$$\mathbf{S}(u,v) = \sum_{i=0}^n \sum_{j=0}^m R_{i,j}^{p,q}(u,v) \mathbf{P}_{i,j} = \frac{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) M_{j,q}(v) \mathbf{w}_{i,j} \mathbf{P}_{i,j}}{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) M_{j,q}(v) \mathbf{w}_{i,j}}. \quad (2.4)$$

Proposición 2.10. Las superficies generadas mediante el producto tensorial mantienen la suavidad de las curvas que la generan en las direcciones de estas.

Demostración. La demostración puede encontrarse en el Anexo A.4.10 □

A continuación se muestran ejemplos de superficies NURBS en la Figura 2.4.



(a) Superficie NURBS ejemplo 1. (b) Superficie NURBS ejemplo 2.

Figura 2.4: Ejemplos de distintas superficies NURBS

Nota: La explicación de las figuras se encuentra en el Anexo A.6.2 y el código fuente de estas en el Anexo A.7.5.

2.4. Refinamiento de curvas y superficies

2.4.1. Inserción de nudos

Este procedimiento se basa en insertar nuevos valores en un vector de nudos, introduciendo nuevos puntos de control y pesos para mantener la misma figura geométrica. La inserción de nudos tiene como finalidad aumentar el control local de las estructuras definidas mediante NURBS, obteniendo un mayor número de funciones base donde se inserta y aumentando por ende el número de puntos de control sobre los que se trabaja. En consecuencia, el sistema adquiere un mayor número de grados de libertad en esta sección local.

A continuación, redefiniremos los elementos necesarios para la inserción:

- **Puntos de control ponderados, $\mathbf{P}_i^w$:** Retomando la elevación previa al espacio extendido $\mathbb{R}^d \times \mathbb{R}$ de la proposición 2.3, Sea $\mathbf{P}_i$ un punto de control y w_i su peso asociado, definimos $\mathbf{P}_i^w = (w_i \mathbf{P}_i, w_i)^T$, $\forall i = 0, \dots, n$.
- **Nuevo nudo, $\xi \in [u_k, u_{k+1})$.**
- **Nuevo vector de nudos refinado, $U^* = \{u_0, u_1, \dots, u_k, \xi, u_{k+1}, \dots, u_{(n+p+1)}\}$.**
- **Nuevos puntos de control ponderados, $\mathbf{Q}_i^w$:** Viene dados por $\mathbf{Q}_i^w = \alpha_i \mathbf{P}_i^w + (1 - \alpha_i) \mathbf{P}_{i-1}^w$, donde α_i se define mediante la fórmula:

$$\alpha_i = \begin{cases} 1 & \text{si } i \leq k - p, \\ \frac{\xi - u_i}{u_{i+p} - u_i} & \text{si } k - p + 1 \leq i \leq k, \\ 0 & \text{si } i > k. \end{cases}$$

Nota: El algoritmo solo modifica los p puntos de control $\mathbf{P}_i$ tales que $k - p + 1 \leq i \leq k$.

- **Nuevas funciones base, $N_{i,p}^*(u)$,** asociadas al nuevo vector de nudos U^* , que cumplen la igualdad: $N_{i,p}(u) = \alpha_i N_{i,p}^*(u) + (1 - \alpha_{i+1}) N_{i+1,p}^*(u)$, $\forall i = 0, \dots, n^5$.

Proposición 2.11. *La curva original ponderada es equivalente a la curva ponderada tras la inserción de un nudo,*

$$\mathbf{C}^w(u) = \sum_{i=0}^n N_{i,p}(u) \mathbf{P}_i^w = \sum_{j=0}^{n+1} N_{j,p}^*(u) \mathbf{Q}_j^w$$

⁵Demostración en el Anexo A.4.11

Demostración. La demostración puede encontrarse en el Anexo A.4.12. □

La inserción de nudos en superficies se realiza de manera direccional, introduciendo únicamente nudos a uno de los dos vectores de nudos, modificando así una de las dos curvas que generan la superficie, dado que cada curva es invariante bajo la inserción de nudos, la superficie generada final es idéntica a la original. Los pesos y puntos de control en forma matricial son tratados como en la demostración de la Proposición 2.10, trabajando con ellos de forma vectorial y relacionándolos con el vector de nudos que recibe la inserción.

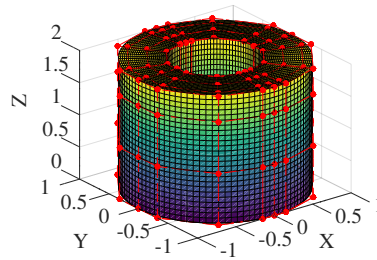


Figura 2.5: Superficie de la figura 2.4b tras inserción de nudos

2.4.2. Elevación del grado polinomial

De manera análoga a la inserción de nudos, la elevación de grado es una estrategia fundamental para el refinamiento de curvas y superficies NURBS. Consiste en aumentar el grado polinomial de p a $p + t$, introduciendo nuevos puntos de control y ajustando el vector de nudos de forma que la geometría se mantenga inalterada. Este proceso permite enriquecer el espacio de representación sin modificar la parametrización ni subdividir los intervalos, lo que se traduce en un mayor número de grados de libertad y en una mayor flexibilidad analítica, especialmente útil para compatibilizar geometrías de distinto orden o mejorar la precisión en métodos numéricos.

A continuación seguiremos el proceso de elevación de grado polinomial para una curva NURBS genérica. Sean $U = \{u_0, u_1, \dots, u_m\}$ un vector de nudos, $u \in [u_0, u_m]$, w un vector de pesos, $\mathbf{P}_i$ la red de $n + 1$ puntos de control y $\mathbf{C}(u)$ la curva NURBS formada por estos elementos de grado polinomial p que queremos elevar al grado $p + t$.

Paso a formas de Bézier locales

Tomamos todos los nudos interiores del vector de nudos e insertamos repeticiones de estos mediante inserción de nudos hasta que todos tengan multiplicidad $p + 1$.

Proposición 2.12. *La curva resultante de este procedimiento es localmente una curva de Bézier para dos valores de nudos distintos consecutivos.*

Elevación del grado en la base de Bernstein

A continuación, para cada una de las curvas de Bézier locales de la proposición 2.12 aplicamos t veces una elevación del grado en la base de Bernstein:

$$\sum_{i=0}^p \mathbf{P}_i B_{i,p}(u) = \sum_{i=0}^{p+1} \left[\frac{i}{p+1} \mathbf{P}_{i-1} + \left(1 - \frac{i}{p+1} \right) \mathbf{P}_i \right] B_{i,p+1}(u). \quad (2.5)$$

Proposición 2.13. *Las curvas de Bézier dadas en la ecuación (2.5) son equivalentes.*

Eliminación de nudos

Como hemos visto, estas curvas de Bézier locales son equivalentes a sus segmentos correspondientes de la curva NURBS. Retomando la formulación general de la curva, ahora con un grado polinomial elevado $\hat{p} = p + t$, es necesario someterla a un proceso de eliminación de nudos para suprimir el exceso de repeticiones introducidas en el primer paso. Es fundamental destacar que no se eliminan todas las copias insertadas; para garantizar que la curva conserve exactamente su suavidad paramétrica original $C^{p-\mu}$ en los puntos de unión, un nudo que inicialmente poseía multiplicidad μ debe adquirir ahora una multiplicidad final de $\mu + t$. Por tanto, este proceso iterativo de reducción se aplicará sobre cada nudo interno estrictamente hasta alcanzar dicha multiplicidad objetivo.

Sea $\bar{u}$ un nudo con multiplicidad original μ al que hemos añadido $r = \hat{p} - \mu - t$ repeticiones en el primer paso. Fijaremos el índice $c = \max\{\eta \mid u_\eta = \bar{u}\} - \hat{p}$ y aplicaremos iterativamente sobre $v \in \{1, \dots, r\}$ el siguiente algoritmo:

1. **Vector de nudos actualizado:** eliminando una de las repeticiones de $\bar{u}$ de este.
2. **Definición de las fronteras:** $f_v = c - v$, $l_v = c + v$.
3. **Puntos de apoyo:** $\mathbf{P}_{f_v}^{(v)} = \mathbf{P}_{f_v}^{(v-1)}$, $\mathbf{P}_{l_v-1}^{(v)} = \mathbf{P}_{l_v}^{(v-1)}$.
4. **Variable de iteración:** $\alpha_\eta^{(v)} = \frac{\bar{u} - u_\eta^{(v)}}{u_{\eta+\hat{p}+1}^{(v)} - u_\eta^{(v)}}$
5. **Iteraciones:** aplicaremos las siguientes asignaciones mientras se cumple la desigualdad $j - i > v$:
 - Para $i = f_v + 1, f_v + 2, \dots$, $\mathbf{P}_i^{(v)} = \frac{\mathbf{P}_i^{(v-1)} - (1 - \alpha_i^{(v)})\mathbf{P}_{i-1}^{(v)}}{\alpha_i^{(v)}}$.
 - Para $j = l_v, l_v - 1, \dots$, $\mathbf{P}_{j-1}^{(v)} = \frac{\mathbf{P}_j^{(v-1)} - \alpha_j^{(v)}\mathbf{P}_j^{(v)}}{1 - \alpha_j^{(v)}}$.
6. **Actualización del resto de puntos:** $\mathbf{P}_\eta^{(v)} = \mathbf{P}_{\eta+1}^{(v-1)} \quad \forall \eta \geq l_v$.

Proposición 2.14. *En cada una de las iteraciones del algoritmo previamente descrito, la curva resultante es equivalente a la curva original.*

Nota Las demostraciones a las proposiciones dadas en esta sección pueden encontrarse en el Anexo A.4.13.

Capítulo 3

Análisis isogeométrico

Aunque las bases matemáticas del diseño CAD mediante NURBS se consolidaron en los años 80 [19, 20], hasta mediados de los 2000 existió un cuello de botella en las simulaciones numéricas de ingeniería: Los modelos creados en CAD debían ser transformados y aproximados por mallas poligonales para poder aplicar FEM [1]. Se estima que en industrias complejas, como automoción o naval, este proceso tomaba hasta el 80 % del tiempo total de análisis [1, 2].

En 2005, Hughes, Cottrell y Bazilevs proponen unificar el diseño CAD y la simulación, empleando las mismas funciones NURBS como base de IGA [1]. Desde entonces, IGA ha trascendido el ámbito académico, consolidándose así en industrias como la aeroespacial, de automoción y la naval [2]. Esta evolución tecnológica se refleja en la creciente adopción de esta metodología en el software comercial de simulación. A las primeras implementaciones en herramientas como LS-DYNA [9], se suma hoy en día su incorporación en plataformas ampliamente utilizadas como Abaqus [22] y Ansys [23]. Además de su integración en estándares de modelado CAD como Rhinoceros 3D [24], así como el desarrollo de entornos de simulación específicamente diseñados para el enfoque isogeométrico, como Coreform [25].

3.1. Formulación variacional

Definimos inicialmente el **dominio paramétrico** $\hat{\Omega}$, correspondiente al intervalo entre los extremos del vector de nudos ($\hat{\Omega} = [u_0, u_m]$ en 1D) o al producto cartesiano de estos intervalos en dimensiones superiores ($\hat{\Omega} = [u_0, u_m] \times [v_0, v_k]$ en 2D). Por simplicidad analítica, es habitual normalizar los vectores de nudos para trabajar sobre el dominio unitario $\hat{\Omega} = [0, 1]^d$, donde d es la dimensión del problema (por ejemplo, un vector $U = \{0, 0, 0, 0, 5, 1, 1, 1\}$ define $\hat{\Omega} = [0, 1]$). A partir de este, el **espacio físico** Ω queda parametrizado mediante el mapeo geométrico $\mathbf{G} : \hat{\Omega} \rightarrow \Omega$, definido en (2.2) en 1D y (2.4) en 2D. Dentro de este espacio físico, un **elemento** $\Omega_e \subset \Omega$ se define como la imagen paramétrica mediante $\mathbf{G}$ de un intervalo de nudos no nulo $[u_{e_u}, u_{(e_u+1)})$ en 1D, o su respectivo producto cartesiano $[u_{e_u}, u_{(e_u+1)}) \times [v_{e_v}, v_{(e_v+1)})$ en 2D.

A continuación en la Figura 3.1 se representa el mapeo $\mathbf{G}$ desde $\hat{\Omega}$ a Ω . Se han empleado grados polinomiales $p = 1$ y $q = 2$, los vectores de nudos $U = \{0, 0, 1, 1\}$ y $V = \{0, 0, 0, 1, 1, 1\}$, la red de puntos de control $\mathbf{P} = \{\{(1, 0), (1, 1), (0, 1)\}, \{(3, 0), (3, 3), (0, 3)\}\}$ y la matriz de pesos $\mathbf{W} = \{\{1, \sqrt{2}/2, 1\}, \{1, \sqrt{2}/2, 1\}\}$. Posteriormente, en la Figura 3.2 se ilustra la correspondencia de las funciones de forma NURBS asociadas, proyectanse hacia Ω mediante la composición con el mapeo inverso $\mathbf{G}^{-1}$.

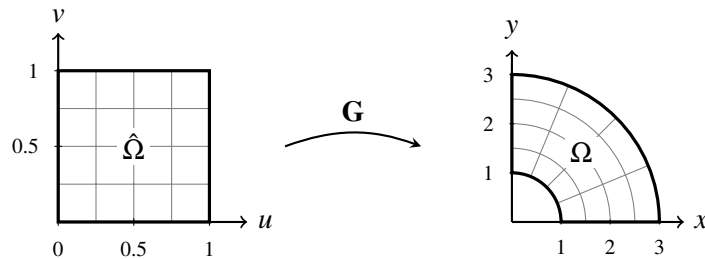


Figura 3.1: Mapeo geométrico $\mathbf{G}$ desde el dominio paramétrico $\hat{\Omega}$ hacia el espacio físico Ω . Se han empleado los siguientes parámetros: Grados polinomiales $p = 1$ (dirección u) y $q = 2$ (dirección v); vectores de nudos $U = \{0, 0, 1, 1\}$ y $V = \{0, 0, 0, 1, 1, 1\}$; red de puntos de control $\mathbf{P} = \{\{(1, 0), (1, 1), (0, 1)\}, \{(3, 0), (3, 3), (0, 3)\}\}$; y matriz de pesos $\mathbf{W} = \{\{1, \sqrt{2}/2, 1\}, \{1, \sqrt{2}/2, 1\}\}$.

Sea ahora N_{el} el número total de intervalos de nudos de longitud no nula, definimos la **mall**a **física** como el conjunto de todos los elementos, $\mathcal{T}_h = \{\Omega_e\}_{e=1}^{N_{el}}$. Para caracterizar el refinamiento de esta partición, definimos el diámetro de un elemento como la máxima distancia euclídea entre dos de sus puntos, $h_e = \text{diam}(\Omega_e) = \sup_{\mathbf{x}, \mathbf{y} \in \Omega_e} \|\mathbf{x} - \mathbf{y}\|_2$. Consecuentemente, el **tamaño global de la malla** queda establecido como el diámetro máximo de entre todos los elementos que la componen, $h = \max_e h_e$.

Finalmente, construimos los espacios de funciones. A lo largo de esta sección, denotaremos con un acento circunflejo ($\hat{\cdot}$) a toda función cuyo dominio sea el paramétrico. Así, dado $\hat{\Omega} = (0, 1)^d$, el **espacio de aproximación** $\hat{V}_h$ definido sobre $\hat{\Omega}$ se formaliza como la envolvente lineal de las funciones base NURBS previamente formuladas (ecuaciones (2.1) y (2.3)) [26]:

$$\hat{V}_h = \left\{ \hat{v} : \hat{\Omega} \rightarrow \mathbb{R} \mid \hat{v}(\xi) = \sum_{A \in \mathcal{J}} c_A \hat{R}_A(\xi), \quad c_A \in \mathbb{R} \right\},$$

donde $\mathcal{J}$ representa el conjunto de índices de la red de control y $\hat{R}_A$ denota la función base paramétrica ($\hat{R}_i^p$ en 1D, o $\hat{R}_{i,j}^{p,q}$ en 2D). Aplicando el mapeo inverso, definimos el **espacio de aproximación** sobre el dominio físico como:

$$V_h = \left\{ v : \Omega \rightarrow \mathbb{R} \mid v \circ \mathbf{G} \in \hat{V}_h \right\}.$$

Las funciones base de este espacio V_h , comúnmente denominadas **funciones de forma**, se obtienen mediante la composición con el mapeo geométrico inverso: $\phi_A = \hat{R}_A \circ \mathbf{G}^{-1}$ para todo índice $A \in \mathcal{J}$.

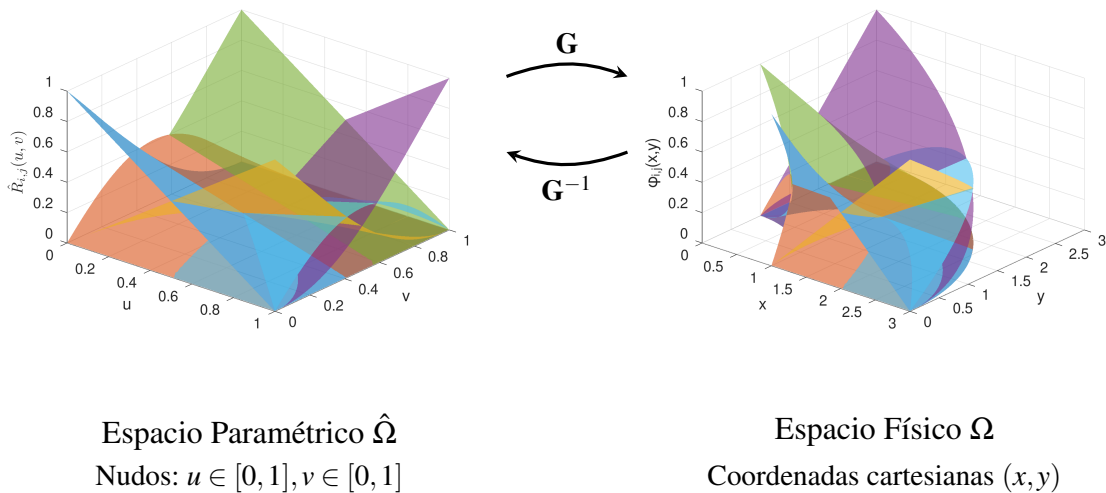


Figura 3.2: Correspondencia de las funciones de forma NURBS de la Figura 3.1. Las funciones paramétricas $\hat{R}_{i,j}^{p,q}(u, v)$ se definen inicialmente sobre el dominio paramétrico $\hat{\Omega}$ (izquierda). Mediante el mapeo geométrico $\mathbf{G}$, el soporte se proyecta hacia el espacio físico Ω (derecha), dando lugar a las funciones físicas $\phi_{i,j}(x, y) = (\hat{R}_{i,j}^{p,q} \circ \mathbf{G}^{-1})(x, y)$.

Concepto Isoparamétrico: Nótese que las funciones base $\hat{R}$ utilizadas para generar el espacio de aproximación V_h son idénticas a las empleadas para construir el mapeo geométrico $\mathbf{G}$. Esta dualidad, donde la geometría y la solución comparten la misma base matemática, es lo que define el **enfoque isoparamétrico** del método.

3.2. Formulación discreta de Galerkin

Partiendo del espacio de aproximación $V_h \subset V$ (mismo V que en la sección 1.4.2), aplicamos el principio de Galerkin al problema variacional (1.1). Buscamos una solución aproximada $u_h \in V_h$ tal que:

$$a(u_h, v_h) = L(v_h), \quad \forall v_h \in V_{h,0} = \{w_h \in V_h \mid w_h = 0 \text{ en } \Gamma\}. \quad (3.1)$$

Expresamos la solución discreta u_h como una combinación lineal de las funciones de forma definidas anteriormente:

$$u_h(\mathbf{x}) = \sum_{j=1}^{N_b} d_j \phi_j(\mathbf{x}),$$

donde d_j son los **variables de control** a determinar y N_b es el número total de funciones base globales.

Sustituyendo u_h en la ecuación (3.1) y tomando sucesivamente cada función de base como función de test ($v_h = \phi_i$ para $i = 1, \dots, N_b$), obtenemos el sistema lineal de ecuaciones:

$$\sum_{j=1}^{N_b} a(\phi_j, \phi_i) d_j = L(\phi_i), \quad \forall i = 1, \dots, N_b.$$

Reescribiendo ahora en forma matricial como $\mathbf{K}\mathbf{d} = \mathbf{F}$, donde $\mathbf{K} \in \mathbb{R}^{N_b \times N_b}$ es la matriz de rigidez global con elementos $K_{i,j} = a(\phi_j, \phi_i)$, $\mathbf{d}$ es el vector de variables de control a determinar y $\mathbf{F}$ es el vector de carga con componentes $F_i = L(\phi_i)$.

3.3. Propiedades del sistema

Proposición 3.1. Las siguientes propiedades se cumplen para problemas en un abierto $\Omega \subset \mathbb{R}^d$, con N_{el} elementos.

1. **Dispersión:** Cada fila tiene a lo sumo $(2p+1)^d$ entradas no nulas.
2. **Simetría:** Los elementos $\mathbf{K}_{(i,j)} = a(\phi_j, \phi_i) = a(\phi_i, \phi_j) = \mathbf{K}_{(j,i)}$, esta propiedad ocurre si y solo si la forma bilineal es simétrica, lo que depende de que el operador diferencial del problema sea autoadjunto.
3. **Condicionamiento:** $\kappa(\mathbf{K}) \approx O(h^{-2})$

Demostración. La demostración puede encontrarse en el Anexo A.4.14. □

3.4. Caso 1D

En el caso unidimensional $\Omega = (a, b)$, la frontera está formada por los extremos del intervalo. Esto reduce el gradiente a la derivada ordinaria y transforma las integrales de contorno en evaluaciones puntuales.

3.4.1. Elementos de la matriz global y el vector de carga

Transformamos la **forma bilineal** basada en las funciones de forma en la siguiente integral basada en las funciones base NURBS,

$$\begin{aligned} a(\phi_j(\mathbf{x}), \phi_i(\mathbf{x})) &= \int_{\Omega} \nabla \phi_j(\mathbf{x}) \cdot \nabla \phi_i(\mathbf{x}) d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} \phi_j(\mathbf{x}) \phi_i(\mathbf{x}) d\Gamma = \\ &= \int_{\hat{\Omega}} \left(\frac{d\hat{R}_j^p(\xi)}{d\xi} J^{-1}(\xi) \right) \left(\frac{d\hat{R}_i^p(\xi)}{d\xi} J^{-1}(\xi) \right) |\det J(\xi)| d\hat{\Omega} + \left[\frac{\alpha}{\beta} \hat{R}_j^p(\xi) \hat{R}_i^p(\xi) \right]_{\xi \in \hat{\Gamma}_R} = \\ &= \int_{\hat{\Omega}} \frac{d\hat{R}_j^p(\xi)}{d\xi} \frac{d\hat{R}_i^p(\xi)}{d\xi} \frac{1}{|\det J(\xi)|} d\hat{\Omega} + \left[\frac{\alpha}{\beta} \hat{R}_j^p(\xi) \hat{R}_i^p(\xi) \right]_{\xi \in \hat{\Gamma}_R}. \end{aligned}$$

Aplicamos lo mismo a la **forma lineal**,

$$\begin{aligned}
L(\phi_i(\mathbf{x})) &= \int_{\Omega} f(\mathbf{x})\phi_i(\mathbf{x}) d\Omega + \int_{\Gamma_N} h(\mathbf{x})\phi_i(\mathbf{x}) d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} \phi_i(\mathbf{x}) d\Gamma = \\
&= \int_{\hat{\Omega}} f(\mathbf{G}(\xi))\hat{R}_i^p(\xi)|\det J(\xi)| d\hat{\Omega} + [h(\mathbf{G}(\xi))\hat{R}_i^p(\xi)]_{\xi \in \hat{\Gamma}_N} + \left[\frac{r}{\beta} \hat{R}_i^p(\xi) \right]_{\xi \in \hat{\Gamma}_R},
\end{aligned}$$

dónde $\mathbf{G}$ es la transformación geométrica definida en (2.2).

3.4.2. Ensamblado de la matriz de rigidez y el vector de cargas

El cálculo de las integrales definidas anteriormente se realizará elemento a elemento, entendiendo por elementos los intervalos no nulos, $\hat{\Omega}_e = [u_\alpha, u_{\alpha+1})$.

Dentro de cada elemento se calcularán todas las combinaciones de funciones base activas. Gracias a las propiedades de soporte local de las funciones base NURBS, sabemos que intervienen a lo sumo $p+1$ funciones por elemento, obteniendo así una **matriz local** $\mathbf{K}^e$ de tamaño $(p+1) \times (p+1)$. A nivel computacional, los índices globales de estas funciones con soporte en dicho elemento se almacenan en un **vector de conectividad**, el cual es fundamental para el ensamblaje en la matriz global.

Teniendo ahora los elementos de esta matriz la forma,

$$\mathbf{K}_{i,j}^e = \int_{u_{e_u}}^{u_{e_u+1}} \frac{d\hat{R}_{j+e_u-p-1}^p(\xi)}{d\xi} \cdot \frac{d\hat{R}_{i+e_u-p-1}^p(\xi)}{d\xi} \frac{1}{|\det J|} d\xi, \quad \forall i, j \in \{1, 2, \dots, p+1\}.$$

Para formalizar el proceso de ensamblaje, definimos la **matriz de rigidez local extendida**, denotada por $\tilde{\mathbf{K}}^e \in \mathbb{R}^{N_b \times N_b}$, la cual se construye explícitamente mediante la asignación

$$\tilde{\mathbf{K}}_{i+e_u-p-1, j+e_u-p-1}^e = \mathbf{K}_{i,j}^e, \quad \forall i, j \in \{1, 2, \dots, p+1\},$$

siendo cero cualquier otro elemento de $\tilde{\mathbf{K}}^e$ cuyos índices no satisfagan esta correspondencia. En la práctica estas matrices no existirán, se sumarán directamente los elementos de $\mathbf{K}^e$ en $\mathbf{K}$. Finalmente la matriz global $\mathbf{K}$ se obtiene al sumar estas matrices extendidas, $\mathbf{K} = \sum_{e=1}^{N_{el}} \tilde{\mathbf{K}}^e$.

Aplicamos el mismo procedimiento ahora a la forma lineal, definimos el **vector de carga local** $\mathbf{F}^e \in \mathbb{R}^{(p+1)}$ cuyos elementos definimos,

$$\mathbf{F}_i^e = \int_{u_{e_u}}^{u_{e_u+1}} f(\mathbf{G}(\xi))\hat{R}_{i+e_u-p-1}^p(\xi)|\det J| d\xi, \quad \forall i \in \{1, 2, \dots, p+1\}.$$

Definimos el **vector de carga local extendido** $\tilde{\mathbf{F}}_i^e \in \mathbb{R}^{N_b}$, construido mediante la asignación

$$\tilde{\mathbf{F}}_{i+e_u-p-1}^e = \mathbf{F}_i^e, \quad \forall i \in \{1, 2, \dots, p+1\},$$

siendo cero cualquier otro elemento de $\tilde{\mathbf{F}}^e$ que no satisfaga la correspondencia. Al igual que en las matrices extendidas anteriores, estos vectores no existirán en la práctica y los elementos de $\mathbf{F}^e$ serán ensamblados directamente sobre $\mathbf{F}$. Para finalizar se obtiene el vector global $\mathbf{F}$ al sumar todos los vectores extendidos, $\mathbf{F} = \sum_{e=1}^{N_{el}} \tilde{\mathbf{F}}^e$.

3.4.3. Imposición de condiciones de contorno.

Tras el ensamblado de la matriz de rigidez global y del vector de carga se implementan las condiciones de contorno,

Al ser en el caso 1D, solo es posible aplicar condiciones de contorno a dos puntos del dominio, explicaremos como se aplican al punto u_0 que será análogo para el punto u_m .

Nota: La explicación de por qué se realizan así las condiciones de contorno puede consultarse en el Anexo A.5.

Tipo Dirichlet

Modificamos la primera (última en el caso u_m) fila de $\mathbf{K}$, de manera que el elemento (1,1) será 1, y el resto pasarán a ser 0, $\mathbf{F}_1$ tomará el valor de la condición de contorno $g(u_0)$.

Con el fin de mantener la simetría del sistema se modificarán las p filas siguientes (anteriores), estableceremos a 0 los elementos de la forma (j,1), y modificaremos los elementos $\mathbf{F}_j$ restándoles el valor $\mathbf{K}_{(j,1)}g(u_0)$.

Tipo Neumann

Actualizaremos el valor de $\mathbf{F}_1 \rightarrow \mathbf{F}_1 + h(u_0)$ ($\mathbf{F}_{N_b} \rightarrow \mathbf{F}_{N_b} + h(u_m)$).

Tipo Robin o mixtas

Actualizaremos $\mathbf{K}_{1,1} \rightarrow \mathbf{K}_{1,1} + \frac{\alpha}{\beta}$, ($\mathbf{K}_{N_b, N_b} \rightarrow \mathbf{K}_{N_b, N_b} + \frac{\alpha}{\beta}$) y $\mathbf{F}_1 \rightarrow \mathbf{F}_1 + \frac{r}{\beta}$, ($\mathbf{F}_{N_b} \rightarrow \mathbf{F}_{N_b} + \frac{r}{\beta}$).

3.5. Caso 2D

Al aplicar la formulación variacional (1.1) al dominio bidimensional $\Omega \subset \mathbb{R}^2$, parametrizamos el problema sobre $\hat{\Omega}$ mediante las variables (ξ, η) . En este caso, el operador gradiente y las integrales sobre el área y su frontera se transforman al espacio paramétrico utilizando la matriz jacobiana $\mathbf{J}$ asociada al mapeo $\mathbf{G}$ (2.4).

3.5.1. Elementos de la matriz global y el vector de carga

Transformamos la **forma bilineal** basada en las funciones de forma en la siguiente integral basada en las funciones base NURBS, **Forma bilineal**:

$$\begin{aligned} a(\phi_{i_2, j_2}(\mathbf{x}), \phi_{i_1, j_1}(\mathbf{x})) &= \int_{\Omega} \nabla \phi_{i_2, j_2}(\mathbf{x}) \cdot \nabla \phi_{i_1, j_1}(\mathbf{x}) d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} \phi_{i_2, j_2}(\mathbf{x}) \phi_{i_1, j_1}(\mathbf{x}) d\Gamma = \\ &= \int_{v_0}^{v_k} \int_{u_0}^{u_m} \left(\mathbf{J}^{-T}(\xi, \eta) \hat{\nabla} \hat{R}_{i_2, j_2}^{p, q}(\xi, \eta) \right)^T \cdot \left(\mathbf{J}^{-T}(\xi, \eta) \hat{\nabla} \hat{R}_{i_1, j_1}^{p, q}(\xi, \eta) \right) |\det \mathbf{J}(\xi, \eta)| d\xi d\eta \\ &\quad + \int_{\hat{\Gamma}_R} \frac{\alpha}{\beta} \hat{R}_{i_2, j_2}^{p, q}(\xi, \eta) \hat{R}_{i_1, j_1}^{p, q}(\xi, \eta) J|_{\hat{\Gamma}}(\xi, \eta) d\hat{\Gamma} = \\ &= \int_{v_0}^{v_k} \int_{u_0}^{u_m} \left(\hat{\nabla} \hat{R}_{i_2, j_2}^{p, q}(\xi, \eta) \right)^T (\mathbf{J}^{-1}(\xi, \eta) \mathbf{J}^{-T}(\xi, \eta)) \left(\hat{\nabla} \hat{R}_{i_1, j_1}^{p, q}(\xi, \eta) \right) |\det \mathbf{J}(\xi, \eta)| d\xi d\eta \\ &\quad + \int_{\hat{\Gamma}_R} \frac{\alpha}{\beta} \hat{R}_{i_2, j_2}^{p, q}(\xi, \eta) \hat{R}_{i_1, j_1}^{p, q}(\xi, \eta) J|_{\hat{\Gamma}}(\xi, \eta) d\hat{\Gamma}. \end{aligned}$$

Aplicamos lo mismo a la **forma lineal**,

$$\begin{aligned} L(\phi_{i_1, j_1}(\mathbf{x})) &= \int_{\Omega} f(\mathbf{x}) \phi_{i_1, j_1}(\mathbf{x}) d\Omega + \int_{\Gamma_N} h(\mathbf{x}) \phi_{i_1, j_1}(\mathbf{x}) d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} \phi_{i_1, j_1}(\mathbf{x}) d\Gamma = \\ &= \int_{v_0}^{v_k} \int_{u_0}^{u_m} f(\mathbf{G}(\xi, \eta)) \hat{R}_{i_1, j_1}^{p, q}(\xi, \eta) |\det \mathbf{J}(\xi, \eta)| d\xi d\eta \\ &\quad + \int_{\hat{\Gamma}_N} h(\mathbf{G}(\xi, \eta)) \hat{R}_{i_1, j_1}^{p, q}(\xi, \eta) J|_{\hat{\Gamma}}(\xi, \eta) d\hat{\Gamma} + \int_{\hat{\Gamma}_R} \frac{r}{\beta} \hat{R}_{i_1, j_1}^{p, q}(\xi, \eta) J|_{\hat{\Gamma}}(\xi, \eta) d\hat{\Gamma}. \end{aligned}$$

3.5.2. Ensamblado de la matriz de rigidez y el vector de cargas

Tomaremos ahora intervalos no nulos de la forma $\hat{\Omega}_e = [u_{e_u}, u_{e_u+1}] \times [v_{e_v}, v_{e_v+1}]$.

Para este caso la **matriz local** $\mathbf{K}^e$ será de tamaño $(p+1)(q+1) \times (p+1)(q+1)$. Con el fin de facilitar la comprensión de la correlación entre funciones base y elementos de la matriz tomaremos una aplicación que relacione los índices de los elementos $\mathbf{K}_{i,j}^e$ con el número de funciones activas en cada dirección del elemento correspondiente:

$$i = I(i_1, j_1) = (i_1 - 1) \cdot (q + 1) + j_1, \quad (3.2)$$

$$j = J(i_2, j_2) = (i_2 - 1) \cdot (q + 1) + j_2, \quad i_1, i_2 \in \{1, 2, \dots, p + 1\}, \quad j_1, j_2 \in \{1, 2, \dots, q + 1\}. \quad (3.3)$$

En esta notación, los índices i y j representan la fila y la columna de la matriz, respectivamente. Por su parte, i_1 e i_2 iteran sobre las funciones activas en la primera dirección paramétrica, mientras que j_1 y j_2 lo hacen en la segunda. Teniendo así la integral que le corresponde al elemento:

$$\mathbf{K}_{I(i_1, j_1), J(i_2, j_2)}^e = \int_{v_{e_v}}^{v_{e_v}+1} \int_{u_{e_u}}^{u_{e_u}+1} \left(\hat{\mathbf{v}} \hat{\mathbf{R}}_{i_2+e_u-p-1, j_2+e_v-q-1}^{p, q} \right)^T (\mathbf{J}^{-1} \mathbf{J}^{-T}) \left(\hat{\mathbf{v}} \hat{\mathbf{R}}_{i_1+e_u-p-1, j_1+e_v-q-1}^{p, q} \right) |\det \mathbf{J}| d\xi d\eta.$$

Definimos de la misma manera ahora la **matriz local extendida** $\tilde{\mathbf{K}}^e$,

$$\tilde{\mathbf{K}}_{I(i_1+e_u-p-1, j_1+e_v-q-1), J(i_2+e_u-p-1, j_2+e_v-q-1)}^e = \mathbf{K}_{I(i_1, j_1), J(i_2, j_2)}^e, \quad i_1, i_2 \in \{1, 2, \dots, p + 1\}, \quad j_1, j_2 \in \{1, 2, \dots, q + 1\}.$$

Formando así la **matriz de rigidez** $\mathbf{K}$ mediante la suma de estas matrices, $\mathbf{K} = \sum_{e=1}^{N_{el}} \tilde{\mathbf{K}}^e$.

Realizamos el mismo procedimiento para el vector de carga:

$$\mathbf{F}_{I(i_1, j_1)}^e = \int_{v_{e_v}}^{v_{e_v}+1} \int_{u_{e_u}}^{u_{e_u}+1} f(\mathbf{G}) \hat{\mathbf{R}}_{i_1, j_1}^{(p, q)} |\det \mathbf{J}| d\xi d\eta.$$

$$\tilde{\mathbf{F}}_{I(i_1+e_u-p-1, j_1+e_v-q-1)}^e = \mathbf{F}_{I(i_1, j_1)}^e, \quad i_1 \in \{1, 2, \dots, p + 1\}, \quad j_1 \in \{1, 2, \dots, q + 1\}.$$

Finalmente obtenemos el vector de carga sumando las contribuciones de cada elemento, $\mathbf{F} = \sum_{e=1}^{N_{el}} \tilde{\mathbf{F}}^e$.

3.5.3. Imposición de condiciones de contorno

Sea $\hat{\Omega}_e = [u_{e_u}, u_{e_u}+1) \times [v_{e_v}, v_{e_v}+1)$ un elemento paramétrico tal que $\mathbf{G}(\hat{\Omega}_e)$ contiene parte de la frontera física de Ω . Dado que la frontera de Ω se parametriza a partir de la frontera del dominio paramétrico, podemos suponer a modo de ejemplo que el elemento $\hat{\Omega}_e$ cumple que esta intersección corresponde únicamente al borde paramétrico inferior. Es decir, la evaluación se realiza sobre $\xi \in [u_{e_u}, u_{e_u}+1)$ fijando $\eta = v_k$ ¹.

Recorreremos las funciones base asociadas a esta frontera, (en este caso recorreremos el índice i_1 y fijamos $j_1 = 1$), para ello proyectamos de vuelta nuestros índices $I_1 = I(i_1, 1)$ para $i_2 = 1, \dots, p + 1$, mediante la ecuación 3.2.

Tipo Dirichlet

1. Para cada índice I_1 :
 - Sustituimos la fila I_1 -ésima de la matriz global $\mathbf{K}$ por ceros, imponiendo 1 en la diagonal ($\mathbf{K}_{I_1, I_1} = 1$).
 - Actualizamos el vector de carga: $\mathbf{F}_{I_1} = g(\mathbf{x}_{I_1})$.
2. Para mantener la simetría del sistema, eliminamos las columnas I correspondientes:
 - Para cada fila distinta de $I_2 \neq I_1$ donde $\mathbf{K}_{I_2, I_1} \neq 0$, modificamos el vector de carga restando la contribución conocida: $\mathbf{F}_K \leftarrow \mathbf{F}_K - \mathbf{K}_{I_2, I_1} \cdot g(\mathbf{x}_{I_1})$.
 - Establecemos $\mathbf{K}_{I_2, I_1} = 0$.

Tipo Neumann

Para cada índice I_1 : $\mathbf{F}_{I_1} \leftarrow \mathbf{F}_{I_1} + \int_{u_m}^{u_{m+1}} h(\mathbf{G}(\xi, v_k)) \cdot \hat{\mathbf{R}}_{i_1, 1}^{p, q}(\xi, v_k) \cdot J_\Gamma(\xi) d\xi$.

Tipo Robin o mixtas

Proyectamos también en este caso $J_1 = (i_2, 1)$ para $i_2 = 1, \dots, p + 1$.

Para cada índice I_1 : $\mathbf{F}_{I_1} \leftarrow \mathbf{F}_{I_1} + \int_{u_m}^{u_{m+1}} \frac{r}{\beta} \cdot \hat{\mathbf{R}}_{i_1, 1}^{p, q}(\xi, v_k) \cdot J_\Gamma(\xi) d\xi$.

Para cada par de índices (I_1, J_1) : $\mathbf{K}_{I_1, J_1} \leftarrow \mathbf{K}_{I_1, J_1} + \int_{u_m}^{u_{m+1}} \frac{\alpha}{\beta} \cdot \hat{\mathbf{R}}_{i_1, 1}^{p, q}(\xi, v_k) \cdot \hat{\mathbf{R}}_{i_2, 1}^{p, q}(\xi, v_k) \cdot J_\Gamma(\xi) d\xi$.

¹Para los elementos esquina basta aplicar el algoritmo dos veces, una para cada borde.

3.6. Convergencia en L2 y H1 y tiempo de computación teórico

3.6.1. Convergencia

Si bien, por el Teorema de Lax-Milgram la existencia y unicidad de la solución variacional del problema de Poisson únicamente requiere que $u \in H^1(\Omega)$, para garantizar las **tasas de convergencia óptimas** en IGA, debemos exigir una mayor regularidad a la solución exacta y ciertas propiedades al mapeo:

- **Regularidad de la solución:** $u \in H^{p+1}(\Omega)$.
- **Malla no degenerada:** $\det(J) > 0$ y $|\det(J)| < \infty$.

Bajo estas condiciones podemos tomar cotas del error en diferentes espacios:

- **Norma en $H^1(\Omega)$:** $\|u - u_h\|_{H^1(\Omega)} \leq C_1 h^p \|u\|_{H^{p+1}(\Omega)}$.
- **Norma en $L^2(\Omega)$:** $\|u - u_h\|_{L^2(\Omega)} \leq C_2 h^{p+1} \|u\|_{H^{p+1}(\Omega)}$.

Donde C_1 y C_2 dependen de la continuidad de las estructuras de NURBS asociadas al problema, a medida que la suavidad crece, estas decrecen.

Demostración. Demostración en el Anexo A.4.15. □

3.6.2. Coste computacional

Separamos el tiempo de computación en sus fases independientes:

- **Fase de evaluación:** Fase previa al ensamblado para dimensiones mayores o iguales que dos, que aprovecha las propiedades del producto tensorial para evitar recalcular evaluaciones de funciones base. Su coste computacional, $\sum_{i=1}^d Q_i \cdot n_i = \mathcal{O}(d \cdot \max_i \{Q_i \cdot n_i\}) = \mathcal{O}(d \cdot h^{-1})$, donde Q_i es el número de evaluaciones direccionales (típicamente $Q_i = p_i + 1$) y n_i el número de divisiones correspondientes a la dimensión i , que es inversamente proporcional al tamaño de la malla.
- **Fase de ensamblado:** Fase en la que se evalúan las reglas de cuadratura numéricas y se ensamblan. Su coste computacional, $\prod_{i=1}^d Q_i \cdot n_i \cdot C = \mathcal{O}(h^{-d})$, donde C es el coste de la integración numérica, cálculo del jacobiano y de la manipulación de la memoria.
- **Fase de resolución:** Una vez ensamblado el sistema lineal $\mathbf{Kd} = \mathbf{F}$ (cuyo coste escala idealmente como $\mathcal{O}(N_b)$), el coste computacional de su resolución viene determinado por el semi-ancho de banda de la matriz (definido formalmente como $B = \max |i - j|$ para todo $\mathbf{K}_{i,j} \neq 0$, es decir, la distancia máxima desde la diagonal principal a un elemento no nulo):
 - **Problemas 1D (Factorización LU):** En una dimensión, el semi-ancho de banda está acotado por p , lo que permite utilizar métodos directos (LU o Cholesky) de manera muy eficiente. En este caso, se alcanza una complejidad computacional esencialmente lineal, $\mathcal{O}(N_b) = \mathcal{O}(h^{-1})$ [28].
 - **Problemas 2D (ICCG):** En dos dimensiones, el semi-ancho de banda crece aproximadamente como $p\sqrt{N_b}$, lo que hace que los métodos directos dejen de ser viables desde el punto de vista de memoria. Por este motivo, se recurre al método del Gradiente Conjugado Precondicionado (ICC). Su coste iterativo depende del condicionamiento del sistema, $\kappa = \mathcal{O}(h^{-2})$, y, debido al mayor solapamiento de las funciones en IGA, la complejidad global tiende (tanto en la práctica como en la teoría) a escalar como $\mathcal{O}(N_b^{1.5}) = \mathcal{O}(h^{-3})$ [29].

Capítulo 4

Resultados.

La explicación de como se ha implementado el programa con el que se han resuelto los problemas que aparecen en este capítulo y de los métodos numéricos empleados para resolver los sistemas matriciales correspondientes puede encontrarse en el anexo A.2.

4.1. Resultados en 1D

Para validar la implementación numérica, se comienza resolviendo el problema de Poisson en una dimensión, cuya formulación variacional y proceso de resolución se han descrito previamente en la sección 3.4. En particular, se considera un problema con condiciones de contorno: $f(x) = 100 \sin(10x)$, $u(0) = 0$, $u(L) = 0$, cuyas solución y derivada analítica son $u(x) = \sin(10x)$, $u'(x) = 10 \cos(10x)$.

Parámetros iniciales

Consideramos como dominio físico una semicircunferencia de radio unidad contenida en el plano OXY y centrada en el origen (véase la Figura 4.2a). Su parametrización exacta se basa en una curva NURBS cuadrática ($p = 2$), construida sobre el dominio paramétrico dado por el vector de nudos $U = \{0, 0, 0, 1/2, 1/2, 1, 1, 1\}$, la red de puntos de control $\mathbf{P} = \{(-1, 0, 0), (-1, 1, 0), (0, 1, 0), (1, 1, 0), (1, 0, 0)\}$ y los pesos $\mathbf{w} = \{1, \sqrt{2}/2, 1, \sqrt{2}/2, 1\}$.

Resultados numéricos

Se muestran a continuación resultados numéricos obtenidos sobre el error en diferentes normas y el tiempo de computación (Tablas 4.1, 4.2 y 4.3) para distintos grados polinomiales y número de elementos, además se aporta una representación de los elementos no nulos de las matrices del sistema para 16 elementos en diferentes grados polinomiales y la proporción de estos (figura 4.1).

Tabla 4.1: Error en norma L^2 .

$p \setminus N_{el}$	16	32	64	128	256
2	0,0513	0,00415	0,000451	$5,43 \cdot 10^{-5}$	$6,73 \cdot 10^{-6}$
3	0,0399	0,000916	$4,18 \cdot 10^{-5}$	$2,39 \cdot 10^{-6}$	$1,51 \cdot 10^{-7}$
4	0,014	0,000169	$3,58 \cdot 10^{-6}$	$1,74 \cdot 10^{-7}$	$3,62 \cdot 10^{-8}$
5	0,0080	$3,44 \cdot 10^{-5}$	$6,40 \cdot 10^{-7}$	$1,39 \cdot 10^{-7}$	$3,48 \cdot 10^{-8}$

Tabla 4.2: Error en norma H^1 .

$p \setminus N_{el}$	16	32	64	128	256
2	5,15	0,827	0,182	0,0442	0,010
3	1,89	0,143	0,0147	0,00174	0,000216
4	1,13	0,0286	0,00133	$7,71 \cdot 10^{-5}$	$4,85 \cdot 10^{-6}$
5	0,51	0,00568	0,000118	$5,42 \cdot 10^{-6}$	$1,10 \cdot 10^{-6}$

Tabla 4.3: Tiempos totales (s).

$p \setminus N_{el}$	16	32	64	128	256
2	0.0034	0.0074	0.0124	0.0245	0.0484
3	0.0043	0.0085	0.0163	0.0329	0.0664
4	0.0055	0.0104	0.0197	0.0387	0.0807
5	0.0067	0.0141	0.0286	0.0582	0.1128

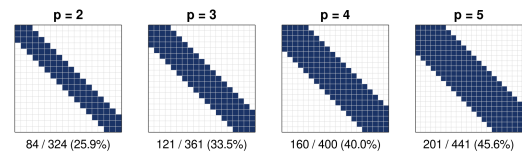


Figura 4.1: Elementos no nulos de la matriz frente al grado polinomial para $N_{el} = 16$ en 1D.

El análisis de las tablas 4.1 y 4.2 muestra que, para grados polinomiales elevados ($p = 4$ y $p = 5$), el error sigue inicialmente las tasas óptimas de convergencia, pero se estanca alrededor de 10^{-8} en la norma L^2 . Este comportamiento no se debe a limitaciones del método, sino a los efectos del condicionamiento del sistema y a la precisión finita de la aritmética en punto flotante. En IGA, el número de condición de la matriz de rigidez depende tanto del tamaño de malla como del grado polinomial, creciendo como $\kappa(\mathbf{K}) \approx \mathcal{O}(h^{-2}p^24^p)$ [31]. Para grados altos, el mayor solapamiento de las funciones base conduce a valores del orden de 10^8 , lo que, en doble precisión ($\epsilon_{\text{mach}} \approx 10^{-16}$), limita la exactitud alcanzable.

La expresión del error total observada puede justificarse mediante la desigualdad triangular en el espacio normado de la solución. Si u es la solución exacta, u_h la solución discreta ideal (en aritmética exacta) y $\tilde{u}_h$ la solución computada, el error total se descompone como:

$$E_{\text{total}} = \|u - \tilde{u}_h\| \leq \|u - u_h\| + \|u_h - \tilde{u}_h\|. \quad (4.1)$$

El primer término de (4.1) corresponde al **error de discretización**, como vimos en la sección 3.6.1, se establece la cota

$$\|u - u_h\|_{L^2} \leq C_1 h^{p+1}, \quad (4.2)$$

donde C_1 es independiente de h .

El segundo término de (4.1) representa el **error algebraico**, asociado a la resolución numérica del sistema $\mathbf{Kc} = \mathbf{F}$ en precisión finita. De acuerdo con los resultados clásicos de estabilidad en álgebra lineal numérica [32], este error está amplificado por el número de condición de la matriz de rigidez, cumpliéndose

$$\|u_h - \tilde{u}_h\| \leq C_2 \kappa(\mathbf{K}) \epsilon_{\text{mach}}, \quad (4.3)$$

donde ϵ_{mach} es la precisión de la máquina ($\approx 1,11 \times 10^{-16}$ en doble precisión) y C_2 depende débilmente del tamaño del sistema.

Combinando ambas contribuciones (4.2) y (4.3) en (4.1), se obtiene la cota global

$$E_{\text{total}} \leq C_1 h^{p+1} + C_2 \kappa(\mathbf{K}) \epsilon_{\text{mach}} = C_1 h^{p+1} + C_2 10^8 10^{-16},$$

que explica el comportamiento observado: para mallas finas, el error deja de estar dominado por la discretización y pasa a estar limitado por los efectos de redondeo.

Por otro lado, la experimentación revela que el estancamiento numérico en la norma H^1 sucede de forma prematura, estableciendo un límite en torno a 10^{-6} . Este comportamiento diferencial respecto a la norma L^2 es una consecuencia analítica directa del proceso de diferenciación numérica en dominios isogeométricos. La semi-norma H^1 evalúa el error sobre el gradiente de la solución. Por la regla de la cadena, el cómputo del gradiente físico requiere la pre-multiplicación por la inversa de la matriz Jacobiana del mapeo, cuya norma espectral escala asintóticamente como $\|\mathbf{J}^{-1}\| \sim \mathcal{O}(h^{-1})$.

En consecuencia, el ruido de redondeo de punto flotante subyacente a la evaluación de la función experimenta una amplificación proporcional a la inversa del tamaño de malla. Para el nivel de refinamiento $N_{el} = 256$ ($h \approx 4 \cdot 10^{-3}$), el factor de amplificación $h^{-1} \approx 2,5 \cdot 10^2$ eleva el suelo de error subyacente de 10^{-8} (observado en L^2) en algo más de dos órdenes de magnitud, situando el umbral de estancamiento teórico para la norma H^1 exactamente en el rango de 10^{-6} , lo cual corrobora con precisión matemática los resultados empíricos observados.

Para visualizar la solución, en la Figura 4.2 se presenta esta. Concretamente, la Figura 4.2a muestra el dominio físico del problema, mientras que la Figura 4.2b ilustra la magnitud de esta, empleando la dimensión Z como recurso puramente visual.

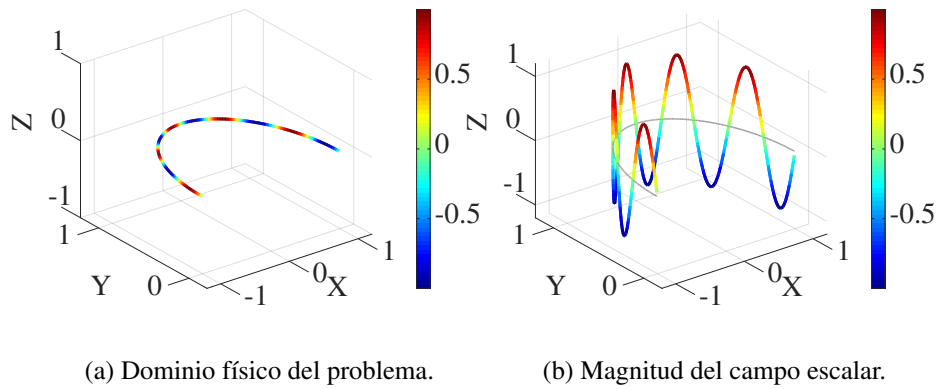


Figura 4.2: Semicircunferencia que constituye la totalidad del dominio físico. En (b) se emplea la dimensión ortogonal (Z) exclusivamente como recurso visual para representar los valores de evaluación de la solución.

4.2. Resultados en 2D

Una vez validada la formulación en una dimensión, se extiende el análisis al problema de Poisson bidimensional. Con el objetivo de comprobar una correcta implementación, se consideran a continuación dos problemas de valores en la frontera definidos sobre dominios físicos con geometrías diferentes.

4.2.1. Problema 1

Planteamos un primer caso de estudio sobre el dominio físico $\Omega = [0, 2] \times [0, 2]$. El sistema se define mediante un problema de Poisson con condiciones de contorno de tipo Dirichlet homogéneas en toda la frontera ($u = 0$ sobre Γ), con término fuente $f(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y)$, la solución exacta viene dada por $u(x, y) = \sin(\pi x) \sin(\pi y)$ y las derivadas parciales son $\frac{\partial u}{\partial x} = \pi \cos(\pi x) \sin(\pi y)$ y $\frac{\partial u}{\partial y} = \pi \sin(\pi x) \cos(\pi y)$.

Parámetros iniciales

La geometría se mapea mediante la identidad entre las coordenadas espaciales y paramétricas. Para ello, empleamos funciones base cuadráticas en ambas direcciones espaciales ($p = q = 2$), construidas sobre los vectores de nudos $U = V = \{0, 0, 0, 2, 2, 2\}$, la red de puntos de control $\mathbf{P} = \{(0, 0), (1, 0), (2, 0)\}, \{(0, 1), (1, 1), (2, 1)\}, \{(0, 2), (1, 2), (2, 2)\}$ y los pesos unitarios ($w_{i,j} = 1$).

Resultados numéricos

En la Tabla 4.4 se presentan los errores de discretización y los costes computacionales (ensamblado y resolución) para $p = q = 2$ en distintas mallas. Los resultados confirman las predicciones teóricas del Análisis Isogeométrico: el error en norma L^2 converge con orden $\mathcal{O}(h^3)$, reduciéndose aproximadamente en un factor de 8 por refinamiento, mientras que la seminorma H^1 sigue el orden esperado $\mathcal{O}(h^2)$.

En cuanto al coste, el tiempo de ensamblado crece linealmente con el número de grados de libertad, mientras que el de resolución presenta un crecimiento superlineal debido al aumento del ancho de banda de la matriz.

Tabla 4.4: Tabla de convergencia y costes computacionales.

Nº Elementos	T. Ensamblado (s)	T. Resolución (s)	Error L^2	Error H^1
16×16	0,0070	0,0013	$4,36 \cdot 10^{-4}$	$2,60 \cdot 10^{-2}$
32×32	0,0251	0,0013	$5,23 \cdot 10^{-5}$	$6,42 \cdot 10^{-3}$
64×64	0,0604	0,0244	$6,46 \cdot 10^{-6}$	$1,60 \cdot 10^{-3}$
128×128	0,2064	0,1083	$8,06 \cdot 10^{-7}$	$3,99 \cdot 10^{-4}$
256×256	0,6724	0,7157	$1,01 \cdot 10^{-7}$	$9,97 \cdot 10^{-5}$

4.2.2. Problema 2

Planteamos un segundo caso de estudio sobre el dominio físico correspondiente a un círculo de radio unidad centrado en el origen en el plano OXY . El sistema se define mediante un problema de Poisson con condiciones de contorno de tipo Dirichlet homogéneas en toda la frontera ($u = 0$ sobre Γ), con término fuente $f(x,y) = 1$, la solución exacta viene dada por $u(x,y) = \frac{1-x^2-y^2}{4}$ y las derivadas parciales son $\frac{\partial u}{\partial x} = -\frac{x}{2}$ y $\frac{\partial u}{\partial y} = -\frac{y}{2}$.

Parámetros iniciales

Empleamos funciones base cuadráticas en ambas direcciones espaciales ($p = q = 2$), construidas sobre los vectores de nudos $U = V = \{0, 0, 0, 1, 1, 1\}$, la red de puntos de control $\mathbf{P} = \{(-1, 0), (-1, -1), (0, -1)\}, \{(-1, 1), (0, 0), (1, -1)\}, \{(0, 1), (1, 1), (1, 0)\}$ y la matriz de pesos asociados $\mathbf{W} = \{1, \sqrt{2}/2, 1\}, \{\sqrt{2}/2, 1, \sqrt{2}/2\}, \{1, \sqrt{2}/2, 1\}$.

Resultados numéricos

Los resultados para el segundo problema, recogidos en la Tabla 4.5, confirman el comportamiento observado anteriormente. El método mantiene las tasas de convergencia óptimas ($\mathcal{O}(h^3)$ en L^2 y $\mathcal{O}(h^2)$ en H^1) pese al cambio de dominio, así como la misma escalabilidad en los tiempos de ensamblado y resolución.

Tabla 4.5: Tabla de convergencia y costes computacionales.

Nº Elementos	T. Ensamblado (s)	T. Resolución (s)	Error L^2	Error H^1
16×16	0,0068	0,0016	$1,42 \cdot 10^{-6}$	$1,21 \cdot 10^{-4}$
32×32	0,0084	0,0198	$1,74 \cdot 10^{-7}$	$3,02 \cdot 10^{-5}$
64×64	0,0663	0,0283	$2,17 \cdot 10^{-8}$	$7,53 \cdot 10^{-6}$
128×128	0,2076	0,1179	$2,71 \cdot 10^{-9}$	$1,88 \cdot 10^{-6}$
256×256	0,6902	0,9705	$3,42 \cdot 10^{-10}$	$4,70 \cdot 10^{-7}$

En la Figura 4.3 se presenta la evaluación de las soluciones para ambos casos de estudio. Específicamente, la Figura 4.3a muestra la superficie de la solución aproximada del problema 1, mientras que la Figura 4.3b ilustra el resultado correspondiente al problema 2. En ambas representaciones, el valor de la solución se ha proyectado sobre la dimensión Z para permitir su visualización.

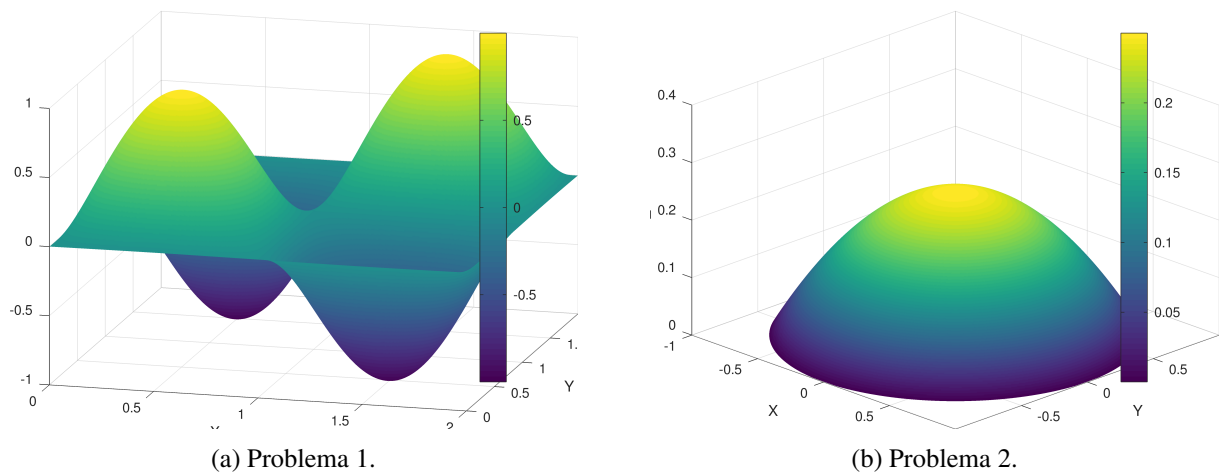


Figura 4.3: Representación gráfica de las soluciones aproximadas obtenidas para ambos problemas. Dado que los dominios físicos están contenidos en el plano $Z = 0$, el valor de la solución se visualiza a través de la dimensión Z .

Capítulo 5

Conclusiones

A lo largo de este trabajo se han establecido las bases matemáticas de IGA, motivado por sus ventajas frente a FEM clásico. El estudio se ha centrado en el desarrollo de su formulación teórica, su implementación en C++ y la validación numérica de los resultados.

A la vista del desarrollo teórico y de los resultados numéricos obtenidos en los capítulos previos, extraemos las siguientes conclusiones:

- **Exactitud Geométrica:** Principal ventaja de IGA frente a FEM, elimina el error de aproximación geométrico y ahorra el desarrollo de una malla intermedia.
- **Alta Continuidad y Suavidad:** Debido a las propiedades de las funciones base NURBS, obtenemos una suavidad de clase $C^{(p-1)}$.
- **Convergencia Numérica:** Los resultados experimentales presentados en el Capítulo 5 confirman las tasas de convergencia en norma H^1 , decrece con orden $\mathcal{O}(h^p)$, y el error en norma L^2 , con orden $\mathcal{O}(h^{p+1})$.
- **Coste Computacional:** El amplio soporte de las bases racionales (activas en $2p + 1$ intervalos por dimensión) incrementa el ancho de banda de las matrices. En la práctica, el ensamblado crece linealmente con los grados de libertad, mientras que la resolución presenta un comportamiento superlineal, fuertemente influido por el grado polinomial p .
- **Implementación:** La implementación de programas que permitan resolver EDPs mediante IGA requieren de una gestión detallada de la información geométrica.

5.1. Trabajo futuro

Otros temas que no se han tratado que tienen una alta importancia en referencia a IGA son:

- **k-refinamiento:** Refinamiento elevando el orden y la continuidad simultáneamente, una característica exclusiva de IGA que maximiza la eficiencia computacional.
- **Refinamiento local (T-Splines):** Superar las limitaciones intrínsecas del producto tensorial de las NURBS implementando T-Splines [5], permitiendo así refinar de manera local un elemento concreto sin afectar a vecinos relativamente lejanos que mediante NURBS sería imposible, empleado en biotecnología para representar estructuras orgánicas complejas localmente. También pueden utilizarse los HB-Splines [16], THB-Splines [17] o los U-Splines [25].
- **Problemas dependientes del tiempo:** Implementaciones discretizando el tiempo mediante métodos como Crank-Nicolson al igual que en FEM, actualmente se están desarrollando discretizaciones isogeométricas espacio-tiempo.
- **Comparación de cotas respecto a FEM:** Resta comprobar si, bajo condiciones idénticas de grado polinomial y número de elementos, el coste computacional resulta equiparable, permitiendo que las cotas de error más óptimas de IGA se traduzcan en una mayor precisión por cada grado de libertad empleado.

Bibliografía

- [1] T. J. R. Hughes, J. A. Cottrell, and Y. Bazilevs. Isogeometric analysis: CAD, finite elements, NURBS, exact geometry and mesh refinement. *Computer Methods in Applied Mechanics and Engineering*, 194(39):4135–4195, 2005. ISSN 0045-7825. doi: <https://doi.org/10.1016/j.cma.2004.10.008>. URL <https://www.sciencedirect.com/science/article/pii/S0045782504005171>.
- [2] J.A. Cottrell, T.J.R. Hughes, and Y. Bazilevs. *Isogeometric Analysis: Toward Integration of CAD and FEA*. Wiley, 2009. ISBN 978-0-470-74873-2. URL <https://books.google.es/books?id=SfOUDAEACAAJ>.
- [3] G.E. Farin. *Curves and Surfaces for CAGD: A Practical Guide*. Computer graphics and geometric modeling. Elsevier Science, 2002. ISBN 978-1-55860-737-8. URL <https://books.google.es/books?id=5HYTP1dIAp4C>.
- [4] L. Piegl and W. Tiller. *The NURBS Book*. Monographs in Visual Communication. Springer Berlin Heidelberg, 2012. ISBN 978-3-642-59223-2. URL <https://books.google.es/books?id=58KqCAAAQBAJ>.
- [5] Y. Bazilevs, V.M. Calo, J.A. Cottrell, J.A. Evans, T.J.R. Hughes, S. Lipton, M.A. Scott, and T.W. Sederberg. Isogeometric analysis using T-splines. *Computer Methods in Applied Mechanics and Engineering*, 199(5-8):229–263, 2010. doi: 10.1016/j.cma.2009.02.036.
- [6] L. Veiga, Annalisa Buffa, Judith Rivas, and G. Sangalli. Some estimates for h–p–k-refinement in Isogeometric Analysis. *Numerische Mathematik*, 118:271–305, January 2009. doi: 10.1007/s00211-010-0338-z.
- [7] P. Antolin, A. Buffa, F. Calabrò, M. Martinelli, and G. Sangalli. Efficient matrix computation for tensor-product isogeometric analysis: The use of sum factorization. *Computer Methods in Applied Mechanics and Engineering*, 285:817–828, 2015. ISSN 0045-7825. doi: <https://doi.org/10.1016/j.cma.2014.12.013>. URL <https://www.sciencedirect.com/science/article/pii/S0045782514004927>.
- [8] J.H. Ahlberg, E.N. Nilson, J.L. Walsh, and R. Bellman. *The Theory of Splines and Their Applications: Mathematics in Science and Engineering: A Series of Monographs and Textbooks, Vol. 38*. Number v. 38. Academic Press, 2016. ISBN 978-1-4832-2295-0. URL <https://books.google.es/books?id=3bZ1DAAAQBAJ>.
- [9] David J. Benson, Yuri Bazilevs, Ming-Chen Hsu, and Thomas J.R. Hughes. A large deformation, rotation-free, isogeometric shell. *Computer Methods in Applied Mechanics and Engineering*, 200(13-16):1367–1378, 2011. doi: 10.1016/j.cma.2010.12.003.
- [10] J.A. Cottrell, A Reali, Yuri Bazilevs, and Thomas Hughes. Isogeometric analysis of structural vibrations. *Computer Methods in Applied Mechanics and Engineering - COMPUT METHOD APPL MECH ENG*, 195, August 2006. doi: 10.1016/j.cma.2005.09.027.

- [11] Yuri Bazilevs, M.-C Hsu, Ido Akkerman, Samuel Wright, Kenji Takizawa, B. Henicke, Timothy Spelman, and Tayfun Tezduyar. 3D simulation of wind turbine rotors at full scale. Part I: Geometry modeling and aerodynamics. *International Journal for Numerical Methods in Fluids*, 65:207 – 235, January 2011. doi: 10.1002/fld.2400.
- [12] Yuri Bazilevs, Jeffrey Gohean, Thomas Hughes, R. Moser, and Yongjie Zhang. Patient-specific isogeometric fluid-structure interaction analysis of thoracic aortic blood flow due to implantation of the Jarvik 2000 left ventricular assist device. *Computer Methods in Applied Mechanics and Engineering*, 198:3534–3550, September 2009. doi: 10.1016/j.cma.2009.04.015.
- [13] K. V. Kostas, A. I. Ginnis, C. G. Politis, and P. D. Kaklis. Ship-hull shape optimization with a T-spline based BEM–isogeometric solver. *Computer Methods in Applied Mechanics and Engineering*, 284:611–622, 2015. ISSN 0045-7825. doi: <https://doi.org/10.1016/j.cma.2014.10.030>. URL <https://www.sciencedirect.com/science/article/pii/S0045782514004009>.
- [14] A. Buffa, G. Sangalli, and R. Vázquez. Isogeometric analysis in electromagnetics: B-splines approximation. *Computer Methods in Applied Mechanics and Engineering*, 199(17):1143–1152, 2010. ISSN 0045-7825. doi: <https://doi.org/10.1016/j.cma.2009.12.002>. URL <https://www.sciencedirect.com/science/article/pii/S0045782509004010>.
- [15] A. Buffa, C. de Falco, and G. Sangalli. IsoGeometric Analysis: Stable elements for the 2D Stokes equation. *International Journal for Numerical Methods in Fluids*, 65(11-12):1407–1422, 2011. doi: <https://doi.org/10.1002/fld.2337>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/fld.2337>. _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/fld.2337>.
- [16] A. A, Carlotta Giannelli, B. B, and Bernd Simeon. A Hierarchical Approach to Adaptive Local Refinement in Isogeometric Analysis. *Computer Methods in Applied Mechanics and Engineering*, 200, December 2011. doi: 10.1016/j.cma.2011.09.004.
- [17] Carlotta Giannelli, Bert Jüttler, and Hendrik Speleers. THB-splines: The truncated basis for hierarchical splines. *Computer Aided Geometric Design*, 29(7):485–498, 2012. ISSN 0167-8396. doi: <https://doi.org/10.1016/j.cagd.2012.03.025>. URL <https://www.sciencedirect.com/science/article/pii/S0167839612000519>.
- [18] Ulrich Langer, Stephen E Moore, and Martin Neumüller. Space-time isogeometric analysis of parabolic evolution equations. *Computer Methods in Applied Mechanics and Engineering*, 306: 342–363, 2016. doi: 10.1016/j.cma.2016.03.042.
- [19] Eric Brechner. Developing and Using CAD/CAM/CAE Systems in Boeing. *IEEE Annals of the History of Computing*, 46(1):38–49, January 2024. ISSN 1058-6180. doi: 10.1109/MAHC.2024.3468340.
- [20] Gerald Farin. A History of Curves and Surfaces in CAGD. *Computer Aided Geometric Design - CAGD*, December 2002. doi: 10.1016/B978-044451104-1/50002-2.
- [21] K.J. Versprille. *Computer Aided Design Applications of the Rational B-spline Approximation Form*. University Microfilms, 1975. URL <https://books.google.es/books?id=D0wvjwEACAAJ>.
- [22] Dassault Systèmes. *Abaqus Analysis User’s Guide*. Dassault Systèmes Simulia Corp., 2016. URL <http://130.149.89.49:2080/v2016/books/usb/default.htm>.
- [23] ANSYS, Inc. Ansys Mechanical - Advanced Meshing and Simulation Technologies, 2024. URL <https://www.ansys.com/products/structures/ansys-mechanical>.
- [24] Robert McNeel & Associates. Rhinoceros 3D, 2024. URL <https://www.rhino3d.com/>

features/.

- [25] Coreform LLC. Coreform IGA: Next-generation isogeometric analysis software, 2025. URL <https://coreform.com/coreform-iga/>.
- [26] Annalisa Buffa, Gregor Gantner, Carlotta Giannelli, Dirk Praetorius, and Rafael Vázquez. Mathematical Foundations of Adaptive Isogeometric Analysis. *Archives of Computational Methods in Engineering*, 29(7):4479–4555, September 2022. ISSN 1886-1784. doi: 10.1007/s11831-022-09752-5. URL <http://dx.doi.org/10.1007/s11831-022-09752-5>.
- [27] P.G. Ciarlet. *The Finite Element Method for Elliptic Problems*. Classics in Applied Mathematics. Society for Industrial and Applied Mathematics, 2002. ISBN 978-0-89871-514-9. URL <https://books.google.es/books?id=1PF-WS0Nl9IC>.
- [28] G.H. Golub and C.F. Van Loan. *Matrix Computations*. Johns Hopkins Studies in the Mathematical Sciences. Johns Hopkins University Press, 2013. ISBN 978-1-4214-0794-4. URL <https://books.google.es/books?id=X5YfsuCWpxMC>.
- [29] Y. Saad. *Iterative Methods for Sparse Linear Systems: Second Edition*. Other Titles in Applied Mathematics. Society for Industrial and Applied Mathematics, 2003. ISBN 978-0-89871-534-7. URL <https://books.google.es/books?id=qtzmkzzqFmcC>.
- [30] Gaël Guennebaud, Benoît Jacob, and others. Eigen v3, 2010. URL <http://eigen.tuxfamily.org>.
- [31] Kapil P. S. Gahalaut. *Isogeometric Analysis: Condition Number Estimates and Fast Solvers*. PhD Thesis, Johannes Kepler University Linz, 2013.
- [32] N.J. Higham. *Accuracy and Stability of Numerical Algorithms: Second Edition*. Other Titles in Applied Mathematics. Society for Industrial and Applied Mathematics, 2002. ISBN 978-0-89871-802-7. URL <https://books.google.es/books?id=7J52J4GrsJkC>.
- [33] M.G. Larson and F. Bengzon. *The Finite Element Method: Theory, Implementation, and Applications*. Texts in Computational Science and Engineering. Springer Berlin Heidelberg, 2013. ISBN 978-3-642-33287-6. URL https://books.google.es/books?id=Vek_AAAAQBAJ.
- [34] H. Brezis. *Functional Analysis, Sobolev Spaces and Partial Differential Equations*. Universitext. Springer New York, 2010. ISBN 978-0-387-70913-0. URL <https://books.google.es/books?id=GAA2Xq0IIGoC>.
- [35] L.C. Evans. *Partial Differential Equations*. Graduate studies in mathematics. American Mathematical Society, 1991. URL <https://books.google.es/books?id=TF8EzQEACAAJ>.
- [36] Josef Kiendl, Kai-Uwe Bletzinger, J. Linhard, and Roland Wüchner. Isogeometric shell analysis with Kirchhoff-Love elements. *Computer Methods in Applied Mechanics and Engineering - COMPUT METHOD APPL MECH ENG*, 198:3902–3914, November 2009. doi: 10.1016/j.cma.2009.08.013.
- [37] R.A. Liming. *Practical Analytic Geometry with Applications to Aircraft*. Macmillan, 1944. ISBN 978-0-598-45557-4. URL <https://books.google.es/books?id=x99EAAAAAAAJ>.
- [38] Dalton Fazanaro, Paulo Amorim, Thiago Franco de Moraes, Jorge Silva, and Helio Pedrini. NURBS Parameterization for Medical Surface Reconstruction. *Applied Mathematics*, 07:137–144, January 2016. doi: 10.4236/am.2016.72013.

- [39] U. Langer and O. Steinbach. *Space-Time Methods: Applications to Partial Differential Equations*. Radon Series on Computational and Applied Mathematics. De Gruyter, 2019. ISBN 978-3-11-054848-8. URL <https://books.google.es/books?id=7DvEDwAAQBAJ>.

Anexos

A.1. Espacios funcionales empleados

- **Espacio de funciones de cuadrado integrable:**

$$L^2(\Omega) = \left\{ v : \Omega \rightarrow \mathbb{R} \mid \int_{\Omega} |v|^2 dx < \infty \right\}.$$

- **Espacio de Sobolev de orden 1:**

$$H^1(\Omega) = \{v \in L^2(\Omega) \mid \nabla v \in [L^2(\Omega)]^n\}.$$

- **Espacio de trazas de orden 1/2:**

$$H^{1/2}(\Gamma_D) = \left\{ v \in L^2(\Gamma_D) \mid \int_{\Gamma_D} \int_{\Gamma_D} \frac{|v(x) - v(y)|^2}{\|x - y\|^n} ds_x ds_y < \infty \right\}.$$

Cabe destacar que ds_x y ds_y denotan la integración respecto a la medida de Hausdorff $(n-1)$ -dimensional $\mathcal{H}^{n-1}$. Para un conjunto $S \subset \mathbb{R}^n$, la medida de Hausdorff d -dimensional se define formalmente como:

$$\mathcal{H}^d(S) = \lim_{\delta \rightarrow 0} \inf \left\{ \sum_{i=1}^{\infty} \alpha(d) \left(\frac{\text{diam}(U_i)}{2} \right)^d \right\},$$

donde el ínfimo se toma sobre todos los recubrimientos numerables $\{U_i\}$ de S tales que $\text{diam}(U_i) < \delta$, siendo $\alpha(d)$ el volumen de la bola unidad en $\mathbb{R}^d$.

A.2. Implementación computacional

A lo largo del siguiente capítulo se mostrará cómo se ha implementado numéricamente los algoritmos propios de IGA.

A.2.1. Lenguaje y librerías utilizados

El lenguaje principal utilizado ha sido C++, los cálculos de las curvas y superficies NURBS, ensamblado y resolución del sistema y cálculo de normas, han sido escritos en este. Octave ha sido empleado como visualizador de los ficheros de salida del código en C++.

Se ha implementado la clase **iMatrix**, que define una matriz de elementos abstractos, así como todos los métodos y operadores necesarios.

Las librerías necesarias para el funcionamiento e implementación son:

- **Eigen [30]:** Matrices *sparse*, optimización en el ensamblado y solvers.
- **Librería estándar:** Para contenedores y algoritmos (vector y algorithm), cálculo del tiempo de ejecución (chrono), funciones matemáticas básicas (CMath) y entrada y salida para creación de ficheros (iostream, fstream, iomanip y string).

A.2.2. Métodos numéricos empleados

Trataremos ahora los métodos numéricos que no son inherentes a IGA que hemos utilizado:

- **Integración numérica:** Se ha utilizado la cuadratura de Gauss-Legendre, con $p_i + 1$ puntos de cuadratura en la dirección de la dimensión i , lo que resulta en $n_G = \prod_{i=1}^d (p_i + 1)$ puntos de cuadratura totales. Esta regla numérica aproxima la integral definida en un intervalo de referencia $[-1, 1]$ mediante una suma ponderada de la forma $\int_{-1}^1 f(\xi) d\xi \approx \sum_{k=1}^{n_G} w_k f(\xi_k)$, donde los puntos de evaluación ξ_k corresponden a las raíces del polinomio de Legendre de grado n , y w_k son sus pesos asociados. Esta regla de integración se adopta de forma estándar en IGA, ya que permite mantener el orden óptimo de convergencia del método [5].

- **Solvers numéricos:** Se ha utilizado el método LU para el caso 1D y el método del gradiente conjugado preconditionado (Cholesky incompleto o Jacobi en su defecto).

En 1D, el tamaño moderado de la matriz permite aplicar LU adaptado a matrices *sparse* del sistema, un solver directo que descompone el sistema en dos matrices triangulares, obteniendo la solución exacta con un coste computacional y de memoria relativamente bajos.

Al aumentar la dimensión del problema, el tamaño del sistema crece drásticamente. Aunque la matriz resultante sigue siendo *sparse*, aplicar un método directo resulta prohibitivo tanto en tiempo de ejecución como en memoria. Por ello, se pasa al gradiente conjugado, un método iterativo adecuado dado que la matriz del sistema es simétrica y definida positiva. Para acelerar la convergencia de este método iterativo se aplican preconditionadores: Cholesky Incompleto, que aproxima la matriz conservando su estructura de ceros original, o Jacobi, que utiliza únicamente la diagonal y presenta un coste por iteración mínimo.

A.2.3. Estructura del programa

Explicaremos el orden de ejecución del programa y que se realiza en cada fase de este.

1. **Estructuras NURBS:** Se crean el espacio físico y paramétrico referente al problema que se quiere tratar, luego se aplica el refinado correspondiente para ajustar el tiempo de ejecución y la precisión requerida.
2. **Fase de evaluaciones:** Fase opcional de optimización computacional que permite precalcular y almacenar las evaluaciones unidimensionales de las funciones base polinómicas (B-splines) y sus derivadas. Para cada dirección paramétrica y cada elemento no nulo del vector de nudos correspondiente (intervalo $[u_m, u_{m+1})$), se mapean los puntos de cuadratura de Gauss-Legendre, definidos originalmente en el elemento de referencia $[-1, 1]$. Este cambio de variable espacial se realiza mediante la transformación afín $\xi(\tilde{\xi}) = \frac{u_{m+1}-u_m}{2}\tilde{\xi} + \frac{u_{m+1}+u_m}{2}$, la cual introduce un jacobiano local $\frac{u_{m+1}-u_m}{2}$ que escala los pesos de integración. Las evaluaciones polinómicas obtenidas en estos puntos mapeados se almacenan en vectores, asignando cada dirección a un vector diferente.
3. **Fase de ensamblado:** Fase en la cual se calculan las integrales numéricas locales correspondientes a cada elemento y se ensamblan en la matriz de rigidez y el vector de carga globales. Si se ha ejecutado la fase previa de evaluaciones, las funciones base exactas (NURBS) se construyen de forma muy eficiente: se aplica el producto tensorial sobre las evaluaciones unidimensionales precalculadas (B-splines), se ponderan con sus respectivos pesos de control y se dividen por la función de peso racional (la suma total de las bases ponderadas en ese punto). Para el caso bidimensional, se ha implementado una estrategia de computación en paralelo, procesando los elementos de forma concurrente. Esto demuestra la independencia espacial de los cálculos a nivel local, lo que permite paralelizar el algoritmo de integración y reducir el tiempo total de esta fase por un factor escalar ligado al número de procesadores.

4. **Imposición de condiciones de contorno:** Se aplican los algoritmos dados en las secciones 3.4.3 y 3.5.3.
5. **Resolución del sistema final:** Se implementa el solver adaptado al tamaño de la matriz, aprovechando en la medida de lo posible su estructura simétrica y su propiedad *sparse*.
6. **Construcción y evaluación de la solución:** Partiendo de la solución del sistema, la cual nos aporta el valor de las variables de control, para evaluar la solución en un punto concreto, se suman las funciones base activas multiplicadas por sus coeficientes correspondientes.
7. **Cálculo del error en normas L^2 y H^1 :** Una vez obtenido el vector solución, se cuantifica la precisión del método evaluando el error de aproximación $e = u - u_h$, donde u es la solución exacta. El error global sobre todo el dominio Ω se calcula sumando las contribuciones locales de cada elemento Ω_e de la malla y aplicando la regla de cuadratura de Gauss-Legendre. Las expresiones numéricas quedan definidas por la siguiente doble serie:

$$\begin{aligned} \|u - u_h\|_{L^2(\Omega)} &= \left(\sum_{e=1}^{N_{el}} \int_{\Omega_e} (u - u_h)^2 d\Omega \right)^{1/2} \approx \left(\sum_{e=1}^{N_{el}} \sum_{q=1}^{N_q} (u(\mathbf{x}_q) - u_h(\mathbf{x}_q))^2 w_q \det(\mathbf{J}_q) \right)^{1/2} \\ \|u - u_h\|_{H^1(\Omega)} &= \left(\sum_{e=1}^{N_{el}} \int_{\Omega_e} ((u - u_h)^2 + |\nabla u - \nabla u_h|^2) d\Omega \right)^{1/2} \approx \\ &\approx \left(\sum_{e=1}^{N_{el}} \sum_{q=1}^{N_q} [(u(\mathbf{x}_q) - u_h(\mathbf{x}_q))^2 + |\nabla u(\mathbf{x}_q) - \nabla u_h(\mathbf{x}_q)|^2] w_q \det(\mathbf{J}_q) \right)^{1/2} \end{aligned}$$

donde N_{el} es el número total de elementos, N_q es el número de puntos de cuadratura por elemento, $|\cdot|$ denota la norma euclídea vectorial, w_q son los pesos de integración de Gauss y $\det(\mathbf{J}_q)$ representa el determinante del Jacobiano total evaluado en el punto de cuadratura, definido como el producto del Jacobiano de la transformación geométrica y el Jacobiano de la transformación afín al punto de cuadratura $\mathbf{x}_q$.

8. **Postprocesamiento (representación gráfica en Octave):** Se toman los ficheros de salida del código en C++ y mediante octave se crean las visualizaciones.

Puede encontrarse código de ejemplo en el Anexo A.7.6 para el caso 1D y 2D.

A.3. Método de los elementos finitos.

Partiendo de la formulación variacional continua, cuya solución única es $u \in H^1(\Omega)$ con $u = g$ en Γ_D , el Método de los Elementos Finitos (FEM) busca construir una aproximación numérica u_h en un subespacio de dimensión finita.

Para ello, se procede mediante dos etapas fundamentales:

- **Discretización del dominio (Ω):** El dominio se divide en una colección de subdominios cerrados y de geometría simple denominados **elementos** K_j (típicamente triángulos o polígonos regulares en 2D). Esta subdivisión conforma la malla $\mathcal{T}_h = \{K_1, K_2, \dots, K_M\}$, la cual recubre el dominio de forma que $\bar{\Omega} \approx \bigcup_{j=1}^M K_j$.
- **Discretización del espacio funcional:** Se define un espacio polinómico a trozos sobre la malla $\mathcal{T}_h$. Para gestionar correctamente la condición de Dirichlet, definimos el espacio discreto global y el subespacio de funciones de prueba nulas en la frontera:

$$\begin{aligned} V_h &= \{v_h \in H^1(\Omega) \mid v_h|_{K_j} \in \mathcal{P}_p(K_j), \forall K_j \in \mathcal{T}_h\}, \\ V_{h,0} &= \{v_h \in V_h \mid v_h = 0 \text{ en } \Gamma_D\}, \end{aligned}$$

donde $p \in \mathbb{N}$ es el grado polinomial elegido y $h = \max_{K_j \in \mathcal{T}_h} \text{diam}(K_j)$ representa el tamaño característico de la malla. La solución aproximada se busca en la variedad discreta $V_{h,g} = \{v_h \in V_h \mid v_h = g_h \text{ en } \Gamma_D\}$, siendo g_h la interpolación polinómica del dato continuo g sobre la frontera.

Definimos adicionalmente los nodos $x_i \in \overline{\Omega}$ y el conjunto nodal $\mathcal{N} = \{x_1, x_2, \dots, x_N\} \subset \overline{\Omega}$, donde N es la dimensión de V_h . La ubicación geométrica de cada nodo x_i depende de la topología del elemento y del grado p . En el FEM clásico (tipo Lagrange), cada nodo está asociado a una única función base global $\phi_i \in V_h$, construyendo la base funcional $\{\phi_1, \phi_2, \dots, \phi_N\}$ que satisface las siguientes propiedades:

- **Propiedad del delta de Kronecker:** $\phi_i(x_j) = \delta_{ij} = \begin{cases} 1 & \text{si } i = j, \\ 0 & \text{si } i \neq j. \end{cases}$
- **Soporte local estricto:** $\text{supp}(\phi_i) = \overline{\bigcup\{K_j \in \mathcal{T}_h \mid x_i \in K_j\}}$, lo que asegura que las matrices del sistema algebraico resultante sean altamente dispersas.

Tras esto, definimos la solución aproximada u_h como una combinación lineal de las funciones de la base funcional:

$$u_h(x) = \sum_{j=1}^N U_j \phi_j(x), \quad U_j \in \mathbb{R},$$

donde los coeficientes U_j representan los grados de libertad del sistema (los valores nodales). Por consiguiente, gracias a la linealidad del operador diferencial, su gradiente se expresa como:

$$\nabla u_h(x) = \nabla \left(\sum_{j=1}^N U_j \phi_j(x) \right) = \sum_{j=1}^N U_j \nabla \phi_j(x).$$

El problema discreto de Galerkin se resume entonces en encontrar la aproximación $u_h \in V_{h,g}$ tal que:

$$a(u_h, v_h) = L(v_h), \quad \forall v_h \in V_{h,0}.$$

Dado que el subespacio de prueba $V_{h,0}$ está generado por las funciones base asociadas a los nodos libres (aquellos donde la solución es incógnita y no pertenecen a Γ_D), basta con exigir que la igualdad se cumpla para cada función de prueba base $\phi_i \in V_{h,0}$. Sustituyendo la expresión de u_h y aplicando la bilinealidad de la forma $a(\cdot, \cdot)$, obtenemos para cada nodo libre i :

$$a \left(\sum_{j=1}^N U_j \phi_j, \phi_i \right) = \sum_{j=1}^N U_j a(\phi_j, \phi_i) = L(\phi_i).$$

Este desarrollo nos conduce directamente a un sistema algebraico de ecuaciones lineales de la forma $\mathbf{KU} = \mathbf{F}$, cuyos principales elementos son¹:

- **La matriz de rigidez $\mathbf{K}$:** con componentes

$$K_{ij} = a(\phi_j, \phi_i) = \int_{\Omega} \nabla \phi_j \cdot \nabla \phi_i d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} \phi_j \phi_i d\Gamma.$$

- **El vector de cargas $\mathbf{F}$:** con componentes

$$F_i = L(\phi_i) = \int_{\Omega} f \phi_i d\Omega + \int_{\Gamma_N} h \phi_i d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} \phi_i d\Gamma.$$

- **El vector de incógnitas nodales $\mathbf{U}$:** $\mathbf{U} = (U_1, U_2, \dots, U_N)^T$.

¹Si se desea conocer más sobre algoritmos de ensamblado de $\mathbf{K}$ y $\mathbf{F}$, consúltese [33], especialmente los capítulos 4 y 5.

Cabe destacar que los coeficientes U_j correspondientes a los nodos ubicados en la frontera Γ_D toman valores fijos y conocidos impuestos por la condición de contorno g_h . Algebraicamente, los términos asociados a estas columnas en la matriz $\mathbf{K}$ se multiplican por sus respectivos valores U_j y se trasladan al lado derecho del sistema, modificando el vector de cargas $\mathbf{F}$ para resolver el sistema estrictamente sobre los grados de libertad desconocidos. Adicionalmente, gracias a la propiedad del delta de Kronecker de la base funcional clásica, se verifica que $u_h(x_i) = U_i$.

Definición. Definimos la norma inducida por la forma bilineal $a(\cdot, \cdot)$, que denotaremos por $\|\cdot\|_V$:

$$\|v\|_V^2 = a(v, v) = \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} v^2 d\Gamma.$$

Lema A.3.1 (Equivalencia de normas). Las normas $\|\cdot\|_V$ y $\|\cdot\|_{H^1(\Omega)}$ son equivalentes sobre el subespacio $V_0(\Omega)$; es decir, existen constantes $C_1, C_2 > 0$ tales que $\forall v \in V_0(\Omega)$ se cumple:

$$C_1 \|v\|_{H^1(\Omega)}^2 \leq \|v\|_V^2 \leq C_2 \|v\|_{H^1(\Omega)}^2.$$

Demostración omitida. La demostración detallada puede consultarse en el Anexo A.4.4.

Teorema A.1. El error $e = u - u_h$ del Método de los Elementos Finitos converge en las normas $\|\cdot\|_{L^2(\Omega)}$ y $\|\cdot\|_{H^1(\Omega)}$, tal que:

- $\|e\|_{H^1(\Omega)}$ se reduce con orden $\mathcal{O}(h^p)$.
- $\|e\|_{L^2(\Omega)}$ se reduce con orden $\mathcal{O}(h^{p+1})$.

Siempre que la solución exacta satisfaga $u \in H^{p+1}(\Omega)$.

Demostración. Separaremos la demostración para cada una de las normas:

- **Convergencia en $\|\cdot\|_{H^1(\Omega)}$:**

Asumiendo que no hay error de aproximación en la frontera de Dirichlet ($u = u_h = g$ en Γ_D), el error pertenece al espacio vectorial de prueba, $e \in V_0(\Omega)$. Dada la coercividad y continuidad de la forma bilineal en $V_0(\Omega)^2$, nos encontramos en las condiciones del **Lema de Céa**, por tanto se cumple:

$$\|u - u_h\|_V \leq \frac{C_{cont}}{C_{coer}} \inf_{v_h \in V_{h,g}} \|u - v_h\|_V,$$

donde C_{cont} es la constante de continuidad y C_{coer} es la constante de coercividad.

Aplicando ahora la equivalencia entre normas $\|\cdot\|_V$ y $\|\cdot\|_{H^1(\Omega)}$ obtenemos que $\exists C_{Céa} \in \mathbb{R}^+$ tal que:

$$\|u - u_h\|_{H^1(\Omega)} \leq C_{Céa} \inf_{v_h \in V_{h,g}} \|u - v_h\|_{H^1(\Omega)}. \quad (\text{A.1})$$

Por último, el uso de la **propiedad de aproximación de los espacios de Sobolev** garantiza que $\exists C_{Sob} \in \mathbb{R}^+$ tal que:

$$\inf_{v_h \in V_{h,g}} \|u - v_h\|_{H^1(\Omega)} \leq C_{Sob} h^p \|u\|_{H^{p+1}(\Omega)}. \quad (\text{A.2})$$

Finalmente, definiendo $C_1 = C_{Céa} C_{Sob}$, el error en $H^1(\Omega)$ converge con orden $\mathcal{O}(h^p)$ siempre que $u \in H^{p+1}(\Omega)$:

$$\|u - u_h\|_{H^1(\Omega)} \leq C_1 h^p \|u\|_{H^{p+1}(\Omega)}.$$

²La demostración de estas propiedades puede ser encontrada en el Anexo A.4.2

- **Convergencia en $\|\cdot\|_{L^2(\Omega)}$** : Para derivar la cota del error $e = u - u_h$ en la norma $L^2(\Omega)$, recurrimos al clásico **argumento de dualidad de Aubin-Nitsche** [27]. Para ello, consideramos el problema dual auxiliar asociado:

Encontrar $w \in V_0(\Omega)$ tal que:

$$a(v, w) = (e, v)_{L^2(\Omega)} = \int_{\Omega} e v d\Omega, \quad \forall v \in V_0(\Omega).$$

Gracias a la coercividad y continuidad de la forma bilineal, y a la continuidad de la forma lineal inducida por el error en L^2 , el **Lema de Lax-Milgram** garantiza que la solución w existe y es única. Su regularidad viene dada por las condiciones de regularidad elíptica asumidas en el Anexo A.4.3, luego $\exists C_{\text{reg}} \in \mathbb{R}^+$ tal que:

$$\|w\|_{H^2(\Omega)} \leq C_{\text{reg}} \|e\|_{L^2(\Omega)}. \quad (\text{A.3})$$

Tomamos ahora como función de prueba $v = e \in V_0(\Omega)$, obteniendo así la identidad:

$$a(e, w) = (e, e)_{L^2(\Omega)} = \|e\|_{L^2(\Omega)}^2.$$

Como la solución discreta u_h satisface la **propiedad de ortogonalidad de Galerkin** sobre el espacio de prueba discreto:

$$a(u - u_h, v_h) = a(e, v_h) = 0, \quad \forall v_h \in V_{h,0}.$$

Esto nos permite restar cualquier función interpolante $w_h \in V_{h,0}$ cumpliéndose la siguiente igualdad:

$$\|e\|_{L^2(\Omega)}^2 = a(e, w) - a(e, w_h) = a(e, w - w_h).$$

Aplicando ahora la **continuidad de la forma bilineal**:

$$\|e\|_{L^2(\Omega)}^2 \leq C_{\text{cont}} \|e\|_{H^1(\Omega)} \|w - w_h\|_{H^1(\Omega)}.$$

Usamos ahora el resultado de convergencia para $\|e\|_{H^1(\Omega)}$ dado por (A.1) y (A.2):

$$\|e\|_{L^2(\Omega)}^2 \leq C_{\text{cont}} \left(C_1 h^p \|u\|_{H^{p+1}(\Omega)} \right) \|w - w_h\|_{H^1(\Omega)}.$$

Terminamos utilizando la **propiedad de aproximación de los espacios de Sobolev**. Sabiendo que $w \in H^2(\Omega)$ debido a la regularidad elíptica, tenemos:

$$\|w - w_h\|_{H^1(\Omega)} \leq C_2 h^{2-1} \|w\|_{H^2(\Omega)} = C_2 h^1 \|w\|_{H^2(\Omega)}.$$

Sustituyendo en la desigualdad:

$$\|e\|_{L^2(\Omega)}^2 \leq C_{\text{cont}} C_1 h^p \|u\|_{H^{p+1}(\Omega)} \left(C_2 h \|w\|_{H^2(\Omega)} \right) = \left(C_{\text{cont}} C_1 C_2 \|u\|_{H^{p+1}(\Omega)} \|w\|_{H^2(\Omega)} \right) h^{p+1}.$$

Aplicando finalmente la cota de regularidad (A.3), y dividiendo en ambos lados de la desigualdad por $\|e\|_{L^2(\Omega)}^3$:

$$\|e\|_{L^2(\Omega)} \leq (C_{\text{reg}} C_{\text{cont}} C_1 C_2) h^{p+1} \|u\|_{H^{p+1}(\Omega)}.$$

Definiendo la constante agregada $C_3 = C_{\text{reg}} C_{\text{cont}} C_1 C_2$, se obtiene así un error en $L^2(\Omega)$ que converge con orden $\mathcal{O}(h^{p+1})$, siempre que $u \in H^{p+1}(\Omega)$:

$$\|u - u_h\|_{L^2(\Omega)} \leq C_3 h^{p+1} \|u\|_{H^{p+1}(\Omega)}.$$

□

³Notar que si $\|e\|_{L^2(\Omega)} = 0$, la cota $\|e\|_{L^2(\Omega)} \leq C h^{p+1}$ se cumple trivialmente $\forall C, h > 0$; por tanto, podemos asumir sin pérdida de generalidad que $\|e\|_{L^2(\Omega)} > 0$.

A.4. Demostraciones omitidas

A.4.1. Paso de Solución Fuerte a Solución Variacional

Demostración. Asumiendo que $u \in C^2(\Omega) \cap C^1(\overline{\Omega})$ es solución del problema fuerte, multiplicamos la ecuación diferencial principal por una función de prueba arbitraria $v \in V_0(\Omega)$ e integramos sobre todo el dominio Ω :

$$\int_{\Omega} (-\Delta u - f)v d\Omega = 0 \implies \int_{\Omega} -v\Delta u d\Omega = \int_{\Omega} fv d\Omega.$$

Utilizamos la identidad del cálculo vectorial para la divergencia del producto de un escalar por un vector, $\nabla \cdot (v\nabla u) = \nabla v \cdot \nabla u + v\Delta u$, lo que nos permite reescribir el término del Laplaciano como:

$$\int_{\Omega} -v\Delta u d\Omega = \int_{\Omega} \nabla u \cdot \nabla v d\Omega - \int_{\Omega} \nabla \cdot (v\nabla u) d\Omega.$$

Al aplicar el **Teorema de la Divergencia (Gauss-Ostrogradsky)** al último término, la integral de volumen se transforma en una integral sobre la frontera Γ :

$$\int_{\Omega} \nabla \cdot (v\nabla u) d\Omega = \int_{\Gamma} (v\nabla u) \cdot \mathbf{n} d\Gamma = \int_{\Gamma} v(\nabla u \cdot \mathbf{n}) d\Gamma.$$

Sustituyendo esta conversión geométrica en la ecuación integral original, obtenemos la formulación variacional preliminar:

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega - \int_{\Gamma} v(\nabla u \cdot \mathbf{n}) d\Gamma = \int_{\Omega} fv d\Omega.$$

Procedemos a separar la frontera Γ en sus tres componentes disjuntas ($\Gamma_D, \Gamma_N, \Gamma_R$):

$$\int_{\Gamma} v(\nabla u \cdot \mathbf{n}) d\Gamma = \int_{\Gamma_D} v(\nabla u \cdot \mathbf{n}) d\Gamma + \int_{\Gamma_N} v(\nabla u \cdot \mathbf{n}) d\Gamma + \int_{\Gamma_R} v(\nabla u \cdot \mathbf{n}) d\Gamma.$$

Evaluamos cada integral utilizando las propiedades del espacio de prueba y las condiciones de contorno:

- Sobre Γ_D : $v = 0$ por definición del espacio $V_0(\Omega)$, luego la integral es nula.
- Sobre Γ_N : $\nabla u \cdot \mathbf{n} = h$.
- Sobre Γ_R : $\alpha u + \beta \nabla u \cdot \mathbf{n} = r \implies \nabla u \cdot \mathbf{n} = \frac{r - \alpha u}{\beta}$.

Sustituyendo estos valores en la identidad de la frontera:

$$\int_{\Gamma} v(\nabla u \cdot \mathbf{n}) d\Gamma = 0 + \int_{\Gamma_N} hv d\Gamma + \int_{\Gamma_R} \left(\frac{r - \alpha u}{\beta} \right) v d\Gamma.$$

Por último, insertamos este desarrollo en la ecuación variacional preliminar y agrupamos los términos que dependen de la incógnita u en el lado izquierdo (forma bilineal) y los datos conocidos en el lado derecho (forma lineal):

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma = \int_{\Omega} fv d\Omega + \int_{\Gamma_N} hv d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma,$$

lo cual coincide exactamente con la ecuación $a(u, v) = L(v)$. □

A.4.2. Existencia y unicidad de la solución variacional (lema de Lax-Milgram)

Teorema A.2. *La existencia y unicidad de la solución del problema variacional están garantizadas por el **Lema de Lax-Milgram**.*

La solución se descompone como $u = u_0 + u_g$, donde $u_g \in H^1(\Omega)$ es una función conocida que satisface la condición de contorno ($u_g = g$ en Γ_D), y $u_0 \in V_0(\Omega)$ es la nueva incógnita, que pertenece a un espacio de Hilbert.

Al sustituir esta descomposición en la formulación débil $a(u, v) = L(v)$ y utilizar la bilinealidad de $a(\cdot, \cdot)$, el problema se reformula como: encontrar $u_0 \in V_0(\Omega)$ tal que

$$a(u_0, v) = L(v) - a(u_g, v) \equiv \tilde{L}(v), \quad \forall v \in V_0(\Omega).$$

*En esta nueva formulación homogeneizada se pueden verificar las hipótesis del **Lema de Lax-Milgram**.*

- **Continuidad de la Forma Bilineal:** $|a(u, v)| \leq (1 + C_{Tr}^2 \left| \frac{\alpha}{\beta} \right|) \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$
- **Coercividad de la Forma Bilineal:** $a(v, v) \geq \|\nabla v\|_{L^2(\Omega)}^2 \geq \frac{1}{C_{p+1}} \|v\|_{H^1(\Omega)}^2.$
- **Continuidad de la Forma Lineal:** $|L(v)| \leq C \|v\|_{H^1(\Omega)}, \quad C \in \mathbb{R}^+.$

Demostraremos ahora cada una de las propiedades:

- **Continuidad de la Forma Bilineal:**

Demostración. aplicaremos la **desigualdad triangular**,

$$|a(u, v)| = \left| \int_{\Omega} \nabla u \cdot \nabla v \, d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} uv \, d\Gamma \right| \leq \left| \int_{\Omega} \nabla u \cdot \nabla v \, d\Omega \right| + \left| \int_{\Gamma_R} \frac{\alpha}{\beta} uv \, d\Gamma \right|.$$

Continuaremos con la **desigualdad de Cauchy-Schwartz** para el primer término,

$$\left| \int_{\Omega} \nabla u \cdot \nabla v \, d\Omega \right| \leq \int_{\Omega} |\nabla u| |\nabla v| \, d\Omega \leq \|\nabla u\|_{L^2(\Omega)} \|\nabla v\|_{L^2(\Omega)} \leq \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Buscamos ahora acotar el segundo término utilizando primero la **desigualdad de Cauchy-Schwartz**:

$$\left| \int_{\Gamma_R} \frac{\alpha}{\beta} uv \, d\Gamma \right| \leq \left| \frac{\alpha}{\beta} \right| \int_{\Gamma_R} |u| |v| \, d\Gamma \leq \left| \frac{\alpha}{\beta} \right| \|u\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)}.$$

Usamos ahora el **teorema de la traza** que nos garantiza la existencia de una constante C_{Tr} tal que,

$$\left| \frac{\alpha}{\beta} \right| \|u\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)} \leq C_{Tr}^2 \left| \frac{\alpha}{\beta} \right| \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Por lo que obtenemos:

$$|a(u, v)| \leq (1 + C_{Tr}^2 \left| \frac{\alpha}{\beta} \right|) \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

□

- **Coercividad de la forma bilineal:**

Demostración. Partimos de la definición de la forma bilineal:

$$a(v, v) = \int_{\Omega} |\nabla v|^2 dx + \int_{\Gamma_R} \frac{\alpha}{\beta} v^2 ds \geq \int_{\Omega} |\nabla v|^2 dx + c_0 \int_{\Gamma_R} v^2 ds.$$

Tomando $C_1 = \min(1, c_0) > 0$, podemos acotar inferiormente la expresión anterior:

$$a(v, v) \geq C_1 \left(\|\nabla v\|_{L^2(\Omega)}^2 + \|v\|_{L^2(\Gamma_R)}^2 \right).$$

Dado que $v \in V_0(\Omega)$ se anula sobre Γ_D , la norma L^2 de su traza sobre $\Gamma_D \cup \Gamma_R$ equivale exactamente a su norma sobre Γ_R :

$$\|v\|_{L^2(\Gamma_D \cup \Gamma_R)}^2 = \int_{\Gamma_D} v^2 ds + \int_{\Gamma_R} v^2 ds = 0 + \|v\|_{L^2(\Gamma_R)}^2.$$

Puesto que $\mathcal{L}^{n-1}(\Gamma_D \cup \Gamma_R) > 0$, la aplicación del teorema de la traza junto a la **desigualdad de Poincaré-Friedrichs generalizada** garantiza la existencia de una constante $C_F > 0$ tal que:

$$\|v\|_{H^1(\Omega)}^2 \leq C_F \left(\|\nabla v\|_{L^2(\Omega)}^2 + \|v\|_{L^2(\Gamma_D \cup \Gamma_R)}^2 \right) = C_F \left(\|\nabla v\|_{L^2(\Omega)}^2 + \|v\|_{L^2(\Gamma_R)}^2 \right).$$

Despejando el término entre paréntesis e introduciéndolo en nuestra primera acotación, concluimos la demostración:

$$a(v, v) \geq \frac{C_1}{C_F} \|v\|_{H^1(\Omega)}^2.$$

Por tanto, la forma bilineal es coerciva con constante de coercividad $\alpha_c = \frac{C_1}{C_F} > 0$. $\square$

■ Continuidad de la Forma Lineal Modificada ($\tilde{L}$):

Demostración. partimos de la definición de la forma lineal homogeneizada $\tilde{L}(v) = L(v) - a(u_g, v)$ y aplicamos la **desigualdad triangular**,

$$|\tilde{L}(v)| \leq |L(v)| + |a(u_g, v)|.$$

Procedemos primero a acotar el término original $|L(v)|$ aplicando nuevamente la **desigualdad triangular**:

$$|L(v)| \leq \left| \int_{\Omega} f v d\Omega \right| + \left| \int_{\Gamma_N} h v d\Gamma \right| + \left| \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma \right|.$$

Acotaremos ahora este sumando término a término. Para el término de volumen:

$$\left| \int_{\Omega} f v d\Omega \right| \leq \|f\|_{L^2(\Omega)} \|v\|_{L^2(\Omega)} \leq \|f\|_{L^2(\Omega)} \|v\|_{H^1(\Omega)}.$$

Para los términos de frontera aplicaremos la desigualdad de Cauchy-Schwartz junto con el **teorema de la traza**:

$$\left| \int_{\Gamma_N} h v d\Gamma \right| \leq \|h\|_{L^2(\Gamma_N)} \|v\|_{L^2(\Gamma_N)} \leq C_{Tr} \|h\|_{L^2(\Gamma_N)} \|v\|_{H^1(\Omega)}.$$

$$\left| \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma \right| \leq \left| \frac{1}{\beta} \right| \|r\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)} \leq \left| \frac{1}{\beta} \right| C_{Tr} \|r\|_{L^2(\Gamma_R)} \|v\|_{H^1(\Omega)}.$$

Sumando estas tres cotas, la parte original de la forma lineal queda acotada por:

$$|L(v)| \leq \left(\|f\|_{L^2(\Omega)} + C_{Tr} \|h\|_{L^2(\Gamma_N)} + C_{Tr} \left| \frac{1}{\beta} \right| \|r\|_{L^2(\Gamma_R)} \right) \|v\|_{H^1(\Omega)}.$$

A continuación, acotamos el término proveniente del levantamiento $|a(u_g, v)|$. Dado que en el primer apartado demostramos la **continuidad de la forma bilineal**, se cumple que:

$$|a(u_g, v)| \leq C_{cont} \|u_g\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)},$$

donde $C_{cont} = \left(1 + C_{Tr}^2 \left|\frac{\alpha}{\beta}\right|\right)$. Dado que la función de levantamiento u_g es un dato fijo y conocido dependiente de la condición de contorno g , su norma $\|u_g\|_{H^1(\Omega)}$ es una constante finita positiva.

Finalmente, uniendo ambas cotas, demostramos la continuidad de la forma lineal homogeneizada:

$$|\tilde{L}(v)| \leq \underbrace{\left(\|f\|_{L^2(\Omega)} + C_{Tr}\|h\|_{L^2(\Gamma_N)} + C_{Tr}\left|\frac{1}{\beta}\right|\|r\|_{L^2(\Gamma_R)} + C_{cont}\|u_g\|_{H^1(\Omega)}\right)}_{C_{\tilde{L}}} \|v\|_{H^1(\Omega)},$$

lo cual concluye que $|\tilde{L}(v)| \leq C_{\tilde{L}}\|v\|_{H^1(\Omega)}$, garantizando la acotación. $\square$

El cumplimiento de estas condiciones para el operador Laplaciano asegura que el problema variacional está **bien planteado**.

A.4.3. Equivalencia de la solución de la formulación variacional y fuerte

Demostración. Asumimos suficiente regularidad en los datos y en el dominio: $\Gamma \in C^2$, $f \in L^2(\Omega)$, $g \in H^{3/2}(\Gamma_D)$, $h \in H^{1/2}(\Gamma_N)$, $r \in H^{1/2}(\Gamma_R)$, lo que por teoría de regularidad elíptica garantiza que la solución débil satisface $u \in H^2(\Omega)$. Recordamos la formulación variacional:

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma = \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma, \quad \forall v \in V_0(\Omega). \quad (\text{A.4})$$

Como $u \in H^2(\Omega)$, podemos aplicar la **identidad de Green** al primer término de la forma bilineal:

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega = - \int_{\Omega} (\Delta u) v d\Omega + \int_{\Gamma} (\nabla u \cdot \mathbf{n}) v d\Gamma.$$

Sustituyendo esto en la ecuación (A.4) y agrupando todos los términos a un lado de la igualdad obtenemos:

$$- \int_{\Omega} (\Delta u) v d\Omega + \int_{\Gamma} (\nabla u \cdot \mathbf{n}) v d\Gamma + \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma - \int_{\Omega} f v d\Omega - \int_{\Gamma_N} h v d\Gamma - \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma = 0.$$

Separamos la integral sobre Γ en sus componentes disjuntas Γ_N , Γ_D y Γ_R . Como $v \in V_0(\Omega)$, se cumple que $v = 0$ en Γ_D , por lo que la integral sobre esta porción se anula:

$$\int_{\Gamma} (\nabla u \cdot \mathbf{n}) v d\Gamma = \int_{\Gamma_N} (\nabla u \cdot \mathbf{n}) v d\Gamma + \int_{\Gamma_R} (\nabla u \cdot \mathbf{n}) v d\Gamma.$$

Reagrupamos ahora los términos que se integran en las mismas regiones:

$$\int_{\Omega} (-\Delta u - f) v d\Omega + \int_{\Gamma_N} (\nabla u \cdot \mathbf{n} - h) v d\Gamma + \int_{\Gamma_R} \left(\nabla u \cdot \mathbf{n} + \frac{\alpha}{\beta} u - \frac{r}{\beta} \right) v d\Gamma = 0.$$

Para concluir, aplicamos el **Lema fundamental del cálculo de variaciones** de manera secuencial:

1. Tomamos $v \in C_c^\infty(\Omega) \subset V_0(\Omega)$, con soporte en el interior de Ω . En este caso, los términos de frontera desaparecen y queda

$$\int_{\Omega} (-\Delta u - f) v d\Omega = 0,$$

lo que implica $-\Delta u = f$ casi en todo Ω .

2. Con esto, el término de volumen se anula para cualquier v . Elegimos ahora $v \in V_0(\Omega)$ que se anule en Γ_R y sea arbitraria en Γ_N . Entonces

$$\int_{\Gamma_N} (\nabla u \cdot \mathbf{n} - h) v d\Gamma = 0,$$

de donde se obtiene $\nabla u \cdot \mathbf{n} = h$ en Γ_N .

3. De forma similar, tomando $v \in V_0(\Omega)$ arbitraria en Γ_R , se obtiene

$$\nabla u \cdot \mathbf{n} + \frac{\alpha}{\beta} u - \frac{r}{\beta} = 0,$$

equivalente a la condición de Robin $\alpha u + \beta \nabla u \cdot \mathbf{n} = r$ en Γ_R .

Finalmente, la condición de Dirichlet ($u = g$ en Γ_D) se satisface por definición de nuestro espacio de soluciones, ya que buscábamos u sujeta a dicha restricción. Por consiguiente, recuperamos la formulación fuerte:

$$\begin{cases} -\Delta u = f & \text{en } \Omega, \\ u = g & \text{en } \Gamma_D, \\ \nabla u \cdot \mathbf{n} = h & \text{en } \Gamma_N, \\ \alpha u + \beta \nabla u \cdot \mathbf{n} = r & \text{en } \Gamma_R. \end{cases}$$

□

A.4.4. Equivalencia de normas

Demostración. Demostraremos ambas cotas por separado para cualquier función $v \in V_0(\Omega)$:

- **Acotación superior** ($\|v\|_V^2 \leq C_2 \|v\|_{H^1(\Omega)}^2$): Iniciamos recordando la definición de cada norma:

- $\|v\|_V^2 = \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} v^2 d\Gamma$
- $\|v\|_{H^1(\Omega)}^2 = \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Omega} v^2 d\Omega$

Aplicaremos la **desigualdad de la traza** para acotar el término de frontera:

$$\int_{\Gamma_R} v^2 d\Gamma = \|v\|_{L^2(\Gamma_R)}^2 \leq C_{Tr}^2 \|v\|_{H^1(\Omega)}^2.$$

Usaremos también la cota trivial para el término de volumen:

$$\int_{\Omega} |\nabla v|^2 d\Omega = \|\nabla v\|_{L^2(\Omega)}^2 \leq \|v\|_{H^1(\Omega)}^2.$$

Sustituyendo ambas desigualdades en la norma de energía, obtenemos finalmente:

$$\|v\|_V^2 \leq \|v\|_{H^1(\Omega)}^2 + \frac{\alpha}{\beta} C_{Tr}^2 \|v\|_{H^1(\Omega)}^2 = \left(1 + C_{Tr}^2 \frac{\alpha}{\beta}\right) \|v\|_{H^1(\Omega)}^2.$$

Luego tomaremos $C_2 = 1 + C_{Tr}^2 \frac{\alpha}{\beta}$ (que coincide con la constante de continuidad de la forma bilineal).

- **Acotación inferior** ($C_1 \|v\|_{H^1(\Omega)}^2 \leq \|v\|_V^2$): Partimos de la definición de la norma de energía y de la condición geométrica $\frac{\alpha}{\beta} \geq 0$, lo que nos permite descartar el término de frontera para minorar la expresión:

$$\|v\|_V^2 = \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} v^2 d\Gamma \geq \int_{\Omega} |\nabla v|^2 d\Omega = \|\nabla v\|_{L^2(\Omega)}^2.$$

Dado que $v \in V_0(\Omega)$ (es decir, $v = 0$ en Γ_D) y sabiendo que $\mathcal{L}^{n-1}(\Gamma_D) > 0$, aplicamos la **desigualdad de Poincaré-Friedrichs**, la cual garantiza la existencia de una constante $C_P > 0$ tal que:

$$\int_{\Omega} v^2 d\Omega \leq C_P \int_{\Omega} |\nabla v|^2 d\Omega \implies \|v\|_{L^2(\Omega)}^2 \leq C_P \|\nabla v\|_{L^2(\Omega)}^2.$$

Sustituyendo esto en la definición de la norma de Sobolev $\|v\|_{H^1(\Omega)}^2$:

$$\|v\|_{H^1(\Omega)}^2 = \|v\|_{L^2(\Omega)}^2 + \|\nabla v\|_{L^2(\Omega)}^2 \leq (C_P + 1) \|\nabla v\|_{L^2(\Omega)}^2.$$

Como ya habíamos establecido que $\|\nabla v\|_{L^2(\Omega)}^2 \leq \|v\|_V^2$, encadenamos las desigualdades:

$$\|v\|_{H^1(\Omega)}^2 \leq (C_P + 1) \|v\|_V^2.$$

Tomando finalmente $C_1 = \frac{1}{C_P + 1}$ (que coincide con la constante de coercividad), concluimos:

$$\|v\|_V^2 \geq C_1 \|v\|_{H^1(\Omega)}^2.$$

□

A.4.5. Continuidad de la forma lineal dual

Demostración. Recordamos que la forma lineal del problema dual auxiliar se define como $L_{\text{dual}}(v) = (e, v)_{L^2(\Omega)}$.

Iniciamos aplicando la **desigualdad de Cauchy-Schwarz** en el espacio de Hilbert $L^2(\Omega)$:

$$|(e, v)_{L^2(\Omega)}| \leq \|e\|_{L^2(\Omega)} \|v\|_{L^2(\Omega)}.$$

Por definición de la norma de Sobolev, sabemos que $\|v\|_{L^2(\Omega)} \leq \|v\|_{H^1(\Omega)}$. Aplicando esta acotación trivial obtenemos:

$$|(e, v)_{L^2(\Omega)}| \leq \|e\|_{L^2(\Omega)} \|v\|_{H^1(\Omega)}.$$

Dado que el error de discretización $e = u - u_h$ es una función fija y conocida en el contexto de este problema dual, y sabiendo que $e \in L^2(\Omega)$, su norma es un valor escalar finito. Por tanto, tomando la constante $C_L = \|e\|_{L^2(\Omega)}$, concluimos la prueba de continuidad:

$$|L_{\text{dual}}(v)| \leq C_L \|v\|_{H^1(\Omega)}.$$

□

A.4.6. Propiedades de Bézier

Funciones base

Demostración. Se demostrarán las propiedades en el orden en el que han sido establecidas:

1. **No negatividad:** dado que $\binom{p}{i} \geq 0$ y $u \in [0, 1]$ implica que $u^i \geq 0$ y $(1 - u)^{p-i} \geq 0$, obtenemos la condición.

2. **Partición de la unidad:** mediante el **Teorema del Binomio de Newton**, que establece que para cualesquiera variables x, y :

$$(x + y)^p = \sum_{i=0}^p \binom{p}{i} x^i y^{p-i}.$$

Si sustituimos $x = u$ y $y = 1 - u$, obtenemos:

$$\sum_{i=0}^p B_{i,p}(u) = \sum_{i=0}^p \binom{p}{i} u^i (1-u)^{p-i} = (u + (1-u))^p = (1)^p = 1.$$

3. **Interpolación de los extremos:** evaluamos en los límites $u = 0$ y $u = 1$. Tomando $0^0 = 1$.

- Para $u = 0$:

$$B_{i,p}(0) = \binom{p}{i} 0^i (1)^{p-i}.$$

Si $i \neq 0$, el término $0^i = 0$, anulando la expresión. El único término no nulo es $i = 0$:

$$B_{0,p}(0) = \binom{p}{0} 0^0 (1)^p = 1 \cdot 1 \cdot 1 = 1.$$

- Para $u = 1$:

$$B_{i,p}(1) = \binom{p}{i} 1^i (0)^{p-i}.$$

Si $i \neq p$, el término $(0)^{p-i} = 0$ (pues $p - i > 0$). El único término no nulo es $i = p$:

$$B_{p,p}(1) = \binom{p}{p} 1^p (0)^0 = 1 \cdot 1 \cdot 1 = 1.$$

4. **Soporte global:** analizamos el intervalo abierto $u \in (0, 1)$. En este intervalo, $u > 0$ y $(1 - u) > 0$. Consecuentemente, $u^i > 0$ y $(1 - u)^{p-i} > 0$ ($\forall i \geq 0$). Luego obtenemos

$$B_{i,p}(u) > 0, \quad \forall i \geq 0, \quad u \in (0, 1).$$

□

Curvas

Demostración. Se demostrarán las propiedades en el orden en el que han sido establecidas:

1. **Interpolación en los extremos:** de las propiedades de las funciones base de Bézier obtenemos que para $u = 0$ se tiene

$$\mathbf{C}(0) = \sum_{i=0}^p B_{i,p}(0) \mathbf{P}_i = 1 \cdot \mathbf{P}_0.$$

Para el caso $u = 1$ tenemos

$$\mathbf{C}(1) = \sum_{i=0}^p B_{i,p}(1) \mathbf{P}_i = 1 \cdot \mathbf{P}_p.$$

2. **Tangencia en los extremos:** partiremos de la derivada de las funciones base,

$$\frac{d}{du} B_{i,p}(u) = p (B_{i-1,p-1}(u) - B_{i,p-1}(u)).$$

Sustituyendo esto en la derivada de la curva:

$$\mathbf{C}'(u) = \sum_{i=0}^p p (B_{i-1,p-1}(u) - B_{i,p-1}(u)) \mathbf{P}_i.$$

Reorganizando el sumatorio obtenemos

$$\mathbf{C}'(u) = p \sum_{i=0}^{p-1} B_{i,p-1}(u)(\mathbf{P}_{i+1} - \mathbf{P}_i).$$

Ahora evaluamos en los extremos usando la propiedad de interpolación de las funciones base:

- En $u = 0$:

$$\mathbf{C}'(0) = p \cdot B_{0,p-1}(0)(\mathbf{P}_1 - \mathbf{P}_0) = p(\mathbf{P}_1 - \mathbf{P}_0).$$

- En $u = 1$:

$$\mathbf{C}'(1) = p \cdot B_{p-1,p-1}(1)(\mathbf{P}_p - \mathbf{P}_{p-1}) = p(\mathbf{P}_p - \mathbf{P}_{p-1}).$$

3. **Invarianza afín:** sea $\Phi(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$ una transformación afín. Aplicamos Φ a la curva:

$$\Phi(\mathbf{C}(u)) = \mathbf{A} \left(\sum_{i=0}^p B_{i,p}(u) \mathbf{P}_i \right) + \mathbf{b}.$$

Por la linealidad de la multiplicación matricial $\mathbf{A}$:

$$\Phi(\mathbf{C}(u)) = \sum_{i=0}^p B_{i,p}(u)(\mathbf{A}\mathbf{P}_i) + \mathbf{b}.$$

Aquí utilizamos la propiedad de **Partición de la Unidad** para multiplicar el vector $\mathbf{b}$ por 1 e introducirlo en el sumatorio:

$$\Phi(\mathbf{C}(u)) = \sum_{i=0}^p B_{i,p}(u)(\mathbf{A}\mathbf{P}_i) + \mathbf{b} = \sum_{i=0}^p B_{i,p}(u)(\mathbf{A}\mathbf{P}_i) + \sum_{i=0}^p B_{i,p}(u)\mathbf{b} = \sum_{i=0}^p B_{i,p}(u)(\mathbf{A}\mathbf{P}_i + \mathbf{b}).$$

Reconocemos que $\mathbf{A}\mathbf{P}_i + \mathbf{b} = \Phi(\mathbf{P}_i)$, por lo tanto:

$$\Phi(\mathbf{C}(u)) = \sum_{i=0}^p B_{i,p}(u)\Phi(\mathbf{P}_i).$$

□

A.4.7. Propiedades de los B-Splines

Funciones base

Demostración. Se demostrarán las propiedades en el orden en el que han sido establecidas:

1. **Soporte local:** procederemos mediante inducción sobre p :

- **Caso base** ($p = 0$): tenemos, $N_{i,0}(u) = 1$ si $u \in [u_i, u_{i+1})$ por definición.
- **Paso de inducción sobre p :** partiendo de la definición mediante el algoritmo de Cox-de Boor:

$$N_{i,p}(u) = \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u). \quad (\text{A.5})$$

Asumimos que $N_{i,p-1}(u)$ tiene soporte en $[u_i, u_{i+p})$ y $N_{i+1,p-1}(u)$ tiene soporte en $[u_{i+1}, u_{i+p+1})$, luego $N_{i,p}(u)$ tiene soporte $[u_i, u_{i+p+1})$.

2. **No negatividad:** procederemos mediante inducción sobre p :

- **Caso base** ($p = 0$): tenemos por definición que $N_i = 0$ o 1 , luego $N_i \geq 0$.

- **Paso de inducción sobre p :** asumimos cierta la hipótesis para $p-1$, tomando $u \in [u_i, u_{i+p+1})$ y aplicando el algoritmo A.5, la demostración se reduce a probar que:

- $\frac{u-u_i}{u_{i+p}-u_i} \geq 0$, trivial pues por como esta definido el vector de nudos, $u_{i+p} \geq u_i$ y como $u \in [u_i, u_{i+p+1})$, $u \geq u_i$
- $\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}} \geq 0$, de manera análoga tenemos $u_{i+p+1} \geq u_{i+1}$ y $u_{i+p+1} \geq u$.

3. Número de bases activas: procederemos mediante inducción sobre p :

- **Caso base ($p=0$):** Sea un intervalo no degenerado $[u_i, u_{i+1})$ con $u_i < u_{i+1}$. Por definición de la función indicatriz, si $u \in [u_i, u_{i+1})$, $N_{i,0}(u) = 1$ y todas las demás $N_{j,0}(u) = 0$ para $j \neq i$. Luego, existe exactamente una $(0+1)$ función base no nula, cuyo índice es $j=i$, cumpliendo la propiedad.
- **Paso de inducción:** Asumimos como hipótesis de inducción que para un grado $p-1$, en el intervalo $[u_i, u_{i+1})$ existen exactamente p funciones base no nulas, correspondientes a los índices $k \in \{i-(p-1), \dots, i\}$.

Por la fórmula de recurrencia de Cox-de Boor, la función $N_{j,p}(u)$ es una combinación lineal de $N_{j,p-1}(u)$ y $N_{j+1,p-1}(u)$. Dado que los coeficientes de la combinación y las propias bases son no negativos en el dominio válido, $N_{j,p}(u)$ será estrictamente positiva en $[u_i, u_{i+1})$ si y solo si al menos una de las dos funciones de grado inferior es no nula en dicho intervalo. Evaluando los índices:

- $N_{j,p-1}(u) \neq 0 \implies j \in \{i-p+1, \dots, i\}$
- $N_{j+1,p-1}(u) \neq 0 \implies j+1 \in \{i-p+1, \dots, i\} \implies j \in \{i-p, \dots, i-1\}$

La unión de ambos conjuntos de índices determina las funciones base de grado p que son activas en el intervalo:

$$j \in \{i-p+1, \dots, i\} \cup \{i-p, \dots, i-1\} = \{i-p, i-p+1, \dots, i\}$$

Este conjunto contiene exactamente $p+1$ índices distintos. Por lo tanto, en cualquier intervalo paramétrico no nulo existen exactamente $p+1$ funciones base activas, completando así la inducción.

4. Partición de la unidad:

- **Caso base ($p=0$):** como hemos probado en el apartado anterior en $p=0$ por cada intervalo $[u_i, u_{i+1})$ hay una sola función base no nula, luego $\forall u \in [u_0, u_m), \exists j \in R^+$ tal que $u \in [u_j, u_{j+1})$, luego,

$$\sum_{i=0}^n N_{i,0}(u) = N_{j,0}(u) = 1.$$

- **Paso de inducción sobre p :** asumimos ahora que se cumple $\sum_{i=0}^n N_{i,p}(u) = 1$, empezamos entonces con el algoritmo de Cox-de Boor A.5 talque,

$$\begin{aligned} \sum_{i=0}^n N_{i,p}(u) &= \sum_{i=0}^n \left[\frac{u-u_i}{u_{i+p}-u_i} N_{i,p-1}(u) + \frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}} N_{i+1,p-1}(u) \right] = \\ &= \sum_{i=0}^n \left[\frac{u-u_i}{u_{i+p}-u_i} N_{i,p-1}(u) \right] + \sum_{i=0}^n \left[\frac{u_{i+p+1}-u}{u_{i+p+1}-u_{i+1}} N_{i+1,p-1}(u) \right] = \\ &= \sum_{i=0}^n \left[\frac{u-u_i}{u_{i+p}-u_i} N_{i,p-1}(u) \right] + \sum_{i=1}^{n+1} \left[\frac{u_{i+p}-u}{u_{i+p}-u_i} N_{i,p-1}(u) \right]. \end{aligned}$$

Tomando que el soporte de $N_{i,p-1}(u)$ es $[u_i, u_i + p]$ y que U es abierto, luego $u_0 = u_1 = \dots = u_p$, por lo que $N_{0,p-1}(u)$, al tener soporte degenerado, es siempre nula, y $N_{n+1,p-1}$ es nula $\forall u \in [u_0, u_n]$ nos queda:

$$\begin{aligned} \sum_{i=0}^n N_{i,p}(u) &= \sum_{i=0}^n \left[\frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) \right] + \left[\frac{u_{i+p} - u}{u_{i+p} - u_i} N_{i,p-1}(u) \right] = \\ &= \sum_{i=0}^n \left[\frac{u - u_i}{u_{i+p} - u_i} + \frac{u_{i+p} - u}{u_{i+p} - u_i} \right] N_{i,p-1}(u) = \sum_{i=0}^n N_{i,p-1}(u) = 1. \end{aligned}$$

□

Curvas B-Spline

Demostración. Se demostrarán las propiedades en el orden en el que han sido establecidas:

1. **Invarianza bajo transformaciones afines:** sea una transformación afín $T(\mathbf{P}) = \mathbf{M}\mathbf{P} + \mathbf{t}$, aplicamos esta a la curva obteniendo:

$$\begin{aligned} T(\mathbf{C}(u)) &= \mathbf{M}\mathbf{C}(u) + \mathbf{t} = \mathbf{M} \left(\sum_{i=0}^n N_{i,p}(u) \mathbf{P}_i \right) + \mathbf{t} = \\ &= \sum_{i=0}^n \mathbf{M}(N_{i,p}(u) \mathbf{P}_i) + 1 \cdot \mathbf{t} = \sum_{i=0}^n N_{i,p}(u) (\mathbf{M}\mathbf{P}_i) + \mathbf{t} \left(\sum_{i=0}^n N_{i,p}(u) \right) = \\ &= \sum_{i=0}^n N_{i,p}(u) (\mathbf{M}\mathbf{P}_i + \mathbf{t}). \end{aligned}$$

Siendo esta última serie la curva generada por los puntos de control transformados mediante la transformación afín.

2. **Continuidad controlable:** Suponemos que u_j tiene multiplicidad $k \leq p$, aplicamos ahora la definición de la derivada de la curva:

$$\mathbf{C}'(u) = \frac{d}{du} \left(\sum_{i=0}^n N_{i,p}(u) \mathbf{P}_i \right) = \sum_{i=0}^n N'_{i,p}(u) \mathbf{P}_i = \sum_{i=0}^n \left[\frac{p}{u_{i+p} - u_i} N_{i,p-1}(u) - \frac{p}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u) \right] \mathbf{P}_i.$$

Reordenamos ahora los índices del término de la derecha de la suma, aplicando las mismas propiedades que en el apartado anterior, obteniendo así:

$$\mathbf{C}'(u) = \sum_{i=0}^{n-1} N_{i,p-1}(u) \underbrace{\left[p \frac{\mathbf{P}_{i+1} - \mathbf{P}_i}{u_{i+p+1} - u_{i+1}} \right]}_{\mathbf{P}_i^{(1)}} = \sum_{i=0}^{n-1} N_{i,p-1}(u) \mathbf{P}_i^{(1)}.$$

Así, de manera recursiva definimos la derivada r -ésima de la curva:

$$\mathbf{C}^{(r)}(u) = \sum_i N_{i,p-r}(u) \mathbf{P}_i^{(r)}, \quad \left(\mathbf{P}_i^{(r)} = p \frac{\mathbf{P}_{i+1}^{(r-1)} - \mathbf{P}_i^{(r-1)}}{u_{i+p+1} - u_{i+1}} \right)$$

Obteniendo así que para $r = p - k$, si $u = u_j$, $N_{j-1,k} = 1$, luego:

$$\begin{aligned} \lim_{u \rightarrow u_j^-} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^-} N_{i,k}(u) = \mathbf{P}_{j-1}^{(r)} \cdot \lim_{u \rightarrow u_j^-} N_{j-1,k}(u) = \mathbf{P}_{j-1}^{(r)}. \\ \lim_{u \rightarrow u_j^+} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^+} N_{i,k}(u) = \mathbf{P}_{j-1}^{(r)} \cdot \lim_{u \rightarrow u_j^+} N_{j-1,k}(u) = \mathbf{P}_{j-1}^{(r)}. \end{aligned}$$

Para $r = p - k + 1$, tenemos que:

$$\begin{aligned} \lim_{u \rightarrow u_j^-} N_{j-1,k-1}(u) &= 1, \quad \lim_{u \rightarrow u_j^+} N_{j-1,k-1}(u) = 0. \\ \lim_{u \rightarrow u_j^-} N_{j,k-1}(u) &= 0, \quad \lim_{u \rightarrow u_j^+} N_{j,k-1}(u) = 1. \end{aligned}$$

Por lo que en este caso:

$$\begin{aligned} \lim_{u \rightarrow u_j^-} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^-} N_{i,k-1}(u) = \mathbf{P}_{j-1}^{(r)} \cdot \lim_{u \rightarrow u_j^-} N_{j-1,k-1}(u) = \mathbf{P}_{j-1}^{(r)}. \\ \lim_{u \rightarrow u_j^+} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^+} N_{i,k-1}(u) = \mathbf{P}_j^{(r)} \cdot \lim_{u \rightarrow u_j^+} N_{j,k-1}(u) = \mathbf{P}_j^{(r)}. \end{aligned}$$

Luego como la continuidad de $\mathbf{C}^{(r)}(u)$ depende de los puntos de control, $\mathbf{C}(u) \notin L^{p-k+1}(\Omega)$.

□

A.4.8. Derivada de las funciones base NURBS

Demostración. Definimos inicialmente al cociente como $W(u) = \sum_{i=0}^n N_{i,p}(u)w_i$.

Obteniendo ahora

$$\begin{aligned} (R_i^p)'(u) &= \frac{w_i N'_{i,p}(u)W(u) - w_i N_{i,p}(u)W'(u)}{(W(u))^2} = \frac{w_i N'_{i,p}(u)W(u)}{(W(u))^2} - \frac{w_i N_{i,p}(u)W'(u)}{(W(u))^2} \\ &= \frac{w_i N'_{i,p}(u)}{W(u)} - \left(\frac{w_i N_{i,p}(u)}{W(u)} \right) \frac{W'(u)}{W(u)} = \frac{w_i N'_{i,p}(u)}{W(u)} - R_i^p(u) \frac{W'(u)}{W(u)} = \frac{w_i N'_{i,p}(u) - R_i^p(u)W'(u)}{W(u)}. \end{aligned}$$

Sustituyendo de vuelta $W(u)$ y su derivada obtenemos la fórmula

$$(R_i^p)'(u) = \frac{w_i N'_{i,p}(u) - R_i^p(u) \sum_{j=0}^n N'_{j,p}(u)w_j}{\sum_{j=0}^n N_{j,p}(u)w_j}.$$

□

A.4.9. Derivada de la curva NURBS

Demostración. Definimos la curva como cociente de funciones $\mathbf{C}(u) = \frac{\mathbf{A}(u)}{W(u)}$, donde

- **Numerador:** $\mathbf{A}(u) = \sum_{i=0}^n N_{i,p}(u)w_i \mathbf{P}_i$.
- **Denominador:** $W(u) = \sum_{i=0}^n N_{i,p}(u)w_i$.

Aplicando ahora la derivada del cociente la parametrización analítica de su primera derivada es:

$$\mathbf{C}'(u) = \frac{\mathbf{A}'(u)W(u) - \mathbf{A}(u)W'(u)}{(W(u))^2} = \frac{\mathbf{A}'(u)}{W(u)} - \left(\frac{\mathbf{A}(u)}{W(u)} \right) \frac{W'(u)}{W(u)}.$$

Manipulando ahora este resultado obtenemos

$$\mathbf{C}'(u) = \frac{\mathbf{A}'(u) - W'(u)\mathbf{C}(u)}{W(u)} = \frac{\sum_{i=0}^n N'_{i,p}(u)w_i \mathbf{P}_i - \mathbf{C}(u) \sum_{i=0}^n N'_{i,p}(u)w_i}{W(u)} = \frac{\sum_{i=0}^n N'_{i,p}(u)w_i (\mathbf{P}_i - \mathbf{C}(u))}{\sum_{i=0}^n N_{i,p}(u)w_i}.$$

□

A.4.10. Comportamiento direccional de las superficies NURBS

Demostración. Tomaremos una dirección arbitraria, v en este caso, y fijaremos $u = u_0$, la demostración es análoga para la dirección u .

$$S(u_0, v) = \frac{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u_0) M_{j,q}(v) \mathbf{w}_{i,j} \mathbf{P}_{i,j}}{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u_0) M_{j,q}(v) \mathbf{w}_{i,j}}.$$

Ahora reagrupamos los valores que dependen de v y los que no:

$$S(u_0, v) = \frac{\sum_{j=0}^m M_{j,q}(v) (\sum_{i=0}^n N_{i,p}(u_0) \mathbf{w}_{i,j} \mathbf{P}_{i,j})}{\sum_{j=0}^m M_{j,q}(v) (\sum_{i=0}^n N_{i,p}(u_0) \mathbf{w}_{i,j})}.$$

Definimos ahora los siguientes elementos:

$$\mathbf{w}_j^* = \sum_{i=0}^n N_{i,p}(u_0) \mathbf{w}_{i,j}, \quad \mathbf{Q}_j = \frac{\sum_{i=0}^n N_{i,p}(u_0) \mathbf{P}_{i,j} \mathbf{w}_{i,j}}{\mathbf{w}_j^*}.$$

Obteniendo al final una definición

$$C(v) = S(u_0, v) = \frac{\sum_{j=0}^m M_{j,q}(v) \mathbf{w}_j^* \mathbf{Q}_j}{\sum_{j=0}^m M_{j,q}(v) \mathbf{w}_j^*}.$$

Obteniendo así la curva de NURBS asociada a la dirección v en el punto u_0 . □

A.4.11. Igualdad de las nuevas funciones base bajo refinamiento

Demostración. Procederemos mediante inducción sobre p :

- **Caso base** ($p = 0$): Sea ξ con valor dentro del intervalo de nudos $[u_k, u_{k+1})$, separamos los casos:
 - $i < k$: En este caso $N_{i,0}(u)$ comparte soporte con $N_{i,0}^*(u)$.
 - $i > k$: En este caso $N_{i,0}(u)$ comparte soporte con $N_{i+1,0}^*(u)$.
 - $i = k$: En este caso $N_{k,0}(u)$ tiene como soporte a la unión de ambos soportes de $N_{k,0}^*(u)$ y $N_{k+1,0}^*(u)$, obteniendo la igualdad:

$$\begin{aligned} N_{k,0}(u) &= 1_{[u_k, u_{k+1})}(u) = 1_{[u_k, \xi)}(u) + 1_{[\xi, u_{k+1})}(u) = \\ &= N_{k,0}^*(u) + N_{k+1,0}^*(u) = \alpha_k N_{k,0}^*(u) + (1 - \alpha_{k+1}) N_{k+1,0}^*(u). \end{aligned}$$

- **Paso de inducción sobre p** : iniciamos asumiendo cierta la hipótesis para $p - 1$ y sustituimos en el algoritmo de Cox-de Boor,

$$\begin{aligned} N_{i,p}(u) &= \frac{u - u_i}{u_{i+p} - u_i} [\alpha_{i,p-1} N_{i,p-1}^*(u) + (1 - \alpha_{i+1,p-1}) N_{i+1,p-1}^*(u)] + \\ &+ \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} [\alpha_{i+1,p-1} N_{i+1,p-1}^*(u) + (1 - \alpha_{i+2,p-1}) N_{i+2,p-1}^*(u)]. \end{aligned}$$

Donde

$$\alpha_{i,p} = \begin{cases} 1 & \text{si } i \leq k - p, \\ \frac{\xi - u_i}{u_{i+p} - u_i} & \text{si } k - p < i \leq k, \\ 0 & \text{si } i > k. \end{cases}$$

Reorganizamos ahora la igualdad agrupando los términos asociados a cada función base distinta:

- Término asociado a $N_{i,p-1}^*$:

$$\frac{u - u_i}{u_{i+p} - u_i} \alpha_{i,p-1} = \alpha_{i,p} \frac{u - u_i^*}{u_{i+p}^* - u_i^*}.$$

- Término asociado a $N_{i+1,p-1}^*$:

$$\frac{u - u_i}{u_{i+p} - u_i} (1 - \alpha_{i+1,p-1}) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} \alpha_{i+1,p-1} = \alpha_{i,p} \frac{u_{i+p+1}^* - u}{u_{i+p+1}^* - u_{i+1}^*} + (1 - \alpha_{i+1,p}) \frac{u - u_{i+1}^*}{u_{i+1+p}^* - u_{i+1}^*}.$$

- Término asociado a $N_{i+2,p-1}^*$:

$$\frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} (1 - \alpha_{i+2,p-1}) = (1 - \alpha_{i+1,p}) \frac{u_{i+p+2}^* - u}{u_{i+p+2}^* - u_{i+2}^*}.$$

Sustituyendo ahora en la igualdad original obtenemos:

$$\begin{aligned} N_{i,p}(u) &= \alpha_{i,p} \underbrace{\left[\frac{u - u_i^*}{u_{i+p}^* - u_i^*} N_{i,p-1}^* + \frac{u_{i+p+1}^* - u}{u_{i+p+1}^* - u_{i+1}^*} N_{i+1,p-1}^* \right]}_{N_{i,p}^*(u)} + \\ &+ (1 - \alpha_{i+1,p}) \underbrace{\left[\frac{u - u_{i+1}^*}{u_{i+1+p}^* - u_{i+1}^*} N_{i+1,p-1}^* + \frac{u_{i+p+2}^* - u}{u_{i+p+2}^* - u_{i+2}^*} N_{i+2,p-1}^* \right]}_{N_{i+1,p}^*(u)}. \end{aligned}$$

□

A.4.12. Equivalencia de curvas bajo refinamiento

Demostración. Separaremos entre índices donde α_j toma valores 0 o 1 y en los que no son tan triviales

- Para los índices $j \leq k - p$, $\alpha_j = 1$, $\mathbf{Q}_j^w = (1)\mathbf{P}_j^w + (1-1)\mathbf{P}_{j-1}^w = \mathbf{P}_j^w$, $N_{j,p} = N_{j,p}^*$.
- Ahora para los índices $j > k$, $\alpha_j = 0$, $\mathbf{Q}_j^w = (0)\mathbf{P}_j^w + (1-0)\mathbf{P}_{j-1}^w = \mathbf{P}_{j-1}^w$, $N_{j-1,p} = N_{j,p}^*$.
- Por último para los índices $i \in \{k - p, k - p + 1, \dots, k\}$, obtenemos la definición de la influencia de estos índices mediante:

$$\mathbf{C}^w(u) = \sum_{i=k-p}^k N_{i,p}(u) \mathbf{P}_i^w.$$

descomponemos ahora las funciones base en las nuevas funciones base:

$$\mathbf{C}^w(u) = \sum_{i=k-p}^k [\alpha_i N_{i,p}^*(u) + (1 - \alpha_{i+1}) N_{i+1,p}^*(u)] \mathbf{P}_i^w.$$

Separamos en la suma en sus dos términos principales y reindexamos:

$$\begin{aligned} &\sum_{i=k-p}^k \alpha_i N_{i,p}^*(u) \mathbf{P}_i^w. \\ &\sum_{i=k-p}^k (1 - \alpha_{i+1}) N_{i+1,p}^*(u) \mathbf{P}_i^w = \sum_{j=k-p+1}^{k+1} (1 - \alpha_j) N_{j,p}^*(u) \mathbf{P}_{j-1}^w. \end{aligned}$$

Como hemos mostrado anteriormente la equivalencia de los términos de la serie para $j = k - p$, $j = k + 1$ y ambos términos aparecen con su función base y punto de control correspondiente una única vez, podemos eliminarlos de la suma y centrarnos en los índices de los que desconocemos si se cumple.

$$\begin{aligned} \mathbf{C}^w(u) &= \sum_{j=k-p+1}^k [\alpha_j N_{j,p}^*(u) \mathbf{P}_j^w + (1 - \alpha_j) N_{j,p}^*(u) \mathbf{P}_{j-1}^w] = \\ &= \sum_{j=k-p+1}^k N_{j,p}^*(u) [\alpha_j \mathbf{P}_j^w + (1 - \alpha_j) \mathbf{P}_{j-1}^w] = \sum_{j=k-p+1}^k N_{j,p}^*(u) \mathbf{Q}_j^w. \end{aligned}$$

□

A.4.13. Propositiones elevación del grado polinomial

■ Proposición 2.12:

Demostración. Aplicamos una reparametrización del segmento de la curva NURBS, tomamos el vector de nudos local $U_{loc} = \{\underbrace{u_k, \dots, u_k}_{p+1}, \underbrace{u_{k+1}, \dots, u_{k+1}}_{p+1}\}$ y su dominio paramétrico original $u \in [u_k, u_{k+1}]$. Sea ahora $\hat{u} = \frac{u - u_k}{u_{k+1} - u_k} \in [0, 1]$, su nuevo vector de nudos reparametrizado es $\hat{U}_{loc} = \{\underbrace{0, \dots, 0}_{p+1}, \underbrace{1, \dots, 1}_{p+1}\}$. Tenemos ahora que:

$$N_{i,p}(\hat{u}) = \frac{\hat{u} - \hat{u}_i}{\hat{u}_{i+p} - \hat{u}_i} N_{i,p-1}(\hat{u}) + \frac{\hat{u}_{i+p+1} - \hat{u}}{\hat{u}_{i+p+1} - \hat{u}_{i+1}} N_{i+1,p-1}(\hat{u})$$

Separamos ahora en casos distintos:

- **Términos internos** ($0 < i < p$): $\frac{\hat{u} - \hat{u}_i}{\hat{u}_{i+p} - \hat{u}_i} = \frac{\hat{u}}{1}, \frac{\hat{u}_{i+p+1} - \hat{u}}{\hat{u}_{i+p+1} - \hat{u}_{i+1}} = \frac{1 - \hat{u}}{1}.$
- **Término extremo por la izquierda** ($i = 0$): $\frac{\hat{u} - \hat{u}_i}{\hat{u}_{i+p} - \hat{u}_i} = \frac{\hat{u} - \hat{u}_i}{0} = 0^4, \frac{\hat{u}_{i+p+1} - \hat{u}}{\hat{u}_{i+p+1} - \hat{u}_{i+1}} = \frac{1 - \hat{u}}{1}.$
- **Término extremo por la derecha** ($i = p$): $\frac{\hat{u} - \hat{u}_i}{\hat{u}_{i+p} - \hat{u}_i} = \frac{\hat{u}}{1}, \frac{\hat{u}_{i+p+1} - \hat{u}}{\hat{u}_{i+p+1} - \hat{u}_{i+1}} = \frac{\hat{u}_{i+p+1} - \hat{u}}{0} = 0^4.$

Procedemos ahora por inducción sobre p :

- **Caso base** ($p = 1$): $\hat{U} = \{0, 0, 1, 1\}$
 - $N_{0,1}(\hat{u}) = (1 - \hat{u})N_{1,0}(\hat{u}) = 1 - \hat{u}.$
 - $N_{1,1}(\hat{u}) = \hat{u}N_{1,0}(\hat{u}) = \hat{u}.$

Ambas coinciden con $B_{0,1}(\hat{u})$ y $B_{1,1}(\hat{u})$ respectivamente.

- **Paso de inducción sobre p :** asumimos cierta la equivalencia para $p - 1$, luego:

$$N_{i,p}(\hat{u}) = \hat{u} \left[\binom{p-1}{i} \hat{u}^i (1 - \hat{u})^{p-1-i} \right] + (1 - \hat{u}) \left[\binom{p-1}{i-1} \hat{u}^{i-1} (1 - \hat{u})^{p-i} \right],$$

factorizamos el término común $\hat{u}^i (1 - \hat{u})^{p-i}$:

$$N_{i,p}(\hat{u}) = \left[\binom{p-1}{i} + \binom{p-1}{i-1} \right] \hat{u}^i (1 - \hat{u})^{p-i},$$

por último aplicamos ahora la **identidad de Pascal**:

$$N_{i,p}(\hat{u}) = \binom{p}{i} \hat{u}^i (1 - \hat{u})^{p-i} = B_{i,p}(\hat{u}).$$

⁴Por la convención de Cox-de Boor, derivada del hecho de que el soporte paramétrico de la función base evaluada es un intervalo de longitud nula, el sumando asociado se anula.

□

■ **Proposición 2.13:**

Demostración. Tomamos la igualdad $1 = u + (1 - u)$ y multiplicamos la curva original por esta:

$$\sum_{i=0}^p \mathbf{P}_i B_{i,p}(u) \cdot [u + (1 - u)] = \sum_{i=0}^p \mathbf{P}_i u B_{i,p}(u) + \sum_{i=0}^p \mathbf{P}_i (1 - u) B_{i,p}(u),$$

aplicamos ahora una transformación de elevación del grado a los polinomios de Bernstein de cada serie:

- **Primer término:** $u B_{i,p}(u) = \binom{p}{i} u^{i+1} (1 - u)^{p-i}$, multiplicamos y dividimos por $\frac{p+1}{i+1}$:

$$\begin{aligned} u B_{i,p}(u) &= \frac{i+1}{p+1} \frac{(p+1)!}{(i+1)!(p-i)!} u^{i+1} (1-u)^{(p+1)-(i+1)} = \\ &= \frac{i+1}{p+1} \binom{p+1}{i+1} u^{i+1} (1-u)^{(p+1)-(i+1)} = \frac{i+1}{p+1} B_{i+1,p+1}(u). \end{aligned}$$

- **Segundo término:** $(1 - u) B_{i,p}(u) = \binom{p}{i} u^i (1 - u)^{p+1-i}$, multiplicamos y dividimos por $\frac{p+1}{p+1-i}$:

$$\begin{aligned} (1 - u) B_{i,p}(u) &= \frac{p+1-i}{p+1} \frac{(p+1)!}{i!(p+1-i)!} u^i (1-u)^{p+1-i} = \\ &= \frac{p+1-i}{p+1} \binom{p+1}{i} u^i (1-u)^{p+1-i} = \frac{p+1-i}{p+1} B_{i,p+1}(u). \end{aligned}$$

Reensamblamos ahora las series y obtenemos:

$$\sum_{i=0}^p \mathbf{P}_i \frac{i+1}{p+1} B_{i+1,p+1}(u) + \sum_{i=0}^p \mathbf{P}_i \frac{p+1-i}{p+1} B_{i,p+1}(u),$$

realizamos el cambio de índice $j = i + 1$ a la primera serie:

$$\sum_{j=1}^{p+1} \mathbf{P}_{j-1} \frac{j}{p+1} B_{j,p+1}(u) + \sum_{i=0}^p \mathbf{P}_i \left(1 - \frac{i}{p+1}\right) B_{i,p+1}(u),$$

notamos que para $j = 0$, $\frac{j}{p+1} = 0$ y para $i = p + 1$, $\left(1 - \frac{i}{p+1}\right) = 0$, luego reescribimos de vuelta j como i y obtenemos la serie:

$$\sum_{i=0}^{p+1} \left[\frac{i}{p+1} \mathbf{P}_{i-1} + \left(1 - \frac{i}{p+1}\right) \mathbf{P}_i \right] B_{i,p+1}(u).$$

□

■ **Proposición 2.14:**

Demostración. Sea la curva al inicio de la iteración v definida como $\sum_{i=0}^{\hat{n}+1} \mathbf{P}_i^{(v-1)} \bar{N}_{i,\hat{p}}(u)$. Sea $\bar{u}$ el nudo que queremos eliminar en esta iteración, existe entonces $u_k \in U$ tal que $\bar{u} = u_k$ y $\bar{u} \in [u_k, u_{k+1})$.

Definimos ahora los escalares asociados a la iteración v :

$$\alpha_i^{(v)} = \begin{cases} 1 & \text{si } i \leq k - \hat{p} \\ \frac{\bar{u} - u_i^{(v)}}{u_{i+\hat{p}}^{(v)} - u_i^{(v)}} & \text{si } k - \hat{p} + 1 \leq i \leq k \\ 0 & \text{si } i \geq k + 1 \end{cases}$$

Notemos las evaluaciones en las fronteras de los índices:

- **Para $i = 0$:** asumiendo una curva válida (donde $k \geq \hat{p}$), se cumple que $0 \leq k - \hat{p}$, luego $\alpha_0^{(v)} = 1$.
- **Para $i = \hat{n} + 1$:** se cumple que $\hat{n} + 1 \geq k + 1$, luego $\alpha_{\hat{n}+1}^{(v)} = 0$.

Tenemos ahora la curva resultante de aplicar la iteración del algoritmo, utilizando la identidad de reducción de la base B-spline⁵:

$$\sum_{i=0}^{\hat{n}} \mathbf{P}_i^{(v)} \left[\alpha_i^{(v)} \bar{N}_{i,\hat{p}}(u) + (1 - \alpha_{i+1}^{(v)}) \bar{N}_{i+1,\hat{p}}(u) \right]$$

Reagrupando en dos series distintas obtenemos:

$$\sum_{i=0}^{\hat{n}} \mathbf{P}_i^{(v)} \alpha_i^{(v)} \bar{N}_{i,\hat{p}}(u) + \sum_{i=0}^{\hat{n}} \mathbf{P}_i^{(v)} (1 - \alpha_{i+1}^{(v)}) \bar{N}_{i+1,\hat{p}}(u)$$

Sustituimos ahora en la segunda serie un cambio de índice $j = i + 1$:

$$\sum_{i=0}^{\hat{n}} \mathbf{P}_i^{(v)} \alpha_i^{(v)} \bar{N}_{i,\hat{p}}(u) + \sum_{j=1}^{\hat{n}+1} \mathbf{P}_{j-1}^{(v)} (1 - \alpha_j^{(v)}) \bar{N}_{j,\hat{p}}(u)$$

Aplicando que $\alpha_{\hat{n}+1}^{(v)} = 0$ en la primera serie, que $1 - \alpha_0^{(v)} = 0$, y reescribiendo el índice mudo j como i en la segunda serie, obtenemos:

$$\sum_{i=0}^{\hat{n}+1} \mathbf{P}_i^{(v)} \alpha_i^{(v)} \bar{N}_{i,\hat{p}}(u) + \sum_{i=0}^{\hat{n}+1} \mathbf{P}_{i-1}^{(v)} (1 - \alpha_i^{(v)}) \bar{N}_{i,\hat{p}}(u) = \sum_{i=0}^{\hat{n}+1} \left[\alpha_i^{(v)} \mathbf{P}_i^{(v)} + (1 - \alpha_i^{(v)}) \mathbf{P}_{i-1}^{(v)} \right] \bar{N}_{i,\hat{p}}(u)$$

Dado que el algoritmo de reensamblaje iterativo descrito en el paso 5 resuelve de manera analítica los barridos izquierdo y derecho, impone la siguiente condición de consistencia para cada índice i :

$$\mathbf{P}_i^{(v-1)} = \alpha_i^{(v)} \mathbf{P}_i^{(v)} + (1 - \alpha_i^{(v)}) \mathbf{P}_{i-1}^{(v)}$$

Sustituyendo esta igualdad en nuestra serie agrupada, la expresión se colapsa de forma exacta a:

$$\sum_{i=0}^{\hat{n}+1} \mathbf{P}_i^{(v-1)} \bar{N}_{i,\hat{p}}(u)$$

La cual es, por definición, la evaluación paramétrica de la curva original al inicio de la iteración. $\square$

A.4.14. Propiedades del sistema

Demostración. Se demostrarán las propiedades en el orden en el que han sido establecidas:

1. **Dispersión:** la matriz de rigidez se define como $\mathbf{K}_{i,j} = a(\phi_j, \phi_i)$. La propiedad fundamental de las funciones base NURBS es que tienen **soporte local**.

Para que $\mathbf{K}_{ij} \neq 0$, la intersección de los soportes debe ser no nula:

$$\text{supp}(\phi_i) \cap \text{supp}(\phi_j) \neq \emptyset.$$

Para un cierto grado polinomial p tenemos que cada función base NURBS tiene soporte en, a lo sumo, $p + 1$ intervalos no nulos del vector de nudos, por lo que obtenemos las siguientes propiedades:

⁵ $N_{i,\hat{p}}(u) = \alpha_i^{(v)} \bar{N}_{i,\hat{p}}(u) + (1 - \alpha_{i+1}^{(v)}) \bar{N}_{i+1,\hat{p}}(u)$

- En 1D, una función base comparte soporte con a lo sumo p funciones base con índices menores, p con índices mayores y consigo misma. Total: $2p + 1$.
- En $\mathbb{R}^d$, si tomamos p como el grado polinomial direccional mayor en cada dirección, mediante estructura de producto tensorial, el número de interacciones es a lo sumo el producto de las interacciones en cada dimensión si tuvieran grado máximo: $(2p + 1)^d$.

Por tanto, el número de entradas no nulas por fila de $\mathbf{K}$ está acotado por $C_p = (2p + 1)^d$. Dado que el número total de incógnitas N_b crece con el refinamiento ($N_{el} \rightarrow \infty$), mientras que C_p permanece constante (fijo p o tomando p como el grado polinomial máximo), se cumple:

$$\lim_{N_{el} \rightarrow \infty} \frac{(2p + 1)^d}{N_b} = \lim_{N_b \rightarrow \infty} \frac{C_p}{N_b} = 0.$$

2. **Simetría:** La entrada de la matriz está dada por $\mathbf{K}_{ij} = a(\phi_j, \phi_i)$. Si el operador diferencial del problema $\mathcal{L}$ es autoadjunto (por ejemplo, $\mathcal{L}u = -\Delta u$), la forma bilineal asociada $a(\cdot, \cdot)$ hereda la propiedad de simetría, tal que:

$$a(u, v) = a(v, u) \quad \forall u, v \in V.$$

Aplicando esto a las funciones de base:

$$\mathbf{K}_{ij} = a(\phi_j, \phi_i) = a(\phi_i, \phi_j) = \mathbf{K}_{ji}.$$

Por consiguiente, la matriz $\mathbf{K}$ es simétrica.

3. **Condicionamiento:** El número de condición espectral se define mediante el cociente de los autovalores extremos $\kappa(\mathbf{K}) = \frac{\lambda_{\max}}{\lambda_{\min}}$. Podemos estimarlos mediante el cociente de Rayleigh asociado a la forma bilineal $a(\cdot, \cdot)$ y la norma euclídea de los coeficientes $\mathbf{c}$:

$$\lambda = \frac{\mathbf{c}^T \mathbf{K} \mathbf{c}}{\mathbf{c}^T \mathbf{c}} = \frac{a(u_h, u_h)}{\|\mathbf{c}\|^2}, \quad \text{donde } u_h = \sum_i c_i \phi_i.$$

Es necesario relacionar la norma L^2 de la solución discreta u_h con la norma euclídea de sus variables de control $\mathbf{c}$. Gracias a la **Estabilidad de Riesz** de la base B-spline y al mapeo geométrico desde el dominio paramétrico, se satisface la equivalencia de normas:

$$C_1 h^d \|\mathbf{c}\|^2 \leq \|u_h\|_{L^2}^2 \leq C_2 h^d \|\mathbf{c}\|^2.$$

Basándonos en esta relación y en las propiedades del espacio de aproximación:

- **Cota superior ($\lambda_{\max}$):** Se utiliza la **desigualdad inversa global** típica en espacios de elementos finitos y splines:

$$\|\nabla u_h\|_{L^2(\Omega)} \leq C_{\text{inv}} h^{-1} \|u_h\|_{L^2(\Omega)}.$$

Elevando al cuadrado para la forma bilineal obtenemos

$$a(u_h, u_h) = \|\nabla u_h\|_{L^2}^2 \lesssim h^{-2} \|u_h\|_{L^2}^2.$$

Finalmente, dividiendo por $\|\mathbf{c}\|^2$:

$$\lambda_{\max} \approx \sup_{\mathbf{c}} \frac{a(u_h, u_h)}{\|\mathbf{c}\|^2} \lesssim h^{d-2}.$$

- **Autovalor mínimo ($\lambda_{\min}$):** Usando la *coercividad* del operador ($a(u_h, u_h) \geq \alpha \|u_h\|_{L^2}^2$):

$$\lambda_{\min} \approx \inf_{u_h} \frac{a(u_h, u_h)}{\|\mathbf{c}\|^2} \gtrsim \frac{\|u_h\|_{L^2}^2}{h^{-d} \|u_h\|_{L^2}^2} \sim O(h^d).$$

Finalmente, el número de condición escala como el cociente de ambas cotas:

$$\kappa(\mathbf{K}) = \frac{\lambda_{\max}}{\lambda_{\min}} \approx C \frac{h^{d-2}}{h^d} = O(h^{-2}).$$

□

A.4.15. Convergencia en IGA

Demostración. Iniciaremos con definiciones y teoremas auxiliares necesarios para la demostración:

Definición (Funcional lineal y espacio dual). Sea V un espacio vectorial sobre el cuerpo de los números reales (en el contexto analítico de este trabajo, $V = H^{p+1}(\hat{\Omega})$). Un funcional es una aplicación $\lambda : V \rightarrow \mathbb{R}$ que asigna a cada elemento del espacio de funciones un único valor escalar.

Se dice que el funcional es lineal si satisface el principio de superposición:

$$\lambda(\alpha u + \beta v) = \alpha \lambda(u) + \beta \lambda(v), \quad \forall u, v \in V, \forall \alpha, \beta \in \mathbb{R}.$$

Al conjunto de todos los funcionales lineales y continuos definidos sobre V se le denomina espacio dual, denotado como V' . En el marco de la cuasi-interpolación isogeométrica, la familia de funcionales $\{\lambda_i\}_{i=1}^n \subset (H^{p+1}(\hat{\Omega}))'$ representa los operadores que extraen la información local de la función continua $\hat{u}$ para determinar los coeficientes discretos (grados de libertad) en la base NURBS.

Definición (Operador de cuasi-interpolación paramétrico). Sea $\hat{V}_h = \text{span}\{\hat{R}_i\}_{i=1}^n$ el espacio discreto generado por las funciones base racionales (NURBS) de grado p sobre el dominio paramétrico $\hat{\Omega}$. Un operador de cuasi-interpolación $\hat{\Pi}_h : H^{p+1}(\hat{\Omega}) \rightarrow \hat{V}_h$ se define como una proyección de la forma:

$$\hat{\Pi}_h \hat{u}(\xi) = \sum_{i=1}^n \lambda_i(\hat{u}) \hat{R}_i(\xi),$$

donde $\{\lambda_i\}_{i=1}^n$ es una familia de funcionales lineales (funcionales duales) que satisfacen las siguientes condiciones analíticas:

1. **Localidad:** La evaluación del funcional $\lambda_i(\hat{u})$ depende únicamente de los valores de $\hat{u}$ en el soporte de la función base correspondiente, es decir, $\text{supp}(\lambda_i) \subseteq \text{supp}(\hat{R}_i)$.
2. **Reproducción polinómica:** El operador es una proyección exacta para el espacio de polinomios de grado hasta p , garantizando que $\hat{\Pi}_h q = q \quad \forall q \in \mathbb{P}_p(\hat{\Omega})$.

Definición (Proyector isogeométrico). Sea $\mathbf{G} : \hat{\Omega} \rightarrow \Omega$ el mapeo geométrico, el cual asumimos como un difeomorfismo con regularidad suficiente. Dada una función exacta $u \in H^{p+1}(\Omega)$, su transformación al dominio paramétrico viene dada por $\hat{u} = u \circ \mathbf{G} \in H^{p+1}(\hat{\Omega})$. Empleando el operador de cuasi-interpolación paramétrico $\hat{\Pi}_h$ previamente definido, el proyector isogeométrico sobre el espacio físico, $\Pi_h : H^{p+1}(\Omega) \rightarrow V_h$, se define mediante la transformación:

$$\Pi_h u = (\hat{\Pi}_h \hat{u}) \circ \mathbf{G}^{-1}.$$

Teorema A.3 (Propiedad de aproximación isogeométrica [5]). *Bajo las hipótesis de que la malla paramétrica no es degenerada y el Jacobiano satisface $\det(\mathbf{J}) > 0$ en $\hat{\Omega}$, el error de interpolación para una función $u \in H^{k+1}(\Omega)$ con $k \leq p$ está acotado por:*

$$\|u - \Pi_h u\|_{H^m(\Omega)} \leq C_{(int,m)} h^{k+1-m} \|u\|_{H^{k+1}(\Omega)}, \quad \text{para } 0 \leq m \leq k+1.$$

donde el entero m representa el orden de diferenciación de la norma de Sobolev en la que se evalúa el error. La constante $C_{(int,m)}$ es independiente de h , pero depende del grado polinomial p , de la regularidad de los vectores de nudos y de las normas infinito del mapeo geométrico, $\|\nabla \mathbf{G}\|_{L^\infty(\hat{\Omega})}$ y $\|\nabla \mathbf{G}^{-1}\|_{L^\infty(\Omega)}$.

Definimos el error de aproximación de Galerkin como $e = u - u_h$.

1. Convergencia en la norma $H^1(\Omega)$:

Partiendo del análisis funcional desarrollado en el Anexo A.1 para la norma H^1 , el error de aproximación puede acotarse mediante el Lema de Céa, lo que conduce a la estimación:

$$\|u - u_h\|_{H^1(\Omega)} \leq C_{Céa} \inf_{v_h \in V_{h,g}} \|u - v_h\|_{H^1(\Omega)}.$$

En el contexto de IGA, este ínfimo se evalúa tomando como función discreta el **proyector isogeométrico** $v_h = \Pi_h u \in V_{h,g}$. Bajo esta elección, y aplicando el Teorema A.3 con $m = 1$ y $k = p$, se obtiene:

$$\inf_{v_h \in V_{h,g}} \|u - v_h\|_{H^1(\Omega)} \leq \|u - \Pi_h u\|_{H^1(\Omega)} \leq C_{(int,1)} h^p \|u\|_{H^{p+1}(\Omega)}.$$

Combinando ambas estimaciones y agrupando las constantes en $C_1 = C_{C\acute{e}a} C_{(int,1)}$, se llega a la cota final:

$$\|u - u_h\|_{H^1(\Omega)} \leq C_1 h^p \|u\|_{H^{p+1}(\Omega)},$$

que confirma una tasa de convergencia óptima de orden $\mathcal{O}(h^p)$ para la solución isogeométrica.

2. Convergencia en la norma $L^2(\Omega)$:

Tal y como se muestra en la demostración general del Anexo A.1 para la norma L^2 , basada en el argumento de dualidad de Aubin-Nitsche [27], la ortogonalidad de Galerkin y la regularidad elíptica del problema dual permiten acotar el error en la norma $L^2(\Omega)$ mediante la siguiente expresión:

$$\|e\|_{L^2(\Omega)}^2 \leq C_{\text{cont}} \|e\|_{H^1(\Omega)} \|w - w_h\|_{H^1(\Omega)}, \quad \forall w_h \in V_{h,0}.$$

Para adaptar este resultado al contexto del Análisis Isogeométrico, se toma como aproximación discreta el proyector isogeométrico de la solución dual. Dado que $w \in V_0(\Omega)$, su proyector conserva la condición nula en la frontera de Dirichlet, es decir, $w_h = \Pi_h w \in V_{h,0}$. Como la regularidad del problema dual garantiza que $w \in H^2(\Omega)$, puede aplicarse el Teorema A.3 de aproximación en espacios NURBS (con $m = 1$ y $k = 1$), obteniendo:

$$\|w - \Pi_h w\|_{H^1(\Omega)} \leq C_{(int,1)} h \|w\|_{H^2(\Omega)}.$$

Sustituyendo esta estimación en la desigualdad anterior y utilizando la regularidad elíptica del problema dual, que implica $\|w\|_{H^2(\Omega)} \leq C_{\text{reg}} \|e\|_{L^2(\Omega)}$, se obtiene:

$$\|e\|_{L^2(\Omega)}^2 \leq C_{\text{cont}} \|e\|_{H^1(\Omega)} \left(C_{(int,1)} C_{\text{reg}} h \|e\|_{L^2(\Omega)} \right).$$

A continuación, se introduce la cota de convergencia en norma $H^1(\Omega)$ previamente demostrada, $\|e\|_{H^1(\Omega)} \leq C_1 h^p \|u\|_{H^{p+1}(\Omega)}$, lo que conduce a:

$$\|e\|_{L^2(\Omega)}^2 \leq C_{\text{cont}} \left(C_1 h^p \|u\|_{H^{p+1}(\Omega)} \right) \left(C_{(int,1)} C_{\text{reg}} h \|e\|_{L^2(\Omega)} \right).$$

Agrupando las constantes y dividiendo ambos lados por $\|e\|_{L^2(\Omega)}$ (suponiendo $\|e\|_{L^2(\Omega)} > 0$), se llega a:

$$\|e\|_{L^2(\Omega)} \leq (C_{\text{cont}} C_{\text{reg}} C_{(int,1)} C_1) h^{p+1} \|u\|_{H^{p+1}(\Omega)}.$$

Definiendo la constante agrupada como $C_2 = C_{\text{cont}} C_{\text{reg}} C_{(int,1)} C_1$, se obtiene finalmente la tasa óptima $\mathcal{O}(h^{p+1})$ para IGA:

$$\|u - u_h\|_{L^2(\Omega)} \leq C_2 h^{p+1} \|u\|_{H^{p+1}(\Omega)}.$$

□

A.5. Base teórica de los algoritmos de condiciones de contorno

Partimos de la partición de la frontera $\Gamma = \overline{\Gamma_D} \cup \overline{\Gamma_R} \cup \overline{\Gamma_N}$, con $\Gamma_D \cap \Gamma_R = \Gamma_D \cap \Gamma_N = \Gamma_R \cap \Gamma_N = \emptyset$.

Partiendo de la formulación variacional:

$$a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} u v d\Gamma, \quad (\text{A.6})$$

$$L(v) = \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma. \quad (\text{A.7})$$

Sea el sistema lineal discreto resultante del ensamblado $\mathbf{Ku} = \mathbf{F}$, donde $\mathbf{u} \in \mathbb{R}^n$.

A.5.1. Tipo Dirichlet

Sea el espacio de funciones test en la condición de Dirichlet $V_{h,0} = \{v \in V_h \mid v|_{\Gamma_D} = 0\}$.

Sin embargo tenemos que para cada función de forma ϕ_k cuya intersección de su soporte con Γ_D es no nula, no son necesariamente nulas en la frontera. Esto implica que estas funciones no pertenecen al espacio test admisible.

Como consecuencia las filas k dadas por $\mathbf{K}_{k,i} = a(\phi_i, \phi_k)$, y su correspondiente elemento del vector de carga $L(\phi_k) = F_k$, no son válidas para el sistema, por ende estas filas las "perdemos", obteniendo ahora un sistema singular no resoluble, al que podemos añadir ahora las igualdades en los puntos de la frontera en las que solo una función base es no nula, de manera que obtenemos un número equivalente de ecuaciones a las que hemos eliminado de la forma $\mathbf{K}_{k,i} = 0, \forall i \neq k, \mathbf{K}_{k,k} = 1, F_k = g(x_k)$.

A.5.2. Tipo Neumann

Observamos en A.7, que las condiciones de contorno de tipo Neumann solo suman la integral referente a esta a la forma lineal, obteniendo así un caso trivial en el que solo debemos sumarle al valor del vector de carga su integral correspondiente a cada elemento en la frontera⁶.

A.5.3. Tipo Robin o mixtas

Análogamente a las condiciones de Neumann, en este caso observamos en A.6 y A.7 afectarán a la matriz de rigidez y al vector de carga, debiendo sumar las integrales correspondientes en ambos términos del sistema.

A.6. Explicación y parámetros de las figuras

A.6.1. Figura 1.1

En referencia a la figura 1.1.

- Subfigura 1.1a: la subfigura ilustra las funciones base clásicas del Método de los Elementos Finitos (FEM) para un grado polinomial lineal ($p = 1$) sobre un dominio discretizado en 6 elementos. En ella se puede apreciar visualmente la continuidad estrictamente C^0 en las fronteras inter-elementales, una limitación geométrica inherente a esta formulación.
- Subfigura 1.1b: la subfigura presenta las funciones base cuadráticas ($p = 2$) de FEM clásico para una discretización de 3 elementos (7 funciones base en total). A diferencia de las bases B-Spline, se observa claramente que estas funciones pueden tomar valores negativos. Asimismo, se evidencia que el incremento del grado polinomial no mejora la regularidad global: aunque las bases son suaves en los nodos interiores de cada elemento, la continuidad decae abruptamente a estrictamente C^0 en los 4 nodos que conforman las fronteras de los elementos.
- Subfigura 1.1c: la subfigura muestra las funciones base NURBS cuadráticas ($p = 2$), definidas sobre el vector de nudos $U = \{0, 0, 0, 0, 4, 0, 8, 1, 1, 1\}$ y con vector de pesos $w = \{1, 4, 5, 1, 1\}$, se puede apreciar en esta la suavidad global que obtienen las funciones base de este tipo.

A.6.2. Figura 2.5

En referencia a la figura 2.4.

- Subfigura 2.4a:
 - Grados polinomiales: $p = 2$ (dirección u) y $q = 3$ (dirección v).

⁶Notar que para el caso 1D esta integral se resume a la evaluación en un punto.

- Vectores de nudos: $U = \{0, 0, 0, 0,5, 0,5, 1, 1, 1\}$ y $V = \{0, 0, 0, 0, 1, 1, 1, 1\}$.
- Red de control **P**: matriz de 5×4 vértices tridimensionales:

$$\mathbf{P} = \begin{pmatrix} (0,1,1) & (0,2,2) & (0,3,2) & (0,4,1) \\ (1,1,1) & (1,2,2) & (1,3,2) & (1,4,1) \\ (2,1,1) & (2,2,3) & (2,3,3) & (2,4,1) \\ (3,1,1) & (3,2,3) & (3,3,3) & (3,4,1) \\ (4,1,1) & (4,2,2) & (4,3,2) & (4,4,1) \end{pmatrix}$$

- Matriz de pesos w :

$$w = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 2 & 1 \\ 1 & 3 & 2 & 1 \\ 1 & 2 & 2 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

■ Subfigura 2.4b:

- Grados polinomiales: $p = 2$ (dirección u) y $q = 1$ (dirección v).
- Vectores de nudos: $U = \{0, 0, 0, 0,25, 0,25, 0,5, 0,5, 0,75, 0,75, 1, 1, 1\}$ y $V = \{0, 0, 0,25, 0,5, 0,75, 1, 1\}$.
- Red de control **P**: matriz de 9×5 vértices tridimensionales que definen la directriz circular y las generatrices lineales:

$$\mathbf{P} = \begin{pmatrix} (1,0,0) & (1,0,2) & (0,5,0,2) & (0,5,0,0) & (1,0,0) \\ (1,1,0) & (1,1,2) & (0,5,0,5,2) & (0,5,0,5,0) & (1,1,0) \\ (0,1,0) & (0,1,2) & (0,0,5,2) & (0,0,5,0) & (0,1,0) \\ (-1,1,0) & (-1,1,2) & (-0,5,0,5,2) & (-0,5,0,5,0) & (-1,1,0) \\ (-1,0,0) & (-1,0,2) & (-0,5,0,2) & (-0,5,0,0) & (-1,0,0) \\ (-1,-1,0) & (-1,-1,2) & (-0,5,-0,5,2) & (-0,5,-0,5,0) & (-1,-1,0) \\ (0,-1,0) & (0,-1,2) & (0,-0,5,2) & (0,-0,5,0) & (0,-1,0) \\ (1,-1,0) & (1,-1,2) & (0,5,-0,5,2) & (0,5,-0,5,0) & (1,-1,0) \\ (1,0,0) & (1,0,2) & (0,5,0,2) & (0,5,0,0) & (1,0,0) \end{pmatrix}$$

- Matriz de pesos w , esencial para la representación cónica exacta de la curvatura:

$$w = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 1 & 1 & 1 & 1 & 1 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 1 & 1 & 1 & 1 & 1 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 1 & 1 & 1 & 1 & 1 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

- Subfigura 2.5: Tras aplicar el algoritmo de refinamiento mediante inserción de nudos en ambas direcciones paramétricas, el espacio de la superficie se enriquece sin alterar su geométrica. Los parámetros resultantes son los siguientes:

- Grados polinomiales (invariantes): $p = 2$ y $q = 1$.

- Vectores de nudos refinados (U' y V'):
 $U' = \{0, 0, 0, 0, 1, 0, 2, 0, 25, 0, 25, 0, 3, 0, 4, 0, 5, 0, 5, 0, 6, 0, 7, 0, 75, 0, 75, 0, 8, 0, 9, 1, 1, 1\}$
 $V' = \{0, 0, 0, 1, 0, 2, 0, 25, 0, 3, 0, 4, 0, 5, 0, 5, 0, 6, 0, 7, 0, 75, 0, 8, 0, 9, 1, 1\}$
- Red de control refinada $\mathbf{P}'$: al proyectar los puntos de control ponderados $\mathbf{P}^w$ desde $\mathbb{R}^4$ a $\mathbb{R}^3$ (dividiendo las tres primeras coordenadas entre el peso w), se obtiene una nueva malla tensorial de 17×14 vértices. Por concisión, se representa su estructura límite:

$$\mathbf{P}' = \begin{pmatrix} (1,0,0) & (1,0,0,8) & \dots & (1,0,0) \\ (0,883,0,283,0) & (0,883,0,283,0,706) & \dots & (0,883,0,283,0) \\ \vdots & \vdots & \ddots & \vdots \\ (1,0,0) & (1,0,0,8) & \dots & (1,0,0) \end{pmatrix}_{17 \times 14}$$

- Matriz de pesos refinada w' , extraída directamente de la cuarta coordenada del espacio proyectivo:

$$w' = \begin{pmatrix} 1 & 1 & \dots & 1 \\ 0,883 & 0,883 & \dots & 0,883 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \dots & 1 \end{pmatrix}_{17 \times 14}$$

A.7. Código omitido

A.7.1. FEM

Código de las funciones base FEM

```

1  clear; clc; close all;
2
3  % --- 1. PARÁMETROS CONFIGURABLES ---
4  L = 1;
5  num_elementos = 6;
6  p = 1;
7
8  % --- 2. GENERACIÓN DE LA MALLA ---
9  num_nodos = num_elementos * p + 1;
10 x_nodos = linspace(0, L, num_nodos);
11 colores = lines(num_nodos);
12
13 % Configuración de la figura
14 hold on; grid on;
15 axis([-0.1 L+0.1 -0.2 1.2]);
16
17 % --- 3. BUCLE DE FUNCIONES BASE ---
18 for i = 1:num_nodos
19     valores_nodales = zeros(1, num_nodos);
20     valores_nodales(i) = 1;
21     x_plot_total = [];
22     y_plot_total = [];
23     for e = 1:num_elementos
24         idx_ini = (e-1)*p + 1;
25         idx_fin = e*p + 1;
26         indices_locales = idx_ini:idx_fin;
27         x_elem = x_nodos(indices_locales);
28         y_elem = valores_nodales(indices_locales);
29         coeficientes = polyfit(x_elem, y_elem, p);
30         xx_elem = linspace(x_elem(1), x_elem(end), 50);
31         yy_elem = polyval(coeficientes, xx_elem);
32         x_plot_total = [x_plot_total, xx_elem];
33         y_plot_total = [y_plot_total, yy_elem];
34     end
35     % --- DIBUJO ---
36     plot(x_plot_total, y_plot_total, 'Color', colores(i,:), 'LineWidth', 2);
37     %try
38     %fill(x_plot_total, y_plot_total, colores(i,:), 'FaceAlpha', 0.1, 'EdgeColor', 'none');
39     %catch
40     %end
41     plot(x_nodos(i), 1, 'o', 'MarkerFaceColor', colores(i,:), 'MarkerEdgeColor', 'k');
42     if p == 1
43         offset = 0.05;
44     else
45         offset = 0.1 + 0.05*mod(i,2);
46     end
47     text(x_nodos(i), 1 + offset, sprintf('\phi_{%d}', i), ...
48         'Color', colores(i,:), 'HorizontalAlignment', 'center', 'FontWeight', 'bold');
49 end
50 for e = 0:num_elementos
51     x_borde = e * (L/num_elementos);
52     line([x_borde x_borde], [-0.2 1.2], 'Color', [0.7 0.7 0.7], 'LineStyle', '--');
53 end
54
55 hold off;
56
57 nombre_archivo = sprintf('bases_fem_p%d.svg', p);
58 set(gcf, 'PaperPositionMode', 'auto');
59 print(gcf, nombre_archivo, '-dsvg');

```

A.7.2. Figuras de Bézier

Código de las curvas racionalizadas de Bernstein

```

1  m = 100;
2  u = 0:1/m:1;
3
4
5  P = [0, 1, 2, 3;
6        0, 5, 0, 4.5;
7        0, 1, 2, 4];
8
9  W = [1.0, 10.0, 7.0, 1.0];
10
11 [aux, n_puntos] = size(P);
12
13 sol = zeros(aux, m+1);
14
15 for k = 1:m+1
16
17     Bold = zeros(n_puntos, 1);
18     Bold(1) = 1;
19
20     for j = 2:n_puntos
21         Bnew = zeros(n_puntos, 1);
22         for i = 1:j-1
23             % B(i,j) = (1-u)*B(i,j-1) + u*B(i-1,j-1)
24             Bnew(i) = Bnew(i) + (1 - u(k)) * Bold(i);
25             Bnew(i+1) = Bnew(i+1) + u(k) * Bold(i);
26         endfor
27         Bold = Bnew;
28     endfor
29
30     %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
31     % RACIONALIZACIÓN
32     %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
33
34     Numerator = zeros(aux, 1);
35     Denominator = 0;
36
37     for j = 1:n_puntos
38
39         WeightedBasis = Bold(j) * W(j);
40         Numerator = Numerator + WeightedBasis * P(:, j);
41         Denominator = Denominator + WeightedBasis;
42     endfor
43
44     sol(:,k) = Numerator / Denominator;
45
46 endfor
47
48 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
49 % PLOTED Y AJUSTE DE EJES DINÁMICOS
50 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
51
52 min_coords = min([P, sol], [], 2);
53 max_coords = max([P, sol], [], 2);
54
55 padding = 0.2;
56
57 min_X = min_coords(1) - padding;
58 max_X = max_coords(1) + padding;
59 min_Y = min_coords(2) - padding;
60 max_Y = max_coords(2) + padding;
61 min_Z = min_coords(3) - padding;
62 max_Z = max_coords(3) + padding;
63
64 figure
65 hold on
66
67 % Plotear el polígono de control (rojo discontinuo con círculos)
68 plot3(P(1,:), P(2,:), P(3,:), 'r--o', 'LineWidth', 1, 'MarkerSize', 5);
69
70 % Plotear la curva racional (azul sólido)
71 plot3(sol(1,:), sol(2,:), sol(3,:), 'b', 'LineWidth', 2);
72
73 % ** Establecer límites dinámicos **
74 xlim([min_X, max_X]);
75 ylim([min_Y, max_Y]);
76 zlim([min_Z, max_Z]);
77
78 xlabel('X');
79 ylabel('Y');
80 zlabel('Z');
81 grid on
82 view(3);
83 hold off
84
85 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
86 % EXPORTACIÓN DE LA FIGURA
87 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
88 print('curva_bezier_racional.svg', '-dsvg');

```

Código de las bases de Bernstein

```

1 function plot_bernstein_bases(p)
2
3     m = 100;
4     u = linspace(0, 1, m+1);
5
6     figure;
7     hold on;
8     grid on;
9
10    colors = hsv(p + 1);
11
12    for i = 0:p
13
14        binom_coeff = nchoosek(p, i);
15        poly_term = u.^i .* (1 - u).^(p - i);
16        B_ip = binom_coeff * poly_term;
17        plot(u, B_ip, 'LineWidth', 2, 'Color', colors(i + 1, :), ...
18             'DisplayName', ['B_{', num2str(i), ', ', num2str(p), '} (u)']);
19    end
20
21    xlim([0, 1]);
22    ylim([0, 1.1]);
23    hold off;
24
25    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
26    % EXPORTACIÓN EN FORMATO SVG
27    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
28
29    filename = ['bernstein_bases_p', num2str(p), '.svg'];
30    print(filename, '-dsvg');
31
32    disp(['Figura guardada como: ', filename]);
33
34 end

```

A.7.3. Código fundamental para el funcionamiento en C++

Clase iMatrix

```

1  /*****
2  *AUTHOR: Carlos Palomera Oliva
3  *DATE: MARCH 2025
4  *****/
5  #include <iostream>
6  #include <stdexcept>
7  #include <algorithm>
8  #include "C:\Librerias\eigen-3.4.0\Eigen\Dense" //windows version
9  // #include "/home/carlo/Librerias/eigen-3.4.0/Eigen/Dense" //linux version
10 // #include "/home/carlo/Librerias/eigen-3.4.0/Eigen/Sparse" //linux version
11 #include <iomanip>
12 #include <vector>
13 #include <cmath>
14 #include <numeric>
15 #include <type_traits>
16
17 #ifndef IMATRIX_H
18 #define IMATRIX_H
19
20 // This file is part of the qBLinAlg linear algebra library.
21
22 /*
23 MIT License
24 Copyright (c) 2023 Michael Bennett
25
26 Permission is hereby granted, free of charge, to any person obtaining a copy of this software
27 and associated documentation files (the "Software"), to deal in the Software without restriction,
28 including without limitation the rights to use, copy, modify, merge, publish, distribute,
29 sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is
30 furnished to do so, subject to the following conditions:
31
32 The above copyright notice and this permission notice shall be included in all copies or
33 substantial portions of the Software.
34
35 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING
36 BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
37 NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,
38 DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
39 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
40 */
41
42
43
44
45 template <class T>
46 class iMatrix{
47 public:
48     // Define the various constructors.
49     iMatrix();
50     iMatrix(int nRows, int nCols);
51     iMatrix(int nRows, int nCols, const T *inputData);
52     iMatrix(int nRows, int nCols, const Eigen::VectorXd &inputData);
53     iMatrix(const iMatrix<T> &inputMatrix);

```

```

54
55 // And the destructor.
56 ~iMatrix();
57
58 // Configuration methods.
59 bool Resize(int numRows, int numCols);
60 void SetToIdentity();
61
62 // Element access methods.
63 T GetElement(int row, int col) const;
64 std::vector<T> GetRow(int row) const;
65 std::vector<T> GetCol(int col) const;
66 bool SetElement(int row, int col, T elementValue);
67 bool SetCol(int col, std::vector<T> elements);
68 bool SetRow(int row, std::vector<T> elements);
69 int GetNumRows() const;
70 int GetNumCols() const;
71 iMatrix<T> GetSubMat(int rowIni, int rowEnd, int colIni, int colEnd) const;
72
73
74 // Overload the assignment operator.
75 iMatrix<T> operator= (const iMatrix<T> &rhs);
76
77
78 // Write
79 inline T& operator()(int i, int j) noexcept {
80     return m_matrixData[i * m_nCols + j];
81 };
82
83 // Read
84 inline const T& operator()(int i, int j) const noexcept {
85     return m_matrixData[i * m_nCols + j];
86 };
87
88
89 iMatrix<T> invert() const { //Uses LU factorization with partial pivoting
90     if (m_nRows != m_nCols) {
91         throw std::runtime_error("Matrix must be square for inversion.");
92     }
93
94     int n = m_nRows;
95     iMatrix<T> A(*this);
96     std::vector<int> piv(n);
97     for (int i = 0; i < n; ++i) piv[i] = i;
98
99     for (int k = 0; k < n; ++k) {
100         T maxVal = std::abs(A(k, k));
101         int pivotRow = k;
102         for (int i = k + 1; i < n; ++i) {
103             T val = std::abs(A(i, k));
104             if (val > maxVal) {
105                 maxVal = val;
106                 pivotRow = i;
107             }
108         }
109         if (maxVal == T(0)) {
110             throw std::runtime_error("Matrix is singular.");
111         }
112
113         if (pivotRow != k) {
114             for (int j = 0; j < n; ++j) {
115                 std::swap(A(k, j), A(pivotRow, j));
116             }
117             std::swap(piv[k], piv[pivotRow]);
118         }
119
120         for (int i = k + 1; i < n; ++i) {
121             A(i, k) /= A(k, k);
122             T factor = A(i, k);
123             for (int j = k + 1; j < n; ++j) {
124                 A(i, j) -= factor * A(k, j);
125             }
126         }
127     }
128
129     iMatrix<T> inv(n, n);
130     for (int col = 0; col < n; ++col) {
131         std::vector<T> b(n, T(0));
132         b[piv[col]] = T(1);
133
134         for (int i = 0; i < n; ++i) {
135             for (int j = 0; j < i; ++j) {
136                 b[i] -= A(i, j) * b[j];
137             }
138         }
139
140         for (int i = n - 1; i >= 0; --i) {
141             for (int j = i + 1; j < n; ++j) {
142                 b[i] -= A(i, j) * b[j];
143             }
144             b[i] /= A(i, i);
145         }
146
147         for (int i = 0; i < n; ++i) {
148             inv(i, col) = b[i];
149         }
150     }
151
152     return inv;
153 }
154
155 // Overload +, - and * operators (friends).
156 template <class U> friend iMatrix<U> operator+ (const iMatrix<U>& lhs, const iMatrix<U>& rhs);
157 template <class U> friend iMatrix<U> operator+ (const U& lhs, const iMatrix<U>& rhs);

```

```

157     template <class U> friend iMatrix<U> operator+ (const iMatrix<U>& lhs, const U& rhs);
158
159     template <class U> friend iMatrix<U> operator- (const iMatrix<U>& lhs, const iMatrix<U>& rhs);
160     template <class U> friend iMatrix<U> operator- (const U& lhs, const iMatrix<U>& rhs);
161     template <class U> friend iMatrix<U> operator- (const iMatrix<U>& lhs, const U& rhs);
162
163     template <class U> friend iMatrix<U> operator* (const iMatrix<U>& lhs, const iMatrix<U>& rhs);
164     template <class U> friend iMatrix<U> operator* (const U& lhs, const iMatrix<U>& rhs);
165     template <class U> friend iMatrix<U> operator* (const iMatrix<U>& lhs, const U& rhs);
166
167     template <class U> friend iMatrix<U> operator* (const std::vector<U>& v1, const std::vector<U>& v2);
168
169     iMatrix<T>& operator+= (const iMatrix<T>& rhs);
170     iMatrix<T>& operator-= (const T& rhs);
171
172     void PrintMatrix();
173     void PrintMatrix(int precision);
174     std::vector<T> Vectorize() const;
175     iMatrix<T> Transpose() const;
176
177
178     bool IsSquare();
179     T Determinant() const;
180
181     // Función estática para calcular el Producto Punto entre dos std::vector<T>.
182     static T dot_product (const std::vector<T>& v1, const std::vector<T>& v2);
183
184     private:
185         int Sub2Ind(int row, int col) const;
186     private:
187         std::vector<T> m_matrixData;
188         int m_nRows, m_nCols;
189 };
190
191
192 /* *****
193 CONSTRUCTOR / DESTRUCTOR FUNCTIONS
194 ***** */
195 template <class T>
196 iMatrix<T>::iMatrix() : m_nRows(1), m_nCols(1)
197 {
198     // Inicializamos el vector con 1 elemento (por defecto 0 o T())
199     m_matrixData.resize(1);
200 }
201
202 // Construct empty matrix (all elements 0)
203 template <class T>
204 iMatrix<T>::iMatrix(int nRows, int nCols) : m_nRows(nRows), m_nCols(nCols)
205 {
206     int nElements = m_nRows * m_nCols;
207     if constexpr (std::is_arithmetic_v<T>) {
208         // Redimensionar e inicializar a 0.0f
209         m_matrixData.resize(nElements, static_cast<T>(0.0f));
210     } else {
211         // Redimensionar e inicializar con el constructor por defecto T()
212         m_matrixData.resize(nElements);
213     }
214 }
215 // Construct from Eigen::VectorXd
216 template <class T>
217 iMatrix<T>::iMatrix(int nRows, int nCols, const Eigen::VectorXd& inputData)
218 : m_nRows(nRows), m_nCols(nCols)
219 {
220     int nElements = m_nRows * m_nCols;
221     if (inputData.size() != nElements)
222         throw std::invalid_argument("Eigen::VectorXd no tiene el tamaño correcto para llenar la matriz.");
223
224     m_matrixData.resize(nElements);
225     for (int i = 0; i < nElements; ++i)
226         m_matrixData[i] = static_cast<T>(inputData[i]);
227 }
228 // Construct from const linear array.
229 template <class T>
230 iMatrix<T>::iMatrix(int nRows, int nCols, const T *inputData) : m_nRows(nRows), m_nCols(nCols)
231 {
232     int nElements = m_nRows * m_nCols;
233     // Usamos el constructor de rango de std::vector para la copia eficiente.
234     m_matrixData = std::vector<T>(inputData, inputData + nElements);
235 }
236
237 // The copy constructor.
238 template <class T>
239 iMatrix<T>::iMatrix(const iMatrix<T> &inputMatrix)
240 : m_nRows(inputMatrix.m_nRows),
241   m_nCols(inputMatrix.m_nCols),
242   m_matrixData(inputMatrix.m_matrixData)
243 {
244     // Cuerpo del constructor vacío (RAII).
245 }
246
247 template <class T>
248 iMatrix<T>::~iMatrix()
249 {
250     // El destructor de std::vector se llama automáticamente (RAII).
251 }
252 /* *****
253 CONFIGURATION FUNCTIONS
254 ***** */
255 template <class T>
256 bool iMatrix<T>::Resize(int numRows, int numCols)
257 {
258     m_nRows = numRows;
259     m_nCols = numCols;

```

```

260     int nElements = (m_nRows * m_nCols);
261
262
263     if constexpr (std::is_arithmetic_v<T>) {
264         m_matrixData.resize(nElements, static_cast<T>(0.0f));
265     } else {
266         m_matrixData.resize(nElements);
267     }
268
269     return true;
270 }
271 template <class T>
272 void iMatrix<T>::SetToIdentity()
273 {
274     if (!IsSquare())
275         throw std::invalid_argument("Cannot form an identity matrix that is not square.");
276
277     for (int row=0; row<m_nRows; ++row)
278     {
279         for (int col=0; col<m_nCols; ++col)
280         {
281             if (col == row)
282                 m_matrixData[Sub2Ind(row,col)] = 1.0;
283             else
284                 m_matrixData[Sub2Ind(row,col)] = 0.0;
285         }
286     }
287 }
288
289 template <class T>
290 int iMatrix<T>::GetNumRows() const
291 {
292     return m_nRows;
293 }
294
295 template <class T>
296 int iMatrix<T>::GetNumCols() const
297 {
298     return m_nCols;
299 }
300 /* *****
301 ELEMENT FUNCTIONS
302 /* *****
303 template <class T>
304 T iMatrix<T>::GetElement(int row, int col) const
305 {
306     int linearIndex = Sub2Ind(row, col);
307     if (linearIndex >= 0)
308         return m_matrixData[linearIndex];
309     else{
310         if constexpr (std::is_arithmetic_v<T>) {
311             return 0; // Para tipos numéricos (int, float, double)
312         } else {
313             return T(); // Para contenedores (std::vector<float>, etc.)
314         }
315     }
316 }
317
318 template <class T>
319 std::vector<T> iMatrix<T>::GetRow(int row) const
320 {
321     int linearIndex = Sub2Ind(row, 0);
322     std::vector<T> AuxRow;
323     for(unsigned int i=0; i<this->GetNumCols(); i++){
324         AuxRow.push_back(m_matrixData[linearIndex]);
325         linearIndex++;
326     }
327     return AuxRow;
328 }
329
330
331 template <class T>
332 bool iMatrix<T>::SetElement(int row, int col, T elementValue)
333 {
334     int linearIndex = Sub2Ind(row, col);
335     if (linearIndex >= 0)
336     {
337         m_matrixData[linearIndex] = elementValue;
338         return true;
339     }
340     else
341     {
342         return false;
343     }
344 }
345
346 template <class T>
347 bool iMatrix<T>::SetCol(int col, std::vector<T> elements){
348     if (col < 0 || col >= m_nCols || static_cast<int>(elements.size()) != m_nRows)
349         return false;
350     for (int i = 0; i < m_nRows; ++i) {
351         m_matrixData[i * m_nCols + col] = elements[i];
352     }
353     return true;
354 }
355
356
357 template <class T>
358 bool iMatrix<T>::SetRow(int row, std::vector<T> elements){
359     int linearIndex = Sub2Ind(row, 0);
360     int aux = this->GetNumCols();
361     if (linearIndex >= 0){
362         for(unsigned int i=0; i<aux; i++){

```

```

363         (*this)(row, i) = elements[i];
364         //this->SetElement(row,i,elements[i]);
365         linearIndex=linearIndex+aux;
366     }
367     return true;
368 }
369 else{
370     return false;
371 }
372 }
373 }
374
375 template <class T>
376 iMatrix<T> iMatrix<T>::GetCol(int col) const
377 {
378     std::vector<T> result(m_nRows);
379     for (int i = 0; i < m_nRows; ++i) {
380         result[i] = m_matrixData[i * m_nCols + col];
381     }
382     return result;
383 }
384
385 template <class T>
386 iMatrix<T> iMatrix<T>::GetSubMat(int rowIni, int rowEnd, int colIni, int colEnd) const
387 {
388     int numRows= rowEnd-rowIni+1;
389     int numCols= colEnd-colIni+1;
390
391     int aux =this->GetNumCols();
392     iMatrix<T> AuxMat (numRows, numCols);
393
394     for(unsigned int i=0; i< numRows; i++){
395         for(unsigned int j=0; j<numCols; j++){
396             AuxMat(i,j)=(*this)(i+rowIni, j+colIni);
397         }
398     }
399
400     return AuxMat;
401 }
402
403 /* *****
404 THE + OPERATOR
405 /* *****
406 // matrix + matrix.
407 template <class T>
408 iMatrix<T> operator+ (const iMatrix<T>& lhs, const iMatrix<T>& rhs)
409 {
410     int numRows = lhs.m_nRows;
411     int numCols = lhs.m_nCols;
412     int numElements = lhs.m_matrixData.size();
413     T *tempResult = new T[numElements];
414     for (int i=0; i<numElements; i++)
415         tempResult[i] = lhs.m_matrixData[i] + rhs.m_matrixData[i];
416
417     iMatrix<T> result (numRows, numCols, tempResult);
418     delete[] tempResult;
419     return result;
420 }
421
422 // scaler + matrix
423 template <class T>
424 iMatrix<T> operator+ (const T& lhs, const iMatrix<T>& rhs)
425 {
426     int numRows = rhs.m_nRows;
427     int numCols = rhs.m_nCols;
428     int numElements = rhs.m_matrixData.size();
429     T *tempResult = new T[numElements];
430     for (int i=0; i<numElements; ++i)
431         tempResult[i] = lhs + rhs.m_matrixData[i];
432
433     iMatrix<T> result (numRows, numCols, tempResult);
434     delete[] tempResult;
435     return result;
436 }
437
438 // matrix + scaler
439 template <class T>
440 iMatrix<T> operator+ (const iMatrix<T>& lhs, const T& rhs)
441 {
442     int numRows = lhs.m_nRows;
443     int numCols = lhs.m_nCols;
444     int numElements = lhs.m_matrixData.size();
445     T *tempResult = new T[numElements];
446     for (int i=0; i<numElements; ++i)
447         tempResult[i] = lhs.m_matrixData[i] + rhs;
448
449     iMatrix<T> result (numRows, numCols, tempResult);
450     delete[] tempResult;
451     return result;
452 }
453
454 /* *****
455 THE - OPERATOR
456 /* *****
457 // matrix - matrix
458 template <class T>
459 iMatrix<T> operator- (const iMatrix<T>& lhs, const iMatrix<T>& rhs)
460 {
461     int numRows = lhs.m_nRows;
462     int numCols = lhs.m_nCols;
463     int numElements = lhs.m_matrixData.size();
464     T *tempResult = new T[numElements];
465     for (int i=0; i<numElements; i++)

```

```

466         tempResult[i] = lhs.m_matrixData[i] - rhs.m_matrixData[i];
467
468         iMatrix result(numRows, numCols, tempResult);
469         delete[] tempResult;
470         return result;
471     }
472
473     // scaler - matrix
474     template <class T>
475     iMatrix<T> operator- (const T& lhs, const iMatrix<T>& rhs)
476     {
477         int numRows = rhs.m_nRows;
478         int numCols = rhs.m_nCols;
479         int numElements = rhs.m_matrixData.size();
480         T *tempResult = new T[numElements];
481         for (int i=0; i<numElements; ++i)
482             tempResult[i] = lhs - rhs.m_matrixData[i];
483
484         iMatrix<T> result(numRows, numCols, tempResult);
485         delete[] tempResult;
486         return result;
487     }
488
489     // matrix - scaler
490     template <class T>
491     iMatrix<T> operator- (const iMatrix<T>& lhs, const T& rhs)
492     {
493         int numRows = lhs.m_nRows;
494         int numCols = lhs.m_nCols;
495         int numElements = lhs.m_matrixData.size();
496         T *tempResult = new T[numElements];
497         for (int i=0; i<numElements; ++i)
498             tempResult[i] = lhs.m_matrixData[i] - rhs;
499
500         iMatrix<T> result(numRows, numCols, tempResult);
501         delete[] tempResult;
502         return result;
503     }
504
505     // scaler * matrix
506     template <class T>
507     iMatrix<T> operator* (const T& lhs, const iMatrix<T>& rhs)
508     {
509         iMatrix<T> result(rhs.m_nRows, rhs.m_nCols);
510         std::transform(
511             rhs.m_matrixData.begin(),
512             rhs.m_matrixData.end(),
513             result.m_matrixData.begin(),
514             [&lhs](const T& element) { return lhs * element; }
515         );
516         return result;
517     }
518
519     /* Old product operator for the scaler
520
521     template <class T>
522     iMatrix<T> operator* (const T& lhs, const iMatrix<T>& rhs)
523     {
524         int numRows = rhs.m_nRows;
525         int numCols = rhs.m_nCols;
526         int numElements = rhs.m_matrixData.size();
527         T *tempResult = new T[numElements];
528         for (int i=0; i<numElements; ++i)
529             tempResult[i] = lhs * rhs.m_matrixData[i];
530
531         iMatrix<T> result(numRows, numCols, tempResult);
532         delete[] tempResult;
533         return result;
534     }
535
536
537     template <class T>
538     iMatrix<T> operator* (const iMatrix<T>& lhs, const T& rhs)
539     {
540         int numRows = lhs.m_nRows;
541         int numCols = lhs.m_nCols;
542         int numElements = lhs.m_matrixData.size();
543         T *tempResult = new T[numElements];
544         for (int i=0; i<numElements; ++i)
545             tempResult[i] = lhs.m_matrixData[i] * rhs;
546
547         iMatrix<T> result(numRows, numCols, tempResult);
548         delete[] tempResult;
549         return result;
550     }
551
552     */
553     // matrix * scaler
554     template <class T>
555     iMatrix<T> operator* (const iMatrix<T>& lhs, const T& rhs)
556     {
557         iMatrix<T> result(lhs.m_nRows, lhs.m_nCols);
558
559         std::transform(
560             lhs.m_matrixData.begin(),
561             lhs.m_matrixData.end(),
562             result.m_matrixData.begin(),
563             [&rhs](const T& element) { return element * rhs; }
564         );
565         return result;
566     }
567 }
568

```

```

569 // matrix * matrix
570 template <class T>
571 iMatrix<T> operator* (const iMatrix<T>& lhs, const iMatrix<T>& rhs)
572 {
573     int r_numRows = rhs.m_nRows;
574     int r_numCols = rhs.m_nCols;
575     int l_numRows = lhs.m_nRows;
576     int l_numCols = lhs.m_nCols;
577
578     if (l_numCols == r_numRows)
579     {
580         iMatrix<T> rhs_T(r_numCols, r_numRows);
581         for (int i = 0; i < r_numRows; ++i) {
582             for (int j = 0; j < r_numCols; ++j) {
583                 rhs_T(j, i) = rhs(i, j);
584             }
585         }
586
587         T *tempResult = new T[lhs.m_nRows * rhs.m_nCols];
588
589         for (int lhsRow=0; lhsRow<l_numRows; lhsRow++)
590         {
591             for (int rhsCol=0; rhsCol<r_numCols; rhsCol++)
592             {
593                 T elementResult = static_cast<T>(0.0);
594                 int lhsLinearIndex = (lhsRow * l_numCols);
595                 int rhsTLinearIndex = (rhsCol * r_numRows);
596
597                 for (int k=0; k<l_numCols; k++)
598                 {
599                     elementResult += (lhs.m_matrixData[lhsLinearIndex + k] * rhs_T.m_matrixData[rhsTLinearIndex + k]);
600                 }
601                 int resultLinearIndex = (lhsRow * r_numCols) + rhsCol;
602                 tempResult[resultLinearIndex] = elementResult;
603             }
604         }
605
606         iMatrix<T> result(l_numRows, r_numCols, tempResult);
607         delete[] tempResult;
608         return result;
609     }
610     else
611     {
612         iMatrix<T> result(1, 1);
613         return result;
614     }
615 }
616 }
617
618
619 template <class T>
620 iMatrix<T> iMatrix<T>::operator= (const iMatrix<T> &rhs)
621 {
622     if (this != &rhs)
623     {
624         m_nRows = rhs.m_nRows;
625         m_nCols = rhs.m_nCols;
626         m_matrixData = rhs.m_matrixData;
627     }
628
629     return *this;
630 }
631
632
633
634 template <class T>
635 iMatrix<T> operator* (const std::vector<T>& v1, const std::vector<T>& v2)
636 {
637     int N = v1.size();
638     int M = v2.size();
639
640     iMatrix<T> result(N, M);
641     for (int i = 0; i < N; ++i)
642     {
643         T vi_i = v1[i];
644
645         for (int j = 0; j < M; ++j)
646         {
647             result(i, j) = vi_i * v2[j];
648         }
649     }
650
651     return result;
652 }
653
654
655 template <class T>
656 iMatrix<T>& iMatrix<T>::operator+= (const iMatrix<T>& rhs)
657 {
658     if (m_nRows != rhs.m_nRows || m_nCols != rhs.m_nCols)
659     {
660         throw std::invalid_argument("Error: Las matrices deben tener las mismas dimensiones para la suma in-place.");
661     }
662
663     int numElements = m_matrixData.size();
664     T* data_lhs = m_matrixData.data();
665     const T* data_rhs = rhs.m_matrixData.data();
666     for (int i = 0; i < numElements; ++i)
667     {
668         data_lhs[i] += data_rhs[i];
669     }
670     return *this;
671 }

```

```

672 }
673
674 template <class T>
675 iMatrix<T>& iMatrix<T>::operator*=(const T& rhs)
676 {
677     int numElements = m_matrixData.size();
678     T* data_lhs = m_matrixData.data();
679     for (int i = 0; i < numElements; ++i)
680     {
681         data_lhs[i] *= rhs;
682     }
683
684     return *this;
685 }
686
687 // A simple function to print a matrix to stdout.
688 template <class T>
689 void iMatrix<T>::PrintMatrix()
690 {
691     int nRows = this->GetNumRows();
692     int nCols = this->GetNumCols();
693     for (int row = 0; row < nRows; ++row)
694     {
695         for (int col = 0; col < nCols; ++col)
696         {
697             std::cout << std::fixed << std::setprecision(3) << (*this)(row, col) << " ";
698         }
699         std::cout << std::endl;
700     }
701 }
702
703 // A simple function to print a matrix to stdout, with specified precision.
704 template <class T>
705 void iMatrix<T>::PrintMatrix(int precision)
706 {
707     int nRows = this->GetNumRows();
708     int nCols = this->GetNumCols();
709     for (int row = 0; row < nRows; ++row)
710     {
711         for (int col = 0; col < nCols; ++col)
712         {
713             std::cout << std::fixed << std::setprecision(precision) << (*this)(row, col) << " ";
714         }
715         std::cout << std::endl;
716     }
717 }
718
719 // Function to test whether the matrix is square.
720 template <class T>
721 bool iMatrix<T>::IsSquare()
722 {
723     if (m_nCols == m_nRows)
724         return true;
725     else
726         return false;
727 }
728
729 template <class T>
730 T iMatrix<T>::Determinant() const
731 {
732     if (!this->IsSquare()){
733         return static_cast<T>(0.0);
734     }
735
736     int N = this->m_nRows;
737     if (N == 1){
738         return (*this)(0, 0);
739     }
740     iMatrix<T> tempMatrix = *this;
741
742     T det = static_cast<T>(1.0);
743     int sign = 1;
744
745     for (int k = 0; k < N; ++k){
746         int pivotRow = k;
747         T maxValue = std::abs(tempMatrix(k, k));
748
749         for (int i = k + 1; i < N; ++i)
750         {
751             if (std::abs(tempMatrix(i, k)) > maxValue)
752             {
753                 maxValue = std::abs(tempMatrix(i, k));
754                 pivotRow = i;
755             }
756         }
757
758         if (pivotRow != k){
759             for (int j = 0; j < N; ++j){
760                 std::swap(tempMatrix(k, j), tempMatrix(pivotRow, j));
761             }
762
763             sign = -sign;
764         }
765
766         if (std::abs(tempMatrix(k, k)) < static_cast<T>(1e-9)){
767             return static_cast<T>(0.0);
768         }
769
770         T pivotValue = tempMatrix(k, k);
771         for (int i = k + 1; i < N; ++i) {
772             T factor = tempMatrix(i, k) / pivotValue;
773
774             for (int j = k; j < N; ++j){

```

```

775         tempMatrix(i, j) -= factor * tempMatrix(k, j);
776     }
777 }
778
779     det *= pivotValue;
780 }
781     return sign * det;
782 }
783
784 template <class T>
785 std::vector<T> iMatrix<T>::Vectorize() const
786 {
787     return m_matrixData;
788 }
789
790 template <class T>
791 iMatrix<T> iMatrix<T>::Transpose() const
792 {
793     iMatrix<T> result(m_nCols, m_nRows); // ← filas y columnas intercambiadas
794
795     // Recorremos todos los elementos de la matriz original
796     for (int i = 0; i < m_nRows; ++i)
797     {
798         for (int j = 0; j < m_nCols; ++j)
799         {
800             // Elemento (i,j) en la original → posición (j,i) en la transpuesta
801             result(j, i) = (*this)(i, j);
802         }
803     }
804
805     return result;
806 }
807
808
809
810 template <class T>
811 T dot_product (const std::vector<T>& v1, const std::vector<T>& v2)
812 {
813     if (v1.size() != v2.size())
814     {
815         throw std::invalid_argument("Error: Los vectores deben tener la misma dimensión para el Producto Punto.");
816     }
817
818     int N = v1.size();
819     T resultValue = static_cast<T>(0.0);
820
821     for (int i = 0; i < N; ++i)
822     {
823         resultValue += v1[i] * v2[i];
824     }
825
826     return resultValue;
827 }
828 /* *****
829 PRIVATE FUNCTIONS
830 /* *****
831 // Function to return the linear index corresponding to the supplied row and column values.
832 template <class T>
833 int iMatrix<T>::Sub2Ind(int row, int col) const
834 {
835     if ((row < m_nRows) && (row >= 0) && (col < m_nCols) && (col >= 0))
836         return (row * m_nCols) + col;
837     else
838         return -1;
839 }
840
841
842
843 #endif

```

Fichero funciones.hpp

```

1  /*****
2  *AUTHOR: Carlos Palomera Oliva
3  *DATE: MARCH 2025
4  *****/
5  #define _USE_MATH_DEFINES
6  #include <iostream>
7  #include <unordered_map>
8  #include <utility>
9  #include <cmath>
10 #include <iomanip>
11 #include <vector>
12 #include <algorithm>
13 #include <fstream>
14 #include <string>
15 #include <limits>
16 #include "iMatrix.cpp"
17 #include "C:\Librerias\eigen-3.4.0\Eigen\Sparse" //windows version
18 #include "C:\Librerias\eigen-3.4.0\Eigen\Dense" //windows version
19 // #include "home\carlo\Librerias\eigen-3.4.0\Eigen/Dense"
20 // #include "home\carlo\Librerias\eigen-3.4.0\Eigen/Sparse"
21 #ifndef FUNCIONES_HPP
22 #define FUNCIONES_HPP
23
24
25
26
27 /*****
28 * Structure of the Data File for Matrix

```

```

29  * Nrow Ncol
30  * vector of size Ncol representing the first variable where elements are separated by space Ex(x1 x2 x3 ...)
31  * vector of size Ncol ...
32  * .
33  * . Nrow times in total
34  * .
35  * .
36  *****/
37  /*
38  *Input: Direction of the data file
39  *Output: iMatrix filled with the control points in data file
40  */
41  iMatrix<double> ReadDataFile_Matrix(std::string direction);
42
43  /*****
44  * Structure of the Data File for Tensors
45  * Ncol NSplits
46  * vector of size Ncol representing the first variable where elements are separated by space Ex(x1 x2 x3 ...)
47  * vector of size Ncol ...
48  * .
49  * . 3 times in total
50  * .
51  * -----
52  * The previous repeated NSplits totals
53  *****/
54  /*
55  *Input: Direction of the data file
56  *Output: vector of iMatrixes filled with the control points in data file
57  */
58  std::vector<iMatrix<double>> ReadDataFile_CtrlPts(std::string direction);
59
60
61  /*
62  *Input: Value of the point where the polinomial is evaluated t, and degree of the polinomial n
63  *Condition: t∈[0,1]
64  *Output: Values of all the Bernstein polinomial of degree n evaluated at t
65  */
66  std::vector<double> BernsteinPolinomial(double t, int n);
67
68
69  /*
70  *Input: iMatrix of control points and parameter of the curve t
71  *Conditions: t∈[0,1]
72  *Output: Value of the Bezier curve of such control points in the point t
73  */
74  std::vector<double> BezierCurve_Point_t(const iMatrix<double> &PtsControl, double t);
75
76
77  /*
78  *Input: iMatrix of control points and number of steps for the curve
79  *Conditions: Steps must be >= 1
80  *Output: Value of the Bezier curve at every point from 0 to 1 separated in N_Steps steps
81  */
82  iMatrix<double> BezierCurve(const iMatrix<double> &PtsControl, double N_Steps);
83
84  /*
85  *Input: Tensor of control points and the 2 parameters of the curve t1, t2
86  *Conditions: t1,t2∈[0,1]
87  *Output: Value of the Bezier surface at of such control points in the points t1 t2
88  */
89  std::vector<double> BezierSurface_Point_t1_t2(const std::vector<iMatrix<double>> &PtsControl, double t1, double t2);
90
91  /*
92  *Input: iMatrix of control points and number of steps for the curve in both parameters
93  *Conditions: N_Steps must be >= 1
94  *Output: Value of the Bezier surface at every point from 0 to 1 separated in N_Steps along both parameters
95  */
96  std::vector<iMatrix<double>> BezierSurface(const std::vector<iMatrix<double>> &PtsControl, double N_Steps);
97
98  /*
99  *Input: Bezier surface in form of a tensor
100  *Output: Writes a file of 3 matrices one for each coordinate
101  */
102  void Write_BezierSurface(const std::vector<iMatrix<double>> &Tensor, std::string name);
103
104  /*
105  *Input: CtrlPts in format described before
106  *Output: Writes a file of 3 matrices one for each coordinate
107  */
108  void Write_CtrlPts(const std::vector<iMatrix<double>> &CtrlPts, std::string name);
109
110
111  /*****
112  *
113  * Classic funtions from the Nurbs Book
114  *
115  *****/
116
117  /*
118  *Input: A KnotVector, t the point of evalution, its polynomial degree, and n=the size of the Nurbs vector-p-2
119  *Output: the span index
120  */
121  int FindSpan(const std::vector<double> &KnotVector, double t, int p, int n);
122
123  /*
124  *Input: i span index, t eval point, p polinomial degree and KnotVector
125  *Output: vector of basis funcs
126  */
127  std::vector<double> BasisFuns(int i, double t, int p, const std::vector<double> &KnotVector);
128
129  /*
130  *Input: i span index, t eval point, p polinomial degree and KnotVector

```

```

132  *Output: iMatrix where the i-th row is the i-th degree basis fun
133  */
134  iMatrix<double> AllBasisFuns(int i, double t, int p, const std::vector<double> &KnotVector);
135
136  /*
137  *Input: i span index, t eval point, p polinomial degree, k derivate degree, KnotVector and an output vector ders;
138  *Output: value of the k-derivate at point t in the ders vector on the k-row of the matrix
139  */
140  iMatrix<double> DerBasisFuns(int span, double t, int p, int k, const std::vector<double> &KnotVector);
141
142  /*
143  *Input: matrix of BasisFunctions or its derivatives and the path where they will be written
144  *Output: Writes the Functions as a matrix on selected path
145  */
146  void Write_BasisFunctions(const iMatrix<double> &BasisFunctions, std::string name);
147
148  /*****
149  *
150  * Classic funtions for p spline curves/surfaces
151  *
152  *****/
153
154  /*
155  *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,t the point of evaluation
156  *Output: Evaluation of the curve at point t
157  */
158  std::vector<double> CurvePoint(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts, double t);
159
160  /*
161  *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,
162  *      t the point of evaluation and d highest derivate order to compute
163  *Output: iMatrix where the i-th row is the evaluation of the i-th derivate
164  */
165  iMatrix<double> CurveDerivsAlg1(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts, double t, int d);
166
167  /*
168  *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,t the point of evaluation,
169  *      d highest derivate order to compute, r1 and r2 specify the intervals of CtrlPoints
170  *Output: iMatrix where the i-th row is the vector of ctrl points
171  */
172  iMatrix<std::vector<double>> CurveDerivsCpts(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts,
173                                             int d, int r1, int r2);
174
175  /*
176  *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,
177  *      t the point of evaluation and d highest derivate order to compute
178  *Output: iMatrix where the i-th row is the evaluation of the i-th derivate
179  */
180  iMatrix<double> CurveDerivsAlg2(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts, double t, int d);
181
182  /*
183  *Input: Curve and path
184  *Output: Writes the values of the Curve in the selected path
185  */
186  void Write_Curve(const iMatrix<double> &Curve, std::string filename);
187
188  /*
189  *Input: n_i=the size of the Nurbs vector_i-p-2, p_i polynomial degree, KnotVector_i, the CtrlPts ,t_i the point of evaluation
190  *Output: Evaluation of the surface at point (t1,t2)
191  */
192  std::vector<double> SurfacePoint(int n1, int p1, const std::vector<double> &KnotVector1, int n2, int p2, const std::vector<double> &KnotVector2,
193                                  const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2);
194
195  /*
196  *Input: n_i=the size of the Nurbs vector_i-p-2, p_i polynomial degree, KnotVector_i, the CtrlPts ,
197  *      t_i the point of evaluation and d maximum order of the derivatives
198  *Output: Evaluation of the derivatives up to degree d (maximum is p or q and would overwrite it if d>p,q) in (t1,t2) where Output[k_1][k_2] is been
199  *      derived k_i times respect to the ith variable
200  */
201  iMatrix<std::vector<double>> SurfaceDerivsAlg1(int n1, int p1, const std::vector<double> &KnotVector1, int n2, int p2,
202                                               const std::vector<double> &KnotVector2, const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2, int d);
203
204  /*****
205  *
206  * Classic and support funtions for NURBS curves/surfaces
207  *
208  *****/
209
210
211  /*
212  *Input: Vector of CtrlPts and their weights at each point in a separated vector
213  *Output: iMatrix that each column is each of the coordinates of each CtrlPt except its last component that is it's respective weight
214  */
215  iMatrix<double> WeightCtrlPts(const iMatrix<double> &CtrlPts, const std::vector<double> &Weights);
216
217  /*
218  *Input: CtrlPts multiplied by their Weight with the respective weight at the end of each column
219  *Output: The original CtrlPts
220  */
221  iMatrix<double> UnWeightCtrlPts(const iMatrix<double> &CtrlPtsW);
222
223  /*
224  *Input: Vector of CtrlPts and their weights at each point in a separated vector
225  *Output: std::vector<iMatrix<double>> that keeps the format of the CtrlPts for Surface
226  *      operations with and extra 4th dimension on each coord thats's occupied by the weight of such pt
227  */
228  std::vector<iMatrix<double>> WeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPts, const iMatrix<double> &Weights);
229
230  /*
231  *Input: CtrlPts multiplied by their Weight with the respective weight at the end of each column
232  *Output: The original CtrlPts
233  */
234

```

```

235 std::vector<iMatrix<double>> UnWeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPtsW);
236
237 /*
238 *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts weighted with theris weight in the last component ,
239 t the point of evaluation
240 *Output: Evaluation of the NURBS curve at point t
241 */
242 std::vector<double> CurvePointRational(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPtsWeighted, double t);
243
244 /*
245 *Input: binomial coeficients k, i
246 *Output: combinatorial number given by upper number k and lower number i
247 */
248 int Binomial(int k, int i);
249
250 /*
251 *Input: i span index, t eval point, p polinomial degree and KnotVector, weights vector
252 *Output: vector of rational basis funcs
253 */
254 std::vector<double> RationalBasisFuns(int i, double t, int p,const std::vector<double> &KnotVector,const std::vector<double> &WeightVector);
255
256 /*
257 *Input: i span index, t eval point, k derivate degree, p polinomial degree and KnotVector, weights vector
258 *Output: vector of rational basis funcs
259 */
260 iMatrix<double> RatDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector,const std::vector<double> &WeightVector);
261
262 /*
263 *Input: i span index, t eval point, k derivate degree, p polinomial degree and KnotVector, iMatrix of the derivates of the non-rational
264 basis funs and weights vector
265 *Output: vector of rational basis funcs
266 */
267 iMatrix<double> RationalizeDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector,const iMatrix<double> &ders ,
268 const std::vector<double> &WeightVector);
269
270 /*
271 *Input: Aders: derivates of the CtrlPtsWeighted using CurvDersAlg1 evaluated at some t, wders: same as Aders but with the weighted vector,
272 maximum degree of the derivate
273 -Aders = CurveDerivsAlg1(n, p,KnotVector, WeightedCtrlPts, t, d);
274 -wders = CurveDerivsAlg1(n, p,KnotVector, WeightVector, t, d);
275 *Output: iMatrix on which each row is the i-th derivate of the rational curve made of the weighted CtrlPts and the weight vector
276 */
277 iMatrix<double> RatCurveDerivs(const iMatrix<double> &Aders,const iMatrix<double> &wders, int d);
278
279 /*
280 *Input: n_i=the size of the Nurbs vector_i-p-2, p_i polinomial degree, KnotVector_i, the CtrlPtsWeighted ,t_i the point of evaluation
281 *Output: Evaluation of the surface at point (t1,t2)
282 */
283 std::vector<double> SurfacePointRational(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,
284 const std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsWeighted, double t1, double t2);
285
286 /*
287 *Input: derivates of the CtrlPtsWeighted using CurvDersAlg1 evaluated at some t, wders: same as Aders but with the weighted vector,
288 maximum degree of the derivate
289 *Output: iMatrix on which each row is the i-th derivate of the rational surface made of the weighted CtrlPts and the weight vector
290 */
291 iMatrix<std::vector<double>> RatSurfaceDerivs(const iMatrix<std::vector<double>> &Aders,const iMatrix<double> &wders,int d);
292
293 /*****
294 *
295 * Fundamental Geometric Algorithms
296 *
297 *****/
298
299 /*
300 *Input: KnotVector and element you want to introduce
301 *Output: index i such that KnotVector[i-1] <= e < KnotVector[i]
302 *Note: Applying insertion with this index using KnotVector.insert(KnotVector.begin()+index+1, r, e) will yield the KnotVector
303 we want (r is number of insertions)
304 */
305 int findInsertionIndex(const std::vector<double> &KnotVector, double e);
306
307 /*
308 *Input: ordered vector and element you want to check it's number of ocurrences
309 *Output: number of times e appears on the vector
310 */
311 int countOccurrences(const std::vector<double> &v, double e);
312
313 /*
314 *Input: A KnotVector,t the point of evaluation, its polynomial degree, and n=the size of the Nurbs vector-p-2
315 *Output: the span index at k, its multiplicity at s
316 */
317 void FindSpanMult(const std::vector<double> &KnotVector, double t, int p, int n,int &k, int &s);
318
319 /*
320 *Input: np number of initial CtrlPts-1, p polynomial degree, the initial KnotVector, the initial CtrlPts Weighted, u value of the inserted knot,
321 index such that KnotVector[k]<= u < KnotVector[k+1], s initial multiplicity of u in KnotVector, r number of insertions of u in Knotvector
322 *Output: New CtrlPts Weighted that presserves the initial curve
323 *Note: !! KnotVector will be updated !!
324 */
325 iMatrix<double> CurveKnotsIns(int np, int p, std::vector<double> &KnotVector,const iMatrix<double> &CtrlPtsW, double u, int index, int s, int r);
326
327 /*
328 *Input: n=the size of the Knot vector-p-2, p polynomial degree, KnotVector, CtrlPts Weigthed, u point of evaluation
329 *Output: Evaluation of the curve given by the KnotVector and the ControlPts at the given point of evaluation u
330 */
331 std::vector<double> CurvePntByCornerCut(int n, int p, std::vector<double> &KnotVector, iMatrix<double> &CtrlPtsW, double u);
332
333 /*
334 *Input: npi size of KnotVectori-pi-2, pi degree of KnotVectori, KnotVectori for i=1,2, CtrlPts Weigthed, dir(true= knot inserted in Knotvector1,
335 false= knot inserted in Knotvector2, uv knot we want to insert, index such that KnotVectori[k]<= u < KnotVectori[k+1], s initial
336 multiplicity of u in KnotVectori, r number of insertions of u in Knotvectori
337 *Output: the KnotVectori will be updated and it will return new CtrlPtsW such that they generate the same surface.

```

```

338 */
339 std::vector<iMatrix<double>> SurfaceKnotIns(int np1, int p1, std::vector<double> &KnotVector1, int np2, int p2,
340     std::vector<double> &KnotVector2, const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
341     double uv, int index, int r, int s);
342
343 /*
344 *Input: n=KnotVector.size()-2-p, p polynomial degree, KnotVector, CtrlPtsW, Insertions: vector of new elements to insert into KnotVector,
345 *       r=Insertion.size()-1.
346 *Output: Update of the KnotVector with the new inserions, CtrlPtsW that generate the same curve with the new KnotVector.
347 */
348 iMatrix<double> RefineKnotVectCurve(int n, int p, std::vector<double> &KnotVector, const iMatrix<double> &CtrlPtsW,
349     const std::vector<double> &Insertions, int r);
350
351 /*
352 *Input: npi size of KnotVector1-1, pi degree of KnotVector1, KnotVector1 for i=1,2, CtrlPts Weighthed, dir(true= knot inserted in KnotVector1,
353 *       false= knot inserted in KnotVector2, Insertions: vector of new elements to insert into KnotVector, r=Insertion.size()-1.
354 *Output: the KnotVector1 will be updated and it will return new CtrlPtsW such that they generate the same surface.
355 */
356 std::vector<iMatrix<double>> RefineKnotVectSurface(int np1, int p1, std::vector<double> &KnotVector1, int np2, int p2,
357     std::vector<double> &KnotVector2, const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
358     const std::vector<double> &Insertions, int r);
359
360 /*****
361 *
362 * Functions for the Assembly of the Matrix
363 *
364 *****/
365
366 /*
367 *Input: n: n2 pts of integration
368 *Output: nodes will contain the roots of the legendre polynomial and weights its weights (all in double precision).
369 */
370 void legendre_pol(int n, Eigen::VectorXd &nodes, Eigen::VectorXd &weights);
371
372 /*
373 *Input: ordered vector
374 *Output: the original vector without duplicates
375 */
376 std::vector<double> removeDuplicates(const std::vector<double> & input);
377
378 /*
379 *Input: number of elements of evaluation
380 *Output: vector of nElements+1 uniformly distributed
381 */
382 std::vector<double> createSubIntervals(const double nElements, const double lower_limit, const double upper_limit);
383
384 /*
385 *Input: v1 vector where we remove the elements that appear in v2, v2 KnotVector
386 *Output: v1 without the elements in v2
387 */
388 std::vector<double> removeMatches(const std::vector<double> & v1, const std::vector<double> & v2);
389
390 /*
391 *Input: n=the size of the Knot vector-p-2, i span index, t eval point, p polinomial degree, number of points of evaluations in the integral,
392 *       KnotVector, upper and lower limits of the integration, weights vector, nodes of the legendre polynomial, CtrlPtsW the control
393 *       points weighted, the function f and the previous evalPoint.
394 *Output: BasisFunsEvals and DerBasisFunsEvals each row will store the evaluation of the value or derivate of the basis fun alongside each
395 *       evaluation point, same for the JacobianEvals, InverseJacobianEvals and funcEvals.
396 */
397 void D1_element_eval(int n, int i, int p, int nEvals, double lower_limit, double upper_limit, const std::vector<double> &KnotVector,
398     const std::vector<double> &weights, const Eigen::VectorXd &nodes, const iMatrix<double> &CtrlPtsW, std::function<double(double)> f,
399     double &preEvalPoint, double &cumLenght, iMatrix<double> &BasisFunsEvals, iMatrix<double> &DerBasisFunsEvals, std::vector<double> & JacobianEvals,
400     std::vector<double> & InverseJacobianEvals, std::vector<double> & funcEvals, double Lenght=1.0);
401
402 /*
403 *Input: p polinomial degree, upper and lower limits of the integration, number of points of evaluations in the integral,
404 *       evaluations of RationalDerivates, evaluation of the Inverse Jacobian and the legendre weights.
405 *Output: Matrix of all the permutuations of evaluations where the elements i,j of the matrix correspond to
406 *       the BasisFuns(init+i)*BasisFuns(init+j) where init=spanIndex()
407 */
408 iMatrix<double> gauss_legendre_cuadrature_integral_bilinearForm(int p, double lower_limit, double upper_limit, int nEvals,
409     const iMatrix<double> &DerBasisFunsEvals, const std::vector<double> & InverseJacobianEvals, const Eigen::VectorXd &weights);
410
411 /*
412 *Input: p polinomial degree, upper and lower limits of the integration, number of points of evaluations in the integral,
413 *       evaluations of RationalBasisFuns, evaluations of the Jacobian, evaluations of the function and the legendre weights.
414 *Output: Vector of evaluations which its components are l(index) where init=spanIndex()
415 */
416 std::vector<double> gauss_legendre_cuadrature_integral_linealForm(int p, double lower_limit, double upper_limit,
417     int nEvals, const iMatrix<double> &BasisFunsEvals, const std::vector<double> & JacobianEvals, const std::vector<double> & funcEvals,
418     const Eigen::VectorXd &weights);
419
420 /*
421 *Input: p polinomial degree, size=LineraForm.size()-1, SparseMatrix with the system already assambled,
422 *       same with the LinearForm, Modify0(1) true if we have dirichlet condition at the start(end) of the boundary,
423 *       vValueAt0(1) value set for the boundary condition, default is set at 0.
424 *Output: Modification of the sparse matrix and the linear form to keep the system as symetric as possible.
425 */
426 void impose_Dirichlet_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm,
427     bool Modify0, bool Modify1, double ValueAt0=0.0, double ValueAt1=0.0);
428
429 /*
430 *Input: size=LineraForm.size()-1, LinearForm, Modify0(1) true if we have dirichlet condition at the start(end) of the boundary,
431 *       vValueAt0(1) value set for the boundary condition, default is set at 0.
432 *Output: Modification of the linear form applying the conditions to the borders.
433 */
434 void impose_Newmann_Condition(int size, Eigen::VectorXd &LinearForm, bool Modify0, bool Modify1, double ValueAt0=0.0, double ValueAt1=0.0);
435
436 /*
437 *Input: p polinomial degree, size=LineraForm.size()-1, SparseMatrix with the system already assambled, same with the LinearForm,
438 *       Modify0(1) true if we have dirichlet condition at the start(end) of the boundary,
439 *       vValueAt0(1) value set for the boundary condition, default is set at 0.
440 *       values alpha and beta come from the expresion alpha*u(x)+beta*u'(x)=ValueAtx.

```

```

441 *Output: Modification of the sparse matrix and the linear form to keep the system as symetric as posible.
442 */
443 void impose_Robin_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm, double alpha,
444     double beta, bool Modify0, bool Modify1, double ValueAt0=0.0, double ValueAt1=0.0);
445
446 /*
447 *Input: vector of evaluations of the function, name of hte file where we write the evaluations
448 *Output: writes the evaluation of the function in a txt
449 */
450 void writeNumericFunction(Eigen::VectorXd funEvals, const std::vector<double> &KnotVector, const std::vector<double> &WeightVector,
451     int n, int p, int nEvals, double lowerLimit, double upperLimit, std::string name,
452     std::vector<double> Intervals, int nElements, double Lenght);
453
454 /*
455 *Input: the function we evaluate, lower and upper limits of evaluation and number of steps of the evaluation, name of the
456     file where we write the evaluations, for the evaluation of the curve point, CtrlPtsW, KnotVector, n = KnotVector.size()-p-2,
457     p polynomial degree.
458 *Output: writes the evaluation of the function in a txt
459 */
460 void writeAnalyticFunction(std::function<double(double)> f, std::function<double(double)> f_Ders, std::vector<double> Intervals, double nEvals,
461     const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector, int n, int p, std::string name, double Lenght, double nElements);
462
463 /*
464 *Input: name of the file, p polynomial degree
465 *Output: vector of evaluations on each elements loaded
466 */
467 std::vector<std::vector<double>> leerArchivo(const std::string& name, int p);
468
469 /*
470 *Input: vector of evaluations of the function, name of hte file where we write the evaluations
471 *Output: writes the evaluation of the derivates of the function in a txt
472 */
473 /*
474 void writeFunctionDers(Eigen::VectorXd funEvals, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPtsW,
475     const std::vector<double> &WeightVector, int n, int p, int nEvals, double lowerLimit, double upperLimit, std::string name);
476 */
477 /*
478 *Input: the function we evaluate, lower and upper limits of evaluation and number of steps of the evaluation, name of hte file
479     where we write the evaluations, for the evaluation of the curve point, CtrlPtsW, KnotVector, n = KnotVector.size()-p-2, p polynomial degree.
480 *Output: writes the evaluation of the function in a txt
481 */
482 /*
483 void writeFunctionDers(std::function<double(double)> f, double lowerLimit, double upperLimit, double nEvals,
484     const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector,
485     int n, int p, std::string name, double Lenght=1.0);
486 */
487 /*
488 *Input: lowerLimit of integration, upperLimit of integration, number of subintervals for the quadrature, CtrlPtsW, Knotvector, n, p
489 *Output: numerical value of the integral between lowerLimit and upperLimit of the norm of the derivate of the curve
490 */
491 /*
492 double IntegrateNormDer(double lowerLimit, double upperLimit, int nEvals, const iMatrix<double> &CtrlPtsW,
493     const std::vector<double> &KnotVector, int n, int p);
494 */
495 /*
496 *Input: e fisical parameter, lowerLimit of integration, upperLimit of integration, number of subintervals for the quadrature,
497     CtrlPtsW, Knotvector, n, p
498 *Output: numerical value parametric space parameter such that the analytic solution of it its equal to the fisical solution at e
499 */
500 inline double Fisical_to_Parametric(double t2, double t1, double lowerLimit, double Lenght,
501     int nEvals, const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector, int n, int p){
502     return IntegrateNormDer(t2,t1,nEvals, CtrlPtsW, KnotVector, n, p)/Lenght;
503 }
504
505 /*
506 *Input: i span index, p polynomial degree, k maximum derivate order, KnotVector, Intervals, nodes and weights
507     of the gauss-legendre cuadrature, vector of storage with spanIndex(memory has to be reserved before execution).
508 *Output: vector that each element is a vector that evaluates the basis functions in each of the corresponding gauss-legendre
509     points of each interval, modification of the vector spanIndex.
510 */
511 std::vector<std::vector<iMatrix<double>>> PreBasis_and_Ders(int p, int k, std::vector<double> KnotVector, std::vector<double> Intervals,
512     Eigen::VectorXd nodes, std::vector<int>& spanIndex);
513
514 /*
515 *Input: span index, nEvals number of points of the gauss-legendre cuadrature alongside the corresponding variable, nodes,
516     vector of the Gauss-Legendre cuadrature, lower_limit, upper_limit of the interval, p polynomial degree, k maximum derivate order, KnotVector,
517     Basis_and_DersY vector of the precomputed Basis and its corresponding derivatives, nodes and weights of the gauss-legendre cuadrature
518     and Weights vector of the corresponding weights alongside the corresponding variables.
519 *Output: vector that each element is thee evaluations of the basis functions and its derivatives up to k-degree in each of the corresponding
520     gauss-legendre points of the interval.
521 */
522 std::vector<iMatrix<double>> RationalizeDersBasisFunsVec(int span, int nEvals, Eigen::VectorXd nodes, double lower_limit, double upper_limit,
523     int p, int k, int n, std::vector<double> KnotVector, std::vector<iMatrix<double>>> Basis_and_DersY, std::vector<double> Weights);
524
525 /*
526 *Input: vector of Intervals, and vector of the correlated indexes for each interval
527 *Output: unordered map such that given a span index returns its corresponding interval.
528 */
529 std::unordered_map<int, std::vector<std::pair<double, double>>> BuildSpanMap(const std::vector<double>& Intervals, const std::vector<int>& spanIndices);
530
531 /*
532 *Input: vector of Intervals, and vector of the correlated indexes for each interval
533 *Output: unordered map such that given a span index returns its corresponding interval.
534 */
535 iMatrix<iMatrix<double>> RationalizeDersBasisFuns2D(const iMatrix<double> &DersBasisX, const iMatrix<double> &DersBasisY,
536     const iMatrix<double> &activeWeights, unsigned int pX, unsigned int pY);
537
538 /*
539 *Input: SolEvals and Surface evaluations of the same points with the filename and the path
540 *Output: writes the corresponding .txt with the Surface and its evaluations.
541 */
542 void ExportToTxt(const iMatrix<double> & SolEvals, const iMatrix<std::vector<double>>& Surf, const std::string& filename);

```

543 #endif

Fichero funciones.cpp

```

1  /*****
2  *AUTHOR: Carlos Palomera Oliva
3  *DATE: MARCH 2025
4  *****/
5  #include "funciones.hpp"
6
7  /***** */
8
9  /***** */
10 iMatrix<double> ReadDataFile_Matrix(std::string direction){
11     double Nrow,Ncol;
12     std::string line;
13     std::ifstream DataSet;
14
15     DataSet.open(direction);
16     if(DataSet.is_open()){
17
18         if(std::getline(DataSet,line, ' ')){
19             Nrow = stod(line);
20         }
21         if(std::getline(DataSet,line, '\n')){
22             Ncol = stod(line);
23         }
24
25         iMatrix<double> matrix(Nrow,Ncol);
26
27
28         for(unsigned int i=0; i<Nrow; i++){
29             for(unsigned int j=0; j<Ncol-1;j++){
30                 if(std::getline(DataSet,line, ' ')){
31                     matrix(i,j)=stod(line);
32                 }
33             }
34             if(std::getline(DataSet,line, '\n')){
35                 matrix(i,Ncol-1)=stod(line);
36             }
37         }
38         DataSet.close();
39         return matrix;
40     }
41     else{
42         throw std::invalid_argument("Cannot open Data File.");
43         iMatrix<double> m;
44         return m;
45     }
46
47
48 }
49 /***** */
50
51 /***** */
52 std::vector<iMatrix<double>> ReadDataFile_CtrlPts(std::string direction){
53     double Ncol,NSplits;
54     std::string line;
55     std::ifstream DataSet;
56
57     DataSet.open(direction);
58     if(DataSet.is_open()){
59
60         if(std::getline(DataSet,line, ' ')){
61             Ncol = stod(line);
62         }
63         if(std::getline(DataSet,line, '\n')){
64             NSplits = stod(line);
65         }
66
67         std::vector<iMatrix<double>> CtrlPts(NSplits);
68         for(unsigned int k=0; k<NSplits;k++){
69
70             CtrlPts[k].Resize(3,Ncol);
71
72             for(unsigned int i=0; i<3; i++){
73
74                 for(unsigned int j=0; j<Ncol-1;j++){
75
76                     if(std::getline(DataSet,line, ' ')){
77
78                         CtrlPts[k](i,j)=stod(line);
79                     }
80                 }
81                 if(std::getline(DataSet,line, '\n')){
82                     CtrlPts[k](i,Ncol-1)=stod(line);
83                 }
84             }
85             getline(DataSet,line,'\n');
86
87         }
88
89
90         DataSet.close();
91         return CtrlPts;
92     }
93     else{
94         throw std::invalid_argument("Cannot open Data File.");
95         std::vector<iMatrix<double>> T;
96         return T;

```

```

97     }
98 }
99
100
101 /***** */
102
103 /***** */
104 std::vector<double> BernsteinPolynomial(double t, int n){
105     std::vector<double> Bnew(n);
106     std::vector<double> Bold(n);
107     std::fill(Bold.begin(), Bold.end(), 0);
108     Bold[0]=1;
109     double t1=1-t;
110     for(unsigned int j=1;j<n;j++){
111
112         std::fill(Bnew.begin(), Bnew.end(), 0);
113
114         for(unsigned int i=0; i<j;i++){
115
116             Bnew[i]=Bnew[i]+t1*Bold[i];
117             Bnew[i+1]=Bnew[i+1]+t*Bold[i];
118         }
119
120         Bold=Bnew;
121     }
122 }
123 return Bnew;
124 }
125
126 /***** */
127
128 /***** */
129 std::vector<double> BezierCurve_Point_t(const iMatrix<double> &PtosControl, double t){
130     int dim = PtosControl.GetNumRows();
131     std::vector<double> Curve(dim);
132     int n=PtosControl.GetNumCols();
133     std::vector<double> B = BernsteinPolynomial(t,n);
134     for(unsigned int j=0; j<dim; j++){
135         for(unsigned int i=0; i<n;i++){
136             Curve[j]=Curve[j]+B[i]*PtosControl(j,i);
137         }
138     }
139     return Curve;
140 }
141
142 /***** */
143
144 /***** */
145 iMatrix<double> BezierCurve(const iMatrix<double> &PtosControl, double N_Steps){
146     iMatrix<double> Curve(PtosControl.GetNumRows(),N_Steps+1);
147     double Step=1/(N_Steps);
148     for(unsigned int i=0; i<N_Steps+1;i++){
149         Curve.SetCol(i,BezierCurve_Point_t(PtosControl,i*Step));
150     }
151     return Curve;
152 }
153
154 /***** */
155
156 /***** */
157
158 /***** */
159 std::vector<double> BezierSurface_Point_t1_t2(const std::vector<iMatrix<double>> &PtosControl, double t1, double t2){
160     int dim = 3;
161     std::vector<double> Curve(dim);
162
163     int N_Splits = PtosControl.size();
164     int n=PtosControl[0].GetNumCols();
165     std::vector<double> B1 = BernsteinPolynomial(t1,N_Splits);
166     std::vector<double> B2 = BernsteinPolynomial(t2,n);
167
168     for(unsigned int k=0; k<N_Splits;k++){
169         for(unsigned int j=0; j<dim; j++){
170             for(unsigned int i=0; i<n;i++){
171                 Curve[j]=Curve[j]+B1[k]*B2[i]*PtosControl[k](j,i);
172             }
173         }
174     }
175
176     return Curve;
177 }
178
179 /***** */
180
181 /***** */
182 std::vector<iMatrix<double>> BezierSurface(const std::vector<iMatrix<double>> &PtosControl, double N_Steps){
183     std::vector<iMatrix<double>> Surface(N_Steps+1);
184     double Step=1/(N_Steps);
185
186     for(unsigned int k=0; k<N_Steps+1;k++){
187         Surface[k].Resize(3,N_Steps+1);
188         for(unsigned int i=0; i<N_Steps+1;i++){
189             Surface[k].SetCol(i,BezierSurface_Point_t1_t2(PtosControl,i*Step,k*Step));
190         }
191     }
192     return Surface;
193 }
194
195 /***** */
196
197 /***** */
198
199 void Write_BezierSurface(const std::vector<iMatrix<double>> &Tensor, std::string name){

```

```

200     std::ofstream fichero;
201     char coord[3]={'X','Y','Z'};
202     int iterj = Tensor[0].GetNumCols();
203     int iterk = Tensor.size();
204
205     for(unsigned int i=0; i<3;i++){
206         fichero.open(name+coord[i]+".txt");
207         for(unsigned int j=0; j<iterj;j++){
208             for(unsigned int k=0; k<iterk;k++){
209                 fichero<<Tensor[k](i,j)<<" ";
210             }
211             fichero<<std::endl;
212         }
213         fichero.close();
214     }
215 }
216
217 /***** */
218
219 /***** */
220 void Write_CtrlPts(const std::vector<iMatrix<double>> &CtrlPts, std::string name){
221     std::ofstream fichero;
222     char coord[3]={'X','Y','Z'};
223     int iterj = CtrlPts[0].GetNumCols();
224     int iterk = CtrlPts.size();
225
226     for(unsigned int i=0; i<3;i++){
227         fichero.open(name+coord[i]+".txt");
228         for(unsigned int j=0; j<iterj;j++){
229             for(unsigned int k=0; k<iterk;k++){
230                 fichero<<CtrlPts[k](i,j)<<" ";
231             }
232             fichero<<std::endl;
233         }
234         fichero.close();
235     }
236 }
237
238
239 }
240
241
242 /*****
243 *
244 * Classic funtions from the Nurbs Book
245 *
246 *
247 *****/
248
249 int FindSpan(const std::vector<double> &KnotVector, double t, int p, int n){
250
251     if (KnotVector[n+1]==t){
252         return n;
253     }
254
255     else{
256         int low =p;
257         int high=n+1;
258         int mid=(low+high)/2;
259         while(t<KnotVector[mid] || t>KnotVector[mid+1]){
260             if (t<double(KnotVector[mid])){
261                 //std::cout<<"Condición t<U[mid]"<< "t="<<t<< " mid="<<mid<< " , U[mid]="<< KnotVector[mid]<<std::endl;
262                 high=mid;
263             }
264             else{
265                 //std::cout<<"Condición t>=U[mid]"<< "t="<<t<< " mid="<<mid<< " , U[mid]="<< KnotVector[mid]<<std::endl;
266                 low=mid;
267             }
268             mid=(low+high)/2;
269         }
270         return mid;
271     }
272 }
273
274 /***** */
275
276 /***** */
277 std::vector<double> BasisFuns(int i, double t, int p,const std::vector<double> &KnotVector){
278     std::vector<double> sol(p+1);
279     std::vector<double> left(p+1);
280     std::vector<double> right(p+1);
281     double temp,saved;
282     //std::cout<<"Entramos en basisFuns"<<std::endl;
283     sol[0]=1.0f;
284     for(unsigned int j=1; j<=p; j++){
285         //std::cout<<"j: " << j<< std::endl;
286         left[j]=t-KnotVector[i+1-j];
287         right[j]=KnotVector[i+j]-t;
288         saved = 0.0f;
289         for(unsigned int r=0; r<j; r++){
290             //std::cout<<"r: " << r<< std::endl;
291             temp= sol[r]/(right[r+1]+left[j-r]);
292             sol[r]=saved+right[r+1]*temp;
293             saved=left[j-r]*temp;
294         }
295         sol[j]=saved;
296     }
297     //std::cout<<"Salimos de basisFuns"<<std::endl;
298     return sol;
299 }
300
301 /***** */
302

```

```

303  /***** */
304
305
306
307  iMatrix<double> AllBasisFuns(int i, double t, int p, const std::vector<double> &KnotVector){
308
309
310      iMatrix<double> sol(p+1,p+1);
311      std::vector<double> left(p+1);
312      std::vector<double> right(p+1);
313      sol(0,0)=1.0f;
314      double temp,saved;
315
316      for(unsigned int j=1; j<=p; j++){
317
318          left[j]=t-KnotVector[i+1-j];
319          right[j]=KnotVector[i+j]-t;
320          saved = 0.0f;
321          for(unsigned int r=0; r<j; r++){
322
323              temp= sol(j-1,r)/(right[r+1]+left[j-r]);
324              sol(j,r)=saved+right[r+1]*temp;
325              saved=left[j-r]*temp;
326          }
327          sol(j,j)=saved;
328
329      }
330
331      return sol;
332  }
333
334  /***** */
335
336  /***** */
337  iMatrix<double> DerBasisFuns(int span, double t, int p, int k, const std::vector<double> &KnotVector){
338      iMatrix<double> ders(k+1,p+1);
339      iMatrix<double> ndu(p+1,p+1);
340      double temp,saved;
341      ndu(0,0)=1.0f;
342      std::vector<double> left(p + 1), right(p + 1);
343
344      for (int j = 1; j <= p; ++j) {
345          left[j] = t - KnotVector[span + 1 - j];
346          right[j] = KnotVector[span + j] - t;
347          double saved = 0.0;
348
349          for (int r = 0; r < j; ++r) {
350              ndu(j,r)= right[r + 1] + left[j - r];
351              double temp= ndu(r,j-1)/ndu(j,r);
352              ndu(r,j)=saved + right[r + 1] * temp ;
353              saved = left[j - r] * temp;
354          }
355          ndu(j,j)=saved;
356      }
357
358      for (int j = 0; j <= p; ++j) {
359          ders(0,j)=ndu(j,p);
360      }
361
362
363      for (int r = 0; r <= p; ++r) {
364          int s1 = 0, s2 = 1;
365          iMatrix<double> a(2,p+1);
366          a(0,0)=1.0f;
367
368          for (int i = 1; i <= k; ++i) {
369              double d = 0.0;
370              int rk = r - 1, pk = p - i;
371              if (r >= i) {
372                  a(s2,0) = a(s1,0)/ndu(pk+1,rk);
373                  d = a(s2,0)*ndu(rk,pk);
374              }
375              int j1 = (rk >= -1) ? 1 : -rk;
376              int j2 = (r - 1 <= pk) ? i - 1 : p - r;
377
378              for (int j = j1; j <= j2; ++j) {
379                  a(s2,j)= (a(s1,j)-a(s1,j-1))/ndu(pk+1,rk+1) ;
380                  d += a(s2,j)*ndu(rk+j,pk);
381              }
382              if (r <= pk) {
383                  a(s2,i)= -a(s1,i-1)/ndu(pk+1,r) ;
384                  d += a(s2,i)*ndu(r,pk);
385              }
386              ders(i,r)=d;
387              std::swap(s1, s2);
388          }
389
390      }
391
392      // Ajustar multiplicadores
393      int r = p;
394      for (int i = 1; i <= k; ++i) {
395          for (int j = 0; j <= p; ++j) {
396              ders(i,j)= ders(i,j)*r;
397          }
398          r *= (p - i);
399      }
400
401      return ders;
402  }
403
404  /***** */
405

```

```

406  /***** */
407  void Write_BasisFunctions(const iMatrix<double> &BasisFunctions, std::string name){
408      std::ofstream fichero;
409      fichero.open(name+".txt");
410      for(unsigned int j=0; j<BasisFunctions.GetNumRows();j++){
411          for(unsigned int k=0; k<BasisFunctions.GetNumCols();k++){
412              fichero<<BasisFunctions(j,k)<<" ";
413          }
414          fichero<<std::endl;
415      }
416      fichero.close();
417  }
418
419  /***** */
420  *
421  * Classic funtions for p spline curves/surfaces
422  *
423  /***** */
424
425  std::vector<double> CurvePoint(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPts, double t){
426      int span =FindSpan(KnotVector, t, p, n);
427      std::vector<double> Funs = BasisFuns(span,t,p,KnotVector);
428
429      int itermax=CtrlPts.GetNumRows();
430      std::vector<double> C(itermax);
431      for(unsigned int i = 0; i<=p;i++){
432          double aux=Funs[i];
433
434          for(unsigned int j=0;j<itermax;j++){
435
436              C[j]=C[j]+ aux*(CtrlPts(j,span-p+i));
437
438          }
439      }
440
441      return C;
442  }
443
444  /***** */
445
446  /***** */
447  iMatrix<double> CurveDerivsAlg1(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPts, double t, int d){
448      int du= std::min(d,p);
449      int span =FindSpan(KnotVector, t, p, n);
450      int itermax=CtrlPts.GetNumRows();
451      //std::cout << "du " << du<< std::endl;
452      iMatrix<double> CK(du+1, itermax); //inicializa a 0 no hace falta dar valores
453      iMatrix<double> nders= DerBasisFuns(span, t, p, du,KnotVector);
454      for(unsigned int k=0; k<=du;k++){
455          for(unsigned int i=0; i<=p;i++){
456              double aux =nders(k,i);
457
458              //CK[k] = CK[k] + nders[k] [j]*P[span-p+j]
459              for(unsigned int j=0;j<itermax;j++){
460
461                  CK(k,j)+=aux*(CtrlPts(j,span-p+i));
462
463              }
464          }
465      }
466      return CK;
467  }
468
469  /***** */
470
471  /***** */
472
473  /***** */
474  iMatrix<std::vector<double>> CurveDerivsCpts(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPts, int d,
475      int r1, int r2){
476      int r= r2-r1;
477      iMatrix<std::vector<double>> sol(d+1,r+1);
478      for(unsigned int i=0;i<=r;i++){
479          sol(0,i)= CtrlPts.GetCol(r1+i);
480      }
481      for(unsigned int k=1;k<=d;k++){
482          int tmp = p-k+1;
483          for(unsigned int i=0;i<=r-k;i++){
484
485              int iter=CtrlPts.GetNumCols(); //dimension del espacio
486              std::vector<double> aux(iter);
487              double denominator=KnotVector[r1+i+p+1]-KnotVector[r1+i+k];
488
489              for(unsigned int j=0; j<iter; j++){
490                  aux[j]=tmp*(sol(k-1,i+1)[j]-sol(k-1,i)[j])/denominator;
491              }
492
493              sol(k,i)= aux ;
494          }
495      }
496
497      return sol;
498  }
499
500  /***** */
501
502  /***** */
503  iMatrix<double> CurveDerivsAlg2(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPts, double t, int d){
504      int du= std::min(d,p);
505      int span =FindSpan(KnotVector, t, p, n);
506      int itermax=CtrlPts.GetNumRows(); //dimension del espacio
507      iMatrix<double> CK(itermax, du); //inicializa a 0 no hace falta dar valores
508      iMatrix<double> nders= AllBasisFuns(span, t, p, KnotVector);

```

```

509     iMatrix<std::vector<double>> DervCtps=CurveDerivsCtps(n,p,KnotVector,CtrlPts,du,span-p,span);
510     for(unsigned int k=0; k<=du;k++){
511         for(unsigned int i=0; i<=p-k;i++){
512             double aux =nders(i,p-k);
513             for(unsigned int j=0; j<itermax;j++){
514
515                 CK(k,j)=CK(k,j)+aux*DervCtps(k,i)[j] ;
516
517             }
518         }
519     }
520     return CK;
521 }
522
523
524 /***** */
525
526 /***** */
527 void Write_Curve(const iMatrix<double> &Curve, std::string filename){
528     std::ofstream fichero;
529     int iterMaxi=Curve.GetNumCols();
530     int iterMaxj=Curve.GetNumRows();
531     fichero.open(filename);
532     for(unsigned int j=0; j<iterMaxj;j++){
533         for(unsigned int i=0; i<iterMaxi;i++){
534             fichero<<Curve(j,i)<<" ";
535         }
536         fichero<< std::endl;
537     }
538     fichero.close();
539 }
540
541 /***** */
542
543 /***** */
544 std::vector<double> SurfacePoint(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,
545     const std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2){
546     int span1=FindSpan(KnotVector1,t1,p1,n1);
547     int span2=FindSpan(KnotVector2,t2,p2,n2);
548     std::vector<double> Basis1=BasisFuns(span1,t1,p1,KnotVector1);
549     std::vector<double> Basis2=BasisFuns(span2,t2,p2,KnotVector2);
550     int ind1 = span1-p1;
551     std::vector<double> Pt= {0,0,0};
552     for(unsigned int l=0; l<=p2; l++){
553
554         std::vector<double> temp= {0,0,0};
555
556         int ind2= span2 - p2+l;
557         for(unsigned int k=0;k<=p1;k++){
558             double auxBasis1= Basis1[k];
559             //std::vector<double> auxCtrlPt = CtrlPts(ind1+k,ind2);
560             for(unsigned int i=0; i<3;i++){
561                 //std::cout<<"i="<<i<<"k="<<k<<"l="<<l<<std::endl;
562                 temp[i]=temp[i]+auxBasis1*CtrlPts[ind1+k](i,ind2);
563             }
564         }
565
566         double auxBasis2=Basis2[l];
567         for(unsigned int i=0; i<3;i++){
568             Pt[i]=Pt[i]+auxBasis2*temp[i];
569         }
570     }
571     return Pt;
572 }
573
574 /***** */
575
576 /***** */
577 iMatrix<std::vector<double>> SurfaceDerivsAlg1(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,
578     const std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2, int d){
579     unsigned int du=std::min(d,p1);
580     unsigned int dv=std::min(d,p2);
581
582     unsigned int dim = CtrlPts[0].GetNumRows();
583     //std::cout<<"dim: "<<dim<<std::endl;
584     iMatrix<std::vector<double>> Sol(du+1,dv+1);
585     const std::vector<double> zero(dim);
586     for(unsigned int k=0; k<=du; k++){
587         for(unsigned int j=0; j<=dv;j++){
588             Sol(k,j)= zero ;
589         }
590     }
591     //std::cout<<"2" <<std::endl;
592     int span1=FindSpan(KnotVector1,t1,p1,n1);
593     int span2=FindSpan(KnotVector2,t2,p2,n2);
594
595     iMatrix<double> Ders1 = DerBasisFuns(span1, t1, p1, du, KnotVector1);
596     iMatrix<double> Ders2 = DerBasisFuns(span2, t2, p2, dv, KnotVector2);
597     iMatrix<double> temp(dim,p2+1);
598     //std::cout<<"3" <<std::endl;
599     for(unsigned int k=0; k<=du; k++){
600         //std::cout<<"4" <<std::endl;
601
602         for(unsigned int s=0;s<=p2;s++){
603             //std::cout << "s: " << s << std::endl;
604             temp.SetCol(s,zero);
605             for(unsigned int r=0; r<=p1; r++){
606                 // std::cout<<"6" <<std::endl;
607                 //temp[s] = temp[s] + Nu[k] [r]*P[uspan-p+r] [vspan-q+s];
608                 double aux= Ders1(k,r);
609                 for(unsigned int j=0; j<dim;j++){
610                     // std::cout<<"7" <<std::endl;
611                     //std::cout<< CtrlPts[span1-p1+r](j,span2 -p2+s) << "|";

```

```

612         temp(j,s)= temp(j,s)+aux*CtrlPts[span1-p1+r](j,span2 -p2+s);
613     }
614
615
616     }
617     //std::cout << "-----" << std::endl;
618 }
619 //std::cout << "8" << std::endl;
620 int dd=std::min(d-k,dv);
621 for(unsigned int l=0; l<= dd; l++){
622     Sol(k,l)=zero;
623     for(unsigned int s=0;s<=p2;s++){
624         double aux = Ders2(l,s);
625         std::vector<double> auxSol= Sol(k,l);
626         std::vector<double> aux2(dim);
627         for(unsigned int j=0; j<dim; j++){
628             aux2[j]=auxSol[j]+aux*temp(j,s);
629         }
630         Sol(k,l)= aux2;
631     }
632 }
633 }
634 }
635
636     return Sol;
637 }
638
639 /*****
640 *
641 * Classic and support funtions for NURBS curves/surfaces
642 *
643 *****/
644 iMatrix<double> WeightCtrlPts(const iMatrix<double> &CtrlPts,const std::vector<double> &Weights){
645     int dim= CtrlPts.GetNumRows()+1;
646     int size = CtrlPts.GetNumCols();
647     iMatrix<double> CtrlPtsWeighted(dim,size);
648     for(unsigned int i=0; i<size;i++){
649         for(unsigned int j=0; j<dim-1; j++){
650             CtrlPtsWeighted(j,i)=CtrlPts(j,i)*Weights[i];
651         }
652         CtrlPtsWeighted(dim-1,i)=Weights[i];
653     }
654
655     return CtrlPtsWeighted;
656 }
657
658 /*****
659
660 *****/
661 iMatrix<double> UnWeightCtrlPts(const iMatrix<double> &CtrlPtsW){
662     int NumPts = CtrlPtsW.GetNumCols();
663     int dim = CtrlPtsW.GetNumRows()-1;
664     iMatrix<double> CtrlPts(dim,NumPts);
665     for(unsigned int j=0; j< NumPts;j++){
666         for(unsigned int i=0; i<dim; i++){
667             if(CtrlPtsW(dim,j)==0){
668                 //nothing, the declaracion automatically sets it to 0
669             }
670             else{
671                 CtrlPts(i,j)=CtrlPtsW(i,j)/CtrlPtsW(dim,j);
672             }
673         }
674     }
675     return CtrlPts;
676 }
677
678 /*****
679
680 *****/
681 /*****
682 std::vector<iMatrix<double>> WeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPts,const iMatrix<double> &Weights){
683     int dim = CtrlPts.size();
684     std::vector<iMatrix<double>> WeightedCtrlPts(dim);
685     for(unsigned int i = 0; i< dim; i++){
686         WeightedCtrlPts[i]=WeightCtrlPts(CtrlPts[i], Weights.GetRow(i));
687     }
688     return WeightedCtrlPts;
689 }
690
691 /*****
692
693 *****/
694 std::vector<iMatrix<double>> UnWeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPtsW){
695     int NumPts2= CtrlPtsW.size();
696     int NumPts1 = CtrlPtsW[0].GetNumCols();
697     int dim = CtrlPtsW[0].GetNumRows()-1;
698
699     std::vector<iMatrix<double>> CtrlPts(NumPts2);
700
701     for(unsigned int k=0; k<NumPts2; k++){
702         CtrlPts[k]=iMatrix<double>(dim, NumPts1);
703         //iMatrix<double> AuxMat (dim,NumPts1);
704
705         for(unsigned int j=0; j< NumPts1;j++){
706
707             for(unsigned int i=0; i<dim; i++){
708
709                 if(CtrlPtsW[k](dim,j)==0){
710
711                     //nothing, the declaracion automatically sets it to 0
712                 }
713                 else{
714

```

```

715         CtrlPts[k](i,j)=CtrlPtsW[k](i,j)/CtrlPtsW[k](dim,j);
716     }
717 }
718 }
719
720
721 }
722
723 return CtrlPts;
724
725 }
726
727 /***** */
728
729 /***** */
730 std::vector<double> CurvePointRational(int n, int p, const std::vector<double> &KnotVector,
731 const iMatrix<double> &CtrlPtsWeighted, double t){
732     int span = FindSpan(KnotVector, t, p, n);
733     std::vector<double> Funs = BasisFuns(span,t,p,KnotVector);
734     int dim= CtrlPtsWeighted.GetNumRows();
735     std::vector<double> Cw(dim, 0.0f);
736     for(unsigned int j=0; j<=p; j++){
737         //Cw=Cw + Funs[j]*CtrlPtsWeighted[span-p+j]
738         for(unsigned int i=0; i<dim; i++){
739             Cw[i]=Cw[i]+Funs[j]*CtrlPtsWeighted(i,span-p+j);
740         }
741     }
742     std::vector<double> C(dim-1);
743     double w = Cw[dim-1];
744
745     for(unsigned int i=0; i<dim-1; i++){
746         C[i]=Cw[i]/w;
747     }
748     return C;
749 }
750
751 /***** */
752
753 /***** */
754 int Binomial(int k, int i){
755     if (i < 0 || i > k) return 0;
756     if (i == 0 || i == k) return 1;
757     if (i > k - i) i = k - i; // simetria
758
759     long long res = 1;
760     for (int j = 1; j <= i; j++) {
761         res *= k - (i - j);
762         res /= j;
763     }
764     return (int)res;
765 }
766
767 /***** */
768
769 /***** */
770 std::vector<double> RationalBasisFuns(int i, double t, int p, const std::vector<double> &KnotVector,
771 const std::vector<double> &WeightVector){
772     std::vector<double> Funs = BasisFuns(i, t, p, KnotVector);
773     int itermax=Funs.size();
774     double coef=0.0f;
775     //std::cout<<itermax<<std::endl;
776     for(int j=0; j<itermax;j++){
777         //std::cout<<"a"<<std::endl;
778         coef=coef+Funs[j]*WeightVector[i-p+j];
779     }
780     for(int j=0; j<itermax;j++){
781         //std::cout<<"b"<<std::endl;
782         Funs[j]=Funs[j]*WeightVector[i-p+j]/coef;
783     }
784
785     return Funs;
786 }
787
788 /***** */
789
790 /***** */
791 iMatrix<double> RatDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector,
792 const std::vector<double> &WeightVector){
793     iMatrix<double> R_ders(k+1, p+1);
794     iMatrix<double> ders = DerBasisFuns(i, t, p, k, KnotVector);
795
796     std::vector<double> Wk(k+1, 0.0);
797     for (int j = 0; j <= k; ++j){
798         for (int iter = 0; iter <= p; iter++){
799             Wk[j] += WeightVector[i-p+iter] * ders(j,iter);
800             //std::cout << "Weight1: " << WeightVector[i-p+iter] << " ";
801         }
802     }
803     //std::cout << std::endl;
804
805     for (int a = 0; a < p+1; ++a) {
806         R_ders(0, a)= ders(0,a)*WeightVector[i-p+a]/Wk[0]; // (D(0, a) * weights[a]) / Wk[0];
807         for (int j = 1; j <= k; ++j) {
808             double sum = 0.0;
809             for (int m = 1; m <= j; ++m){
810                 sum += Binomial(j, m) * Wk[m] * R_ders(j - m, a);
811             }
812             double num = ders(j, a) * WeightVector[i-p+a] - sum;
813             R_ders(j, a)= num / Wk[0];
814         }
815     }
816 }
817

```

```

818     return R_ders;
819 }
820 }
821
822 /***** */
823
824 /***** */
825 iMatrix<double> RationalizeDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector,
826 const iMatrix<double> &ders, const std::vector<double> &WeightVector){
827     iMatrix<double> R_ders(k+1, p+1);
828
829
830     std::vector<double> Wk(k+1, 0.0);
831     for (int j = 0; j <= k; ++j){
832         for (int iter = 0; iter <= p; iter++){
833             Wk[j] += WeightVector[i-p+iter] * ders(j,iter);
834         }
835     }
836
837     for (int a = 0; a < p+1; ++a) {
838         R_ders(0, a) = ders(0,a)*WeightVector[i-p+a]/Wk[0]; // (D(0, a) * weights[a]) / Wk[0];
839         for (int j = 1; j <= k; ++j) {
840             double sum = 0.0;
841             for (int m = 1; m <= j; ++m){
842                 sum += Binomial(j, m) * Wk[m] * R_ders(j - m, a);
843             }
844             double num = ders(j, a) * WeightVector[i-p+a] - sum;
845             R_ders(j, a) = num / Wk[0];
846         }
847     }
848 }
849
850     return R_ders;
851 }
852 }
853
854 /***** */
855
856 /***** */
857 iMatrix<double> RatCurveDerivs(const iMatrix<double> &Aders, const iMatrix<double> &wders, int d){
858     int NumCols=Aders.GetNumCols();
859     iMatrix<double> CK(d+1,NumCols);
860     //CK[0]=Aders[0]/wders[0]
861     //std::vector<double> init(NumCols);
862     for(unsigned int k=0; k<NumCols; k++){
863         CK(0,k)=Aders(0,k)/wders(0,0);
864     }
865
866
867     for(unsigned int k =1; k<=d; k++){
868         std::vector<double> v = Aders.GetRow(k);
869         for(int i=1; i<=k; i++){
870             int Bin=Binomial(k,i);
871             //std::cout << "k: " << k <<std::endl;
872             //std::cout << "i: " << i <<std::endl;
873             //std::cout << "binomial: " << Bin <<std::endl;
874             //v = v - Binomial(k, i) * wders[i] * CK[k - i];
875             for(unsigned int j=0; j<NumCols; j++){
876                 v[j] -= Bin*wders(i,0)*CK(k-i,j);
877             }
878         }
879         //CK[k] = v / wders[0];
880         for(unsigned int j=0; j<NumCols; j++){
881             v[j] /= wders(0,0);
882         }
883         CK.SetRow(k,v);
884     }
885
886     return CK;
887 }
888 }
889
890 /***** */
891
892 /***** */
893 std::vector<double> SurfacePointRational(int n1, int p1, const std::vector<double> &KnotVector1, int n2, int p2,
894 const std::vector<double> &KnotVector2, const std::vector<iMatrix<double>> &CtrlPtsWeighted, double t1, double t2){
895     int span1=FindSpan(KnotVector1,t1,p1,n1);
896     int span2=FindSpan(KnotVector2,t2,p2,n2);
897     std::vector<double> Basis1=BasisFuns(span1,t1,p1,KnotVector1);
898     std::vector<double> Basis2=BasisFuns(span2,t2,p2,KnotVector2);
899     int ind1 = span1-p1;
900     int dim = CtrlPtsWeighted[0].GetNumRows();
901     std::vector<double> Pt(dim);
902     iMatrix<double> auxtemp(p2+1,dim);
903     for(unsigned int l=0; l<=p2; l++){
904
905         std::vector<double> temp(dim);
906
907         int ind2= span2 - p2+1;
908         for(unsigned int k=0;k<=p1;k++){
909             double auxBasis1= Basis1[k];
910             //std::vector<double> auxCtrlPt = CtrlPts(ind1+k,ind2);
911             for(unsigned int i=0; i<dim;i++){
912                 //std::cout<<"i="<<i<<","k="<<k<<","l="<<l<<std::endl;
913                 temp[i]=temp[i]+auxBasis1*CtrlPtsWeighted[ind1+k](i,ind2);
914             }
915         }
916         auxtemp.SetRow(l, temp);
917     }
918 }
919 }
920

```

```

921     for(unsigned int l=0; l<=p2;l++){
922         double auxBasis2=Basis2[l];
923         for(unsigned int i=0; i<dim;i++){
924             Pt[i]=Pt[i]+auxBasis2*auxtemp(l,i);
925         }
926     }
927
928     std::vector<double> C(dim-1);
929     double w = Pt[dim-1];
930
931     for(unsigned int i=0; i<dim-1; i++){
932         C[i]=Pt[i]/w;
933     }
934     return C;
935 }
936
937 /***** */
938
939 /***** */
940 iMatrix<std::vector<double>> RatSurfaceDerivs(const iMatrix<std::vector<double>> &Aders,const iMatrix<double> &wders,int d){
941
942     int dim=Aders(0,0).size();
943     iMatrix<std::vector<double>> SKL(d+1,d+1);
944
945     for(unsigned int k=0; k<=d; k++){
946         for(unsigned int l=0; l<=d-k;l++){
947             //v = Aders [k] [l] ;
948             std::vector<double> v = Aders(k,l);
949
950             for(unsigned int j=1; j<=l;j++){
951                 double binaux=Binomial(l,j);
952                 double Wdersaux=wders(0,j);
953
954
955
956                 for(unsigned int iter=0; iter < dim; iter++){
957
958                     //v = v - Bin[l] [j]*wders[0] [j]*SKL[k] [l-j];
959                     v[iter]=v[iter]-binaux*Wdersaux*SKL(k,l-j)[iter];
960
961                 }
962
963             }
964
965
966             for(unsigned int i=1; i<=k; i++){
967                 double binaux=Binomial(k,i);
968                 double Wdersaux=wders(i,0);
969
970                 for(unsigned int iter=0; iter < dim; iter++){
971                     //v = v - Bin[k] [i]*wders[i] [0]*SKL[k-i] [l];
972                     v[iter]=v[iter]-binaux*Wdersaux*SKL(k-i,l)[iter];
973                 }
974
975                 //v2 = 0.0;
976                 std::vector<double> v2(Aders(0,0).size());
977                 for(unsigned int j=1; j<=l;j++){
978                     binaux=Binomial(l,j);
979                     Wdersaux=wders(i,j);
980
981                     for(unsigned int iter=0; iter < dim; iter++){
982                         //v2 = v2 + Bin[l] [j]*wders[i] [j]*SKL[k-i] [l-j];
983                         v2[iter]=v2[iter]+binaux*Wdersaux*SKL(k-i,l-j)[iter];
984                     }
985
986                 }
987                 binaux=Binomial(k,i);
988                 for(unsigned int iter=0; iter < dim; iter++){
989                     //v = v - Bin[k] [i]*v2;
990                     v[iter]=v[iter]-binaux*v2[iter];
991                 }
992
993             }
994
995             //SKL[k] [l] = v/wders[0] [0];
996             std::vector<double> auxvec(dim);
997             for(unsigned int iter=0; iter < dim; iter++){
998                 auxvec[iter]=v[iter]/wders(0,0);
999             }
1000
1001             SKL(k,l)=auxvec;
1002         }
1003     }
1004
1005     return SKL;
1006 }
1007
1008 /*****
1009 *
1010 * Fundamental Geometric Algorithms and support functions
1011 *
1012 *****/
1013 int findInsertionIndex(const std::vector<double>& v, double e){
1014     auto it = std::upper_bound(v.begin(), v.end(), e);
1015     return it - v.begin()-1;
1016 }
1017
1018 /***** */
1019
1020 /***** */
1021
1022 int countOccurrences(const std::vector<double>& v, double e) {
1023     double eps = 1e-6f;

```

```

1024     int left = 0;
1025     int right = v.size() - 1;
1026     int index = -1;
1027
1028     //binary search
1029     while (left <= right) {
1030         int mid = left + (right - left) / 2;
1031         if (std::fabs(v[mid] - e) < eps) {
1032             index = mid;
1033             right = mid - 1;
1034         } else if (v[mid] < e) {
1035             left = mid + 1;
1036         } else {
1037             right = mid - 1;
1038         }
1039     }
1040
1041     if (index == -1) return 0; // Not found case
1042
1043     //linear count
1044     int count = 0;
1045     while (index + count < v.size() && std::fabs(v[index + count] - e) < eps) {
1046         count++;
1047     }
1048
1049     return count;
1050 }
1051
1052 /***** */
1053
1054 /***** */
1055 void FindSpanMult(const std::vector<double> &KnotVector, double t, int p, int n, int &k, int &s){
1056     double eps = 1e-6f;
1057     k=FindSpan(KnotVector,t,p,n);
1058     s=0;
1059     for (int i = k; i >= 0 && std::fabs(KnotVector[i] - t) < eps; --i) s++;
1060     for (int i = k + 1; i <= n + p + 1 && std::fabs(KnotVector[i] - t) < eps; ++i) s++;
1061 }
1062
1063 /***** */
1064
1065 /***** */
1066 iMatrix<double> CurveKnotsIns(int np, int p, std::vector<double> &KnotVector, const iMatrix<double> &CtrlPtsW,
1067     double u, int index, int s, int r){
1068     //int mp = np+p+1;
1069     int dim=CtrlPtsW.GetNumRows();
1070     int L;
1071     double alpha;
1072
1073     //New CtrlPts
1074     iMatrix<double> NewCtrlPtsW(dim, CtrlPtsW.GetNumCols()+r);
1075
1076     for(unsigned int i=0; i<=index-p; i++){
1077         NewCtrlPtsW.SetCol(i, CtrlPtsW.GetCol(i));
1078     }
1079
1080     for(unsigned int i=index-s; i<=np; i++){
1081         NewCtrlPtsW.SetCol(i+r, CtrlPtsW.GetCol(i));
1082     }
1083
1084     iMatrix<double> AuxCtrlPts(dim,p+1);
1085
1086     for(unsigned int i=0; i<=p-s; i++){
1087         AuxCtrlPts.SetCol(i, CtrlPtsW.GetCol(index-p+i));
1088     }
1089
1090     for(unsigned int j=1; j<=r; j++){
1091
1092         L=index-p+j;
1093
1094         for(int i=0; i<=p-j-s; i++){
1095             alpha= (u-KnotVector[L+i])/(KnotVector[i+index+1]-KnotVector[L+i]);
1096
1097             for(unsigned int iter=0; iter<dim; iter++){
1098
1099                 AuxCtrlPts(iter,i)=alpha*AuxCtrlPts(iter,i+1)+(1.0f-alpha)*AuxCtrlPts(iter,i);
1100             }
1101         }
1102
1103         NewCtrlPtsW.SetCol(L, AuxCtrlPts.GetCol(0));
1104
1105         NewCtrlPtsW.SetCol(index+r-j-s, AuxCtrlPts.GetCol(p-j-s));
1106     }
1107
1108     for(unsigned int i=L+1; i<=index-s; i++){
1109         NewCtrlPtsW.SetCol(i, AuxCtrlPts.GetCol(i-L));
1110     }
1111
1112     //Update KnotVector
1113     KnotVector.insert(KnotVector.begin()+index+1, r, u);
1114
1115     return NewCtrlPtsW;
1116 }
1117
1118 /***** */
1119
1120 /***** */
1121
1122 std::vector<double> CurvePntByCornerCut(int n, int p, std::vector<double> &KnotVector, iMatrix<double> &CtrlPtsW, double u){
1123     int dim = CtrlPtsW.GetNumRows()-1;
1124     std::vector<double> aux(dim+1);
1125
1126

```

```

1127     if(u==KnotVector[0]){
1128         std::vector<double> aux(dim);
1129         for(unsigned int i=0; i< dim; i++){
1130             aux[i]=CtrlPtsW(i,0)/CtrlPtsW(dim,0);
1131         }
1132         return aux;
1133     }
1134     if(u==KnotVector[n+p+1]){
1135         std::vector<double> aux(dim);
1136         for(unsigned int i=0; i< dim; i++){
1137             aux[i]=CtrlPtsW(i,n)/CtrlPtsW(dim,n);
1138         }
1139         return aux;
1140     }
1141     int k,s;
1142     FindSpanMult(KnotVector, u, p, n, k, s);
1143     int r = p-s;
1144     //std::cout<< "r: " << r << std::endl;
1145     iMatrix<double> AuxCtrlPts(dim+1,r+1);
1146     for(unsigned int i=0; i<=r; i++){
1147         AuxCtrlPts.SetCol(i,CtrlPtsW.GetCol(k-p+i));
1148     }
1149
1150
1151
1152     for(unsigned int j=1; j<=r; j++){
1153         for(unsigned int i=0; i<=r-j; i++){
1154             double alpha= (u-KnotVector[k-p+j+i])/(KnotVector[i+k+1]-KnotVector[k-p+j+i]);
1155
1156             for(unsigned int iter=0; iter< dim+1; iter++){
1157                 aux[iter]=alpha*AuxCtrlPts(iter,i+1)+(1.0f-alpha)*AuxCtrlPts(iter,i);
1158             }
1159             AuxCtrlPts.SetCol(i,aux);
1160             //AuxCtrlPts.PrintMatrix();
1161         }
1162     }
1163     std::vector<double> sol(dim);
1164
1165     for(unsigned int i=0; i<dim; i++){
1166         sol[i]=AuxCtrlPts(i,0)/AuxCtrlPts(dim,0);
1167     }
1168     return sol;
1169 }
1170
1171
1172
1173 /*****
1174
1175 *****/
1176
1177 /*****
1178 *****/
1179
1180 std::vector<iMatrix<double>> SurfaceKnotIns(int np1, int p1, std::vector<double> &KnotVector1, int np2, int p2,
1181 std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
1182 double uv, int index, int r, int s){
1183
1184     if(dir==true){//KnotVector1 will be updated
1185         int dim=CtrlPtsW[0].GetNumRows();
1186         iMatrix<double> Aux(dim,p1-s+1);
1187         int numCtrlPts1=CtrlPtsW.size()+1;
1188         int L;
1189         iMatrix<double> alpha(p1-s+1,r+1);
1190         std::vector<iMatrix<double>> NewCtrlPts(numCtrlPts1);
1191         for(unsigned int i=0; i<numCtrlPts1; i++){
1192             NewCtrlPts[i]=iMatrix<double>(dim, CtrlPtsW[0].GetNumCols());
1193         }
1194
1195         for(unsigned int j=1; j<=p1-j-s; j++){
1196             L=index-p1+j;
1197             for(unsigned int i=0; i<=p1-j-s; i++){
1198                 alpha(i,j)= (uv-KnotVector1[L+i])/(KnotVector1[i+index+1]-KnotVector1[L+i]);
1199             }
1200         }
1201
1202         for(unsigned int row=0; row<= np2; row++){
1203             for(unsigned int i=0; i<=index-p1; i++){
1204                 NewCtrlPts[i].SetCol(row,CtrlPtsW[i].GetCol(row));
1205             }
1206             for(unsigned int i=index-s; i<=np1; i++){
1207                 NewCtrlPts[i+r].SetCol(row,CtrlPtsW[i].GetCol(row));
1208             }
1209             for(unsigned int i=0; i<=p1-s; i++){
1210                 Aux.SetCol(i,CtrlPtsW[index-p1+i].GetCol(row));
1211             }
1212         }
1213
1214         for(unsigned int j=1; j<=r; j++){
1215             L=index-p1+j;
1216             for(unsigned int i=0; i<= p1-j-s; i++){
1217                 std::vector<double> aux(dim);
1218                 for(unsigned int iter=0; iter<dim; iter++){
1219                     aux[iter]= alpha(i,j)*Aux(iter,i+1)+(1.0f-alpha(i,j))*Aux(iter,i);
1220                 }
1221                 Aux.SetCol(i,aux);
1222             }
1223             NewCtrlPts[L].SetCol(row,Aux.GetCol(0));
1224             NewCtrlPts[index+r-j-s].SetCol(row,Aux.GetCol(p1-j-s));
1225         }
1226         for(unsigned int i=L+1; i<index-s; i++){
1227             NewCtrlPts[i].SetCol(row,Aux.GetCol(i-L));
1228         }
1229     }
1230     KnotVector1.insert(KnotVector1.begin()+index+1, r, uv);
1231     return NewCtrlPts;
1232 }
1233
1234 else{//KnotVector2 will be updated
1235     int dim = CtrlPtsW[0].GetNumRows();

```

```

1230     iMatrix<double> Aux(dim, p2 - s + 1);
1231     int numCtrlPts2 = CtrlPtsW[0].GetNumCols() + 1;
1232     int L;
1233     iMatrix<double> alpha(p2 - s + 1, r + 1);
1234     std::vector<iMatrix<double>> NewCtrlPts(CtrlPtsW.size());
1235
1236     for (unsigned int i = 0; i < CtrlPtsW.size(); i++) {
1237         NewCtrlPts[i] = iMatrix<double>(dim, numCtrlPts2);
1238     }
1239
1240     for (unsigned int j = 1; j <= p2 - j - s; j++) {
1241         L = index - p2 + j;
1242         for (unsigned int i = 0; i <= p2 - j - s; i++) {
1243             alpha(i, j) = (uv - KnotVector2[L + i]) / (KnotVector2[i + index + 1] - KnotVector2[L + i]);
1244         }
1245     }
1246
1247     for (unsigned int col = 0; col <= np1; col++) {
1248         for (unsigned int i = 0; i <= index - p2; i++) {
1249             NewCtrlPts[col].SetCol(i, CtrlPtsW[col].GetCol(i));
1250         }
1251         for (unsigned int i = index - s; i <= np2; i++) {
1252             NewCtrlPts[col].SetCol(i + r, CtrlPtsW[col].GetCol(i));
1253         }
1254
1255         for (unsigned int i = 0; i <= p2 - s; i++) {
1256             Aux.SetCol(i, CtrlPtsW[col].GetCol(index - p2 + i));
1257         }
1258
1259         for (unsigned int j = 1; j <= r; j++) {
1260             L = index - p2 + j;
1261             for (unsigned int i = 0; i <= p2 - j - s; i++) {
1262                 std::vector<double> aux(dim);
1263                 for (unsigned int d = 0; d < dim; d++) {
1264                     aux[d] = alpha(i, j) * Aux(d, i + 1) +
1265                         (1.0f - alpha(i, j)) * Aux(d, i);
1266                 }
1267                 Aux.SetCol(i, aux);
1268             }
1269
1270             NewCtrlPts[col].SetCol(L, Aux.GetCol(0));
1271             NewCtrlPts[col].SetCol(index + r - j - s, Aux.GetCol(p2 - j - s));
1272         }
1273
1274         for (unsigned int i = L + 1; i < index - s; i++) {
1275             NewCtrlPts[col].SetCol(i, Aux.GetCol(i - L));
1276         }
1277     }
1278
1279     KnotVector2.insert(KnotVector2.begin() + index + 1, r, uv);
1280
1281     return NewCtrlPts;
1282 }
1283
1284 }
1285
1286
1287 /***** */
1288
1289 /***** */
1290
1291 iMatrix<double> RefineKnotVectCurve(int n, int p, std::vector<double> &KnotVector,
1292     const iMatrix<double> &CtrlPtsW, const std::vector<double> &Insertions, int r){
1293     int m=n+p+1;
1294     int dim=CtrlPtsW.GetNumRows();
1295     std::vector<double> NewKnotVector(m+r+2);
1296     //int numCtrlPts=CtrlPtsW.GetNumCols();
1297     iMatrix<double> NewCtrlPts(dim,n+r+2);
1298     int a=FindSpan(KnotVector,Insertions[0],p,n);
1299     int b=FindSpan(KnotVector,Insertions[r],p,n)+1;
1300
1301     for(unsigned int j=0;j<=a-p;j++){
1302         NewCtrlPts.SetCol(j,CtrlPtsW.GetCol(j));
1303     }
1304     for(unsigned int j=b-1;j<=n;j++){
1305         NewCtrlPts.SetCol(j+r+1,CtrlPtsW.GetCol(j));
1306     }
1307     for(unsigned int j=0; j<=a;j++){
1308         NewKnotVector[j]=KnotVector[j];
1309     }
1310     for(unsigned int j=b+p; j<=m;j++){
1311         NewKnotVector[j+r+1]=KnotVector[j];
1312     }
1313
1314
1315     int i=b+p-1;
1316     int k=b+p+r;
1317     for(int j=r; j>=0; j--){
1318         while(Insertions[j]<=KnotVector[i]&&i>a){
1319             NewCtrlPts.SetCol(k-p-1, CtrlPtsW.GetCol(i-p-1));
1320             NewKnotVector[k]=KnotVector[i];
1321             k--;
1322             i--;
1323         }
1324         NewCtrlPts.SetCol(k-p-1,NewCtrlPts.GetCol(k-p));
1325         for(unsigned int l=1; l<=p;l++){
1326             int ind = k-p+1;
1327             double alpha=NewKnotVector[k+1]-Insertions[j];
1328             if (alpha==0.0){
1329                 NewCtrlPts.SetCol(ind-1,NewCtrlPts.GetCol(ind));
1330             }
1331             else{
1332                 alpha=alpha/(NewKnotVector[k+1]-KnotVector[i-p+1]);

```

```

1333         std::vector<double> aux(dim);
1334         for(unsigned int iter=0; iter<dim; iter++){
1335             aux[iter]=alpha*NewCtrlPts(iter,ind-1)+(1.0f-alpha)*NewCtrlPts(iter,ind);
1336         }
1337         NewCtrlPts.SetCol(ind-1,aux);
1338     }
1339 }
1340 }
1341 NewKnotVector[k]=Insertions[j];
1342 k--;
1343 }
1344 KnotVector=NewKnotVector;
1345 return NewCtrlPts;
1346 }
1347
1348 /***** */
1349
1350 /***** */
1351 std::vector<iMatrix<double>> RefineKnotVectSurface(int np1, int p1, std::vector<double> &KnotVector1,
1352 int np2, int p2, std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
1353 const std::vector<double> Insertions, int r){
1354     int dim=CtrlPtsW[0].GetNumRows();
1355     if(dir==true){
1356         //int L;
1357         int numCtrlPts1=CtrlPtsW.size()+r+1;
1358         int numCtrlPts2=CtrlPtsW[0].GetNumCols();
1359         int a=FindSpan(KnotVector1,Insertions[0],p1,np1);
1360         int b=FindSpan(KnotVector1,Insertions[r],p1,np1)+1;
1361         int m=np1+p1+1;
1362         std::vector<double> NewKnotVector(m+r+2);
1363         std::vector<iMatrix<double>> NewCtrlPts(numCtrlPts1);
1364         for(unsigned int i=0; i<numCtrlPts1; i++){
1365             NewCtrlPts[i]=iMatrix<double>(dim, numCtrlPts2);
1366         }
1367
1368         for(unsigned int row=0; row<=m;row++){
1369             for(unsigned int k=0; k<=a-p1;k++){
1370                 NewCtrlPts[k].SetCol(row,CtrlPtsW[k].GetCol(row));
1371             }
1372             for(unsigned int k=b-1; k<=np1; k++){
1373                 NewCtrlPts[k+r+1].SetCol(row,CtrlPtsW[k].GetCol(row));
1374             }
1375
1376
1377         }
1378         for(unsigned int j=0; j<=a;j++){
1379             NewKnotVector[j]=KnotVector1[j];
1380         }
1381         for(unsigned int j=b+p1; j<=m; j++){
1382             NewKnotVector[j+r+1]=KnotVector1[j];
1383         }
1384
1385         int i=b+p1-1;
1386         int k=b+p1+r;
1387
1388         for(int j=r; j>=0; j--){
1389             while(Insertions[j]<=KnotVector1[i]&&i>a){
1390                 for(unsigned int row=0; row<=m;row++){
1391                     NewCtrlPts[k-p1-1].SetCol(row,CtrlPtsW[i-p1-1].GetCol(row));
1392                 }
1393                 NewKnotVector[k]=KnotVector1[i];
1394                 k--;
1395                 i--;
1396             }
1397
1398             for(unsigned int row=0; row<=m;row++){
1399                 NewCtrlPts[k-p1-1].SetCol(row,NewCtrlPts[k-p1].GetCol(row));
1400             }
1401
1402             for(unsigned int l=1; l<=p1; l++){
1403                 int ind=k-p1+1;
1404                 double alpha=NewKnotVector[k+1]-Insertions[j];
1405                 if(alpha==0.0){
1406                     //NewCtrlPts.SetCol(ind-1,NewCtrlPts.GetCol(ind));
1407                     for(unsigned int row=0; row<=m;row++){
1408                         NewCtrlPts[ind-1].SetCol(row,NewCtrlPts[ind].GetCol(row));
1409                     }
1410                 }
1411                 else{
1412                     alpha=alpha/(NewKnotVector[k+1]-KnotVector1[i-p1+1]);
1413                     for(unsigned int row=0; row<=m;row++){
1414                         std::vector<double> aux(dim);
1415                         for(unsigned int iter=0; iter<dim; iter++){
1416                             aux[iter]=alpha*NewCtrlPts[ind-1](iter,row)+(1.0f-alpha)*NewCtrlPts[ind](iter,row);
1417                         }
1418                         NewCtrlPts[ind-1].SetCol(row,aux);
1419                     }
1420                 }
1421             }
1422
1423         }
1424         NewKnotVector[k]=Insertions[j];
1425         k--;
1426     }
1427
1428     KnotVector1=NewKnotVector;
1429     return NewCtrlPts;
1430 }
1431 else{
1432     int numCtrlPts1 = CtrlPtsW.size();
1433     int numCtrlPts2 = CtrlPtsW[0].GetNumCols() + r + 1;
1434     int a = FindSpan(KnotVector2, Insertions[0], p2, np2);
1435     int b = FindSpan(KnotVector2, Insertions[r], p2, np2) + 1;

```

```

1436     int m = np2 + p2 + 1;
1437
1438     std::vector<double> NewKnotVector(m + r + 2);
1439     std::vector<iMatrix<double>> NewCtrlPts(numCtrlPts1);
1440
1441     for (int i = 0; i < numCtrlPts1; i++) {
1442         NewCtrlPts[i] = iMatrix<double>(dim, numCtrlPts2);
1443     }
1444
1445     for (int col = 0; col <= a - p2; col++) {
1446         for (int i = 0; i < numCtrlPts1; i++) {
1447             NewCtrlPts[i].SetCol(col, CtrlPtsW[i].GetCol(col));
1448         }
1449     }
1450
1451     for (int col = b - 1; col <= np2; col++) {
1452         for (int i = 0; i < numCtrlPts1; i++) {
1453             NewCtrlPts[i].SetCol(col + r + 1, CtrlPtsW[i].GetCol(col));
1454         }
1455     }
1456
1457     for (int j = 0; j <= a; j++) {
1458         NewKnotVector[j] = KnotVector2[j];
1459     }
1460
1461     for (int j = b + p2; j <= m; j++) {
1462         NewKnotVector[j + r + 1] = KnotVector2[j];
1463     }
1464
1465     int i = b + p2 - 1;
1466     int k = b + p2 + r;
1467
1468     for (int j = r; j >= 0; j--) {
1469         while (Insertions[j] <= KnotVector2[i] && i > a) {
1470             for (int row = 0; row < numCtrlPts1; row++) {
1471                 NewCtrlPts[row].SetCol(k - p2 - 1, CtrlPtsW[row].GetCol(i - p2 - 1));
1472             }
1473             NewKnotVector[k] = KnotVector2[i];
1474             k--;
1475             i--;
1476         }
1477
1478         for (int row = 0; row < numCtrlPts1; row++) {
1479             NewCtrlPts[row].SetCol(k - p2 - 1, NewCtrlPts[row].GetCol(k - p2));
1480         }
1481
1482         for (int l = 1; l <= p2; l++) {
1483             int ind = k - p2 + l;
1484             double alpha = NewKnotVector[k + l] - Insertions[j];
1485             if (alpha == 0.0f) {
1486                 for (int row = 0; row < numCtrlPts1; row++) {
1487                     NewCtrlPts[row].SetCol(ind - 1, NewCtrlPts[row].GetCol(ind));
1488                 }
1489             } else {
1490                 alpha = alpha / (NewKnotVector[k + l] - KnotVector2[i - p2 + l]);
1491                 for (int row = 0; row < numCtrlPts1; row++) {
1492                     std::vector<double> aux(dim);
1493                     for (int iter = 0; iter < dim; iter++) {
1494                         aux[iter] = alpha * NewCtrlPts[row](iter, ind - 1) + (1.0f - alpha) * NewCtrlPts[row](iter, ind);
1495                     }
1496                     NewCtrlPts[row].SetCol(ind - 1, aux);
1497                 }
1498             }
1499         }
1500
1501         NewKnotVector[k] = Insertions[j];
1502         k--;
1503     }
1504
1505     KnotVector2 = NewKnotVector;
1506     return NewCtrlPts;
1507 }
1508
1509 }
1510
1511
1512 /*****
1513 *
1514 * Functions for the Assembly of the Matrix
1515 *
1516 *****/
1517 void legendre_pol(int n, Eigen::VectorXd &nodes, Eigen::VectorXd &weights) {
1518
1519     Eigen::VectorXd beta(n-1);
1520     for (int i = 0; i < n-1; i++) {
1521         double k = i + 1;
1522         beta(i) = k / std::sqrt(4.0 * k * k - 1);
1523     }
1524
1525     Eigen::MatrixXd J = Eigen::MatrixXd::Zero(n, n);
1526     for (int i = 0; i < n-1; i++) {
1527         J(i, i+1) = beta(i);
1528         J(i+1, i) = beta(i);
1529     }
1530
1531     Eigen::SelfAdjointEigenSolver<Eigen::MatrixXd> solver(J);
1532     nodes = solver.eigenvalues();
1533
1534     Eigen::MatrixXd V = solver.eigenvectors();
1535     weights.resize(n);
1536     for (int i = 0; i < n; i++) {
1537         weights(i) = 2.0 * V(0, i) * V(0, i);
1538     }

```

```

1539 }
1540
1541
1542 /***** */
1543
1544 /***** */
1545 std::vector<double> removeDuplicates(const std::vector<double>& input) {
1546     std::vector<double> result = input;
1547     auto last = std::unique(result.begin(), result.end());
1548     result.erase(last, result.end());
1549     return result;
1550 }
1551
1552 /***** */
1553
1554 /***** */
1555 std::vector<double> createSubIntervals(const double nElements, const double lower_limit, const double upper_limit){
1556     std::vector<double> sol(nElements+1);
1557     double Lenght = upper_limit - lower_limit;
1558     double Step=Lenght/nElements;
1559     for(unsigned int i=0; i<=nElements; i++){
1560         sol[i]=lower_limit + i*Step;
1561     }
1562     return sol;
1563 }
1564
1565 /***** */
1566
1567 /***** */
1568 std::vector<double> removeMatches(const std::vector<double>& v1, const std::vector<double>& v2 ){
1569     if (v1.empty()) return {};
1570
1571     double start = v1.front();
1572     double step = v1.size() > 1 ? v1[1] - v1[0] : 1;
1573
1574     std::vector<char> toRemove(v1.size(), 0);
1575
1576     for (size_t i = 0; i < v2.size(); ++i) {
1577         double val = v2[i];
1578         if (val < start) continue;
1579
1580         double diff = val - start;
1581         double idxReal = diff / step;
1582         long long idx = llround(idxReal);
1583
1584         if (std::fabs(idxReal - idx) > 1e-9) continue;
1585
1586         if (idx >= 0 && static_cast<size_t>(idx) < v1.size()) {
1587             toRemove[idx] = 1;
1588         }
1589     }
1590
1591     std::vector<double> result;
1592     result.reserve(v1.size());
1593     for (size_t i = 0; i < v1.size(); ++i) {
1594         if (!toRemove[i]) result.push_back(v1[i]);
1595     }
1596     return result;
1597 }
1598
1599
1600 /***** */
1601
1602 /***** */
1603 void D1_element_eval(int n, int i, int p, int nEvals, double lower_limit, double upper_limit, const std::vector<double> &KnotVector,
1604     const std::vector<double> &Weights, const Eigen::VectorXd &nodes, const iMatrix<double> &CtrlPtsW, std::function<double(double)> f,
1605     double &preEvalPoint, double &cumLenght, iMatrix<double> &BasisFunsEvals, iMatrix<double> &DerBasisFunsEvals,
1606     std::vector<double> & JacobianEvals, std::vector<double> & InverseJacobianEvals, std::vector<double> & funcEvals, double Lenght){
1607
1608     BasisFunsEvals.Resize(p+1,nEvals);
1609     DerBasisFunsEvals.Resize(p+1,nEvals);
1610     JacobianEvals.resize(nEvals);
1611     InverseJacobianEvals.resize(nEvals);
1612     funcEvals.resize(nEvals);
1613     double auxeval1= (upper_limit - lower_limit)/2;
1614     double auxeval2= (upper_limit + lower_limit)/2;
1615
1616     for(unsigned int k=0; k<nEvals; k++){
1617         //std::cout << k << std::endl;
1618         double EvalPoint = auxeval1*nodes(k)+auxeval2;
1619         //std::cout <<"EvalPoint: " << EvalPoint<< std::endl;
1620         iMatrix<double> AuxMat1 = RatDersBasisFuns(i,EvalPoint, p, 1, KnotVector, Weights);
1621         BasisFunsEvals.SetCol(k, AuxMat1.GetRow(0));
1622         DerBasisFunsEvals.SetCol(k, AuxMat1.GetRow(1));
1623         //std::cout << "i1" << std::endl;
1624         iMatrix<double> Aders, wders;
1625         iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, EvalPoint, 1);
1626         //std::cout << "i2" << std::endl;
1627         int auxInt=Allders.GetNumCols()-1;
1628         Aders = Allders.GetSubMat(0,1,0, auxInt-1);
1629         wders = Allders.GetSubMat(0,1,auxInt, auxInt);
1630         iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
1631         //std::cout << "i3" << std::endl;
1632
1633         std::vector<double> AuxVec= AuxMat2.GetRow(1);
1634         double AuxVal=0;
1635         for(unsigned int iter=0; iter< AuxVec.size(); iter++){
1636             AuxVal+= AuxVec[iter]*AuxVec[iter];
1637         }
1638         AuxVal=sqrt(AuxVal);
1639         //std::cout << "i4" << std::endl;
1640         JacobianEvals[k]=AuxVal;
1641     }

```

```

1642     InverseJacobianEvals[k]=1/AuxVal;
1643     double FisicalP=Fisical_to_Parametric(preEvalPoint, EvalPoint, 0, Lenght, 20, CtrlPtsW, KnotVector, n, p);
1644     cumLenght += FisicalP;
1645     preEvalPoint=EvalPoint;
1646     //std::cout <<std::setprecision(7) << "EvalPoint= " << cumLenght << std::endl;
1647     //funcEvals[k]=f(EvalPoint)/(M_PI *M_PI ); //Case Jacobian is constant at 1
1648     //funcEvals[k] = 10*10*M_PI*M_PI*sin(10*M_PI*CurvePointRational(n, p, KnotVector, CtrlPtsW, EvalPoint)[0]);
1649     //funcEvals[k] = f(CurvePointRational(n, p, KnotVector, CtrlPtsW, EvalPoint)[0]); //Case unidimensional curve
1650     funcEvals[k]= f(cumLenght); //Generic case (optimization needed)
1651 }
1652 }
1653 }
1654
1655 /***** */
1656
1657 /***** */
1658 iMatrix<double> gauss_legendre_cuadrature_integral_bilinearForm(int p, double lower_limit, double upper_limit, int nEvals,
1659     const iMatrix<double> &DerBasisFunsEvals, const std::vector<double> & InverseJacobianEvals, const Eigen::VectorXd &weights){
1660     iMatrix<double> sol(p+1,p+1);
1661     double AuxConstant = (upper_limit-lower_limit)/2;
1662
1663     for(unsigned int i=0; i<=p; i++){
1664         for(unsigned int j=0; j<=p; j++){
1665             double AuxVal=0;
1666             for(unsigned int iter=0; iter<nEvals; iter++){
1667                 AuxVal+=weights[iter]*DerBasisFunsEvals(i, iter)*DerBasisFunsEvals(j, iter)*InverseJacobianEvals[iter];
1668             }
1669             AuxVal=AuxConstant*AuxVal;
1670
1671             sol(i,j)=AuxVal;
1672             //sol(j,i,AuxVal);
1673         }
1674     }
1675     return sol;
1676 }
1677
1678 /***** */
1679
1680 /***** */
1681 std::vector<double> gauss_legendre_cuadrature_integral_linealForm(int p, double lower_limit, double upper_limit,
1682     int nEvals, const iMatrix<double> &BasisFunsEvals, const std::vector<double> & JacobianEvals,
1683     const std::vector<double> & funcEvals, const Eigen::VectorXd &weights){
1684     std::vector<double> sol(p+1);
1685     double AuxConstant = (upper_limit-lower_limit)/2;
1686
1687     for(unsigned int i=0; i<=p; i++){
1688         double AuxVal=0;
1689         for(unsigned int iter=0; iter<nEvals; iter++){
1690             AuxVal+=weights[iter]*BasisFunsEvals(i, iter)*JacobianEvals[iter]*funcEvals[iter];
1691         }
1692         sol[i]=AuxConstant*AuxVal;
1693     }
1694     return sol;
1695 }
1696
1697 /***** */
1698
1699 /***** */
1700 void impose_Dirichlet_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm,
1701     bool Modify0, bool Modify1, double ValueAt0, double ValueAt1){
1702     if (Modify0){
1703         global.coeffRef(0, 0) = 1.0;
1704         LinearForm(0)=ValueAt0;
1705         for(unsigned int i=1; i<=p; i++){
1706             global.coeffRef(i, 0) = 0.0;
1707             LinearForm(i)-=ValueAt0*global.coeffRef(0, i);
1708             global.coeffRef(0, i) = 0.0;
1709         }
1710     }
1711     if (Modify1){
1712         global.coeffRef(size, size) = 1.0;
1713         LinearForm(size)=ValueAt1;
1714         for(int i=size-1; i>=size-p; i--){
1715             global.coeffRef(size, i) = 0.0;
1716             LinearForm(i)-=ValueAt1*global.coeffRef(i, size);
1717             global.coeffRef(i, size) = 0.0;
1718         }
1719     }
1720 }
1721
1722 /***** */
1723
1724 /***** */
1725 void impose_Neumann_Condition(int size, Eigen::VectorXd &LinearForm, bool Modify0, bool Modify1, double ValueAt0, double ValueAt1){
1726     if (Modify0){
1727         LinearForm(0)-=ValueAt0;
1728     }
1729     if (Modify1){
1730         LinearForm(size)+=ValueAt1;
1731     }
1732 }
1733
1734 /***** */
1735
1736 /***** */
1737 void impose_Robin_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm,
1738     double alpha, double beta, bool Modify0, bool Modify1, double ValueAt0, double ValueAt1){
1739     double BetaInverse=1/beta;

```

```

1745
1746     if (Modify0){
1747         global.coeffRef(0, 0) -= alpha/beta;
1748         LinearForm(0) -= ValueAt0/beta;
1749     }
1750
1751     if (Modify1){
1752         global.coeffRef(size, size) += alpha/beta;
1753         LinearForm(0) += ValueAt1/beta;
1754     }
1755 }
1756
1757 /***** */
1758
1759 /***** */
1760
1761 void writeNumericFunction(Eigen::VectorXd funEvals, const std::vector<double> &KnotVector, const std::vector<double> &WeightVector, int n,
1762     int p, int nEvals, double lowerLimit, double upperLimit, std::string name, std::vector<double> Intervals, int nElements, double Lenght){
1763
1764     std::ofstream fichero1, fichero2;
1765     fichero1.open(name+ "_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p) + "_testCircle.txt");
1766     fichero2.open(name+ "Ders_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p) + "_testCircle.txt");
1767
1768     // Configurar alta precisión
1769     fichero1 << std::scientific << std::setprecision(15);
1770     fichero2 << std::scientific << std::setprecision(15);
1771     Eigen::VectorXd nodes, weights;
1772     legendre_pol(nEvals, nodes, weights);
1773
1774     for(unsigned int iter=0; iter<Intervals.size()-1; iter++){
1775
1776         double auxeval1= (Intervals[iter+1] - Intervals[iter])/2;
1777         double auxeval2= (Intervals[iter+1] + Intervals[iter])/2;
1778         //double preVal=Intervals[iter];
1779
1780
1781         for(unsigned int i=0; i<nEvals; i++){
1782             double t=0.5*(auxeval1)*nodes(i)+auxeval2;
1783
1784             int span= FindSpan(KnotVector, t, p, n);
1785             //std::cout << "span: " << span << std::endl;
1786             double AuxVal1=0;
1787             double AuxVal2=0;
1788             //std::vector<double> eval= RatDersBasisFuns(span, t, p, 0, KnotVector, WeightVector).GetRow(0);
1789             iMatrix<double> evalMat= RatDersBasisFuns(span, t, p, 1, KnotVector, WeightVector);
1790             std::vector<double> eval1=evalMat.GetRow(0);
1791             std::vector<double> eval2=evalMat.GetRow(1);
1792             //std::cout << "t= " << t << std::endl;
1793             for(int j=0; j<p; j++){
1794                 //std::cout << "funEvals[" << span-p+j << "] = " << funEvals[span-p+j] << std::endl;
1795                 //std::cout << "eval[" << j << "] = " << eval[j] << std::endl;
1796                 AuxVal1+=funEvals[span-p+j]*eval1[j];
1797                 AuxVal2+=funEvals[span-p+j]*eval2[j];
1798             }
1799             fichero1 << AuxVal1 << " ";
1800             fichero2 << AuxVal2 << " ";
1801         }
1802     }
1803     fichero1.close();
1804     fichero2.close();
1805 }
1806
1807 /***** */
1808
1809 /***** */
1810
1811 void writeAnalyticFunction(std::function<double(double)> f, std::function<double(double)> f_Ders, std::vector<double> Intervals,
1812     double nEvals, const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector, int n, int p,
1813     std::string name, double Lenght, double nElements){
1814
1815     std::ofstream fichero1, fichero2;
1816     fichero1.open(name+ "_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p) + "_Analytic_testCircle.txt");
1817     fichero2.open(name+ "_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p) + "Ders_Analytic_testCircle.txt");
1818
1819     // Configurar alta precisión
1820     fichero1 << std::scientific << std::setprecision(15);
1821     fichero2 << std::scientific << std::setprecision(15);
1822     double preVal=Intervals[0];
1823     double cumLenght=0;
1824     Eigen::VectorXd nodes, weights;
1825     legendre_pol(nEvals, nodes, weights);
1826
1827     for(unsigned int iter=0; iter<Intervals.size()-1; iter++){
1828
1829         double auxeval1= (Intervals[iter+1] - Intervals[iter])/2;
1830         double auxeval2= (Intervals[iter+1] + Intervals[iter])/2;
1831         //double preVal=Intervals[iter];
1832
1833         for(unsigned int i=0; i<nEvals; i++){
1834             iMatrix<double> Aders, wders;
1835             double t=0.5*(auxeval1)*nodes(i)+auxeval2;
1836             iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, t, 1);
1837             //std::cout << "i2" << std::endl;

```

```

1848     int auxInt=Allders.GetNumCols()-1;
1849     Aders = Allders.GetSubMat(0,1,0, auxInt-1);
1850     wders = Allders.GetSubMat(0,1,auxInt, auxInt);
1851     iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
1852     //std::cout << "i3" << std::endl;
1853
1854     std::vector<double> AuxVec= AuxMat2.GetRow(1);
1855     double Jacobian=0;
1856     for(unsigned int i=0; i< AuxVec.size(); i++){
1857         Jacobian+= AuxVec[i]*AuxVec[i];
1858     }
1859     Jacobian=sqrt(Jacobian);
1860     double FisicalP=Fisical_to_Parametric(preVal, t, 0, Lenght, 20, CtrlPtsW, KnotVector, n, p);
1861     cumLenght+=FisicalP;
1862     //double auxval = CurvePointRational(n, p, KnotVector, CtrlPtsW, lowerLimit + i*Step)[0];
1863     //std::cout << "point is: "<< lowerLimit + (auxeval1*nodes(i)+auxeval2) << std::endl;
1864     //fichero << f(auxeval1*nodes(i)+auxeval2) << " "; //case Jacobian is constant at value 1
1865     //fichero << f(CurvePointRational(n, p, KnotVector, CtrlPtsW, auxeval1*nodes(i)+auxeval2)[0])
1866     //<< " "; //case jacobian is not constant but f only takes 1 dimensional parameters
1867     fichero1 << f(cumLenght)<< " "; //optimization needed, not necessary to
1868                                     //compute the lagranje pol pts in each iteration
1869     fichero2 << f_Ders(cumLenght)*Jacobian<< " ";
1870     preVal=t;
1871     //fichero1 << f(auxeval1*nodes(i)+auxeval2)<< " ";
1872     //optimization needed, not necessary to compute the lagranje pol pts in each iteration
1873     //fichero2 << f_Ders(auxeval1*nodes(i)+auxeval2)*Jacobian<< " ";
1874     //std::cout << CurvePointRational(n, p, KnotVector, CtrlPtsW, time[i])[0] << std::endl;
1875     //fichero1 << f(CurvePointRational(n, p, KnotVector, CtrlPtsW, time[i])[0]) << " ";
1876     //fichero2 << f_Ders(CurvePointRational(n, p, KnotVector, CtrlPtsW, time[i])[0])*Jacobian << " ";
1877
1878     }
1879 }
1880 fichero1.close();
1881 fichero2.close();
1882 std::cout << "Longitud total arco recorrida: " << cumLenght << std::endl;
1883
1884 }
1885
1886 /***** */
1887
1888 /***** */
1889 std::vector<std::vector<double>> leerArchivo(const std::string& name, int p) {
1890     std::vector<std::vector<double>> resultado;
1891     std::ifstream archivo(name);
1892
1893     if (!archivo.is_open()) {
1894         std::cerr << "Error: No se pudo abrir el archivo " << name << std::endl;
1895         return resultado;
1896     }
1897
1898     std::string linea;
1899     std::vector<double> filaActual;
1900
1901     std::stringstream buffer;
1902     buffer << archivo.rdbuf();
1903     std::string contenido = buffer.str();
1904
1905     std::istringstream ss(contenido);
1906     double numero;
1907
1908     while (ss >> numero) {
1909         filaActual.push_back(numero);
1910
1911         if (filaActual.size() == p) {
1912             resultado.push_back(filaActual);
1913             filaActual.clear();
1914         }
1915     }
1916
1917     if (!filaActual.empty()) {
1918         resultado.push_back(filaActual);
1919     }
1920
1921     archivo.close();
1922     return resultado;
1923 }
1924 /***** */
1925
1926 /***** */
1927 /*
1928 void writeFunctionDers(Eigen::VectorXd funEvals,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPtsW ,
1929                      const std::vector<double> &WeightVector, int n, int p, int nEvals, double lowerLimit, double upperLimit, std::string name){
1930     std::ofstream fichero;
1931     fichero.open(name);
1932     Eigen::VectorXd nodes, weights;
1933     legendre_pol(nEvals, nodes, weights);
1934     double auxeval1= (upperLimit - lowerLimit)/2;
1935     double auxeval2= (upperLimit + lowerLimit)/2;
1936
1937
1938     //double Step=(upperLimit-lowerLimit)/(funEvals.size()-p);
1939     for(unsigned int i=0; i<nEvals; i++){
1940         double t=auxeval1*nodes(i)+auxeval2;
1941         //std::cout << "i4" << std::endl;
1942
1943         int span= FindSpan(KnotVector, t, p, n);
1944         double AuxVal=0;
1945         std::vector<double> eval= RatDersBasisFuns(span, t , p, 1, KnotVector, WeightVector).GetRow(1);
1946         for(int j=0; j<p; j++){
1947             AuxVal+=funEvals[span-p+j]*eval[j];
1948         }
1949
1950         fichero << AuxVal << " ";

```

```

1951     }
1952     fichero.close();
1953 }
1954 */
1955 /***** */
1956
1957 /***** */
1958 /*
1959 void writeFunctionDers(std::function<double(double)> f, double lowerLimit, double upperLimit, double nEvals, const iMatrix<double> &CtrlPtsW,
1960                      const std::vector<double> &KnotVector, int n, int p, std::string name, double Lenght){
1961     std::ofstream fichero;
1962     fichero.open(name);
1963     Eigen::VectorXd nodes, weights;
1964     legendre_pol(nEvals, nodes, weights);
1965     double auxeval1= (upperLimit - lowerLimit)/2;
1966     double auxeval2= (upperLimit + lowerLimit)/2;
1967     for(unsigned int i=0; i< nEvals; i++){
1968         double t= auxeval1*nodes(i)+auxeval2;
1969         iMatrix<double> Aders, wders;
1970         iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, t, 1);
1971         //std::cout << "i2" << std::endl;
1972         int auxInt=Allders.GetNumCols()-1;
1973         Aders = Allders.GetSubMat(0,1,0, auxInt-1);
1974         wders = Allders.GetSubMat(0,1,auxInt, auxInt);
1975         iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
1976         //std::cout << "i3" << std::endl;
1977
1978         std::vector<double> AuxVec= AuxMat2.GetRow(1);
1979         double Jacobian=0;
1980         for(unsigned int i=0; i< AuxVec.size(); i++){
1981             Jacobian+= AuxVec[i]*AuxVec[i];
1982         }
1983         Jacobian=sqrt(Jacobian);
1984         //double auxVal = f(CurvePointRational(n, p, KnotVector, CtrlPtsW, t)[0])*Jacobian; //Case curve is 1 dimensional
1985         //std::cout << "tders = " << Fisical_to_Parametric(t, 0, Lenght, p+2, CtrlPtsW, KnotVector, n, p) << std::endl;
1986         double auxVal = f(Fisical_to_Parametric(t, 0, Lenght, 20, CtrlPtsW, KnotVector, n, p))*Jacobian; //Generic case
1987
1988         //double auxval = CurvePointRational(n, p, KnotVector, CtrlPtsW, lowerLimit + i*Step)[0];
1989         //std::cout << "point is: "<< lowerLimit + (auxeval1*nodes(i)+auxeval2) << std::endl;
1990
1991         //fichero << f(auxeval1*nodes(i)+auxeval2) << " "; //case Jacobian is constant at value 1
1992         fichero << auxVal << " "; //case jacobian is not constant but f only takes 1 dimensional parameters
1993     }
1994
1995     fichero.close();
1996 }
1997 */
1998
1999 /***** */
2000
2001 /***** */
2002 double IntegrateNormDer(double lowerLimit, double upperLimit, int nEvals, const iMatrix<double> &CtrlPtsW,
2003                      const std::vector<double> &KnotVector, int n, int p){
2004     Eigen::VectorXd nodes, weights;
2005     legendre_pol(nEvals, nodes, weights);
2006     double auxeval1= (upperLimit - lowerLimit)/2;
2007     double auxeval2= (upperLimit + lowerLimit)/2;
2008     double sol=0;
2009
2010     for(unsigned int i=0; i< nEvals; i++){
2011
2012         double t= auxeval1*nodes(i)+auxeval2;
2013         //std::cout << "t= " << t << std::endl;
2014         iMatrix<double> Aders, wders;
2015         //std::cout << "i0" << std::endl;
2016         iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, t, 1);
2017         //std::cout << "i1" << std::endl;
2018         //Allders.PrintMatrix();
2019         //std::cout << "i2" << std::endl;
2020         int auxInt=Allders.GetNumCols()-1;
2021         Aders = Allders.GetSubMat(0,1,0, auxInt-1);
2022         wders = Allders.GetSubMat(0,1,auxInt, auxInt);
2023         iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
2024         //std::cout << "i3" << std::endl;
2025
2026         std::vector<double> AuxVec= AuxMat2.GetRow(1);
2027         double Jacobian=0;
2028         for(unsigned int iter=0; iter< AuxVec.size(); iter++){
2029             Jacobian+= AuxVec[iter]*AuxVec[iter];
2030         }
2031         Jacobian=sqrt(Jacobian);
2032         sol+=Jacobian*weights[i];
2033
2034         //double auxval = CurvePointRational(n, p, KnotVector, CtrlPtsW, lowerLimit + i*Step)[0];
2035         //std::cout << "point is: "<< lowerLimit + (auxeval1*nodes(i)+auxeval2) << std::endl;
2036         //fichero << f(auxeval1*nodes(i)+auxeval2) << " "; //case Jacobian is constant at value 1
2037
2038     }
2039
2040     return sol*auxeval1;
2041 }
2042
2043 /***** */
2044
2045 /***** */
2046 std::vector<std::vector<iMatrix<double>>> PreBasis_and_Ders(int p, int k, std::vector<double> KnotVector,
2047                 std::vector<double> Intervals, Eigen::VectorXd nodes, std::vector<int> & spanIndex){
2048     int nIntervals =Intervals.size()-1;
2049     int nEvals=nodes.size();
2050     int n= KnotVector.size()-p-2;
2051     std::vector<std::vector<iMatrix<double>>> sol(nIntervals);
2052     for(unsigned int i=0; i< nIntervals; i++){
2053         double lower_limit=Intervals[i];

```

```

2054     double upper_limit=Intervals[i+1];
2055     double auxeval1= (upper_limit - lower_limit)/2;
2056     double auxeval2= (upper_limit + lower_limit)/2;
2057     int span= FindSpan(KnotVector, auxeval2, p, n);
2058     spanIndex[i]=span;
2059     std::vector<iMatrix<double>> auxVec(nEvals);
2060     for(unsigned int j=0; j<nEvals; j++){
2061         double EvalPoint = auxeval1*nodes(j)+auxeval2;
2062
2063         //std::cout <<"EvalPoint: " << EvalPoint<< std::endl;
2064         auxVec[j] = DerBasisFuns(span,EvalPoint, p, k, KnotVector);
2065     }
2066     sol[i]=auxVec;
2067 }
2068 return sol;
2069 }
2070
2071 /***** */
2072
2073 /***** */
2074 std::vector<iMatrix<double>> RationalizeDersBasisFunsVec(int span, int nEvals, Eigen::VectorXd nodes,
2075 double lower_limit, double upper_limit, int p, int k, int n, std::vector<double> KnotVector,
2076 std::vector<iMatrix<double>> Basis_and_DersY, std::vector<double> Weights){
2077     std::vector<iMatrix<double>> sol(nEvals);
2078     double auxeval1= (upper_limit - lower_limit)/2;
2079     double auxeval2= (upper_limit + lower_limit)/2;
2080     for(unsigned int iter=0; iter<nEvals; iter++){
2081         double EvalPoint = auxeval1*nodes(iter)+auxeval2;
2082         sol[iter]=RationalizeDersBasisFuns(span, auxeval1*nodes(iter)+auxeval2, p, k, KnotVector, Basis_and_DersY[iter], Weights);
2083     }
2084     return sol;
2085 }
2086
2087 /***** */
2088
2089 /***** */
2090 std::unordered_map<int, std::vector<std::pair<double, double>>> BuildSpanMap(const std::vector<double>& Intervals, const std::vector<int>& spanIndices){
2091     std::unordered_map<int, std::vector<std::pair<double, double>>> map;
2092     size_t nIntervals = spanIndices.size(); // debe ser nodes.size() - 1
2093
2094     for (size_t i = 0; i < nIntervals; ++i) {
2095         int span = spanIndices[i];
2096         std::pair<double, double> interval = { Intervals[i], Intervals[i+1] }; // extremos reales
2097         map[span].push_back(interval);
2098     }
2099
2100     return map;
2101 }
2102
2103 /***** */
2104
2105 /***** */
2106 iMatrix<iMatrix<double>> RationalizeDersBasisFuns2D(const iMatrix<double> &DersBasisX, const iMatrix<double> &DersBasisY,
2107 const iMatrix<double> &activeWeights, unsigned int pX, unsigned int pY) {
2108     // Matriz de salida 2x2 de matrices
2109     iMatrix<iMatrix<double>> Rationalized(2, 2);
2110
2111     // Inicializamos las submatrices (pY+1)x(pX+1)
2112     for (int i = 0; i < 2; ++i){
2113         for (int j = 0; j < 2; ++j){
2114             Rationalized(i, j).Resize(pY + 1, pX + 1);
2115         }
2116     }
2117
2118     // Calcular la superficie de pesos y sus derivadas
2119     double W = 0.0;
2120     double Wx = 0.0;
2121     double Wy = 0.0;
2122
2123     for (unsigned int j = 0; j <= pY; ++j)
2124     {
2125         for (unsigned int i = 0; i <= pX; ++i)
2126         {
2127             double w = activeWeights(j, i);
2128             double Nx = DersBasisX(0, i);
2129             double Ny = DersBasisY(0, j);
2130             double Nx_x = DersBasisX(1, i);
2131             double Ny_y = DersBasisY(1, j);
2132
2133             W += Nx * Ny * w;
2134             Wx += Nx_x * Ny * w;
2135             Wy += Nx * Ny_y * w;
2136         }
2137     }
2138
2139     // Racionalización de funciones base y derivadas
2140     for (unsigned int j = 0; j <= pY; ++j)
2141     {
2142         for (unsigned int i = 0; i <= pX; ++i)
2143         {
2144             double w = activeWeights(j, i);
2145             double Nx = DersBasisX(0, i);
2146             double Ny = DersBasisY(0, j);
2147             double Nx_x = DersBasisX(1, i);
2148             double Ny_y = DersBasisY(1, j);
2149
2150             double N = Nx * Ny * w;
2151
2152             // Función base racionalizada
2153             Rationalized(0, 0)(j, i) = N / W;
2154
2155             // Derivada respecto a X
2156             Rationalized(1, 0)(j, i) = w * (Nx_x * Ny * W - Nx * Ny * Wx) / (W * W);

```

```

2157         // Derivada respecto a Y
2158         Rationalized(0, 1)(j, i) = w * (Nx * Ny_y * W - Nx * Ny * Wy) / (W * W);
2159     }
2160 }
2161
2162     return Rationalized;
2163 }
2164
2165 /*****
2166 /*****
2167 void ExportToTxt(const iMatrix<double>& SolEvals, const iMatrix<std::vector<double>>& Surf, const std::string& filename){
2168     std::ofstream file(filename);
2169     if (!file.is_open()) {
2170         std::cerr << "Error abriendo archivo para escritura: " << filename << std::endl;
2171         return;
2172     }
2173
2174     size_t nx = SolEvals.GetNumRows();
2175     size_t ny = SolEvals.GetNumCols();
2176     //std::cout << "1";
2177     for (size_t i = 0; i < nx; ++i) {
2178         //std::cout << "2";
2179         for (size_t j = 0; j < ny; ++j) {
2180
2181             const auto& xyz = Surf(i, j);
2182             //std::cout << xyz.size();
2183             double val = SolEvals(i, j);
2184             file<< std::setprecision(std::numeric_limits<double>::max_digits10) << xyz[0] << " " << xyz[1] << " " << xyz[2] << " " << val << "\n";
2185         }
2186     }
2187
2188     file.close();
2189     std::cout << "Datos exportados correctamente a " << filename << std::endl;
2190 }
2191
2192

```

A.7.4. Curvas de NURBS

Código del dibujo de las curvas de NURBS

```

1  RUTA_ARCHIVO_CURVA = PATH + 'EjCurva3d.txt';
2  NOMBRE_ARCHIVO_SVG = 'CurvaNurbs_3D_Plot.svg';
3
4  PC = [
5      10, 0, 0;
6      15, 10, 5;
7      5, 15, 20;
8      0, 5, 10;
9      10, 0, 0
10 ];
11
12 try
13     D = load(RUTA_ARCHIVO_CURVA);
14 catch
15     error('ERROR: No se pudo cargar el archivo. Comprueba que la ruta sea correcta: %s',
16         ↳ RUTA_ARCHIVO_CURVA);
17 end
18 [M, N] = size(D);
19
20 C = D';
21
22 figure;
23 hold on;
24 grid on;
25
26
27 plot3(C(:,1), C(:,2), C(:,3), 'b-', 'LineWidth', 2);
28 plot3(PC(:,1), PC(:,2), PC(:,3), 'r--', 'LineWidth', 1, 'DisplayName', 'Polígono de Control');
29 plot3(PC(:,1), PC(:,2), PC(:,3), 'ro', 'MarkerSize', 4, 'MarkerFaceColor', 'r', 'DisplayName',
30     ↳ 'Puntos de Control');
31
32 min_coords = min([C; PC]);
33 max_coords = max([C; PC]);
34 margen = 2;

```

```

34 axis([min_coords(1)-margen max_coords(1)+margen ...
35       min_coords(2)-margen max_coords(2)+margen ...
36       min_coords(3)-margen max_coords(3)+margen]);
37
38 view(3);
39 set(gca, 'Box', 'on');
40 print(NOMBRE_ARCHIVO_SVG, '-dsvg');
41 disp(['Gráfico final guardado en: ', fullfile(pwd, NOMBRE_ARCHIVO_SVG)]);

```

Código de las derivadas de las funciones base de NURBS

```

1  int p=3;
2  std::vector<double> Nurbs={0,0,0,0,0.5,1,1,1,1};
3  std::vector<double> weights={1,1,5,1,1};
4
5  double N_Steps = 1000;
6  std::vector<double> eval(Nurbs.size()-p);
7  iMatrix<double> ders(p+1,p+1);
8  iMatrix<double> sol(Nurbs.size()-p,N_Steps+1);
9  //std::vector<double> Nurb(NurbsVector.size());
10 double Step=1/(N_Steps);
11 auto start = std::chrono::high_resolution_clock::now();
12 for(unsigned int i = 0; i<N_Steps+1; i++){
13
14     double t=Step*i;
15     //std::cout << "Iteración iniciada " << i<< std::endl;
16     int span = FindSpan(Nurbs, t, p, Nurbs.size()-2-p);
17     ders = RatDersBasisFuns(span,t,p,1,Nurbs, weights);
18     ders.PrintMatrix();
19     std::cout<<"-----"<<std::endl;
20     for(int j = span-p; j<=span;j++){
21         if(j>=0) {
22             sol(j,i)=ders(1,j-span+p);
23         }
24         else{
25             //Caso error.
26         }
27     }
28 }
29 }
30
31 Write_BasisFunctions(sol, "path+name");
32 auto end = std::chrono::high_resolution_clock::now();
33 auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start);
34
35 std::cout << "Tiempo tomado: " << duration.count() << " nanosegundos\n";

```

Código de las funciones base de NURBS

```

1  int p=3;
2  double N_Steps = 1000;
3  std::vector<double> KnotVector={0,0,0,0,0.5,1,1,1,1};
4  std::vector<double> WeightVector={1,1,5,1,1};
5  iMatrix<double> sol(KnotVector.size()-p-1,N_Steps+1);
6
7  double Step=1/(N_Steps);
8  for(unsigned int i = 0; i<N_Steps+1; i++){
9      double t=Step*i;
10     int span = FindSpan(KnotVector, t, p, KnotVector.size()-2-p);
11     std::vector<double> eval= RationalBasisFuns(span, t , p, KnotVector, WeightVector);
12     for(int j = span-p; j<=span;j++){
13         if(j>=0) {

```

```

14         sol(j,i)=eval[j-span+p];
15     }
16     else{
17         //Caso error.
18     }
19 }
20 }
21 Write_BasisFunctions(sol, "path+name");

```

A.7.5. Superficies de NURBS

Código del dibujo de las superficies de NURBS

```

1  dataX = dlmread(PATH + 'SuperficieRationalEj2X.txt', ' ');
2  dataY = dlmread(PATH + 'SuperficieRationalEj2Y.txt', ' ');
3  dataZ = dlmread(PATH + 'SuperficieRationalEj2Z.txt', ' ');
4
5  hold on
6  surf(dataX, dataY, dataZ);
7
8  dataCtrlX = dlmread(PATH + 'Ej2X.txt', ' ');
9  dataCtrlY = dlmread(PATH + 'Ej2Y.txt', ' ');
10 dataCtrlZ = dlmread(PATH + 'Ej2Z.txt', ' ');
11
12 [n1 n2] = size(dataCtrlX);
13
14 for i=1:n1
15     plot3(dataCtrlX(i,:),dataCtrlY(i,:),dataCtrlZ(i,:), color='r');
16 endfor
17
18 for j=1:n2
19     plot3(dataCtrlX(:,j),dataCtrlY(:,j),dataCtrlZ(:,j), color='r');
20 endfor
21
22 scatter3(dataCtrlX(:), dataCtrlY(:), dataCtrlZ(:), 30, 'k', 'filled', 'MarkerEdgeColor', 'w');
23 view(30, 45);
24 grid on;
25
26 print -dsvg 'Superficie_NURBS_Ej1.svg';
27 hold off;

```

Código del cálculo de las superficies de NURBS

```

1  std::vector<iMatrix<double>> CtrlPts = ReadDataFile_CtrlPts("path+nameCtrlPts");
2  iMatrix<double> Weights =ReadDataFile_Matrix("path+nameWeights");
3  std::vector<iMatrix<double>> CtrlPtsW=WeightCtrlPtsSurf(CtrlPts,Weights);
4  std::vector<double> V1={0,0,0,0.5,0.5,1,1,1};
5  std::vector<double> V2={0,0,0,0,1,1,1,1};
6  int p1=2;
7  int p2=3;
8  double N_Steps_1 = 25;
9  double N_Steps_2 = 25;
10
11 double Step_1=1/N_Steps_1;
12 double Step_2=1/N_Steps_2;
13
14 std::vector<iMatrix<double>> eval(N_Steps_1+1);
15 for(unsigned int i=0; i<N_Steps_1+1;i++){
16
17     iMatrix<double> auxMatrix(3,N_Steps_2+1);
18     for(unsigned int j=0; j< N_Steps_2+1;j++){
19

```

```

20         std::vector<double> aux = SurfacePointRational(V1.size()-2-p1,p1,V1,
21                                                         V2.size()-2-p2,p2,V2,CtrlPtsW,i*Step_1,j*Step_2);
22         for(unsigned int k=0; k<3;k++){
23             auxMatrix(k,j)=aux[k];
24         }
25     }
26     eval[i]=auxMatrix;
27 }
28
29 Write_BezierSurface(eval,"path+nameSurf" );
30 Write_CtrlPts(CtrlPts,"path+nameCtrlPts");

```

A.7.6. Implementaciones de IGA

Caso 1D, Solución

```

1
2
3 bool PrintMatixes=false;
4 bool PrintIntermediateValues=false;
5 bool PrintPrevalues=false;
6
7
8 int p=2;
9 double nElements = 512;
10 //variables que no cambian en cada ejecución del bucle o que se declaran fuera para ir actualizandolas
11 iMatrix<double> CtrlPts = ReadDataFile_Matrix(PATH + "EjCirculo.txt");
12 if(PrintPrevalues){
13     std::cout<< "-----CtrlPts-----" << std::endl;
14     CtrlPts.PrintMatrix();
15 }
16 std::vector<double> KnotVector={0,0,0,0.5,0.5,1,1,1};
17 double lower_limit=KnotVector[0];
18 double upper_limit=KnotVector[KnotVector.size()-1];
19 std::vector<double> Weights={1,sqrt(2)/2,1,sqrt(2)/2,1};
20
21
22 iMatrix<double> WeightedCtrlPts=WeightCtrlPts(CtrlPts,Weights);
23
24 if(PrintPrevalues){
25     std::cout<< "-----WeightedCtrlPts-----" << std::endl;
26     WeightedCtrlPts.PrintMatrix();
27 }
28 int n= KnotVector.size()-p-2;
29 if(PrintPrevalues){
30     std::cout<< "-----KnotVectorOld-----" << std::endl;
31     for(unsigned int i=0; i<KnotVector.size(); i++){
32         std::cout << std::setprecision(7) << KnotVector[i] << " ";
33     }
34     std::cout << std::endl;
35     std::cout << "n value:" << n << std::endl;
36 }
37 std::vector<double> Insertions=createSubIntervals(nElements, lower_limit, upper_limit);
38 Insertions=removeMatches(Insertions, KnotVector);
39 iMatrix<double> NewCtrlPts=RefineKnotVectCurve(n,p,KnotVector, WeightedCtrlPts, Insertions, Insertions.size()-1);
40 if(PrintPrevalues){
41     std::cout<< "-----NewCtrlPtsW-----" << std::endl;
42     NewCtrlPts.PrintMatrix(7);
43 }
44 int size = KnotVector.size()-p-1;
45 std::function<double(double)> f = [](double x) {
46     return 100 std::sin( 10*M_PI * x);
47 };
48
49 auto analyticSol = [](double x){
50     //double pi= std::numbers::pi;
51     return sin(10*M_PI*x);
52 };
53
54 auto analyticSolDers = [](double x){
55     //double pi= std::numbers::pi;
56     return 10*cos(10*M_PI*x);
57 };
58
59 n = KnotVector.size()-p-2;
60
61
62 if(PrintPrevalues){
63     std::cout<< "-----KnotVectorNew-----" << std::endl;
64     for(unsigned int i=0; i<KnotVector.size(); i++){
65         std::cout << std::setprecision(7) << KnotVector[i] << " ";
66     }
67 }
68 //double Lenght=IntegrateNormDer(lower_limit,upper_limit, 5000, NewCtrlPts, KnotVector, n, p);
69 double Lenght = M_PI;
70 std::cout << std::endl;
71 std::cout << "n value:" << n << std::endl;
72
73 Weights=NewCtrlPts.GetRow(NewCtrlPts.GetNumRows()-1);
74 if(PrintPrevalues){
75     std::cout<< "-----Weights-----" << std::endl;

```

```

76     for(unsigned int i=0; i<Weights.size(); i++){
77         std::cout <<std::setprecision(7) << Weights[i] << " , ";
78     }
79     std::cout << std::endl;
80 }
81 int dim=NewCtrlPts.GetNumRows()-1;
82 int numElements= NewCtrlPts.GetNumCols();
83
84 int nEvals=p+1;
85
86
87 Eigen::SparseMatrix<double> global(size, size);
88 Eigen::VectorXd LinearForm(size);
89 LinearForm.setZero(); // Inicializa en cero
90
91 std::vector<double> Intervals=removeDuplicates(KnotVector);
92
93
94 if(PrintPreValues){
95     std::cout<< "-----Intervals-----" << std::endl;
96     for(unsigned int i=0; i<Intervals.size(); i++){
97         std::cout << Intervals[i] << " , ";
98     }
99     std::cout << std::endl;
100 }
101 Eigen::VectorXd nodes, weights;
102 legendre_pol(nEvals, nodes, weights);
103 if(PrintPreValues){
104     std::cout << "Nodos (raices):\n" << nodes.transpose() << "\n";
105     std::cout << "Pesos:\n" << weights.transpose() << "\n";
106 }
107 typedef Eigen::Triplet<double> T;
108 std::vector<Eigen::Triplet<double>> tripletList;
109 double preEvalPoint=lower_limit;
110 double cumLenght=0;
111
112 //iteraciones
113 auto start = std::chrono::high_resolution_clock::now();
114 for(unsigned int index=0; index<Intervals.size()-1; index++){
115     double lowerLimit=Intervals[index], upperLimit=Intervals[index+1];
116     int span = FindSpan(KnotVector, lowerLimit,p,n);
117     std::vector<int> global_indices(p+1);
118     for(unsigned int w=0; w<p; w++){
119         global_indices[w]=span-p+w;
120     }
121     if(PrintIntermediateValues){
122
123         std::cout << "global_indices: (" ;
124         for(unsigned int v=0; v<p; w++){
125             std::cout <<global_indices[w] << " , ";
126         }
127         std::cout << global_indices[p] << ")" <<std::endl;
128         std::cout << "Evaluation Interval: (" << Intervals[index] << " , " << Intervals[index+1]<< ")" <<std::endl;
129         std::cout<< "-----" << std::endl;
130     }
131
132     iMatrix<double> BasisFunsEvals;
133     iMatrix<double> DerBasisFunsEvals;
134     std::vector<double> JacobianEvals;
135     std::vector<double> InverseJacobianEvals;
136     std::vector<double> funcEvals;
137
138     D1_element_eval(n, span,p,nEvals, lowerLimit, upperLimit, KnotVector, Weights, nodes, NewCtrlPts,
139         f, preEvalPoint, cumLenght, BasisFunsEvals, DerBasisFunsEvals, JacobianEvals, InverseJacobianEvals, funcEvals, Lenght);
140     if(PrintIntermediateValues){
141         std::cout<< "-----BasisFunsEvals-----" << std::endl;
142         BasisFunsEvals.PrintMatrix(7);
143         std::cout<< "-----DerBasisFunsEvals-----" << std::endl;
144         DerBasisFunsEvals.PrintMatrix(7);
145         std::cout<< "-----JacobianEvals-----" << std::endl;
146         for(unsigned int i=0; i<JacobianEvals.size(); i++){
147             std::cout<< std::setprecision(7) << JacobianEvals[i] << " , ";
148         }
149         std::cout << std::endl;
150         std::cout<< "-----InverseJacobianEvals-----" << std::endl;
151         for(unsigned int i=0; i<InverseJacobianEvals.size(); i++){
152             std::cout<< std::setprecision(7) << InverseJacobianEvals[i] << " , ";
153         }
154         std::cout << std::endl;
155         std::cout<< "-----funcEvals-----" << std::endl;
156         for(unsigned int i=0; i<funcEvals.size(); i++){
157             std::cout<< std::setprecision(7) << funcEvals[i] << " , ";
158         }
159         std::cout << std::endl;
160         std::cout<< "-----Element-----" << std::endl;
161     }
162
163     iMatrix<double> Element=gauss_legendre_cuadrature_integral_bilinearForm(p,lowerLimit,upperLimit,nEvals, DerBasisFunsEvals, InverseJacobianEvals, weights);
164
165     if(PrintIntermediateValues){
166         Element.PrintMatrix(7);
167         std::cout<< "-----" << std::endl;
168     }
169
170     for (int i = 0; i <= p; ++i){
171         for (int j = 0; j <= p; ++j){
172             tripletList.push_back(T(global_indices[i], global_indices[j], Element(i, j)));
173         }
174     }
175
176     std::vector<double> LinearElement=gauss_legendre_cuadrature_integral_linealForm(p,lowerLimit, upperLimit, nEvals,
177         BasisFunsEvals, JacobianEvals, funcEvals, weights);
178

```

```

179     if(PrintIntermediateValues){
180         std::cout<< "-----LinearElement-----" << std::endl;
181         for(unsigned int i=0; i<LinearElement.size(); i++){
182             std::cout<< std::setprecision(7) << LinearElement[i] << ", ";
183         }
184         std::cout << std::endl;
185     }
186     for(unsigned int i=0; i<=p; i++){
187         LinearForm(global_indices[i])+=LinearElement[i];
188     }
189 }
190
191 global.setFromTriplets(tripletList.begin(), tripletList.end());
192
193
194 if(PrintMatixes){
195     std::cout<< "-----GlobalMat-----" << std::endl;
196     for (int k = 0; k < global.outerSize(); ++k){
197         for (Eigen::SparseMatrix<double>::InnerIterator it(global, k); it; ++it){
198             std::cout << "(" << it.row() << ", " << it.col() << "): " << it.value() << "\n";
199         }
200     }
201     //std::cout<<Eigen::MatrixXd(global) <<std::endl;
202     std::cout<< "-----GlobalLinearForm-----" << std::endl;
203     for (int k = 0; k < LinearForm.size(); ++k){
204         std::cout << LinearForm[k] << ", ";
205     }
206     std::cout << std::endl;
207 }
208
209 //Compute of the Inverse jacobians for boundary conditions on dirhclet/Robin conditions
210 iMatrix<double> Aders, wders;
211 iMatrix<double> Allders0 = CurveDerivsAlg1(n, p, KnotVector, NewCtrlPts, 0, 1);
212 int auxInt=Allders0.GetNumCols()-1;
213 Aders = Allders0.GetSubMat(0,1,0, auxInt-1);
214 wders = Allders0.GetSubMat(0,1,auxInt, auxInt);
215 iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
216 std::vector<double> AuxVec= AuxMat2.GetRow(1);
217 double JacobianEvals0=0;
218 for(unsigned int i=0; i< AuxVec.size(); i++){
219     JacobianEvals0+= AuxVec[i]*AuxVec[i];
220 }
221
222
223 double InverseJacobianEvals0=1/JacobianEvals0;
224
225
226 iMatrix<double> Allders1 = CurveDerivsAlg1(n, p, KnotVector, NewCtrlPts, 1, 1);
227 Aders = Allders1.GetSubMat(0,1,0, auxInt-1);
228 wders = Allders1.GetSubMat(0,1,auxInt, auxInt);
229 AuxMat2 = RatCurveDerivs(Aders, wders, 1);
230 AuxVec= AuxMat2.GetRow(1);
231 double JacobianEvals1=0;
232 for(unsigned int i=0; i< AuxVec.size(); i++){
233     JacobianEvals1+= AuxVec[i]*AuxVec[i];
234 }
235 double InverseJacobianEvals1=1/JacobianEvals1;
236
237 //apply boundary conditions
238 impose_Dirichlet_Condition(p, size-1, global, LinearForm, true, true);
239 //impose_Neumann_Condition(size-1, LinearForm, true, true);
240 //double alpha = 3;
241 //double beta= 5;
242 //impose_Robin_Condition(p, size-1, global, LinearForm, alpha, beta, true, false);
243
244 if(PrintMatixes){
245     std::cout<< "-----GlobalMatPostCond-----" << std::endl;
246     for (int k = 0; k < global.outerSize(); ++k){
247         for (Eigen::SparseMatrix<double>::InnerIterator it(global, k); it; ++it){
248             std::cout << setprecision(std::numeric_limits<double>::max_digits10) << "(" << it.row() << ", " << it.col() << "): " << it.value() << "\n";
249         }
250     }
251 }
252
253 if(PrintMatixes){
254     std::cout<< "-----GlobalLinearFormPostCond-----" << std::endl;
255     for (int k = 0; k < LinearForm.size(); ++k){
256         std::cout << setprecision(std::numeric_limits<double>::max_digits10) << LinearForm[k] << ", ";
257     }
258     std::cout << std::endl;
259 }
260
261
262 Eigen::SparseLU<Eigen::SparseMatrix<double>> solver;
263 solver.compute(global);
264 if(solver.info() != Eigen::Success) {
265     std::cout << "Error 1" << std::endl;
266 }
267
268 Eigen::VectorId u = solver.solve(LinearForm);
269 if(solver.info() != Eigen::Success) {
270     std::cout << "Error 2" << std::endl;
271 }
272 auto end = std::chrono::high_resolution_clock::now();
273 auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start);
274 std::cout << "Tiempo tomado: " << duration.count() << " nanosegundos\n";
275
276
277 //std::cout<<"Longitud de la curva= "<< setprecision(std::numeric_limits<double>::max_digits10) << Lenght <<std::endl;
278 std::string Text=PATH + "EjSinNonConst"; //_h=" + to_string(static_cast<int>(nElements)) + "_p="+ to_string(p)+"_test2.txt"
279
280 writeNumericFunction(u, KnotVector, Weights, n, p, nEvals, lower_limit, upper_limit, Text,Intervals, nElements, M_PI);
281 //std::cout<<"Longitud de la curva= "<< setprecision(std::numeric_limits<double>::max_digits10) << Lenght <<std::endl;

```

```

282 writeAnalyticFunction(analyticSol, analyticSolDers, Intervals, nEvals, NewCtrlPts, KnotVector, n, p, Text, Lenght, nElements);
283
284 //std::cout<<"Longitud de la curva= "<< setprecision(std::numeric_limits<double>::max_digits10) << cumLenght <<std::endl;
285
286 iMatrix<double> sol(dim,nEvals*nElements);
287 std::vector<double> eval(dim);
288
289 for(unsigned int iter = 0; iter<nElements; iter++){
290     double auxeval1= (Intervals[iter+1] - Intervals[iter])/2;
291     double auxeval2= (Intervals[iter+1] + Intervals[iter])/2;
292
293     for(unsigned int j=0; j<nEvals; j++){
294         double t=0.5*(auxeval1)*nodes(j)+auxeval2;
295         //std::cout << i << std::endl;
296         eval=CurvePointRational(KnotVector.size()-2-p,p,KnotVector,NewCtrlPts,t);
297         //std::cout << eval[0] << "|" << eval[1] << std::endl;
298         sol.SetCol(iter*nEvals + j,eval);
299     }
300 }
301
302 Write_Curve(sol,PATH + "CurvaEjCircle_h="+ std::to_string(static_cast<int>(nElements)) + "_p=" + to_string(p) + ".txt");
303

```

Caso1D, Cálculo de Normas

```

1
2 int h_val = 32768;
3 int p_val = 2;
4 std::string nameAnalytic=PATH + "EjSinNonConst_h="+ to_string(h_val) + "_p=" + to_string(p_val) + "_Analytic_testCircle.txt";
5 std::string nameNumeric=PATH + "EjSinNonConst_h="+ to_string(h_val) + "_p=" + to_string(p_val) + "_testCircle.txt";
6 std::string nameAnalyticDers=PATH + "EjSinNonConst_h="+ to_string(h_val) + "_p=" + to_string(p_val) + "Ders_Analytic_testCircle.txt";
7 std::string nameNumericDers=PATH + "EjSinNonConstDers_h="+ to_string(h_val) + "_p=" + to_string(p_val) + "_testCircle.txt";
8 std::vector<std::vector<double>> AnalyticSol = leerArchivo(nameAnalytic, p_val+1);
9 std::vector<std::vector<double>> NumericSol = leerArchivo(nameNumeric, p_val+1);
10 std::vector<std::vector<double>> AnalyticSolDers = leerArchivo(nameAnalyticDers, p_val+1);
11 std::vector<std::vector<double>> NumericSolDers = leerArchivo(nameNumericDers, p_val+1);
12
13 Eigen::VectorXd nodes, weights;
14 legendre_pol(p_val+1, nodes, weights);
15
16 std::vector<double> Intervals = createSubIntervals(h_val, 0, 1);
17
18 double L2sqr=0;
19 double H1sqr=0;
20 double cumSum=0;
21 double cumSumDers=0;
22
23
24 for(unsigned int iter=0; iter<h_val; iter++){
25     double a = Intervals[iter];
26     double b = Intervals[iter+1];
27     double h_elemento = (b - a)/2.0;
28
29     cumSum=0;
30     cumSumDers=0;
31
32     for(unsigned int i=0; i<p_val; i++){
33         cumSum+=(AnalyticSol[iter][i]-NumericSol[iter][i])*(AnalyticSol[iter][i]-NumericSol[iter][i])*weights(i)* h_elemento;
34         cumSumDers+=(AnalyticSolDers[iter][i]-NumericSolDers[iter][i])*(AnalyticSolDers[iter][i]-NumericSolDers[iter][i])*weights(i)* h_elemento;
35     }
36
37     L2sqr+=cumSum;
38     H1sqr+=cumSumDers;
39 }
40
41 double L2norm=std::sqrt(L2sqr);
42 double H1norm=std::sqrt(L2sqr+H1sqr);
43 std::cout <<"Norma L2=" << L2norm << std::endl;
44 std::cout <<"Norma H1=" << H1norm << std::endl;
45 std::cout<< "fin 29 " << std::endl;
46

```

Caso 2D

```

1 bool PrintMatrixes=false;
2 bool PrintMatrixes2=false;
3 bool ShowValues=false;
4 bool ShowIterations=false;
5 bool ShowIterations2=false;
6 bool ShowConditions=false;
7 bool ShowSolutions=false;
8
9 //Initial definitions of variables
10 int pY= 2;
11 int pX= 2;
12 int hY= 512;
13 int hX= 512;
14 int nEvalsY=pY+1;
15 int nEvalsX=pX+1;
16 int subMatSize=nEvalsX*nEvalsY;
17 int nElementsY=hY*pY;
18 int nElementsX=hX*pX;
19 int size = nElementsY*nElementsX;
20
21 Eigen::SparseMatrix<double> global(size, size);

```

```

22 Eigen::VectorXd LinearForm(size);
23 std::vector<double> KnotVectorY = {0,0,0,1,1,1};
24 std::vector<double> KnotVectorX = {0,0,0,1,1,1};
25
26 double lower_limitY=KnotVectorY[0];
27 double upper_limitY=KnotVectorY[KnotVectorY.size()-1];
28 double lower_limitX=KnotVectorX[0];
29 double upper_limitX=KnotVectorX[KnotVectorX.size()-1];
30
31
32
33 auto analyticSol = [](double x, double y){
34     //double pi=3.141592653589793;
35     return (1-x*x-y*y)/4;
36 };
37
38 auto analyticSol_dx = [](double x, double y) {
39     //double pi = 3.141592653589793;
40     return -2*x/4;
41 };
42
43 auto analyticSol_dy = [](double x, double y) {
44     //double pi = 3.141592653589793;
45     return -2*y/4;
46 };
47
48 auto f = [](double x, double y){
49     //double pi=3.141592653589793;
50     return 1;
51 };
52
53 std::vector<double> data={1,std::sqrt(2)/2,1,
54                          std::sqrt(2)/2,1,std::sqrt(2)/2,
55                          1,std::sqrt(2)/2,1,
56
57                          };
58
59 iMatrix<double> Weights(3,3, data.data());
60 std::vector<iMatrix<double>> CtrlPts = ReadDataFile_CtrlPts(PATH + "EjSuperficieCircle.txt");
61 std::vector<iMatrix<double>> CtrlPtsW=WeightCtrlPtsSurf(CtrlPts,Weights);
62 Eigen::VectorXd nodesY, nodesX, weightsY, weightsX;
63 legendre_pol(nEvalsY, nodesY, weightsY);
64 legendre_pol(nEvalsX, nodesX, weightsX);
65
66
67 if(ShowValues){
68     std::cout<< "-----KnotVectorY-----" << std::endl;
69     for(unsigned int i=0; i<KnotVectorY.size(); i++){
70         std::cout << KnotVectorY[i] << ", ";
71     }
72     std::cout << std::endl;
73
74     std::cout<< "-----KnotVectorX-----" << std::endl;
75     for(unsigned int i=0; i<KnotVectorX.size(); i++){
76         std::cout << KnotVectorX[i] << ", ";
77     }
78     std::cout << std::endl;
79
80
81     std::cout << "lower_limitY= " << lower_limitY << std::endl;
82     std::cout << "upper_limitY= " << upper_limitY << std::endl;
83     std::cout << "lower_limit2= " << lower_limitX << std::endl;
84     std::cout << "upper_limit2= " << upper_limitX << std::endl;
85
86     std::cout<<"---CtrlPts---" << std::endl;
87     for(unsigned int i=0; i<CtrlPts.size(); i++){
88         CtrlPts[i].PrintMatrix();
89         std::cout<<"-----" << std::endl;
90     }
91     std::cout<<"---Matrix_Pesos---" << std::endl;
92     Weights.PrintMatrix();
93     std::cout<<"---CtrlPtsW---" << std::endl;
94
95     for(unsigned int i=0; i<CtrlPtsW.size(); i++){
96         CtrlPtsW[i].PrintMatrix();
97         std::cout<<"-----" << std::endl;
98     }
99 }
100
101 std::vector<double> InsertionsY=createSubIntervals(hY, lower_limitY, upper_limitY);
102 InsertionsY=removeMatches(InsertionsY, KnotVectorY);
103
104
105 std::vector<double> InsertionsX=createSubIntervals(hX, lower_limitX, upper_limitX);
106 InsertionsX=removeMatches(InsertionsX, KnotVectorX);
107
108
109
110 std::vector<iMatrix<double>> NewCtrlPtsWintermediate= RefineKnotVectSurface(KnotVectorY.size()-2-pY,pY,KnotVectorY,KnotVectorX.size()-2-pX,pX,
111                                                                    KnotVectorX, CtrlPtsW, true, InsertionsY, InsertionsY.size()-1);
112 std::vector<iMatrix<double>> NewCtrlPtsW= RefineKnotVectSurface(KnotVectorY.size()-2-pY,pY,KnotVectorY,KnotVectorX.size()-2-pX,pX,
113                                                                    KnotVectorX, NewCtrlPtsWintermediate, false, InsertionsX, InsertionsX.size()-1);
114
115
116 NewCtrlPtsWintermediate.clear();
117
118 iMatrix<double> NewWeights(NewCtrlPtsW.size(),NewCtrlPtsW[0].GetNumCols());
119 for(unsigned int i=0; i<NewWeights.GetNumRows(); i++){
120     for(unsigned int j=0; j<NewWeights.GetNumCols(); j++){
121         NewWeights(i,j)=NewCtrlPtsW[i](3,j);
122     }
123 }
124 std::vector<double> IntervalsY=removeDuplicates(KnotVectorY);

```

```

125 std::vector<double> IntervalsX=removeDuplicates(KnotVectorX);
126
127
128
129
130
131 if(ShowValues){
132
133     std::cout<< "-----InsertionsY-----" << std::endl;
134     for(unsigned int i=0; i<InsertionsY.size(); i++){
135         std::cout << InsertionsY[i] << " ";
136     }
137     std::cout << std::endl;
138
139     std::cout<< "-----InsertionsX-----" << std::endl;
140     for(unsigned int i=0; i<InsertionsX.size(); i++){
141         std::cout << InsertionsX[i] << " ";
142     }
143     std::cout << std::endl;
144
145
146     std::cout<<"-----NewCtrlPtsW-----" << std::endl;
147     for(unsigned int i=0; i<NewCtrlPtsW.size(); i++){
148         NewCtrlPtsW[i].PrintMatrix();
149         std::cout<<"-----" << std::endl;
150     }
151
152     std::cout<<"-----NewWeights-----" << std::endl;
153     NewWeights.PrintMatrix();
154
155     std::cout<< "-----NewKnotVectorY-----" << std::endl;
156     for(unsigned int i=0; i<KnotVectorY.size(); i++){
157         std::cout << KnotVectorY[i] << " ";
158     }
159     std::cout << std::endl;
160
161     std::cout<< "-----NewKnotVectorX-----" << std::endl;
162     for(unsigned int i=0; i<KnotVectorX.size(); i++){
163         std::cout << KnotVectorX[i] << " ";
164     }
165     std::cout << std::endl;
166
167     std::cout<< "-----IntervalsY-----" << std::endl;
168     for(unsigned int i=0; i<IntervalsY.size(); i++){
169         std::cout << IntervalsY[i] << " ";
170     }
171     std::cout << std::endl;
172
173     std::cout<< "-----IntervalsX-----" << std::endl;
174
175     for(unsigned int i=0; i<IntervalsX.size(); i++){
176         std::cout << IntervalsX[i] << " ";
177     }
178     std::cout << std::endl;
179 }
180
181
182
183 //int nBasisX=NewCtrlPtsW.size();
184 //int nBasisY=NewCtrlPtsW[0].GetNumCols();
185 int nX= KnotVectorX.size()-pX-2;
186 int nY= KnotVectorY.size()-pY-2;
187 int nLoc=(pX+1)*(pY+1);
188 std::vector<Eigen::Triplet<double>> tripletList;
189 tripletList.reserve(hX * hY * nLoc * nLoc);
190
191 //for paralelization
192 int nthreads = omp_get_max_threads();
193 std::vector<std::vector<Eigen::Triplet<double>>> threadTriplets(nthreads);
194 std::vector<std::vector<double>>> threadLinearForms(nthreads, std::vector<double>(size, 0.0));
195
196 // Heuristica de reserva por hilo para evitar muchas realocaciones.
197 // Ajusta segun memoria / experiencia.
198 for (int t = 0; t < nthreads; ++t) {
199     threadTriplets[t].reserve((hX * hY * nLoc * nLoc) / nthreads / 4 + 1000);
200 }
201
202 Eigen::MatrixXd GaussWeights = weightsX * weightsY.transpose();
203 /*
204 cout << "weightsY: " << weightsY << std::endl;
205 cout << "weightsX: " << weightsX << std::endl;
206 cout << "GaussWeights: " << std::endl << GaussWeights << std::endl;
207 */
208 auto start = std::chrono::high_resolution_clock::now();
209 //Precomputation of BasisFuns and its derivatives.
210 std::vector<int> SpanIndexY(IntervalsY.size()-1);
211 std::vector<int> SpanIndexX(IntervalsX.size()-1);
212
213 std::vector<std::vector<Matrix<double>>> Basis_and_DersY=PreBasis_and_Ders(pY, 1, KnotVectorY, IntervalsY, nodesY, SpanIndexY);
214 std::vector<std::vector<Matrix<double>>> Basis_and_DersX=PreBasis_and_Ders(pX, 1, KnotVectorX, IntervalsX, nodesX, SpanIndexX);
215
216 if(ShowValues){
217     std::cout<< "-----Basis_and_DersY-----" << std::endl;
218     for(unsigned int i=0; i< Basis_and_DersY.size(); i++){
219         std::cout<< "-----" << std::endl;
220         for(unsigned int j=0; j< nEvalsY; j++){
221             Basis_and_DersY[i][j].PrintMatrix();
222             std::cout<< "-----" << std::endl;
223         }
224         std::cout<< "-----" << std::endl;
225     }
226     std::cout<< "-----Basis_and_DersX-----" << std::endl;
227     for(unsigned int i=0; i< Basis_and_DersX.size(); i++){

```

```

228     std::cout<< " - - - - - " << std::endl;
229     for(unsigned int j=0; j< nEvalsX; j++){
230         Basis_and_DersX[i][j].PrintMatrix();
231         std::cout<< " - - - - - " << std::endl;
232     }
233     std::cout<< "-+--+--+--+--+--+--+--+--+-" << std::endl;
234 }
235
236 }
237 //std::unordered_map<int, std::vector<std::pair<double,double>>> spanMapY=BuildSpanMap(IntervalsY, SpanIndexY);
238 //std::unordered_map<int, std::vector<std::pair<double,double>>> spanMapX=BuildSpanMap(IntervalsX, SpanIndexX);
239
240 // Double index matrix for easier and faster assembly
241 std::vector<double> detg_vals;
242 #pragma omp parallel for collapse(2) schedule(static)
243 // Assembly of the matrix
244 for (unsigned int i = 0; i < hX; i++) {
245     for (unsigned int j = 0; j < hY; j++) {
246
247         int tid = omp_get_thread_num();
248         // DEBUG: vector local por hilo
249         std::vector<double> local_detg_vals;
250
251
252         // Local buffers referidos por tid
253         auto &localTriplets = threadTriplets[tid];
254         auto &localLinear = threadLinearForms[tid];
255
256         iMatrix<double> K_e(subMatSize, subMatSize);
257         std::vector<double> F_e((pX+1)*(pY+1));
258         if (ShowIterations) {
259             std::cout<<"-----Active Intervals-----"<< std::endl;
260             std::cout<< "IntervalX: (" << IntervalsX[i] << " " << IntervalsX[i+1]<<")"<< std::endl;
261             std::cout<< "IntervalY: (" << IntervalsY[j] << " " << IntervalsY[j+1]<<")"<< std::endl;
262             std::cout<< "SpanIndexX: (" << SpanIndexX[i] <<")"<< std::endl;
263             std::cout<< "SpanIndexY: (" << SpanIndexY[j] <<")"<< std::endl;
264         }
265         double auxeval1X= (IntervalsX[i+1] - IntervalsX[i])/2;
266         double auxeval2X= (IntervalsX[i+1] + IntervalsX[i])/2;
267         double auxeval1Y= (IntervalsY[j+1] - IntervalsY[j])/2;
268         double auxeval2Y= (IntervalsY[j+1] + IntervalsY[j])/2;
269
270
271         int spanU = SpanIndexX[i];
272         int spanV = SpanIndexY[j];
273         iMatrix<double> activeWeights=NewWeights.GetSubMat(spanU-pX,spanU, spanV-pY, spanV).Transpose();
274         auto &DersBasisX_vec = Basis_and_DersX[i]; // vector<iMatrix<double>> size nEvalsX
275         auto &DersBasisY_vec = Basis_and_DersY[j]; // vector<iMatrix<double>> size nEvalsY
276         //std::vector<double> WeightsY=NewWeights.GetSubMat(0,NewWeights.GetNumRows()-1, spanV-pY, spanV-pY).GetCol(0);
277         //std::vector<double> WeightsX=NewWeights.GetSubMat(spanU-pX, spanU-pX,0,NewWeights.GetNumCols()-1).GetRow(0);
278         //std::vector<iMatrix<double>> RationalizedDersBasisY=RationalizeDersBasisFunsVec(spanV, nEvalsY, nodesY, IntervalsY[j],IntervalsY[j+1], pY, 1, nY,
279         // KnotVectorY, Basis_and_DersY[j], WeightsY);
280         //std::vector<iMatrix<double>> RationalizedDersBasisX=RationalizeDersBasisFunsVec(spanU, nEvalsX,nodesX, IntervalsX[i], IntervalsX[i+1],pX, 1, nX ,
281         // KnotVectorX, Basis_and_DersX[i], WeightsX);
282         if (ShowIterations2) {
283             std::cout<<"-----RationalizedDersBasisY-----"<< std::endl;
284             for(unsigned int iter=0; iter<nEvalsY; iter++){
285                 //RationalizedDersBasisY[iter].PrintMatrix();
286                 std::cout<<"-----"<< std::endl;
287             }
288             std::cout<<"-----RationalizedDersBasisX-----"<< std::endl;
289             for(unsigned int iter=0; iter<nEvalsX; iter++){
290                 //RationalizedDersBasisX[iter].PrintMatrix();
291                 std::cout<<"-----"<< std::endl;
292             }
293         }
294
295         //Emulation of computations of the matrix
296         //iMatrix<iMatrix<double>> SurfBasisRat, SurfBasisRatX, SurfBasisRatY;
297         for(int a = 0; a < nEvalsX; ++a){
298
299             double EvalPointX=auxeval1X*nodesX(a)+auxeval2X;
300             //std::cout << "EvalPointX: " << EvalPointX << " ";
301             for(int b = 0; b < nEvalsY; ++b){
302                 double EvalPointY=auxeval1Y*nodesY(b)+auxeval2Y;
303                 //std::cout << "EvalPointY: " << EvalPointY << " ";
304                 iMatrix<std::vector<double>> Allders = SurfaceDerivsAlg1(KnotVectorY.size()-2-pY,pY,KnotVectorY,KnotVectorX.size()-2-pX,pX,KnotVectorX,NewCtrlPtsW,
305                     EvalPointY, EvalPointX, 1);
306
307
308                 iMatrix<std::vector<double>> Aders(2, 2);
309                 iMatrix<double> Wders(2, 2);
310                 for(unsigned int iteri=0; iteri<=1; iteri++){
311                     for(unsigned int iterj=0; iterj<=1; iterj++){
312                         Aders(iteri,iterj)= std::vector<double>(Allders(iteri,iterj).begin(), Allders(iteri,iterj).begin()+3);
313                         Wders(iteri,iterj)=Allders(iteri,iterj)[3];
314                     }
315                 }
316                 iMatrix<iMatrix<double>> RationalizedBasis2d=RationalizeDersBasisFuns2D(DersBasisX_vec[a], DersBasisY_vec[b],activeWeights,pX,pY);
317                 std::vector<double> SurfBasisRat=(RationalizedBasis2d(0,0)).Vectorize();
318                 std::vector<double> SurfBasisRatX=(RationalizedBasis2d(1,0)).Vectorize();
319                 std::vector<double> SurfBasisRatY=(RationalizedBasis2d(0,1)).Vectorize();
320                 iMatrix<std::vector<double>> SurfDers = RatSurfaceDerivs(Aders, Wders, 1);
321
322                 iMatrix<double> PartialXX=SurfBasisRatX*SurfBasisRatX;
323                 iMatrix<double> PartialXY=SurfBasisRatX*SurfBasisRatY;
324                 iMatrix<double> PartialYY=SurfBasisRatY*SurfBasisRatY;
325                 double g11=dot_product(SurfDers(1,0),SurfDers(1,0));
326                 double g12=dot_product(SurfDers(1,0),SurfDers(0,1));
327                 double g22=dot_product(SurfDers(0,1),SurfDers(0,1));
328                 double detg=g11*g22-g12*g12;
329                 double J=std::sqrt(detg);
330                 double Jinv=1/J;

```

```

331     double dOmega = auxeval1X * auxeval1Y * GaussWeights(a,b);
332     double dA = dOmega*J;
333     double auxconstant = dOmega * Jinv;
334     double g11c= g22*auxconstant;
335     double g12c= -g12*auxconstant;
336     double g22c= g11*auxconstant;
337     //std::cout << " <<<<<<<<< " << std::endl;
338     local_detg_vals.push_back(detg);
339     //iMatrix<double> Kq = g22c*PartialXX + g12c*(PartialXY + PartialXY.Transpose()) + g11c*PartialYY;
340     iMatrix<double> Kq(subMatSize, subMatSize);
341     Kq = PartialXX;
342     Kq *= g22c;
343
344     iMatrix<double> Temp = PartialXY + PartialXY.Transpose();
345     Temp *= g12c;
346     Kq += Temp;
347
348     Temp = PartialYY;
349     Temp *= g11c;
350     Kq += Temp;
351
352     K_e += Kq;
353     for (int A = 0; A < SurfBasisRat.size(); ++A) {
354         F_e[A] += SurfBasisRat[A] * f(EvalPointX, EvalPointY) * dA;
355     }
356 }
357 }
358
359 // Emulation of local load vector (all ones)
360
361 // Rellenar triplets en matriz global
362 for(int a = 0; a <= pX; ++a){
363
364     for(int b = 0; b <= pY; ++b){
365         int i_local = b*(pX+1) + a;
366         int I_global = (spanU - pX + a) + (spanV - pY + b)*nElementsX;
367         localLinear[I_global] += F_e[i_local];
368         //std::cout<< "-----" << std::endl;
369         //std::cout << "I_global= " << I_global << std::endl;
370
371         //LinearForm[I_global] += F_e[i_local];
372
373         for(int c = 0; c <= pX; ++c){
374             for(int d = 0; d <= pY; ++d){
375
376                 int j_local = d*(pX+1) + c;
377                 int J_global = (spanU - pX + c) + (spanV - pY + d)*nElementsX;
378
379                 //std::cout << "J_global= " << J_global << std::endl;
380                 double val = K_e(i_local, j_local);
381                 if(val != 0.0){
382                     //localTriplets.emplace_back(I_global, J_global, val);
383                     localTriplets.emplace_back(I_global, J_global, val);
384                 }
385             }
386         }
387     }
388 }
389
390 #pragma omp critical
391 {
392     detg_vals.insert(detg_vals.end(),
393                     local_detg_vals.begin(),
394                     local_detg_vals.end());
395 }
396 }
397 }
398
399 for (int t = 0; t < nthreads; ++t) {
400     // mover triplets al tripletList (efficient push_back many)
401     tripletList.insert(tripletList.end(), threadTriplets[t].begin(), threadTriplets[t].end());
402
403     // sumar fuerza local en LinearForm
404     for (int idx = 0; idx < size; ++idx) {
405         LinearForm[idx] += threadLinearForms[t][idx];
406     }
407 }
408 global.setFromTriplets(tripletList.begin(), tripletList.end());
409
410 auto end = std::chrono::high_resolution_clock::now();
411 auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start);
412 std::cout << "Tiempo tomado: " << duration.count() << " nanosegundos\n";
413
414 if(PrintMatrixes){
415     std::cout<< "-----GlobalMat-----" << std::endl;
416     for (int k = 0; k < global.outerSize(); ++k){
417         for (Eigen::SparseMatrix<double>::InnerIterator it(global, k); it; ++it){
418             std::cout << "(" << it.row() << ", " << it.col() << ") : " << it.value() << "\n";
419         }
420     }
421     //std::cout<<Eigen::MatrixXd(global) <<std::endl;
422 }
423
424 if(PrintMatrixes){
425     std::cout<< "-----GlobalLinearForm-----" << std::endl;
426     for (int k = 0; k < LinearForm.size(); ++k){
427         std::cout << LinearForm[k] << ", ";
428     }
429     std::cout << std::endl;
430 }
431
432 // ----- Imposicion de Dirichlet homogeneo (u = 0) -----
433 std::vector<char> isDirichlet(size, false);

```

```

434 std::vector<int> dirichletNodes;
435 dirichletNodes.reserve(2 * (nElementsX + nElementsY));
436
437 for (int j = 0; j < nElementsY; ++j) {
438     for (int i = 0; i < nElementsX; ++i) {
439         int idx = i + j * nElementsX;
440         if (i == 0 || i == nElementsX - 1 || j == 0 || j == nElementsY - 1) {
441             isDirichlet[idx] = 1;
442             dirichletNodes.push_back(idx);
443         }
444     }
445 }
446 if (ShowConditions) {
447     std::cout << "Numero de nodos Dirichlet detectados: " << dirichletNodes.size() << std::endl;
448 }
449
450 for (int col = 0; col < global.outerSize(); ++col) {
451     for (Eigen::SparseMatrix<double>::InnerIterator it(global, col); it; ++it) {
452         int row = it.row();
453         int c = it.col();
454         if (isDirichlet[row] && row != c) {
455             it.valueRef() = 0.0;
456         }
457     }
458 }
459
460 for (int k = 0; k < (int)dirichletNodes.size(); ++k) {
461     int colIdx = dirichletNodes[k];
462     for (Eigen::SparseMatrix<double>::InnerIterator it(global, colIdx); it; ++it) {
463         int row = it.row();
464         if (row != colIdx) {
465             it.valueRef() = 0.0;
466         }
467     }
468 }
469
470 for (int k = 0; k < (int)dirichletNodes.size(); ++k) {
471     int idx = dirichletNodes[k];
472     global.coeffRef(idx, idx) = 1.0;
473     LinearForm[idx] = 0.0;
474 }
475
476 if (PrintMatrixes2) {
477     std::cout << "-----GlobalMat-----" << std::endl;
478     for (int k = 0; k < global.outerSize(); ++k) {
479         for (Eigen::SparseMatrix<double>::InnerIterator it(global, k); it; ++it) {
480             std::cout << "(" << it.row() << ", " << it.col() << ") : " << it.value() << "\n";
481         }
482     }
483     std::cout << "-----GlobalLinearForm-----" << std::endl;
484     for (int k = 0; k < LinearForm.size(); ++k) {
485         std::cout << LinearForm[k] << ", ";
486     }
487     std::cout << std::endl;
488 }
489
490 global.makeCompressed();
491
492 if (ShowSolutions) {
493     std::cout << "Resolviendo sistema (K u = F) con Conjugate Gradient..." << std::endl;
494 }
495
496 int maxIter = std::max(100, std::min(5 * size, 500000));
497 auto tsolve_start = std::chrono::high_resolution_clock::now();
498 Eigen::VectorXd solution;
499 // IncompleteCholesky como preconditionador (SPD)
500 try {
501     using Precond = Eigen::IncompleteCholesky<double>;
502     Eigen::ConjugateGradient<Eigen::SparseMatrix<double>, Eigen::Lower|Eigen::Upper, Precond> solver;
503     solver.compute(global);
504     if (solver.info() != Eigen::Success) {
505         throw std::runtime_error("Precond compute fallo (IC).");
506     }
507     solver.setTolerance(1e-8);
508     solver.setMaxIterations(maxIter);
509     solution = solver.solve(LinearForm);
510     auto tsolve_end = std::chrono::high_resolution_clock::now();
511
512     double elapsed_s = std::chrono::duration_cast<std::chrono::duration<double>>(tsolve_end - tsolve_start).count();
513     std::cout << "Tiempo solución: " << elapsed_s << " s\n";
514     if (ShowSolutions) {
515         std::cout << "Solver info (IC precondition): " << solver.info() << "\n";
516         std::cout << "Iterations: " << solver.iterations() << ", Estimated error: " << solver.error() << "\n";
517         std::cout << "Solución: " << solution.transpose() << std::endl;
518     }
519 }
520
521 } catch (const std::exception &e) {
522     // Fallback: usar preconditionador diagonal (Jacobi) si IC falla
523     if (ShowSolutions) {
524         std::cerr << "Fallo preconditionador IncompleteCholesky o excepción: " << e.what() << "\n";
525         std::cerr << "Usando ConjugateGradient + DiagonalPreconditioner (fallback)." << std::endl;
526     }
527     Eigen::ConjugateGradient<Eigen::SparseMatrix<double>, Eigen::Lower|Eigen::Upper, Eigen::DiagonalPreconditioner<double>> solver2;
528     solver2.compute(global);
529     solver2.setTolerance(1e-8);
530     solver2.setMaxIterations(2*maxIter);
531     solution = solver2.solve(LinearForm);
532
533     auto tsolve_end2 = std::chrono::high_resolution_clock::now();

```

```

537     double elapsed_s2 = std::chrono::duration_cast<std::chrono::duration<double>>(tsolve_end2 - tsolve_start).count();
538     std::cout << "Tiempo solution: " << elapsed_s2 << " s\n";
539     if (ShowSolutions){
540         std::cout << "Solver info (Diag precondition): " << solver2.info() << "\n";
541         std::cout << "Iterations: " << solver2.iterations() << ", Estimated error: " << solver2.error() << "\n";
542         std::cout << "Solution: " << solution.transpose() << std::endl;
543     }
544 }
545
546 //----- Evaluacion en los elementos de la Solucion -----
547
548
549
550 iMatrix<double> MatSol(nElementsX, nElementsY, solution);
551 //iMatrix<double> MatSol=MatSol1.Transpose();
552 iMatrix<double> SolEvals(nEvalsX*hX, nEvalsY*hY);
553 iMatrix<double> SolEvalsX(nEvalsX*hX, nEvalsY*hY);
554 iMatrix<double> SolEvalsY(nEvalsX*hX, nEvalsY*hY);
555 iMatrix<double> SolAnalytic(nEvalsX*hX, nEvalsY*hY);
556 iMatrix<double> SolAnalyticX(nEvalsX*hX, nEvalsY*hY);
557 iMatrix<double> SolAnalyticY(nEvalsX*hX, nEvalsY*hY);
558 iMatrix<std::vector<double>> Surf(nEvalsX*hX, nEvalsY*hY);
559 for (unsigned int i = 0; i < hX; i++) {
560     for (unsigned int j = 0; j < hY; j++) {
561
562         double auxeval1X= (IntervalsX[i+1] - IntervalsX[i])/2;
563         double auxeval2X= (IntervalsX[i+1] + IntervalsX[i])/2;
564         double auxeval1Y= (IntervalsY[j+1] - IntervalsY[j])/2;
565         double auxeval2Y= (IntervalsY[j+1] + IntervalsY[j])/2;
566
567
568         int spanU = SpanIndexX[i];
569         int spanV = SpanIndexY[j];
570         iMatrix<double> activeWeights=NewWeights.GetSubMat(spanU-pX,spanU, spanV-pY, spanV).Transpose();
571         auto &DersBasisX_vec = Basis_and_DersX[i]; // vector<iMatrix<double>> size nEvalsX
572         auto &DersBasisY_vec = Basis_and_DersY[j]; // vector<iMatrix<double>> size nEvalsY
573         for(int a = 0; a < nEvalsX; ++a){
574
575             double EvalPointX=auxeval1X*nodesX(a)+auxeval2X;
576             //std::cout << "EvalPointX: " << EvalPointX << " ";
577             for(int b = 0; b < nEvalsY; ++b){
578                 double EvalPointY=auxeval1Y*nodesY(b)+auxeval2Y;
579                 iMatrix<iMatrix<double>> RationalizedBasis2d=RationalizeDersBasisFuns2D(DersBasisX_vec[a], DersBasisY_vec[b],activeWeights,pX,pY);
580
581                 iMatrix<double> Rbasis = RationalizedBasis2d(0, 0);
582                 iMatrix<double> RbasisX = RationalizedBasis2d(1, 0);
583                 iMatrix<double> RbasisY = RationalizedBasis2d(0, 1);
584                 iMatrix<double> LocalSol = MatSol.GetSubMat(spanU - pX, spanU, spanV - pY, spanV);
585                 double u_h = 0.0;
586                 double u_hX= 0.0;
587                 double u_hY= 0.0;
588                 for (int ii = 0; ii < Rbasis.GetNumRows(); ++ii) {
589                     for (int jj = 0; jj < Rbasis.GetNumCols(); ++jj) {
590                         u_h += Rbasis(ii, jj) * LocalSol(jj, ii);
591                         u_hX+= RbasisX(ii,jj) * LocalSol(jj, ii);
592                         u_hY+= RbasisY(ii,jj) * LocalSol(jj, ii);
593                     }
594                 }
595                 SolEvals(i * nEvalsX + a, j * nEvalsY + b) = u_h;
596                 SolEvalsX(i * nEvalsX + a, j * nEvalsY + b) = u_hX;
597                 SolEvalsY(i * nEvalsX + a, j * nEvalsY + b) = u_hY;
598                 std::vector<double> SurfPoint=SurfacePointRational(nY, pY, KnotVectorY, nX, pX, KnotVectorX, NewCtrlPtsW, EvalPointY, EvalPointX);
599                 Surf(i*nEvalsX + a, j*nEvalsY + b)=SurfPoint;
600                 SolAnalytic(i*nEvalsX+a, j*nEvalsY +b) = analyticSol(SurfPoint[0], SurfPoint[1]);
601                 SolAnalyticX(i*nEvalsX+a, j*nEvalsY +b) = analyticSol_dx(SurfPoint[0], SurfPoint[1]);
602                 SolAnalyticY(i*nEvalsX+a, j*nEvalsY +b) = analyticSol_dy(SurfPoint[0], SurfPoint[1]);
603                 /*
604                 std::cout<<"-----"<<std::endl;
605                 std::cout<< "SurfPoint[0]=" << SurfPoint[0]<< " , EvalPointX="<< EvalPointX <<std::endl;
606                 std::cout<< "SurfPoint[1]=" << SurfPoint[1]<< " , EvalPointY="<< EvalPointY <<std::endl;
607                 std::cout << "Analytic= " << SolAnalytic(i*nEvalsX+a, j*nEvalsY +b) << " , Numeric= " << SolEvals(i*nEvalsX + a, j*nEvalsY + b) << std::endl;
608                 */
609             }
610         }
611     }
612 }
613 //MatSol.PrintMatrix();
614
615 //ExportToTxt(SolEvals, Surf, PATH + "surfaceCircle3d_hX="+ std::to_string(static_cast<int>(hX)) + "_pX=" + to_string(pX) + "_hY="+ std::to_string(static_cast<int>(hY)) +
616 // " _p=" + to_string(pY) + " data.txt");
617
618 //-----L2 norm calculations-----
619
620 double L2_error_sq = 0.0;
621 double H1_semi_error_sq = 0.0;
622
623 for (unsigned int i = 0; i < hX; i++) {
624     double auxeval1X= (IntervalsX[i+1] - IntervalsX[i])/2;
625     double auxeval2X= (IntervalsX[i+1] + IntervalsX[i])/2;
626     for (unsigned int j = 0; j < hY; j++) {
627         double auxeval1Y= (IntervalsY[j+1] - IntervalsY[j])/2;
628         double auxeval2Y= (IntervalsY[j+1] + IntervalsY[j])/2;
629         double h_elementos = auxeval1X*auxeval1Y;
630         double cumSumElements=0.0;
631         double cumSumH1 = 0.0;
632         for (int a = 0; a < nEvalsX; ++a) {
633             double EvalPointX=auxeval1X*nodesX(a)+auxeval2X;
634             for (int b = 0; b < nEvalsY; ++b) {
635                 // --- Diferencial parametrico ---
636
637
638
639

```

```

640     double EvalPointY=auxeval1Y*nodesY(b)+auxeval2Y;
641     iMatrix<std::vector<double>> Allders = SurfaceDerivsAlg1(KnotVectorY.size()-2-pY,pY,KnotVectorY,KnotVectorX.size()-2-pX,pX,
642                                     KnotVectorX,NewCtrlPtsW, EvalPointY, EvalPointX, 1);
643
644
645     iMatrix<std::vector<double>> Aders(2, 2);
646     iMatrix<double> Wders(2, 2);
647     for(unsigned int iteri=0; iteri<=1; iteri++){
648         for(unsigned int iterj=0; iterj<=1; iterj++){
649             Aders(iteri,iterj)= std::vector<double>(Allders(iteri,iterj).begin(), Allders(iteri,iterj).begin()+3);
650             Wders(iteri,iterj)=Allders(iteri,iterj)[3];
651         }
652     }
653     iMatrix<std::vector<double>> SurfDers= RatSurfaceDerivs(Aders, Wders, 1);
654
655     // Punto fisico (x, y, z) de la superficie S
656     std::vector<double> SurfPoint = SurfDers(0, 0);
657     double x_phys = SurfPoint[0];
658     double y_phys = SurfPoint[1];
659
660     // Gradiente fisico de la solucion analitica (grad x u)
661     double du_dx_analytic = -x_phys / 2.0;
662     double du_dy_analytic = -y_phys / 2.0;
663
664     // Gradiente parametrico de la solucion analitica (grad xi u) usando la Regla de la Cadena
665     // dS/dxi = SurfDers(0, 1) = (dx/dxi, dy/dxi, dz/dxi)
666     std::vector<double> S_xi = SurfDers(0, 1);
667     double du_dxi_analytic = du_dx_analytic * S_xi[0] + du_dy_analytic * S_xi[1];
668     // (El termino dz/dxi * du/dz es 0 ya que u no depende de z)
669
670     // dS/deta = SurfDers(1, 0) = (dx/deta, dy/deta, dz/deta)
671     std::vector<double> S_eta = SurfDers(1, 0);
672     double du_deta_analytic = du_dx_analytic * S_eta[0] + du_dy_analytic * S_eta[1];
673
674
675     double g11=dot_product(SurfDers(1,0),SurfDers(1,0));
676     double g12=dot_product(SurfDers(1,0),SurfDers(0,1));
677     double g22=dot_product(SurfDers(0,1),SurfDers(0,1));
678
679     double detg=g11*g22-g12*g12;
680     double J=std::sqrt(detg);
681     double Jinv=1/J;
682
683     double g11c_J = g22 * Jinv; // g^11 * J
684     double g12c_J = -g12 * Jinv; // g^12 * J
685     double g22c_J = g11 * Jinv; // g^22 * J
686     //double dXiEta = auxeval1X * auxeval1Y * GaussWeights(a,b);
687     double diff = SolEvals(i*nEvalsX + a, j*nEvalsY + b) - SolAnalytic(i*nEvalsX + a, j*nEvalsY + b);
688     double diff_dx = SolEvalsX(i*nEvalsX + a, j*nEvalsY + b) - du_dxi_analytic; // d/dxi
689     double diff_dy = SolEvalsY(i*nEvalsX + a, j*nEvalsY + b) - du_deta_analytic; // d/deta
690
691
692     // --- Aportacion al error L2 ---
693     cumSumElements += diff * diff * GaussWeights(a,b)*J;
694     cumSumH1 += (diff_dx * diff_dx *g22c_J +2.0*g12c_J*diff_dx*diff_dy + diff_dy * diff_dy*g11c_J) * GaussWeights(a,b);
695 }
696
697 L2_error_sq+=cumSumElements*h_elementos;
698 H1_semi_error_sq += cumSumH1 * h_elementos;
699 }
700 }
701
702 double L2_error = std::sqrt(L2_error_sq);
703 double H1_error = sqrt(L2_error_sq + H1_semi_error_sq);
704 std::cout << "Norma L2 = " << L2_error << std::endl;
705 std::cout << "Norma H1 = " << H1_error << std::endl;

```